

Tecnología Meta4: Manual de referencia

Lenguaje LN4



© 2020 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: enero de 2003.

Fecha de la última publicación: noviembre de 2020.

Tabla de contenidos

■	Introducción	1
■	Lenguaje LN4 y estructuras de nodo	2
	Llamadas a los valores de los elementos de las estructuras de nodo.	2
■	Elementos básicos del lenguaje	5
	Variables y constantes	5
	Tipos de datos	5
	Fecha	6
	Número	6
	Cadena	6
	Llamadas a funciones	7
	Retorno de valores.	7
	Operadores LN4	7
	Comentarios en las fórmulas	9
	Expresiones regulares	9
■	Estructuras de control.	11
	Sentencias condicionales	12
	Sentencia If ... Then ... EndIf	12
	Sentencia If ... Then ... Else ... EndIf	13
	Sentencia If ...Then ...Elself ...Else ...EndIf	15
	Sentencias de repetición	16
	Sentencia Do ... Until condición	16
	Sentencia While condición ... Wend	17
	Sentencia For ... Next	18
■	Manejo de elementos	19
	Elementos del sistema	19
	Sintaxis de lectura y asignación de elementos	19
	Atributos y Atributos-Métodos	19
	Ejecución de métodos y conceptos	22
	Ejecución de un grupo de conceptos	22
■	Ejecución de procesos con o sin tramos	24
■	Funciones LN4	25
	Argumentos de las funciones LN4	25
	Tipos de funciones	26
	Funciones básicas	27

BeginVMTransaction ()	28
Tipos de transacciones en Meta4Object Engine	28
Métodos transaccionales	29
Posibles situaciones con transacciones de Meta4Object Engine y su resolución	29
ConfigCSTimeOut (Tipo, Marca)	32
DialogBox (Estilo, Título, Cabecera,...)	33
EndVMTransaction (M4_COMMIT/M4_ROLLBACK / M4_ROLLBACK_RESUME)	34
ExistNodeItemIndex (Nodo, Posición, Filtro) o (Nodo, IDElemento)	34
Flatten (Nodo destino)	35
GetChannelNumberOfNodes ([M4Obj])	36
GetCsTimeOut (Tiempo, Tipo)	36
GetRegistryValue (DirectorioRegistro, VarNombre)	37
GetServerValue (DirectorioOBL, VarNombre)	37
GetSliceMode ()	37
GetUniqueID ()	38
GetVariantType (var)	38
IsNull (var)	38
JoinSlices ()	39
MessageBox (Título, Cabecera,...)	39
Null ()	39
NullValue ()	40
SetCsTimeOut (Tiempo, Tipo)	40
SetCSTimeoutFunction (NombreDLL, NombreFunción)	41
SetSliceMode (ModoTramo)	41
Sleep	42
TickCount	42
Unflatten (elemento...[opcional])	42
Funciones de cadenas	43
Ascii (Cadena)	44
Chr (Código)	44
ConvertCase (Cadena, Low_up)	45
IndexOf (Cadena, Subcadena, ÍndiceInicial)	46
LastIndexOf (Cadena, SubCadena, [ÍndiceInicial])	46
Length (var)	47
MatchRegExp (Cadena, ExpReg, [SensibleMayúsculasMinúsculas])	48
Mid (Valor, Inicio, longitud)	48
Replace (Cadena, SubcadenaAntigua, SubcadenaNueva)	48
StrIn (Cadena1, Cadena2)	49
Trim (Cadena, Dirección)	50
Funciones de conversión de tipos	51
ToDate (Valor)	51

ToDouble (Valor)	51
ToInt (Valor)	52
ToString (Valor, [Precisión])	53
Funciones matemáticas	54
Abs (Fecha o Número)	54
Bound (Valor, limInf, limSup)	55
Div (Número1, Número2)	56
Format (Número, Precision)	56
Fraction (Número)	57
Log (Number, [Base])	57
Max (Value1, Value2, ... , Valuen)	58
Min (Valor1, Valor2, ... , Valorn)	58
Percentage (Porcentaje, Total)	59
Power (Número1, Número2)	60
Rnd (Max)	60
Round (Valor, Precisión)	60
Truncate (Valor, Precisión)	62
Funciones de fecha y hora	64
AddMonths (Fecha, NúmeroMeses)	65
AddYears (Fecha, NúmeroAños)	66
CommonTime (InicFecha1 , Fecha de fin1 , InicFecha2 , Fecha de fin2)	66
CommonTimeStamp (FechaInicio ₁ , FechaFin1 , FechaInicio2 , FechaFin2)	66
Date30 (Fecha)	67
Date30Ex (Fecha, [FebruaryAs30])	67
DateAdd (Fecha1, Unidades, TipoUnidad)	67
DateDiff (Fecha1, Fecha2, TipoUnidad, [InicioSemana])	68
DateToAllNumbers (Fecha, var1, var2, var3, var4, var5, var6)	70
DateToNumbers (Fecha, var1, var2, var3)	71
DayOfTheWeek (Fecha)	72
DaysOfMonth (Fecha)	72
DaysOfYear (Número)	73
DxCompare (Cadena1, Cadena2)	74
DxDate (Día, Mes, Año)	74
DxDate30 (Día, Mes, Año)	74
DxDate30Ex (Día, Mes, Año, [FebruaryAs30])	75
DxDatElso (Fecha)	75
DxDatElsoD (Fecha)	76
DxDatElsoTS (Fecha)	76
DxDay (Fecha)	77
DxDaysOfMonth (Mes, Año)	77
DxInc (Valor)	77

DxMonth (Fecha)	78
DxRound (Valor, Precisión)	78
DxRound50 (Valor)	79
DxRoundCent (Valor)	79
DxTruncaCent (Valor)	79
DxYear (Fecha)	80
Months (Fecha1, Fecha2)	80
NumbersToDate (Número1, Número2, ..., Número6)	81
Today ()	82
TodayGMT ()	82
TodayNow ()	82
TodayNowGMT ()	83
Years (Fecha1, Fecha2)	83
Funciones de cambio de moneda	84
Meta4Object de cambio de moneda	84
Nodo Currency	85
Nodo Rates	85
Nodo Aux_Cur_Ex	86
Elementos de tipo moneda	86
Funciones LN4 para el cambio de monedas	87
CurExchangeRecord (MonedaDestino, bNuevoRegistro, [FechaCambio], [TipoCambio])	87
CurExchangeRecordRounded (MonedaDestino, bNuevoRegistro, [FechaCambio], [TipoCambio])	88
CurGetDecimals (MonedaOrigen)	89
CurGetExchange (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)	89
CurGetExchangeCurrency (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)	90
CurGetExchangeCurrencyRounded (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)	90
CurGetExchangeRounded (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)	91
Ejemplos de funcionamiento del cambio de moneda	91
Funciones de archivos	96
ClearFiles ()	96
CloseFile (Manejador)	96
ConcatFile (Fichero, Valor, [AddEOL] Encoding)	97
DestroyFile (NombreFichero)	98
InitFile (Fichero, Valor, [AddEOL], Encoding)	98
OpenFile (NombreFichero, &Manejador)	99
ReadFile (Manejador, Tipo, &CadenaDevuelta)	99
Funciones de lectura de argumentos	100

GetArgument (Arg_pos)	100
GetArgumentNumber ()	100
Funciones de fechas de ejecución.	100
AppEndDate ()	101
AppRunDate ()	101
AppStartDate ()	101
GetEndDate ()	101
GetRunDate ()	101
GetRunningEndDate ()	101
GetRunningRunDate ()	102
GetRunningStartDate ()	102
GetStartDate ()	103
SetEndDate (Fecha)	103
SetRunDate (Fecha)	103
SetStartDate (Fecha)	104
Funciones para manejo de registros	104
AddFilter (Código/operación, [Estático], [Nombre del argumento_i], [Valor del argumento_i]...)	106
AddRegister ()	107
AddSortFilter ()	108
Begin ()	108
BSearch (Índice, Valores...)	109
Call (Argumentos,..., Nodo, Elemento)	109
ChannelAttribCall (Argumentos,..., Instancia de Meta4Object, Nodo, Elemento, Atributo) ..	110
ChannelAttribGetSliceValue(Instancia de Meta4Object, Nodo, Elemento, Tramo, Atributo) ..	111
ChannelAttribGetValue (Instancia de Meta4Object, Nodo, Elemento, Atributo)	111
ChannelCall (Argumentos,..., Instancia de Meta4Object, Nodo, Elemento)	112
CleanFilter ()	113
CloneRegister	113
CopyBranch (FileName , [Record] , [Node] , [M4Obj] , ...)	113
CopyRegister (Meta4Object, Nodo, [ÍndiceRegistro], [Filtro del elemento origen], [Filtro del elemento destino])	113
Count ()	114
DeleteAllRegisters ()	115
DeleteRegister ()	115
DestroyAllRegisters ()	115
DestroyBlock ()	115
DestroyRegister ()	116
End ()	116
ExecuteBranch (Ámbito, Nombre del elemento)	116
ExportXML (M4Obj, MapType, XML, Validate, DTD, Public, Handle, UseSchema, Namespace,	

...)	117
FindRegister (Elemento ₁ , Valor ₁ , Criterio ₁ , ..., Elemento _n , Valor _n , Criterio _n , [ÍndiceRegistro])	117
GetChannelValue (Instancia de Meta4Object, Nodo, Elemento, Registro, [Tramo])	123
GetCurrent ()	123
GetFilterArgument (ID_Filtro, ID_Argumento)	124
GetValue (Nodo, Elemento, Registro, [Tramo])	124
Gonext ()	125
GoPrevious ()	125
InsertRegister ()	126
IsBlockUpdated(ai_bRecursive)	127
IsDeleted ()	128
IsEOF ()	128
IsNew ()	128
IsNewAndDeleted()	129
IsNodeUpdated(ai_bRecursive)	129
IsUpdated ()	129
IsUpdatedDB ()	129
LoadXML (M4Obj, MapType, XML, Validate, First, Last, Handle, UseSchema, [Consistency], ...)	130
MoveTo (ÍndiceRegistro)	130
MoveToEOF ()	131
PasteBranch (FileName , NewRecord , KeepMarks , DeleteFile , [Record] , [Node] , [M4Obj] , ...)	132
RefreshFilter ()	132
RemoveFilter ()	132
SendBlockMark (Modo de envío)	133
SendNodeMark (Modo de envío)	133
SendRegisterMark (Modo de envío)	133
SetChannelValue (Instancia de Meta4Object, Nodo, Elemento, Registro, [Tramo], Valor)	133
SetFilterArgument (ID_Filtro, ID_Arg, Arg_Val, ...)	134
SetInserted ()	134
SetPriority (Prioridad, Nodo, Elemento, Registro, [Tramo])	134
SetValue (Nodo, Elemento, Registro, [Tramo], Valor)	135
SetValuePriority (Prioridad, Nodo, Elemento, Registro, [Tramo], Valor)	138
Sort ([Índice])	139
Funciones para la base de datos lógica.	139
Tipos de transacciones	140
Métodos transaccionales	141
BeginDBTransaction	141
BeginTransaction ()	141

Delete_blk ()	142
Delete_prg ()	142
EndDBTransaction (M4_COMMIT / M4_ROLLBACK / EXECUTE_POSTVALIDATION)	143
EndTransaction (M4_COMMIT / M4_ROLLBACK / EXECUTE_POSTVALIDATION / M4_ROLLBACK_RESUME)	143
ExecuteDirectSQL (ID_Conexión, Máscara,...)	144
ExecuteRealSQL (ID_Conexión,...)	145
ExecuteSQL ()	145
GetAutoloadMode ()	145
GetNodeAutoloadMode (Modo)	145
Insert_blk ()	145
Insert_prg ()	146
Load_blk ()	146
Load_prg ()	147
Persist_tree ()	147
SetAutoloadMode (Modo)	148
SetLoadRange ([M4Obj] , [Node] , RangeSize , RangeStart , ...)	149
SetLoadTimeOut([M4ObjectInstance], [Node], TimeOut)	150
SetNodeAutoloadMode (Modo)	151
Update_blk ()	152
Update_prg ()	152
Funciones de Meta4Objects	153
AddAsPartner ()	153
CheckModify ()	154
GetHandle ()	154
GetProperty (Propiedad)	154
IsDataCacheable ()	154
Load ()	155
LoadFromFile (NombreArchivo)	155
Persist ()	155
PersistToFile ([NombreArchivo])	155
Release ()	155
RemoveAsPartner ()	155
SetProperty (Propiedad, Valor)	156
Unload ()	156
Funciones básicas de llamadas de nivel 2	156
ApplyBookMark (BookMark)	156
CloseChannel (IdInstancia)	157
DefineInstance (Instancia, Meta4Object, PolíticaCarga, PolíticaCompartición)	157
DefineInstanceEx (Instancia, Meta4ObjectNombre, PolíticaCarga, PolíticaCompartición, [IDOrganización], [TipoOrganización], [CSTipoAcceso], [DataLanguage], [Role],...)	158

EraseAllL2Instances	158
GetBookMark (Type)	159
InsertChannel	160
OpenChannel (Meta4Object, IDInstancia, Ámbito, ModoApertura)	160
OpenChannelEx (Meta4Object, IdInstancia, Ambito, ModoApertura, [OrganizaciónID], [TipoOrganización], [CSTipoAcceso])	161
ReleaseBookMark (BookMark)	162
Funciones para ejecución de código JIT	162
clcCheckSyntax ()	163
clcCompile ()	163
clcExecuteJIT (Manejador, ValArg1, ValArg2,...)	163
clcExecuteLN4JIT (CódigoLN4, Arg1, ValArg1, Arg2, ValArg2...)	163
clcGetPolish ()	163
clcOldTItolDTI ()	164
clcPrepareJIT (CódigoLN4, &Manejador, Arg1, Arg2,...)	164
clcReleaseJIT (Manejador)	164
clcRestoreSource ()	164
clcSourceTold ()	164
clcSourceToNumber ()	164
clcSourceToString ()	165
CompileBlock ()	165
LocalTest ()	165
Funciones de traza	165
DumpChannel ()	165
DumpNode ()	166
DumpRegister ()	166
DumpStatistics (NombreArchivo, Meta4Object, [Nivel])	167
GetErrorString (Módulo, Submódulo, Código, [Argumento 1])	167
GetLog (Posición, &Tipo, &Módulo, &Submódulo, &Código, &TextoAdicional)	168
GetLogSize ()	169
PurgeLog (NumElimina)	169
SetLog (Tipo, Módulo, Submódulo, Código, [Argumento])	169
Funciones para transacciones de memoria	170
CheckPointBlock ()	170
CheckPointChannel ()	171
CheckPointNode ()	171
CheckPointRegister ()	171
UndoBlock ()	172
UndoChannel ()	172
UndoNode ()	172
UndoRegister ()	172

Funciones de lectura de metadatos	172
GetChannelData ([M4Obj] , DataType , ...)	173
GetChannelId	174
GetNodeData ([M4Obj] , [Node] , DataType , [DumpError] , ...)	174
GetNodeIndex (Nodo)	175
GetNodeItemArgument (IdArg, TipoArg, Prec, IsRefArg, ArgPos, Nodo, Posición, Filtro) o (IdArg, TipoArg, Prec, IsRefArg, ArgPos, Nodo, Posición, IDElemento)	175
GetNodeItemArgumentNumber (NúmeroFijo, HasVarArg, NúmeroRef, Nodo, Posición, Filtro) o (NúmeroFijo, HasVarArg, NúmeroRef, Nodo, IDElemento)	176
GetNodeItemArgumentPos (Nodo, Posición, Filtro, IDArg) o (Nodo, IDElemento, IDArg)	177
GetNodeItemData([M4Obj], Nodo, Posición, Filtro, TipoDato) o ([M4Obj], Nodo, IDElemento, TipoDato)	177
GetNodeItemId (Nodo, Posición, Filtro)	179
GetNodeItemIndex (Nodo, Posición, Filtro) o (Nodo, IDElemento)	179
GetNodeItemInternalType (Nodo, Posición, Filtro) o (Nodo, IDElemento)	180
GetNodeItemM4Type (Nodo, Posición, Filtro) o (Nodo, IDElemento)	181
GetNodeItemPrecision (Nodo, Posición, Filtro) o (Nodo, IDElemento)	182
GetNodeItemPrecisionEx (Precision, Nodo, Posición, Filtro) o (Precision, Nodo, IDElemento) 183	
GetNodeItemReadFieldId (Nodo, Posición, Filtro) o (Nodo, IDElemento)	184
GetNodeItemReadObjectAlias (Nodo, Posición, Filtro) o (Nodo, IDElemento)	184
GetNodeItemReadObjectId (Nodo, Posición, Filtro) o (Nodo, IDElemento)	185
GetNodeItemScale (Nodo, Posición, Filtro) o (Nodo, IDElemento)	185
GetNodeItemScope (Nodo, Posición, Filtro) o (Nodo, IDElemento)	186
GetNodeItemSliceTotalize (Nodo, Posición, Filtro) o (Nodo, IDElemento)	187
GetNodeItemType (Nodo, Posición, Filtro) o (Nodo, Elemento)	187
GetNodeItemWriteFieldId (Nodo, Posición, Filtro) o (Nodo, IDElemento)	188
GetNodeItemWriteObjectAlias (Nodo, Posición, Filtro) o (Nodo, IDElemento)	188
GetNodeItemWriteObjectId (Nodo, Posición, Filtro) o (Nodo, IDElemento)	189
GetNodeNumberOfItems (Nodo, Filtro)	190
GetNodeReadObjectId (NodeId o NodeIndex)	191
GetNodeWriteObjectId (NodeId o NodeIndex)	191
GetThisItemId	191
GetThisNodeId	191
Funciones de nómina y del sistema de asignación de valores	191
Sistema de asignación de valores	192
ApplyTable (NodoValores, NodoHistórico, NodoOrganización, [HistóricoCP, Valor sCP], iNumCP, [ApplyAlways])	193
CreateOrgChart ()	196
ExecuteGroup (Nombre de grupo)	196
PrInitApplyValue (Id_DM, Grupo)	196
PrSetDates (FechaInicio, FechaFin, [FechaEjecución])	197

PrSetPeriods (FechaInicio, FechaFin, FechaEjecución, ModoTramo)	197
ResetexecuteGroup()	198
TransferApplyValue (Host_Meta4Object, id, Host_nodo_id, [DMD, MonedaDestino, FechaIntercambio, TipoIntercambio])	198
YTDSearch (Meta4Object, Nodo, FechaInicio, FechaFin, FechaEntrada, FechaPago, PagoFrec, TipoPago, TipoBúsqueda, RevBeh, SelPays, TotOp)	199
YTDSearchFiltered (Meta4Object, Nodo, FechaInicio, FechaFin, FechaEntrada, FechaPago, PagoFrec, TipoPago, TipoBúsqueda, RevBeh, SelPays, TotOp, [IDElemento, Valor, Función])	199
Especificación del filtro	200
Ejecución con tramos	200
Diferentes comportamientos con distintas llamadas	200
Funciones para estadísticas	200
SttGetLevel ()	201
SttPersistToFile (NombreArchivo)	201
SttSetLabel (Etiqueta)	201
SttSetLevel (Nivel)	202
Funciones avanzadas	202
CrossTab (IDMeta4Object, IDHost, Columna1, [columna2, ...], [Índice])	202
LookUp (PCP1, PCP2...ES1, SCP1, SI1, SCP2..., NúmeroSecundario, ElementoFiltroaAplicar, NodoUsuario, FechaInicioApl, FechaFinApl, FechaInicioCorr, FechaFinCorr)	202
Aplicación de fechas	204
Filtro en RAM	204
Tratamiento del argumento ElementoFiltroaAplicar	204
Tratamiento del elemento definido por el usuario	205
Funcionalidad añadida de LookUp	205
Pivot (IDMeta4Object, IDHost, DominioDestino, [Índice])	208
Funcionamiento del pivotado	209
■ Macros LN4	213
Macros básicas	214
M4_TRUE	214
M4_FALSE	214
M4_SHORT_DATE_FORMAT	214
M4_LONG_DATE_FORMAT	214
M4_SUCCESS	215
M4_ERROR	215
M4_ZERO	215
M4_POSITIVE	215
M4_NEGATIVE	215
Macros de búsqueda	215
M4_EQUAL	216
M4_DISTINCT	216

M4_GREATER	216
M4_SMALLER	216
M4_GREATER_OR_EQUAL	217
M4_SMALLER_OR_EQUAL	217
M4_EQUAL_OR_NULL	217
M4_DISTINCT_OR_NULL	217
M4_GREATER_OR_NULL	217
M4_SMALLER_OR_NULL	217
M4_GREATER_OR_EQUAL_OR_NULL	217
M4_SMALLER_OR_EQUAL_OR_NULL	217
M4_CASE_EQUAL	218
M4_CASE_DISTINCT	218
M4_CASE_GREATER	218
M4_CASE_SMALLER	218
M4_CASE_GREATER_OR_EQUAL	218
M4_CASE_SMALLER_OR_EQUAL	218
M4_CASE_EQUAL_OR_NULL	218
M4_CASE_DISTINCT_OR_NULL	219
M4_CASE_GREATER_OR_NULL	219
M4_CASE_SMALLER_OR_NULL	219
M4_CASE_GREATER_OR_EQUAL_OR_NULL	219
M4_CASE_SMALLER_OR_EQUAL_OR_NULL	219
M4_REGULAR_EXPRESSION	219
M4_REGULAR_EXPRESSION_OR_NULL	219
M4_CASE_REGULAR_EXPRESSION	220
M4_CASE_REGULAR_EXPRESSION_OR_NULL	220
Macros para cuadros de diálogo	220
M4_BTN_OK	220
M4_BTN_OK_CANCEL	220
M4_BTN_ABORT_RETRY_IGNORE	220
M4_BTN_YES_NO_CANCEL	221
M4_BTN_YES_NO	221
M4_BTN_RETRY_CANCEL	221
Macros para gestionar botones en cuadros de diálogo	221
M4_OK	221
M4_CANCEL	221
M4_ABORT	222
M4_RETRY	222
M4_IGNORE	222
M4_YES	222
M4_NO	222

Macros de funciones de totalización	222
M4_TOTAL_COUNT	223
M4_TOTAL_SUM	223
M4_TOTAL_AVG	223
M4_TOTAL_MAX	223
M4_TOTAL_MIN	223
M4_TOTAL_FIRST	223
M4_TOTAL_LAST	223
Macros para transacciones en base de datos	224
M4_COMMIT	224
M4_ROLLBACK	224
M4_EXECUTE_POSTVALIDATION	224
Macros para manipulación de cadenas	224
M4_TRIM_LEFT	225
M4_TRIM_ALL	225
M4_TRIM_RIGHT	225
M4_LOWERCASE	225
M4_UNCHANGED	225
M4_UPPERCASE	225
Macros de tipo y ámbito de elementos	225
M4_SCOPE_ALL	226
M4_SCOPE_REGISTER	226
M4_SCOPE_BLOCK	226
M4_SCOPE_NODE	226
M4_TYPE_ALL	226
M4_TYPE_METHOD	226
M4_TYPE_PROPERTY	226
M4_TYPE_FIELD	227
M4_TYPE_CONCEPT	227
Macros de constantes CHR	227
M4_NEW_LINE	227
M4_TAB	227
M4_DOUBLE_QUOTE	227
M4_CR	227
Macros de constantes de indirección de atributos (lectura)	228
M4_ATT_SYS_SLICE_NUMBER	228
M4_ATT_SYS_START_DATE	228
M4_ATT_SYS_END_DATE	228
M4_ATT_SYS_FIRST_SLICE	228
M4_ATT_SYS_LAST_SLICE	229
M4_ATT_SYS_OLD_VALUE	229

M4_ATT_SYS_BLOB_DESCRIPTION	229
M4_ATT_SYS_BLOB_MASK	229
M4_ATT_SYS_AUX_ITEM_ID	229
Macros de constantes de indirección de atributos (ejecución)	229
M4_ATT_SYS_ADD_NEW_SLICE	230
M4_ATT_SYS_CREATE_SLICE	230
M4_ATT_SYS_SPLIT_SLICE	230
M4_ATT_SYS_TOTALIZE_ITEMS	230
M4_ATT_SYS_TOTALIZE_SLICES	230
M4_ATT_SYS_CREATE_BLOB_FILE	230
M4_ATT_SYS_SET_BLOB_DESCRIPTION	230
Macros de autocarga	231
M4_AUTOLOAD_OFF	231
M4_AUTOLOAD_BLOCK	231
M4_AUTOLOAD_PRG	231
M4_AUTOLOAD_NODESAYS	231
Macros de autocarga para llamadas de nivel 2	231
M4_OPEN_MODE_NO_LOAD	232
M4_OPEN_MODE_LOAD	232
M4_OPEN_MODE_AUTOLOAD_BLOCK	232
M4_OPEN_MODE_AUTOLOAD_PRG	232
M4_OPEN_MODE_AUTOLOAD_NODESAYS	232
Macros de tipo de acceso	232
M4_CSTYPE_FRONT	233
M4_CSTYPE_BACK	233
M4_CSTYPE_BOTH	233
M4_CSTYPE_REAL_BOTH	233
M4_CSTYPE_FATHER	233
Macros de constantes de log	233
M4_ERRORLOG	234
M4_WARNINGLOG	234
M4_DEBUGINFOLOG	234
Macros de constantes de ordenación	234
M4_ASCENDING	234
M4_DESCENDING	234
Macros de valores de modo de envío	234
M4_SEND_NO_BRANCH	235
M4_SEND_BRANCH	235
M4_SEND_RESET	235
Macros de constantes de fecha	235
M4_MINUS_INF	235

M4_PLUS_INF	235
Macros de tipos variantes	235
M4_VARIANT_TYPE_NULL	236
M4_VARIANT_TYPE_NUMBER	236
M4_VARIANT_TYPE_STRING	236
M4_VARIANT_TYPE_DATE	236
Macros de políticas de compartición	236
M4_INSTANCE_NOT_SHARED	237
M4_INSTANCE_LOCAL_SHARED	237
M4_INSTANCE_GLOBAL_SHARED	237
Macros de funciones de fecha (suma/diferencia)	237
M4_YEAR	237
M4_MONTH	238
M4_DAY	238
M4_WEEK	238
M4_WEEKDAY	238
M4_HOUR	238
M4_MINUTE	238
M4_SECOND	238
Macros matemáticas	239
M4_MATH_PI	239
M4_MATH_E	239
Macros de metadatos de elementos	239
M4_ITEM_TYPE	240
M4_ITEM_SCOPE	240
M4_ITEM_M4TYPE	240
M4_ITEM_PRECISION	240
M4_ITEM_INDEX	240
M4_ITEM_SCALE	240
M4_ITEM_INTERNAL_TYPE	240
M4_ITEM_READ_OBJECT_ALIAS	241
M4_ITEM_WRITE_OBJECT_ALIAS	241
M4_ITEM_READ_FIELD_ID	241
M4_ITEM_WRITE_FIELD_ID	241
M4_ITEM_READ_OBJECT_ID	241
M4_ITEM_WRITE_OBJECT_ID	241
M4_ITEM_SLICE_TOTALIZE	241
M4_ITEM_NAME	241
M4_ITEM_ORDER	242
M4_ITEM_SYNONYM	242
M4_ITEM_ID	242

M4_ITEM_NUMBER	242
M4_ITEM_IS_PK	242
Macros de tipo de metadato de un Meta4Object	242
M4_M4OBJECT_NODE_NUMBER	243
M4_M4OBJECT_CREATION_TYPE	243
M4_M4OBJECT_NAME	243
M4_M4OBJECT_ID	243
M4_M4OBJECT_INSTANCE_NUMBER	243
M4_M4OBJECT_INSTANCE_HANDLE	243
Macros de tipo de metadato de un nodo	244
M4_NODE_READ_OBJECT_ID	245
M4_NODE_WRITE_OBJECT_ID	245
M4_NODE_ID	245
M4_NODE_NAME	245
M4_NODE_INDEX	245
M4_NODE_IS_ROOT	245
M4_NODE_IS_REAL_ROOT	246
M4_NODE_PARENT	246
M4_NODE_SENTENCE	246
M4_NODE_MAX_NUM_ROWS_DB	246
M4_NODE_AUTO_FILTER	246
Macros de Unicode	246
M4_ENCODING_NATIVE	247
M4_ENCODING_ANSI	247
M4_ENCODING_UTF8	247
Macros de comportamiento societario en niveles 2	247
M4_ORGANIZATION_L2_TYPE_NONE	247
M4_ORGANIZATION_L2_TYPE_MONO	248
M4_ORGANIZATION_L2_TYPE_MULTI	248
M4_ORGANIZATION_L2_TYPE_FATHER	248
M4_ORGANIZATION_L2_TYPE_ANY	248
Macros para el manejo de marcadores	248
M4_BOOKMARK_POSITION	248
M4_BOOKMARK_PK	249
■ Uso de Meta4Objects especializados con LN4	250
Imprimir PDF desde un Meta4Object de datos	251
Crear código para generar un XFDF desde un Meta4Object de datos	252
Llamar al método PRINT_PDF	253
Uso del API para modificar documentos PDF	254
Estructura de nodo PDF	254
Método OPEN_DOCUMENT	255

Método GET_PAGE_NUMBER	255
Método GET_PAGE_HEIGHT	255
Método GET_PAGE_WIDTH	256
Método SCALE_PAGE	256
Método SET_CURRENT_PAGE	256
Método SET_FONT	256
Método SET_COLOR	256
Método ADD_TEXT	256
Método ADD_IMAGE	257
Método ADD_NEW_PAGE	257
Método ADD_RECTANGLE	257
Método CHECK_STATUS	258
Método PROC_XFDF	258
Método PROC_XFA	258
Método PROTECT_PDF	258
Método CLOSE_DOCUMENT	258
Propiedades de los métodos anteriores	259
Uso del API para visualizar documentos en Word y convertirlos a PDF	260
Parámetros de entrada	260
Parámetros de salida	261
Ejemplo de llamada desde una presentación	261
Utilidad de conversión de Word a PDF	263
Uso del API para mostrar y guardar un documento en el gestor documental	263
Uso del API para activar la gestión de anexos	264
Parámetros de entrada	265
Parámetros de salida	265
Ejemplo de llamada desde una presentación	266
Uso del API para personalizar la información de firma en los documentos PDF	267
Estructura de nodo SRTC_MERGE_DOC_AND_SIGN	267
Proceso para cambiar los valores por defecto de las propiedades de la estructura de nodo ..	269
Proceso para cambiar la información de cada firmante	269
Uso del API para crear parámetros de aplicación	270
Uso del API para acceder a las ejecuciones de una planificación multisocietaria	271
Estructura de nodo SRTC_JS_SCHEDULINGS_API	271
Método CALC_ORGANIZATION_FROM_PARAM	272
Método LOAD_SCHED_EXE_BY_TASK_PARAM	272
Método DELETE_SCHED_TASK	273
Estructura de nodo SRTC_SCHEDULINGS_BY_PARAM	273
Método LOAD_RESOURCES	273
Estructura de nodo SRTC_SCHEDULING_RESOURCES	274

Ejemplo de consulta del estado de una ejecución	274
Uso del API para transferir la información de rol-sociedad de una sociedad a otra	277
Método TRANSFER_PERSON_ROLES_BET_ORG	277
Uso del API para abrir usuarios del portal en una sociedad	278
Uso del API para dar de baja usuarios asociados a personas	280
Uso del API para reabrir usuarios asociados a una persona en una fecha	282
Uso del API para borrar los usuarios asociados a una persona	283
Uso del API para borrar un rol-sociedad a un usuario	284
Uso del API para crear usuarios desde el alta de empleados	285
Uso del API para establecer conexión por SSO e impedir conexión con usuario y contraseña ..	286
Uso del API para la descarga asíncrona de ficheros de forma segura	287
Método API_CHECK_ASYNC_CONFIG(Ref AO_AVAILABLE)	288
Método API_PREPARE_FILE_ASYNC(AI_FILE_PATH, AI_FILE_NAME, Ref AO_FILE_KEY)	289
Método API_GET_FILE_ASYNC(AI_FILE_KEY)	289
Método API_CREATE_FILE_GROUP_ASYNC(Ref AO_GROUP_KEY)	289
Método API_ADD_FILE_TO_GROUP_ASYNC(AI_FILE_KEY, AI_GROUP_KEY)	289
Método API_GET_FILE_GROUP_ASYNC(AI_GROUP_KEY)	290
Uso del API para ejecutar gráficos	290
Uso del API para reducir el tamaño y peso de las imágenes	292
Uso del API para ejecutar cartas y salida directa a Excel	293
Uso del API para manejar archivos Excel en el servidor de aplicaciones	294
Estructura de nodo SNTC_EXCEL_JAVA_WORKBOOK	295
Método OPEN_WORKBOOK	296
Método SAVE_WORKBOOK_AS	296
Método CLOSE_WORKBOOK	296
Estructura de nodo SNTC_EXCEL_JAVA_WORKSHEET	296
Método GET_WORKSHEET	296
Método GET_VALUE	297
Método SET_VALUE	298
Método SET_COLUMN_HIDDEN	298
Método SET_ROW_HIDDEN	298
Método COPY_CELL	299
Método COPY_ROW	299
Uso del API para la apertura parcial de documentos Excel	299
Uso del API para proteger archivos Excel contra modificaciones	300
Uso del API para eliminar hojas de cálculo	300
Uso del API para crear un archivo Excel vacío	300
Uso del API para convertir archivos Excel en PDF	301
Estructura de nodo SRTC_EXCEL_CONVERTER	301

Método API_CONVERT_TO_PDF	301
Método API_CONVERT_TO_PDF_A	302
Estructura de nodo EXCEL_CPP	302
Método CONVERT_TO_PDF	302
Estructura de nodo SNTC_EXCEL_CPP	302
Método CONVERT_TO_PDF	302
Uso del API para transcodificar objetos lógicos.	303
Estructura de nodo CSP_PRE_TRANSCOD_VAL2	303
Método CSP_PRE_TRANSCOD_VAL	303
Método CSP_PRE_TRANSCOD_VAL_EXT	304
Uso del API para transcodificar valores de Meta4Objects.	305
Estructura de nodo SRTC_DYN_TRANSCOD	305
Método SRTC_CHANGE_ITEM_TRANSCOD	305
Método SRTC_ITEM_VALUE_TRANSCOD	306
Uso del API para comprimir y descomprimir archivos en formato ZIP	307
Método M4GENERATEZIP	307
Método M4EXPANDZIP	308
Ejemplos de uso del API para comprimir y descomprimir archivos ZIP	308
Uso del API para encriptar, desencriptar, firmar y verificar archivos	310
Método API_ENCRYPT_FILE(ARG_FILE, ARG_KEY, ARG_OUT_FILE)	311
Método API_DECRYPT_FILE(ARG_FILE, ARG_EXT, ARG_OUT_FILE)	311
Método API_SIGN_FILE(ARG_FILE, ARG_OUT_FILE)	311
Método API_VERIFY_FILE(ARG_FILE, ARG_KEY_ID, ARG_EXT, ARG_OUT_FILE) ..	312
Método API_ENCRYPT_SIGN_FILE(ARG_FILE, ARG_KEY_ID, ARG_OUT_FILE)	312
Método API_DECRYPT_VERIFY_FILE(ARG_FILE, ARG_EXT, ARG_KEY_ID, ARG_OUT_FILE)	312
Método API_KF_ENCRYPT_FILE(ARG_FILE, ARG_KEY_FILE, ARG_OUT_FILE)	313
Método API_KF_DECRYPT_FILE(ARG_FILE, ARG_KEY_FILE, ARG_PWD, ARG_EXT, ARG_OUT_FILE)	313
Método API_KF_SIGN_FILE(ARG_FILE, ARG_KEY_FILE, ARG_PWD, ARG_OUT_FILE) ..	313
Método API_KF_VERIFY_FILE(ARG_FILE, ARG_KEY_FILE, ARG_EXT, ARG_OUT_FILE) .	314
Método API_KF_ENCRYPT_SIGN_FILE(ARG_FILE, ARG_ENCR_KEY_FILE, ARG_SIGN_KEY_FILE, ARG_PWD, ARG_OUT_FILE)	314
Método API_KF_DECRYPT_VERIFY_FILE(ARG_FILE, ARG_DECR_KEY_FILE, ARG_PWD, ARG_VERIFY_KEY_FILE, ARG_EXT, ARG_OUT_FILE)	314
Uso del API para generar la información asociada a los componentes de un concepto final	315
Cómo pasar valores a parámetros para consultas nuevas	315
Cómo llamar a una consulta desde una opción de menú	317

Manual de referencia del lenguaje LN4

Introducción

El lenguaje de programación LN4 es un potente y versátil lenguaje de desarrollo, exclusivo de aplicaciones de Meta4 como Meta4 PeopleNet y Meta4 KnowNet, que ofrece diversas posibilidades al desarrollador.

Entre sus características más destacables, se pueden citar:

- Asignación dinámica de tipos de datos, sin declarar las variables de manera explícita.
- Estructuras de control propias de cualquier lenguaje de programación, tales como:
 - Ejecución condicional: If...then...else
 - Sentencias repetitivas:
 - While...wend
 - For...next
 - etc.
- Facilidades para el diseño de fórmulas que calculen el valor de los conceptos, o realicen cualquier tipo de cálculo sobre datos.
- Utilización de elementos de las estructuras de nodo de cualquier Meta4Object en los cálculos y procesos, lo que permite referenciar valores asignados a otros conceptos o campos en el resto de estructuras de nodo (EN) de la aplicación.

El objetivo del presente manual consiste en describir todos los elementos del lenguaje, así como la forma de usar éstos para sacar el mayor provecho de los mismos.

Lenguaje LN4 y estructuras de nodo

Aunque los elementos de las estructuras de nodo pueden ser también de tipo **campo** (contienen valores extraídos directamente de la base de datos) o de tipo **propiedad** (contienen únicamente valores que se asignen a cada una de las propiedades), el código LN4 puede estar asociado únicamente a los elementos de tipo concepto o de tipo método.

Los **conceptos** adquieren un valor que se genera a partir de la fórmula LN4 a la que estén asociados o aquel valor que se le asigne como si de una propiedad se tratase.

Por su parte, los **métodos** equivalen a las funciones y procedimientos de los lenguajes de programación. En este caso, ejecutan una serie de instrucciones LN4 y pueden utilizarse entre otras acciones para actualizar los elementos de tipo campo de aquellas estructuras de nodo que pertenecen al mismo Meta4Object u otro diferente.

La ejecución de los conceptos o métodos puede realizarse a nivel de:

- **Nodo**

En este caso, hay que estar posicionado en el nodo específico sobre el que se va a actuar, el cual debe estar activo.

- **Bloque**

Este debe estar activo, ya que en principio no debería afectar al resto de bloques.

- **Registro**

Este debe estar activo, ya que en principio no debería afectar al resto de los registros.

Llamadas a los valores de los elementos de las estructuras de nodo

Desde las fórmulas creadas con LN4 se pueden utilizar:

- Los valores asignados a los elementos de la estructura de nodo en la que se define el método o concepto.
- Los valores asignados a los elementos de otras estructuras de nodos que estén definidas en el mismo Meta4Object que la estructura de nodo en la que se define el método o concepto.
- Los elementos de tipo método de la misma estructura de nodo u otras estructuras del mismo Meta4Object.

-
- Los elementos de tipo método de la misma estructura de nodo u otras estructuras de distintos Meta4Objects.

Estas asignaciones se graban físicamente en la base de datos cuando se ejecuta la función `Persist_tree ( )`. Esto sucede únicamente en el caso de los campos, pero no en el resto de elementos.

Para llevar a cabo las llamadas a los elementos de la estructura de nodo, se utilizará la siguiente sintaxis:

- Para referenciar un elemento de la estructura de nodo en la que se está creando el concepto o método, basta con escribir el nombre del elemento. El intérprete de LN4 toma el valor que se haya asignado al elemento en el registro que se encuentre activo.

Si se quiere referenciar un elemento de la estructura de nodo en un registro distinto del registro activo, se debe cambiar el registro activo mediante la función `MoveTo (ÍndiceRegistro)`.

- Para referenciar un elemento de otro nodo que forme parte del mismo Meta4Object, basta con escribir el alias de la estructura de nodo asociada al nodo, seguido de un punto y del nombre del elemento que se quiere utilizar.

`AliasEN.NombreItem`

o

`EN.NombreItem`

El alias debe ser un identificador que distinga de forma unívoca a cada nodo dependiente de una misma estructura de nodo.

- Para referenciar un elemento de un registro específico en un nodo incluido en el mismo Meta4Object, es necesario indicar detrás del nombre de la estructura de nodo, entre corchetes, el índice del registro cuyo elemento se quiere utilizar. A continuación se debe escribir un punto y el nombre del elemento.

`NombreEN[índice].NombreItem`

o

`AliasEN[índice].Item`

- Para referenciar un elemento con tramos delimitados por una fecha de inicio y una fecha de fin:
 - Si se trata de un elemento de la estructura de nodos sobre el que se está trabajando, es suficiente con escribir el nombre del elemento y, entre corchetes, el número del tramo o una fecha del rango al que se desea hacer referencia.

Los tramos se numeran comenzando por 0 para el primer tramo, el 1 para el segundo, y así sucesivamente. De esta forma, por ejemplo, la asignación de un valor al tramo del elemento `Salario_Base` situado en el registro activo comprendido entre las fechas 01-03-1999 y 15-03-1999, se podría hacer de la siguiente forma:

```
Salario_Base [ {1999-03-10} ] = 120000
```

Si se conoce el lugar que ocupa el tramo, sería posible utilizar su índice en lugar de una fecha. Así pues, si el tramo comprendido entre el 01-03-1999 y el 15-03-1999 fuese el segundo tramo, se le podría asignar un valor mediante la correspondiente instrucción:

```
Salario_Base [ 1 ] = 120000
```

- Para asignar valor a este mismo elemento y tramo, en el registro situado en séptimo lugar de la estructura de nodo llamada `Acumulado_Largo`, se debería utilizar la instrucción:

```
Acumulado_Largo [ 6 ].Salario_Base [ {1999-03-10} ]
```

Se debe emplear esta misma sintaxis para recuperar el valor almacenado en un elemento con tramos y asignarlo a variables.

Por ejemplo, la siguiente instrucción asignaría a la variable `SalBase` el valor del segundo tramo del elemento `Salario_Base` del quinto registro del nodo `Acumulado_Largo`.

```
SalBase = Acumulado_Largo[4].Salario_Base[1]
```

Elementos básicos del lenguaje

Variables y constantes

El nombre de las variables debe comenzar por una letra o el carácter de subrayado, seguido por un conjunto de letras y/o números. En ninguno de ambos casos es posible incluir espacios en blanco o caracteres especiales.



Para nombrar a las variables, es recomendable utilizar identificadores fáciles de recordar, con la condición de que no excedan los 255 caracteres. No obstante, por usabilidad se recomienda un máximo de 100 caracteres.

Las variables LN4 no necesitan ser declaradas de manera explícita, sino que basta con inicializarlas asignándoles un valor. Asimismo, tampoco están tipificadas, es decir, el tipo de dato de una variable cambia dependiendo de los valores que se le asignen.

La consecuencia inmediata es que si se intenta leer el contenido de cualquiera de estas variables antes de haberles asignado un valor de tipo fecha, cadena o número, se produce un error.

Tipos de datos

Una variable puede almacenar un valor dependiendo del tipo de dato con el que haya sido inicializada. Este tipo puede ser uno de los siguientes:

- Fecha
- Número
- Cadena

Fecha

Este tipo de variable aparece en formato TS (Iso TimeStamp) y entre llaves. No es necesario especificar todo el conjunto de datos.

EJEMPLO

```
{1997-12-01 12:59:59}
```

Número

Este tipo de variable permite utilizar los formatos decimal (con un punto para separar la parte entera de la decimal), de notación científica, y hexadecimal (éste último sólo para números enteros).

La notación científica permite expresar cantidades muy grandes de forma sencilla, anotando los números como potencia de base diez, lo que facilita la realización de operaciones matemáticas en problemas científicos.

En el sistema decimal, cualquier número real puede expresarse mediante la denominada notación científica normalizada. Para expresar un número en notación científica normalizada, se multiplica o se divide por 10 tantas veces como sea necesario para que todos los dígitos aparezcan a la derecha del punto decimal y que el primer dígito después del punto no sea cero.

EJEMPLO

$$732.5051 = 0.7325051 \times 10^3$$
$$-0.005612 = -0.5612 \times 10^{-2}$$

Cadena

El texto que aparezca en esta variable debe ir entre comillas.

Llamadas a funciones

Las llamadas a una función cualquiera se realizan de forma similar a otros lenguajes de programación. Dependiendo de la función, se deberán o no indicar argumentos en el momento de realizar la llamada.

```
Opción 1: [Variable =] nombre_función (argumentos)
Opción 2: nombre_función ( )
```

Retorno de valores

Para obtener el valor de una variable, se utiliza la siguiente sintaxis:

```
return (expresión)
```

Operadores LN4

El lenguaje LN4 recoge un conjunto de operadores que permiten realizar operaciones de diverso tipo sobre variables. Cada uno de estos operadores pertenece a una de las siguientes categorías:

- Operadores matemáticos binarios
- Operadores matemáticos unarios
- Operadores lógicos
- Operadores de comparación

La siguiente tabla recoge los distintos operadores que pertenecen a cada categoría:

Operadores matemáticos binarios	+	Suma
	-	Resta
	*	Multiplicación
	/	División
Operadores matemáticos unarios	-	Indicador de número negativo
Operadores lógicos	NOT	Devuelve VERDADERO si es cierta la condición situada a la izquierda del operador y falsa la condición situada a la derecha del operador.
	AND	Devuelve VERDADERO si son ciertas las dos condiciones situadas a la izquierda y a la derecha del operador.
	OR	Devuelve VERDADERO si es cierta al menos una de las condiciones situadas a la izquierda o a la derecha del operador.
Operadores de comparación	=	Igual que
	<>	Distinto de
	>	Mayor que
	<	Menor que
	>=	Mayor o igual que
	<=	Menor o igual que

Comentarios en las fórmulas

El lenguaje LN4 permite introducir comentarios en las fórmulas. Estos deben ir precedidos por unos caracteres especiales que indican al compilador que debe hacer caso omiso del texto comprendido entre ellos.

A la hora de incluir un comentario en el código de LN4, se pueden dar dos situaciones:

- Si el comentario no ocupa más de una línea, debe ir precedido por el carácter `'`, o por los caracteres `//`.
- Si el comentario ocupa más de una línea, debe ir precedido por los caracteres `/*`, y detrás de la última palabra del comentario se deben añadir los caracteres `*/`.

Expresiones regulares

Las expresiones regulares son expresiones que permiten establecer criterios de selección o búsquedas sobre el conjunto de elementos con los que se trabaja.

Con LN4 existe la posibilidad de trabajar con expresiones regulares, tanto para permitir búsquedas potentes con la función `FindRegister ( )`, como para buscar expresiones regulares dentro de un texto cualquiera con la función `MatchRegister ( )`.

La sintaxis de estas expresiones regulares es la siguiente:

Expresión	Explicación de la expresión utilizada
()	Este operador permite agrupar parte de una expresión regular y que se repita una o más veces cierta subexpresión regular. Así por ejemplo, la expresión <code>a(bc)+</code> encontrará una <code>a</code> seguida de uno o más bloques de <code>bc</code> .
	Este operador permite establecer un OR entre dos subexpresiones. Por ejemplo, <code>ab ac</code> recuperará las secuencias <code>ab</code> y <code>ac</code> .
+, *	El operador <code>*</code> indica que la expresión que le precede puede aparecer cero o más veces y el operador <code>+</code> indica que aparece una o más. Por ejemplo, <code>ab*</code> encontrará una <code>a</code> seguida de ninguna o varias <code>b</code> .
?	Este operador indica que la expresión que le precede es opcional. Por ejemplo, <code>p?sico</code> encontrará <code>psico</code> y <code>sico</code> .

\$ y ^	El operador \$ indica que la expresión que le precede debe ser el final del texto y el operador ^ indica que la expresión que le sigue debe ser la primera del texto. Por ejemplo, ^en encontrará los textos que empiecen por en, no otros. Por ejemplo, adiós\$ sólo encontrará los textos que acaban con la palabra adiós.
.	El operador . es un comodín que permite encontrar cualquier carácter. Por ejemplo, a.*b encontrará las cadenas que empiecen por a y acaben en b.
\w, \W	El operador \w encuentra cualquier carácter que pueda formar parte de una palabra, es decir, letras mayúsculas/minúsculas y números. El operador \W encuentra los que no pueden formar parte de una palabra. A\wB encontrará acb, pero no a b
\< \>	Si \< precede a una subexpresión, indica que la subexpresión debe encontrarse al principio de una palabra. Si \> sucede a una subexpresión, sólo encontrará las palabras que acaben con esa expresión. Por ejemplo, \<mega sólo encuentra las palabras que empiecen por mega. Por ejemplo, mente\> sólo recuperará las palabras que acaben en mente.
Carácter \	Este operador \ permite encontrar el carácter de escape.

Estructuras de control

El lenguaje LN4 incluye diversas estructuras de control que permiten, entre otras funciones, establecer ejecuciones cíclicas de acciones, seleccionar elementos que cumplan ciertas condiciones, etc. Estas estructuras de control, cuya sintaxis y modo de uso veremos a continuación, se dividen en:

- Sentencias condicionales
 - *If... Then, If... Then... Else*, etc.
- Bucles de repetición de sentencias tales como
 - *Do... Until*
 - *While... Wend*
 - *For... Next*

Dentro de estas estructuras de control existe la posibilidad de indicar dos acciones en la misma línea, separándolas únicamente por ":".

EJEMPLO

```
If A < 10 then
    a=12 : b=16
Else
    MessageBox ( "Aviso", "El valor de A es igual o mayor que 10 ")
EndIf
```

Sentencias condicionales

Sentencia *If ... Then ... EndIf*

EJEMPLO

```
If condición Then
...
EndIf
```

La sentencia *If ... Then ... EndIf* se utiliza para ejecutar una serie de sentencias cuando se cumplan una o varias condiciones, que se establecen como criterios de selección.

El orden de esta sentencia es el siguiente:

- Las condiciones se escriben inmediatamente después de la palabra reservada *If*
- Las sentencias que se han de ejecutar si se cumplen las condiciones, deben escribirse entre las palabras reservadas *Then* y *EndIf*.



Es posible anidar tantas sentencias *If... Then... EndIf* como sea necesario.

- La palabra reservada *EndIf* debe escribirse sin interponer un espacio en blanco entre *End* e *If*, ya que de hacerse así se produciría un error.
- En caso de que se indiquen dos o más condiciones detrás de la palabra reservada *If*, éstas deben unirse mediante los operadores lógicos AND, OR y/o NOT:
 - Si se unen varias condiciones mediante el operador **AND**, las sentencias indicadas entre las palabras reservadas *Then* y *EndIf* se ejecutarán sólo si se cumplen todas las condiciones.

EJEMPLO

Sea el siguiente fragmento de código LN4:

```
If A < 10 AND A > 5 Then
    MessageBox ( "Aviso", " El valor de A es " , A )
EndIf
```

En este caso, si la variable A recoge un valor comprendido entre 5 y 10, se mostrará el valor de la variable en un cuadro de diálogo. Sin embargo, si el valor de A no está comprendido en este intervalo, no se ejecutará ninguna acción.

- Si se indica como condición precedida por el operador **NOT**, las sentencias indicadas entre las palabras reservadas *Then* y *EndIf* se ejecutarán si no se cumplen las condiciones indicadas.
- Si se unen dos condiciones mediante el operador lógico **OR**, las sentencias indicadas entre las palabras reservadas *Then* y *EndIf* se ejecutarán si se cumple al menos una de las condiciones.

EJEMPLO

Sea el siguiente fragmento de código LN4:

```
If A < 10 OR B > 5 Then
    MessageBox ( "Aviso", " El valor de A es " , A )
EndIf
```

En este caso, si la variable A contiene un valor superior a 10, o la variable B contiene un valor superior a 5, entonces se mostrará el valor de la variable A en un cuadro de diálogo.

Si no se cumple ninguna de estas dos condiciones, no se ejecutará ninguna acción.

Sentencia If ... Then ... Else ... EndIf

EJEMPLO

```
If condición then
    ...
Else
    ...
EndIf
```

Esta estructura se utiliza para seleccionar el conjunto de sentencias que se ejecutarán, en función del valor de la condición o condiciones indicadas tras la palabra reservada *If*.



Es posible indicar más de una condición, uniéndolas con los operadores lógicos AND y OR.

- Si se cumplen las condiciones, se ejecutarán las sentencias que se hayan escrito entre las palabras reservadas *Then* y *Else*.
- Si no se cumple la condición o condiciones indicadas, se ejecutarán las sentencias que se hayan escrito entre las palabras reservadas *Else* y *EndIf*.

Si la estructura ***If...Then...Else*** se ha escrito en una sola línea, no es necesario finalizar con la palabra ***EndIf***.

EJEMPLO

El código de este ejemplo realiza lo siguiente:

- Si la variable A recoge un valor situado en el rango delimitado por los números 5 y 10, aparece un mensaje en pantalla con el valor de la variable.
- Si el valor de A no está comprendido en este intervalo, se le suma 10 al valor de la variable y se mostrará un mensaje en pantalla con el valor obtenido tras realizar esta operación.

```
If A > 5 AND A < 10 Then
    MessageBox ( "Aviso", "El valor de A es ", A )
Else
    A = A + 10
    MessageBox ( "Aviso", "El valor de A tras sumarle 10 es ", A )
EndIf
```

Sentencia If ...Then ...Elseif ...Else ...Endif

EJEMPLO

```
If condición then
...
Elseif condición then
...
Else
...
Endif
```

Esta estructura de control se utiliza para realizar una serie de acciones en función de las condiciones indicadas. Así, si se cumple la condición o condiciones indicadas tras la palabra reservada *If*, se ejecutarán únicamente las sentencias escritas entre la palabra reservada *Then* y el primer *Elseif*.

Si no se cumple la primera condición o condiciones, se evaluarán las condiciones que se indican detrás de la primera palabra reservada *Elseif*. Si se cumplen éstas, se ejecutará el código que se haya escrito entre las palabras reservadas *Elseif* y *Else*. Si no se cumple, se ejecutará el código escrito entre las palabras reservadas *Else* y *Endif*.

Dentro de esta estructura es posible incluir tantas cláusulas *Elseif* como se consideren necesarias. También es posible omitir la cláusula *Else*, para que así no se ejecute ninguna opción en el caso de que no se cumplan las condiciones indicadas en las cláusulas *If* y *Elseif*.

EJEMPLO

El código del siguiente ejemplo realiza las siguientes acciones.

- Si A recoge un valor comprendido entre 5 y 10, se muestra en pantalla un cuadro de diálogo con su valor.
- Si el valor de A es mayor que 10, se eleva su valor al cuadrado y se muestra en pantalla el resultado. En cualquier otro caso, es decir, si A recoge un valor menor que 5, se le suma 15.
- En todos los casos se mostrará un cuadro de diálogo que indica que las operaciones han finalizado.

```
If A > 5 AND A < 10 Then
    MessageBox ( "Aviso", "El valor de A es ", A )
ElseIf A > 10 Then
    A = Power ( A, 2 )
    MessageBox ( "Aviso", "El valor de A al cuadrado es ", A )
```

```

Else
    A = A + 15
EndIf
MessageBox ( "Aviso", "Las operaciones han finalizado" )

```

Estas sentencias también pueden escribirse anidando sentencias If ... Then, de la siguiente forma:

```

If A > 5 Then
    If A < 10 Then
        MessageBox ( "Aviso", "El valor de A es ", A )
    Else
        A = Power ( A, 2 )
        MessageBox ( "Aviso", "El valor de A al cuadrado es ", A )
    EndIf
Else
    A = A + 15
EndIf
MessageBox ( "Aviso", "Las operaciones han finalizado" )

```

Sentencias de repetición

Sentencia Do ... Until condición

EJEMPLO

```

Do
    ...
Until condición

```

La sentencia *Do ... Until* ejecuta el conjunto de sentencias situado entre las palabras reservadas *Do* y *Until*, hasta que se cumpla la condición o condiciones indicadas tras la palabra reservada *Until*.



En el caso de que se indiquen dos o más condiciones, éstas deben unirse mediante los operadores lógicos AND y/o OR.

EJEMPLO

El siguiente código recorre los campos de una EN hasta alcanzar la variable lógica EOF. Mientras no se encuentre, se suma el valor del campo SALARIO_BASE de todos los registros de la EN al campo TOTAL_SALARIO de la misma estructura de nodos.

```
TotalSalario = 0
Begin ( )
Do
    TotalSalario = TotalSalario + SALARIO_BASE
Gonext ( )
Until IsEOF ( ) = M4_TRUE
MessageBox ("Aviso", "El total de los salarios es ", TotalSalario )
```

Sentencia While condición ... Wend

EJEMPLO

```
While condición
...
Wend
```

La sentencia *While ... Wend* se utiliza siempre que se tenga que repetir una serie de instrucciones, mientras se cumpla una o varias condiciones establecidas previamente.

- Si se unen dos condiciones mediante el operador AND, las instrucciones se ejecutarán mientras se cumplan las dos condiciones.
 - Si se unen dos condiciones mediante el operador OR, las instrucciones se ejecutarán mientras se cumpla al menos una de las condiciones.
-

EJEMPLO

El siguiente código recorre los registros de la estructura de nodos asociada al nodo activo, hasta que se encuentre un registro cuyo campo SALARIO_BASE sea mayor que 600.000, o alcance la variable lógica EOF.

Para cada registro se suma el valor del campo SALARIO_BASE en la variable TotalSalario. Finalmente, se muestra el resultado en un cuadro de diálogo.

```
SalarioBase = 0
Begin ( )
```

```
While Not IsEOF ( )=M4_TRUE AND SALARIO_BASE < 600000
    SalarioBase = SalarioBase + SALARIO_BASE
Gonext ( )
Wend
MessageBox ("Aviso", "El total del salario base es ", SalarioBase )
```

Sentencia For ... Next

EJEMPLO

```
For variable = n to m
...
Next
```

La estructura *For ... Next* repite una o varias instrucciones el número de veces que se indique detrás de la palabra reservada *For*.

Cada vez que se repite el conjunto de instrucciones, se incrementa un contador interno, cuyo valor se guarda en la variable de control. Este contador puede utilizarse, por ejemplo, para recorrer los registros de una estructura de nodos.

Si se hace así, recuerde que los registros de una EN se numeran comenzando por el número 0. No es posible modificar el incremento del contador en la línea del *For*; si se necesita incrementar este valor con un número distinto a 1, se debe hacer aumentando la variable con el valor deseado en el cuerpo del bucle.

La sentencia *For...Next* sólo permite incrementar la variable indicada de uno en uno, aunque es posible incrementar el valor de la variable utilizada dentro de la sentencia *For...Next* un número mayor.

EJEMPLO

El siguiente código muestra seis veces un cuadro de diálogo. El cuadro de diálogo indica en pantalla el valor que recoge la variable *n* en cada momento.

```
For n = 1 To 6
    MessageBox ( "Aviso", "El contador n recoge el valor ", n )
Next
```

Manejo de elementos

Elementos del sistema

Un elemento del sistema es cualquier elemento que pertenezca a una estructura de nodos determinada. El fin de estos elementos es disponer de un grupo de ellos que mapeen directamente un conjunto de funciones de C++ para ejecutar en el Meta4Object Engine.

Este conjunto de elementos proporcionan funcionalidades básicas, tales como, búsquedas, navegación, ejecución, etc.

Sintaxis de lectura y asignación de elementos

A continuación se detalla la sintaxis necesaria para realizar una asignación de valor a los elementos y proceder posteriormente a su lectura:

- elemento
- elemento[tramo]
- nodo.elemento
- nodo[registro].elemento
- nodo[registro].elemento[tramo]

Por ejemplo, la asignación de valores a un elemento correspondiente a un nodo se haría del siguiente modo:

```
nodo.elemento = valor
```

Por último, para hacer referencia a un Meta4Object distinto al actual, se debe utilizar la siguiente sintaxis:

```
Alias ! .
```

Atributos y Atributos-Métodos

En el apartado Sintaxis de lectura y asignación de elementos se indica cómo asignar valores a elementos con tramos y cómo recuperar los valores que contienen los distintos tramos de un elemento.

Las propiedades y métodos de ambos se indican en la tabla siguiente:

Propiedades	Aplicable a	Tarea
SysSliceNumber	Elemento	Devuelve el número de tramos que contiene el elemento.
SysStartDate	Tramo	Fecha de inicio del tramo.
SysEndDate	Tramo	Fecha de fin del tramo.
SysFirstSlice	Elemento	Índice del primer tramo del elemento.
SysLastSlice	Elemento	Índice del último tramo del elemento.
SysOldValue	Elemento y tramos	Devuelve el valor antiguo del elemento cuando se le haya cambiado el valor desde LN4.
SysAddNewSlice (fecha1, fecha2)	Elemento	Añade un nuevo tramo al elemento. Recibe como argumentos la fecha de inicio y de fin del tramo.
SysBlobDescription	Elemento	Descripción blob del elemento.
SysBlobMask	Elemento	Máscara blob del elemento (entero).
SysAuxiliarItemID	Elemento	ID Elemento auxiliar (cadena)
SysAddNewSlice (start_date, end_date)	Elemento y tramos	Añade un nuevo tramo al elemento.
SysGetItemName (Nombre)	Elemento	Devuelve el valor del elemento en un argumento (por referencia)
SysCreateSlice (start_date, end_date, value)	Elemento y tramos	Añade un nuevo tramo al elemento y le asigna un valor.
SysSplitSlice (date)	Tramos	Divide el tramo en dos por esta fecha, manteniendo el valor acorde al comportamiento.
SysTotalItems (function)	Elemento	Totaliza los elementos.
SysTotalSlices (function)	Elemento	Totaliza los tramos.
SysCreateBlobFile (Directory, extension)	Elemento	Crea un archivo blob para el elemento en el directorio con esta extensión.
SysSetBlobDescription (description)	Elemento	Asigna la descripción blob de este elemento.

Para emplear estas propiedades y métodos, debe escribirse el nombre del elemento seguido del identificador del tramo (cuando la propiedad sea aplicable a tramos), seguido de dos puntos (..), y el nombre de la propiedad o método que se quiere emplear.

EJEMPLO

Item..SysSliceNumber	Devuelve el número de tramos del elemento
Item[Tramo]..SysStartDate	Devuelve la fecha de inicio del tramo que se indica. Si no se indica ningún tramo, devuelve la fecha de inicio del tramo actual.
Item[Tramo]..SysEndDate	Devuelve la fecha de fin del tramo que se indica. Si no se indica ningún tramo, devuelve la fecha de fin del tramo activo.
Item..SysFirstSlice	Devuelve el índice del primer tramo del elemento. Siempre devolverá 0, incluso si el elemento no contiene ningún tramo.
Item..SysLastSlice	Devuelve el índice del último tramo del elemento. Por ejemplo, si el elemento tiene 6 tramos, esta propiedad recogerá el valor 5. Si el elemento tuviese un tramo, devolverá el valor 0. Si el elemento no tiene tramos, devolverá 0.
Item..SysAddSlice ({1999-07-01},{1999-07-31})	Crea un nuevo tramo en un elemento con fecha de inicio 01-07-1999 y fecha de fin 31-07-1999.
Item = 34000 'Cambiamos el valor de elemento intValorAnterior = Item..SysOldValue	Devuelve el valor que tenía el elemento antes de asignarle el valor 34.000. Esto sucede porque el valor no está activo, bien porque no se ha acometido una transacción de memoria con el nuevo valor, o porque no se ha almacenado en la base de datos, y por tanto, muestra el valor previo por ser el que se encuentra en memoria.

Ejecución de métodos y conceptos

Los conceptos adquieren el valor que se genera a partir de la fórmula LN4 a la que estén asociados o el valor que se le asigne como si de una propiedad se tratase.

Los métodos equivalen a las funciones y procedimientos de los lenguajes de programación. Estos ejecutan una serie de instrucciones LN4 y pueden utilizarse, entre otras cosas, para actualizar los elementos de tipo campo de las estructuras de nodo que pertenecen al mismo Meta4Object u otro diferente.

Los métodos y los conceptos pueden ejecutarse de dos formas:

- Desde la interfaz de la aplicación, mediante un botón que aparece en pantalla al acceder a la información identificativa de un nodo o una presentación.
- Desde el código LN4 de otros métodos y conceptos.

Para llamar a otros métodos o conceptos del mismo nodo o de un nodo diferente, se utilizará la sintaxis que se indica en el apartado Llamadas a los valores de los elementos de las estructuras de nodo.

Este tipo de operaciones se pueden realizar también entre varios Meta4Objects.

Ejecución de un grupo de conceptos

En el caso de los conceptos, es posible ejecutar varios conceptos a la vez. Para ello se utilizará la función LN4 ExecuteGroup (Nombre de grupo). Al llamar a esta función, LN4 ejecutará uno tras otro todos los conceptos o métodos que estén incluidos en un mismo grupo.

Un grupo puede incluir un número ilimitado de conceptos, pero un concepto sólo puede pertenecer a un grupo, todo ello dentro de la misma estructura de nodo.

La inclusión de un concepto en un grupo se indicará al definir los elementos de la estructura de nodo desde el Diseñador de estructuras de nodo, rellenando la propiedad **Grupo de ejecución** dentro del **Editor de Meta4Objects**.

El modelo de datos actual no incluye ninguna tabla para registrar los grupos de conceptos, es decir, los valores que se indiquen en la propiedad **Grupo de ejecución** no son clave externa de ninguna otra tabla de la base de datos.

El orden en el que se ejecutarán los conceptos lo determina el programa de forma dinámica a partir de las dependencias que existan entre los conceptos y los métodos del grupo que se quiere ejecutar.

El lenguaje LN4 no permite alterar el orden de ejecución elegido por el Meta4Object Engine. Además, se debe evitar la existencia de ciclos en el proceso, ya que se produciría un fallo y se detendría la ejecución.

Es posible ejecutar más de una vez la función ExecuteGroup (Nombre de grupo) en un mismo método. En este caso, el programa no determinará el orden en el que se tienen que ejecutar los distintos grupos. Por lo tanto, podría darse el caso de que se ejecutase en primer lugar un grupo que incluye un concepto en cuya fórmula se está haciendo referencia a un concepto cuyo cálculo se va a procesar en un grupo posterior.

El programador debe controlar estas circunstancias y establecer un orden de ejecución de grupos correcto, para evitar así posibles inconsistencias.

Ejecución de procesos con o sin tramos

Durante el periodo de ejecución de un proceso, la fórmula de un concepto o la de un método puede haber cambiado. También puede suceder que cambie el valor de alguno de los datos con los que trabaja el proceso de cálculo, por ejemplo, en el caso del cálculo de la nómina, puede cambiar la categoría laboral de un empleado, o el departamento al que está adscrito.

Se definen como **elementos con tramos** aquéllos cuyo valor o fórmula puede cambiar a lo largo del tiempo. Cada tramo del elemento está asociado a un periodo de validez que queda delimitado por una fecha de inicio y una fecha de fin.

Si se ejecuta el proceso sin tramos, éste calcula el valor de los distintos conceptos y métodos utilizando las fórmulas o reglas entre cuyas fechas de inicio y de fin esté comprendida la fecha de ejecución. De esta forma, para cada concepto o método, sólo se ejecuta una fórmula.

LN4 ofrece una serie de funciones que permiten tener acceso a las fechas de inicio y de fin del periodo, así como a la fecha de ejecución. Aunque estas fechas siempre se establecerán desde la interfaz del programa, LN4 incluye funciones para poder cambiar estas fechas durante la ejecución de un proceso.

Funciones LN4

Argumentos de las funciones LN4

Las funciones LN4 pueden recibir de 0 a n argumentos de entrada, dependiendo de cada caso, escritos entre paréntesis y separados por comas.

Dentro de la lista de argumentos de una función LN4 se pueden incluir:

- Constantes: números, literales y fechas.
- Variables que recojan datos de diverso tipo (fecha, número, cadena).
- Llamadas a otras funciones LN4.

Para escribir correctamente la sintaxis adecuada, es necesario tener en cuenta ciertos aspectos:

- Si se indica como argumento de una función una constante de tipo cadena de caracteres, se debe delimitar el valor del argumento con comillas dobles.

EJEMPLO

```
EMPLEADOS.StrNombreelemento = "argumento"
```

-
- Si se indica como argumento de una función una constante de tipo número, se debe escribir el número tal cual, sin emplear ningún signo adicional. Es posible indicar el valor de este argumento utilizando el formato científico. Para más información sobre este formato, consulte el apartado [Tipos de datos](#).

EJEMPLO

```
EMPLEADO.Salario = 3000000
```

-
- Si se introduce como argumento una constante de tipo fecha, debe utilizarse la siguiente sintaxis:

```
{yyyy-mm-dd HH:MM:SS}
```

A la hora de utilizar este tipo de dato como argumento de una función, debe tener en cuenta los siguientes aspectos:

- La fecha debe ir delimitada por llaves "{ }".

- El año debe escribirse con cuatro dígitos; para el mes, el día, las horas, los minutos y los segundos se utilizarán dos dígitos.
- El año, el mes y el día se separan mediante el carácter "-".
- Las horas, minutos y segundos se separan mediante el signo ":".
- Entre el día y la hora debe dejarse un espacio en blanco.
- Es posible omitir las horas, minutos y segundos. En este caso, el programa empleará la hora 00, el minuto 00 y el segundo 00.
- Si se introduce como argumento de una función el valor de una variable, basta con escribir el nombre de la variable.
- También es posible indicar como argumento de una función el valor de un elemento de la misma estructura de nodos o de otra estructura, ya sea del mismo Meta4Object o de otro:
 - Si el elemento es de tipo campo, la función recibirá como argumento el valor que se haya asignado a ese elemento en la base de datos.
 - Si el elemento es de tipo método, la función recibirá como argumento el valor que se obtenga al ejecutar el método que se introduzca como argumento. En este caso hay dos posibilidades:
 - Se pasa el valor del concepto al método sin ser ejecutado, mediante *método (concepto)*.
 - Se ejecuta primero el concepto y luego se pasa su valor al método, con *método (concepto())*.

EJEMPLO

Vamos a diseñar una fórmula que obtenga el valor máximo de dos valores. Estos dos valores se obtendrán calculando el valor absoluto de dos elementos incluidos en otras estructuras de nodo del mismo Meta4Object.

Para ello se emplea la función *Max ()*, tal y como se indica en la siguiente sentencia:

```
Max ( Abs ( TI1.elemento), Abs (TI2.elemento) )
```

Tipos de funciones

Las funciones que incorpora el lenguaje LN4 permiten llevar a cabo diversas acciones sobre los registros de un nodo, por ejemplo:

- Añadir registros en una base de datos lógica (memoria)
- Borrar registros en una base de datos lógica (memoria)
- Moverse entre registros

-
- Buscar los registros que cumplen unas condiciones determinadas, etc.

Cada una de ellas pertenece a uno de los siguientes grupos:

- Funciones básicas
- Funciones de cadenas
- Funciones de conversión de tipos
- Funciones matemáticas
- Funciones de fecha y hora
- Funciones de cambio de moneda
- Funciones de archivos
- Funciones de lectura de argumentos
- Funciones de fechas de ejecución
- Funciones para manejo de registros
- Funciones para la base de datos lógica
- Funciones de Meta4Objects
- Funciones básicas de llamadas de nivel 2
- Funciones para ejecución de código JIT
- Funciones de traza
- Funciones para transacciones de memoria
- Funciones de lectura de metadatos
- Funciones de nómina y del sistema de asignación de valores
- Funciones para estadísticas
- Funciones avanzadas

Funciones básicas

Las funciones básicas son:

- BeginVMTransaction ()
- ConfigCSTimeOut (Tipo, Marca)
- DialogBox (Estilo, Título, Cabecera....)
- EndVMTransaction (M4_COMMIT/M4_ROLLBACK / M4_ROLLBACK_RESUME)
- ExistNodeItemIndex (Nodo, Posición, Filtro) o (Nodo, IDElemento)
- Flatten (Nodo destino)
- GetChannelNumberOfNodes ([M4Obj])

- GetCsTimeOut (Tiempo, Tipo)
- GetRegistryValue (DirectorioRegistro, VarNombre)
- GetServerValue (DirectorioOBL, VarNombre)
- GetSliceMode ()
- GetUniqueID ()
- GetVariantType (var)
- IsNull (var)
- JoinSlices ()
- MessageBox (Título, Cabecera....)
- Null ()
- NullValue ()
- SetCsTimeOut (Tiempo, Tipo)
- SetCSTimeoutFunction (NombreDLL, NombreFunción)
- SetSliceMode (ModoTramo)
- Sleep
- TickCount
- Unflatten (elemento...[opcional])

BeginVMTransaction ()

Esta función comienza una transacción de Meta4Object Engine.

Para información complementaria, consulte los apartados siguientes:

- Tipos de transacciones en Meta4Object Engine
- Métodos transaccionales
- Posibles situaciones con transacciones de Meta4Object Engine y su resolución

Tipos de transacciones en Meta4Object Engine

En el Meta4Object Engine existen dos tipos de transacciones distintas:

- Por un lado, están las transacciones clásicas de base de datos, para trabajar con las cuales, el usuario dispone de las funciones *BeginDBTransaction ()* y *EndDBTransaction (Commit/Rollback/Execute_postvalidation)*

-
- Por otro lado, están las transacciones propias del Meta4Object Engine, que permiten trabajar con métodos de cuyo correcto funcionamiento no se está seguro (como el caso de la función *Persist*).

En el caso de que se produzca alguna anomalía en la ejecución de estos métodos, se podrá seguir ejecutando el código.

Las funciones para trabajar con estas transacciones son *BeginVMTransaction ()* y *EndVMTransaction (Commit / Rollback)*

Existe otro conjunto de funciones que son las más usadas y que inician simultáneamente una transacción de Meta4Object Engine y de base de datos. Se trata de *BeginTransaction ()* y *EndTransaction ()*. Estas funciones están definidas como elementos del sistema, por lo que se puede tener acceso a ellas desde cualquier estructura de nodo.

Métodos transaccionales

Los métodos transaccionales son aquellos que inician una transacción de Meta4Object Engine antes de iniciar su ejecución, de forma que si falla algo en ese método, se detiene su ejecución, pero la del método que lo llamó sigue en la instrucción siguiente a la llamada al elemento transaccional.



Trabajar con métodos transaccionales es lo más indicado a la hora de trabajar con transacciones.

En el caso de que el elemento transaccional no funcione correctamente, se devuelve como resultado, así como en las variables por referencia, si las hay, el valor *M4_ERROR*.

Un ejemplo de método transaccional es *Persist_tree ()*.

Posibles situaciones con transacciones de Meta4Object Engine y su resolución

En este apartado, se exponen varios ejemplos que corresponden a distintas situaciones que se pueden dar en una transacción, así como su comportamiento.

EJEMPLO 1

En este ejemplo, una vez que el método no funciona correctamente, deja de ejecutarse el código, lo que supone que no se pueda ejecutar la línea correspondiente a *MessageBox()*.

Es común pensar que se va a ejecutar a partir de la función *EndTransaction*, pero no es así.

En efecto, una vez que el método falla, el Meta4Object Engine se fija en el método en el que se inició la transacción para seguir posteriormente ejecutando desde el elemento del que depende.

```
BeginTransaction
    Método_que_falla()
EndTransaction (commit)
MessageBox ("el método funciona correctamente")
```

EJEMPLO 2

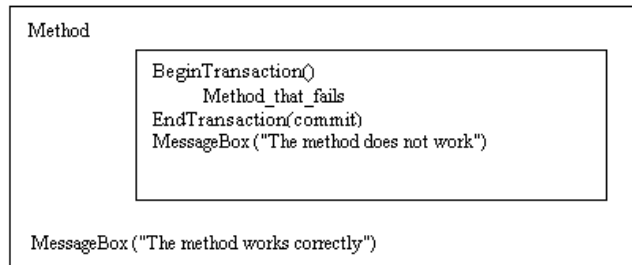
En este segundo ejemplo no existe otro método que llame, por lo tanto se detiene la ejecución.

```
Método_transaccional_que_falla ( )
    MessageBox ("El método funciona correctamente")
```

EJEMPLO 3

En este tercer ejemplo, una vez que falla el método transaccional, se sigue ejecutando el código por el método que lo llamó.

De este modo, ejecutaría la función MessageBox ().



En este caso, al fallar el método, se recupera hasta el anterior al que inició la transacción, por lo que el segundo MessageBox sí se ejecuta, no así el primero.

EJEMPLO 4

En este caso, se utilizan las transacciones para anular la ejecución de un método en el caso de que se cumpla una condición.

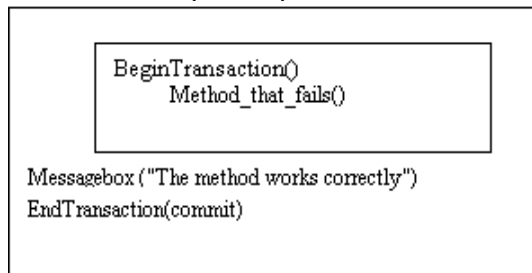
La transacción que se está cerrando en este ejemplo sin éxito, es una transacción que se abre implícitamente antes de comenzar la ejecución de ningún método, de forma que si se cierra su ejecución, se recupera el primer método antes de la llamada.

```
If count ( ) < 1 then
    SetLog (...)
    EndTransaction (Rollback)
Else
    Procesa ( )
Endif
...
```

Si este método formase parte de una cadena de llamadas y pasase por la función `EndTransaction (Rollback)` se comportaría como cualquier otro método que falla (por ejemplo el almacenamiento, la lectura de EOF...) y se recuperaría hasta el anterior al que inició la transacción.

EJEMPLO 5

En este caso, sí se ejecutaría el *MessageBox*, ya que al fallar, se recupera hasta el padre del elemento que empezó la transacción.



A pesar de funcionar como se espera, es poco elegante e induce a errores ya que no queda simétrica la gestión de inicio/fin de la transacción. Además, cuando se ejecuta *EndTransaction* ya no hay ninguna transacción abierta, puesto que la transacción se cerró al producirse el error, quedando como única transacción la implícita que fue abierta antes de la ejecución del primer método.

En el resto de situaciones, no se produce esta circunstancia, porque al producirse un error antes de ejecutar *EndTransaction*, éste no llega a ejecutarse, y el Meta4Object Engine cierra la transacción al recuperarse del error o cancela totalmente la ejecución.

ConfigCSTimeOut (Tipo, Marca)

Esta función configura internamente el tiempo límite de transacción. Los argumentos que recibe son los siguientes:

- **Tipo:** el tipo por defecto es M4_RESET_NEXT_TIMEOUT. Este argumento puede tomar dos valores más:
 - 0: Reinicia el siguiente tiempo límite por defecto a Tipo=1.
 - 1: Habilita/Deshabilita la ventana que indica el tiempo límite.
- **Marca:** este argumento puede tomar dos valores:
 - 0: Deshabilita la ventana que indica el tiempo límite.
 - 1: Habilita la ventana que indica el tiempo límite.

A continuación se explica la posible combinación de sus argumentos:

Type = M4_RESET_NEXT_TIMEOUT

Flag = M4_TRUE

Es equivalente a un SetCsTimeout con el valor por defecto. Se incluye por comodidad.

Type = M4_RESET_NEXT_TIMEOUT

Flag = M4_FALSE

No realiza nada, se incluye por completitud.

Type = M4_ENABLE_TIMEOUT_WINDOW

Flag = M4_TRUE

Permite que aparezca la ventana que indica el tiempo límite.

Type= M4_ENABLE_TIMEOUT_WINDOW

Flag = M4_FALSE

Elimina la ventana de tiempo de espera.

EJEMPLO

```
ConfigCSTimeOut (  
    VALUE NUMBER Type,  
    [VALUE NUMBER Flag]  
)
```

DialogBox (Estilo, Título, Cabecera,...)

Esta función muestra un mensaje en pantalla. Los argumentos segundo y tercero deben ser una cadena de caracteres, pero no así los siguientes.



Para ver más información sobre este punto, haga clic en [Macros para gestionar botones en cuadros de diálogo](#). Estas macros ofrecen diversas combinaciones posibles de botones que aparecerán en pantalla, por ejemplo: *Yes, No, Cancel*.

El estilo es una constante definida previamente, que define el estilo de pantalla que se muestra al usuario. Este argumento determina la presencia del botón y devuelve el valor correspondiente al botón presionado.

El título es un argumento de tipo cadena, que contiene el título de la pantalla que se va a mostrar; mientras que la cabecera es un argumento de tipo cadena que contiene el inicio del mensaje que se muestra en pantalla.

Existen macros que permiten definir los botones que se desea que aparezcan en pantalla, y otras macros que permiten saber qué botones se han presionado.

En una instalación en servidor, la función *Dialogbox*, al igual que la función *MessageBox*, asume que se ha hecho clic en *OK*, ya que no es lógico escribir código LN4 utilizando cualquiera de estas dos funciones para que se ejecute en la parte servidor.

En ambos casos, se muestra un aviso o error donde se indica que se asume que el usuario ha hecho clic en el botón *OK*.

EJEMPLO

```
DialogBox (  
    VALUE NUMBER Style,  
    VALUE VARIABLE STRING Title,  
    VALUE VARIABLE STRING Heading  
)
```

EndVMTransaction (M4_COMMIT/M4_ROLLBACK / M4_ROLLBACK_RESUME)

Finaliza una transacción de Meta4Object Engine. Recibe un único argumento, con dos posibles valores:

- **M4_COMMIT**

Finaliza la transacción en curso de Meta4Object Engine y continua la ejecución de LN4 normalmente.

- **M4_ROLLBACK**

Finaliza la transacción en curso de Meta4Object Engine y finaliza la ejecución de LN4 devolviendo un error.

- **M4_ROLLBACK_RESUME**

Este argumento actúa de forma similar al argumento Rollback con la diferencia de que en este caso no finaliza la ejecución del LN4, esta se retoma a partir del final de la transacción (si la hubiese).



Tenga en cuenta que este comportamiento no se cumple en las transacciones anidadas, ya que la aplicación no soporta este tipo de transacciones (solo existe una transacción, que es la primera que se abre).

EJEMPLO

```
EndVMTransaction(  
    VALUE VARIABLE STRING Commit/RollBack  
)
```

ExistNodeltemIndex (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función devuelve el índice correspondiente a un elemento si existe. En esta función se entiende por índice el ordinal que posee cada uno de los elementos en su nodo según el filtro aplicado.

Los argumentos correspondientes a esta función:

-
- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
 - **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
 - **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
ExistNodeItemIndex (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

No se genera ningún tipo de aviso si no existe el elemento.

Flatten (Nodo destino)

Esta función es de ámbito registro. Se utiliza cuando se dispone de un registro con una serie de elementos, que a su vez tienen tramos. Esta función convierte ese registro en un conjunto de ellos y elimina los tramos, ya que la base de datos no permite su almacenamiento.



En ocasiones se necesita consultar la información recogida en los tramos, que podríamos denominar 3D a modo de ejemplo.

Se puede decir que la función *Flatten* convierte datos de 3D a 2D para tener acceso al contenido de los tramos. Entendemos por 2D un conjunto de datos expuestos en forma de tabla con filas y columnas. Esta conversión se hace sin pérdida de información.

Existe la posibilidad de volver al estado original después de ejecutar la función *Flatten*. Esto se llevaría a cabo mediante la función Unflatten (elemento...[opcional]).

EJEMPLO

Si no se especifica el nodo destino, se trabajará con el mismo nodo.

```
Flatten (
    [VALUE VARIABLE STRING DestinationNode]
)
```

GetChannelNumberOfNodes ([M4Obj])

Esta función devuelve el número de nodos de un Meta4Object determinado. Si no se especifica un Meta4Object, la función devuelve el número de nodos del Meta4Object activo.

GetCsTimeOut (Tiempo, Tipo)

Esta función indica el tiempo límite que tiene una transacción cliente-servidor, de tal forma que si se supera ese tiempo, se indica al cliente que se ha superado y se le pregunta si desea anular la transacción.

El primer argumento indica en segundos el tiempo límite que se ha fijado para realizar una transacción completa.

El segundo indica el tipo de tiempo límite, y puede tomar los siguientes valores:

- M4_DEFAULT_CS_TIMEOUT (0)
Ajusta el valor para los tiempos límite de ejecución OLTP/PROXY. Éste es el valor por defecto.
- M4_NEXT_CS_TIMEOUT (1)
Ajusta el valor del tiempo límite para la siguiente transacción. Una vez ejecutada, vuelve al valor por defecto.
- M4_GENERIC_CS_TIMEOUT (2)
Ajusta el valor para las transacciones que no son ni PROXY ni OLTP.
Ejemplo: Logon, metadatos, espacios proxy, etc.

EJEMPLO

```
GetCSTimeOut (
    VALUE NUMBER, Type
)
```

GetRegistryValue (DirectorioRegistro, VarNombre)

Esta función lee una variable de configuración del registro.

Recibe como argumentos la ubicación dentro del registro de la variable de configuración que se va a leer, y el nombre de dicha variable de configuración.

EJEMPLO

```
GetRegistryValue (
    VALUE FIXED STRING Path,
    VALUE FIXED STRING Variable
)
```

GetServerValue (DirectorioOBL, VarNombre)

Esta función lee una variable de configuración correspondiente al servidor.

Recibe como argumentos la ubicación dentro del servidor de la variable de configuración que se va a leer, y el nombre de dicha variable de configuración.

EJEMPLO

```
GetServerValue (
    VALUE FIXED STRING Path,
    VALUE FIXED STRING Variable
)
```

GetSliceMode ()

Esta función devuelve el modo de ejecución con tramos para llevar a cabo su cálculo.

EJEMPLO

```
GetSliceMode (
    VALUE NUMBER SliceMode
)
```

GetUniqueID ()

Esta función genera un identificador único de 20 caracteres.

GetVariantType (var)

Esta función dice si un valor que se ha indicado como argumento es de tipo cadena, fecha, número o nulo. Es útil en aquellas ocasiones en las que se indique como argumento un valor cuyo tipo se desconozca, para, en función de ello, actuar de un modo u otro.

EJEMPLO

```
GetVariantType (
    VALUE VARIANT Value
)
```

IsNull (var)

Esta función recibe como argumento una variable y su ejecución puede devolver dos posibles resultados:

- Si el valor que recibe como argumento contiene un valor nulo, devuelve el valor M4_TRUE.
- Si el valor que contiene la variable o el campo no es nulo, devuelve el valor M4_FALSE.

EJEMPLO

```
/* Este ejemplo muestra un mensaje en pantalla */
/* dependiendo de si la variable recoge un valor nulo o no*/
If IsNull ( Nombre)=M4_TRUE Then
    MessageBox ("Titulo", Nombre + " contiene un valor nulo")
```

```
Else
    MessageBox("Título", "El valor de Nombre es " + Nombre)
EndIf
```

JoinSlices ()

Esta función permite enlazar todos los registros de un bloque activo, creando así un nuevo registro y borrando todos los originales. Todos los elementos del nuevo registro tendrán un tramo para cada periodo de intersección.

El resultado de los valores del elemento será la suma de los registros si el tipo es numérico, o el último registro si el tipo es cadena.

MessageBox (Título, Cabecera,...)

La función *MessageBox ()* muestra un mensaje en pantalla. *MessageBox ()* sólo puede utilizarse en la parte cliente para generar un único tipo de ventana de diálogo. La ventana incluirá un solo botón con el literal *OK*.

La función *MessageBox ()* recibe como mínimo dos argumentos. El primero se escribirá en la barra de título de la ventana y debe ser una cadena. El resto de argumentos se escribirán en el interior de la ventana, constituyendo así el cuerpo del mensaje.

EJEMPLO

```
MessageBox (
    VALUE VARIABLE STRING Title,
    VALUE VARIABLE STRING Heading
)
```

Null ()

La función *Null ()* no realiza ninguna acción.

NullValue ()

Asigna el valor NULO a una variable, de modo que todas las operaciones que se hagan posteriormente con ésta devolverán el valor NULO, independientemente del tipo de dato que recogiese la variable. La función no recibe argumentos, y su sintaxis es:

```
Variable = NullValue ( )
```

EJEMPLO

```
Variable = NullValue ( ) 'Se asigna a Variable el valor NULO.
Salario= 200000          'Se asigna a Salario el valor 200.000
Total = Salario + Variable 'Se suma el valor de Salario e Variable
' como Variable recoge el valor NULO, la suma ' ' devolverá NULO
MsgBox ("Meta4", "El total es: " + Str ( Total ) ) ' La ventana de
diálogo muestra el mensaje "El total es"
```

SetCsTimeOut (Tiempo, Tipo)

Esta función modifica el tiempo límite que tiene una transacción cliente/servidor para ejecutarse, de tal forma que si se supera ese tiempo, se indica al cliente que se ha sobrepasado y se le pregunta si desea anular la transacción.

El primer argumento indica el tiempo límite que se ha fijado para realizar una transacción completa, facilitado en segundos.

El segundo argumento es opcional y puede tomar los siguientes valores:

- M4_DEFAULT_CS_TIMEOUT (0)
Ajusta el valor para los tiempos de espera de ejecución OLTP/PROXY. Éste es el valor por defecto.
- M4_NEXT_CS_TIMEOUT (1)
Ajusta el valor para la siguiente transacción. Una vez ejecutada, vuelve al valor por defecto.
- M4_GENERIC_CS_TIMEOUT (2)
Ajusta el valor para las demás transacciones que no son ni PROXY ni OLTP. Ejemplo: Logon, metadatos, espacios proxy, etc.

EJEMPLO

```
SetCsTimeOut (
```

```
VALUE NUMBER Time,  
VALUE NUMBER Type  
)
```

SetCSTimeoutFunction (NombreDLL, NombreFunción)

Esta función se utiliza para indicar qué función de una determinada DLL se quiere ejecutar en el caso de que se supere el tiempo definido como límite. El primer argumento indica la DLL a la que pertenece la función que se quiere ejecutar, mientras que el segundo contiene la función propiamente dicha.

La función exportada por la DLL tiene que tener el siguiente prototipo:

```
typedef m4int32_t (*ClTimeoutFunction) (m4int32_t);
```

La variable ***m4int32_t*** es un entero con signo de 4 bytes, cuyo argumento es el valor del *timeout* actual en milésimas de segundo.

El valor de retorno puede ser:

- Menor que 0: indica que la ejecución se anula.
- Igual a 0: indica que el tiempo de espera es infinito o que no existe un tiempo máximo.
- Mayor que 0: indica el tiempo para el siguiente tiempo de espera.

EJEMPLO

```
SetCSTimeoutFunction (  
    VALUE VARIABLE STRING Function,  
    VALUE VARIABLE STRING DllName  
)
```

SetSliceMode (ModoTramo)

Esta función asigna al modo de ejecución con tramos el valor *True* o *False*, necesario para realizar cálculos posteriores sobre él.

EJEMPLO

```
SetSliceMode (  

```

```
VALUE NUMBER SliceMode  
)
```

Sleep

Interrumpe la ejecución del proceso durante el número de segundos especificados en el argumento.

Su argumento es:

- **Seconds:** Número de segundos que se interrumpe la ejecución. Se ignoran los decimales.

EJEMPLO

```
Sleep( 4 )
```

TickCount

Devuelve el número de milisegundos que han transcurrido desde que se inició la aplicación.

No tiene argumentos.

EJEMPLO

```
iMilliseconds = TickCount()
```

Unflatten (elemento...[opcional])

Esta función es de ámbito bloque. Su forma de actuación consiste en tomar un conjunto de registros y comprobar si tienen determinada estructura, de modo que si uno de los elementos tiene el tipo interno *StartDate* y otro elemento tiene el tipo interno *EndDate*, junta el conjunto de registros en uno solo que disponga de tramos.

Las fechas de inicio y de fin son las que van a determinar las fechas de inicio y

fin del tramo. Es importante tener en cuenta que no puede haber solapamiento de fechas.

Se puede indicar a esta función que agrupe un conjunto de registros utilizando como criterio un determinado elemento.

Por ejemplo, si de un conjunto formado por veinte registros se tienen diez de ellos con un valor de elemento y el resto con otro valor de elemento, crearía dos registros distintos con tramos, uno para cada grupo de elementos.

Si no se tiene activado el modo tramos y no se tiene más de un registro, la función no actuaría. Si, por el contrario, el modo Tramos está activo y sólo hay un registro, se crearía un registro con tramos.

EJEMPLO

```
Unflatten (  
    [VALUE VARIABLE STRING Item]  
)
```

Funciones de cadenas

Las funciones de cadenas son:

- Ascii (Cadena)
- Chr (Código)
- ConvertCase (Cadena, Low up)
- IndexOf (Cadena, Subcadena, ÍndiceInicial)
- LastIndexOf (Cadena, SubCadena, [ÍndiceInicial])
- Length (var)
- MatchRegExp (Cadena, ExpReg, [SensibleMayúsculasMinúsculas])
- Mid (Valor, Inicio, longitud)
- Replace (Cadena, SubcadenaAntigua, SubcadenaNueva)
- StrIn (Cadena1, Cadena2)
- Trim (Cadena, Dirección)

Ascii (Cadena)

Esta función devuelve el código Unicode del carácter que aparece en primer lugar del argumento.

Su funcionamiento es opuesto a la función [Chr \(Código\)](#).



El código ASCII está incluido dentro del código Unicode.

EJEMPLO

```
number = Ascii(A)
number acaba valiendo "65"
```

Chr (Código)

Esta función devuelve un valor de tipo cadena que corresponde al carácter cuyo código Unicode (que incluye ASCII) es el argumento especificado. Si este código es nulo, la función devuelve como resultado un valor nulo.

Su funcionamiento es opuesto a la función [Ascii \(Cadena\)](#).

Existen una serie de constantes definidas para poder utilizarlas con la función Chr () y que se enumeran a continuación:

- M4_NEW_LINE
- M4_TAB
- M4_DOUBLE_QUOTE
- M4_CR

Para obtener más información sobre este tipo de constantes, haga clic en [Macros de constantes CHR](#).

EJEMPLO

```
char = Chr(65)
char acaba valiendo "A"
```

ConvertCase (Cadena, Low_up)

Esta función convierte una cadena cualquiera a mayúsculas o minúsculas. Esto se debe a una costumbre muy extendida de convertir cualquier información que se incluye en la base de datos a mayúsculas, ya que las comparaciones son más sencillas y no hay que utilizar ninguna de las constantes establecidas del tipo *M4_Case*.

Los argumentos correspondientes a la función son:

- **Cadena:** contiene la cadena que se desea convertir a mayúsculas o minúsculas.

Si la cadena indicada es nula, esta función devuelve como resultado un valor nulo.

- **Low_up:** indica el tipo de conversión que se desea realizar sobre la cadena, y puede tomar los siguientes valores:

- LowerCase
Convierte la cadena a minúsculas.
- UnChanged
La cadena no tiene ningún cambio.
- UpperCase
Convierte la cadena a mayúsculas.

Se deben tener en cuenta además los siguientes aspectos:

- Si el contenido de este argumento es nulo, la función devuelve como resultado la cadena.
- Si el contenido de este argumento no es válido, devuelve la cadena sin realizar ningún cambio.

Para más información sobre este tipo de constantes, haga clic en [Macros LN4](#).

EJEMPLO

```
ConvertCase (  
    VALUE VARIABLE STRING String,  
    VALUE NUMBER Low_up  
)
```

IndexOf (Cadena, Subcadena, ÍndiceInicial)

Esta función busca la subcadena que se indica como segundo argumento, dentro de la cadena que se indica como primer argumento, utilizando un índice definido que se pasa como tercer argumento.

La búsqueda empieza desde el principio de la cadena, en la posición indicada por el índice, y se dirige hasta el final. Si este tercer argumento es nulo, tomará como valor el 0.

La función devuelve un resultado distinto dependiendo de los valores de los argumentos:

- Para buscar la cadena "p", por ejemplo, dentro de una cadena especificada a partir de la posición 5, la función devuelve el número del carácter donde se encuentra la subcadena dentro del índice
- Si existe algún error en su ejecución, devuelve el valor *M4_ERROR*.
- Si la cadena es nula, devuelve como resultado un valor nulo.
- Si la subcadena es nula, devuelve como resultado *M4_ERROR*.

EJEMPLO

```
IndexOf (  
    VALUE VARIABLE STRING String,  
    VALUE VARIABLE STRING SubString,  
    VALUE NUMBER InitialIndex  
)
```

LastIndexOf (Cadena, SubCadena, [ÍndiceInicial])

Esta función, al igual que *Funciones para manejo de registros*, busca una subcadena dentro de una cadena utilizando un índice definido. La diferencia entre estas dos funciones estriba en que *LastIndexOf* empieza la búsqueda por el final de la cadena.

Los argumentos tienen idéntico significado a los de *IndexOf()*, y al igual que en ésta, la función devuelve un resultado distinto dependiendo de los valores de los argumentos:

-
- Si se busca, por ejemplo, la cadena "p" dentro de una cadena especificada a partir de la posición 5, la función devuelve el número de caracteres donde se encuentra la subcadena dentro del índice.
 - Si existe algún error en su ejecución, devuelve el valor *M4_ERROR*.
 - Si la cadena es nula, devuelve como resultado un valor nulo.
 - Si la subcadena es nula, devuelve como resultado *M4_ERROR*.

Por ejemplo, si se busca la cadena "p" dentro de una cadena especificada a partir de la posición 5, la función devuelve el número de caracteres donde se encuentra la subcadena dentro del índice.

EJEMPLO

```
LastIndexOf (  
    VALUE VARIABLE STRING String,  
    VALUE VARIABLE STRING SubString,  
    [VALUE NUMBER InitialIndex]  
)
```

Length (var)

Esta función devuelve como resultado el número de caracteres de una cadena, que puede ser de tipo texto, fecha o número.

Esta función puede devolver diferentes resultados:

- Si el valor es NULO, devuelve NULL.
- Si el valor es un número, se convierte a una cadena con 8 decimales.

EJEMPLO

```
Length (  
    VALUE VARIANT Value  
)
```

MatchRegExp (Cadena, ExpReg, [SensibleMayúsculasMinúsculas])

Esta función indica si una cadena establecida como primer argumento coincide con una expresión regular que se le indique como segundo argumento.

El tercer argumento es opcional, e indica con *True* o *False* si se quiere que esta función sea sensible o no a mayúsculas y minúsculas. El comportamiento por defecto es que sí lo es.

La función devuelve el índice del primer carácter que cumpla con la expresión regular. Si no lo cumple, devolverá M4_ERROR.

Mid (Valor, Inicio, longitud)

La función *Mid* () extrae de la cadena de caracteres que recibe como primer argumento, los caracteres comprendidos entre las posiciones indicadas en el segundo y el tercer argumento.

La función recibe tres argumentos:

- Valor: el primero, es la cadena de caracteres original.
- Inicio: el segundo, indica la posición a partir de la cual se comenzarán a extraer caracteres. Las posiciones de la cadena comienzan en 0 y van hasta la longitud de la cadena menos uno.
- Longitud: el tercero, indica el número de caracteres a extraer, incluido el inicial.

EJEMPLO

La función *Mid* ("Cálculo de la cotización", 5, 7) devuelve la subcadena "lo de l", es decir, los caracteres comprendidos entre las posiciones 5 (empezando desde 0) y 11, ambas incluidas.

Replace (Cadena, SubcadenaAntigua, SubcadenaNueva)

Esta función reemplaza una subcadena perteneciente a una cadena concreta por otra subcadena nueva que ha sido indicada como argumento.

La función devuelve la cadena obtenida como resultado de las operaciones o, en caso de producirse algún error, devolverá como resultado el valor M4_ERROR.

La función recibe tres argumentos, que corresponden respectivamente a la cadena original, la subcadena que se va a sustituir, y la subcadena que se va a insertar en lugar de la anterior.

Se deben tener en cuenta además los siguientes aspectos:

- Si la cadena que se va a modificar es NULL, devuelve NULL.
- Si la subcadena que se va a eliminar es NULL, devuelve la cadena.
- Si la subcadena que se va a agregar es NULL, devuelve la cadena.

EJEMPLO

```
Replace (
    VALUE VARIABLE STRING String,
    VALUE VARIABLE STRING OldSub,
    VALUE VARIABLE STRING NewSub
)
```

StrIn (Cadena₁, Cadena₂)

Esta función busca la cadena de caracteres que se le asigna como segundo argumento dentro de la cadena de caracteres que recibe como primer argumento. Si la cadena que recibe como segundo argumento está incluida en la que recibe como primer argumento, la función devolverá la posición de la cadena. Si no está incluida, devolverá M4_ERROR.

Si se llama a la función asignándole como argumentos datos de tipo número o fecha, *StrIn* convertirá estos argumentos en una cadena de caracteres, sin que se produzca ningún error.

EJEMPLO

Las siguientes llamadas a la función StrIn () devolverán los valores que se indican en la columna de la derecha:

Llamadas a la función StrIn ()	Valor devuelto
StrIn ("nómina", "casa")	M4_ERROR
StrIn ("nómina", "mina")	Devuelve la posición de la cadena

StrIn ("salario base", "salario")	Devuelve la posición de la cadena
StrIn ("redondeo a 30 días", 30)	Devuelve la posición de la cadena
StrIn ("1999-03-01 es la fecha", {1999-03-01})	Devuelve la posición de la cadena
StrIn ("cotización", "salario")	M4_ERROR
StrIn ("cotización", 30)	M4_ERROR

Trim (Cadena, Dirección)

Esta función elimina los espacios en blanco situados antes o después de una cadena. Sus argumentos son los siguientes:

- **Cadena:** contiene la cadena origen de la cual se tienen que eliminar los espacios en blanco existentes antes o después, en función del argumento *Dirección*.

Tenga en cuenta que si la cadena es nula, la función devuelve el valor nulo.

- **Dirección:** se indica el lado de la cadena donde se deben eliminar los espacios en blanco.

Se deben tener en cuenta los siguientes aspectos:

- Si la dirección es nula, devuelve la cadena.
- Si la dirección no es válida, devuelve la misma cadena.

Este argumento puede tomar los valores dados por las siguientes macros:

- M4_TRIM_LEFT: Se eliminan los espacios que existan en el lado izquierdo de la cadena.
- M4_TRIM_RIGHT: Se eliminan los espacios que existan en el lado derecho de la cadena.
- M4_TRIM_ALL: Se eliminan los espacios que existan a ambos lados de la cadena.

EJEMPLO

```
Trim (
    VALUE VARIABLE STRING String,
    VALUE NUMBER Direction
```

)

Funciones de conversión de tipos

Las funciones de conversión de tipos son:

- ToDate (Valor)
- ToDouble (Valor)
- ToInt (Valor)
- ToString (Valor, [Precisión])

ToDate (Valor)

Convierte la cadena de caracteres o el número que recibe como argumento a un dato de tipo fecha.

EJEMPLO

Las siguientes llamadas a la función ToDate () devolverán los valores que se indican en la columna de la derecha.

Llamadas a la función ToDate ()	Valores devueltos
ToDate ("1999-07-22")	{ 1999-07-22 }
ToDate ("1990-12-01 15:00:00")	{ 1990-12-01 15:00:00 }
ToDate ("1992-10-01")	{ 1992-10-01 }
ToDate ("1990-08-12 10:00:00")	{ 1990-08-12 10:00:00 }

ToDouble (Valor)

Esta función convierte el valor que recibe como argumento a un número en doble precisión, con parte entera y decimal.

Por defecto, el programa utiliza seis cifras en la parte decimal a la hora de la impresión o presentación en pantalla.

EJEMPLO

Las siguientes llamadas a la función ToDouble () devolverían los valores que se indican en la columna de su derecha:

Llamadas a la función ToDouble ()	Valores devueltos
ToDouble (23)	23.000000
ToDouble (89.6)	89.600000
ToDouble (-21)	-21.000000
ToDouble (90)	90.000000

ToInt (Valor)

Esta función devuelve la parte entera de un número con parte decimal, sin realizar ningún tipo de redondeo.

Si la función recibe como argumento un dato que no es de tipo número, por ejemplo una fecha o una cadena de caracteres, intentará convertirla a un número entero. Si no consigue convertir el valor, devolverá el valor 0.0.

EJEMPLO

Las siguientes llamadas a la función ToInt () devuelven el valor que se indica en la columna de su derecha.

Llamadas a la función ToInt ()	Valores devueltos
ToInt (23.56)	23.00
ToInt (32.99)	32.00
ToInt (-15.24)	-15.00
ToInt (27.986)	27.00

ToString (Valor, [Precisión])

Esta función convierte a cadena de caracteres el número o fecha que recibe como primer argumento.

El segundo argumento es opcional, y dependiendo del tipo del primer argumento contendrá datos distintos:

- Si el primer argumento es un número, contiene el número de decimales del número.
- Si el primer argumento es una fecha, indica si es formato corto o largo.

Aunque LN4 gestiona los tipos de datos de forma dinámica, en ocasiones puede ser conveniente realizar conversiones entre tipos.

Por ejemplo, si se quieren concatenar los valores de dos variables que recogen datos de tipo numérico para formar una única cadena de caracteres, será necesario convertir antes las variables al tipo de dato Cadena, ya que LN4 utiliza el mismo operador para concatenar cadenas y para sumar números.

Si la conversión no se realiza con éxito, la función devolverá el valor 0.0.

EJEMPLO

Las siguientes llamadas a la función ToString () devolverán los valores que se indican en la columna de su derecha:

Llamadas a la función ToString ()	Valores devueltos
ToString (34.3)	"34.3"
ToString (-13.5)	"-13.5"
ToString ({ 1999-07-22 11:30:00 })	"1999-07-22 11:30:00"
ToString (45)	"45.000000"

Funciones matemáticas

Las funciones matemáticas son:

- Abs (Fecha o Número)
- Bound (Valor, limInf, limSup)
- Div (Número1, Número2)
- Format (Número, Precision)
- Fraction (Número)
- Log (Number, [Base])
- Max (Value1, Value2, ... , Valuen)
- Min (Valor1, Valor2, ... , Valorn)
- Percentage (Porcentaje, Total)
- Power (Número1, Número2)
- Rnd (Max)
- Round (Valor, Precisión)
- Truncate (Valor, Precisión)

Abs (Fecha o Número)

La función *Abs ()* puede recibir como argumento un valor de tipo número o fecha.

- Si recibe un valor de tipo número, *Abs ()* devolverá el valor absoluto del número que se le asigna como argumento.
- Si recibe un valor de tipo fecha, *Abs ()* devolverá el año, el mes y el día, e igualará a 0 las horas, los minutos y los segundos.

EJEMPLO

Las siguientes llamadas a la función *Abs ()* devolverán los valores indicados en la columna de la derecha.

Llamadas a la función Abs ()	Valores devueltos
Abs (-78.15)	78.15
Abs (69)	69

Abs (-69.42)	69.42
Abs ({1999-07-22 15:30:10})	{1999-07-22 00:00:00}

Bound (Valor, limInf, limSup)

La función *Bound* () recibe tres argumentos, que pueden ser de tipo numérico o de tipo fecha.

El primer argumento es un valor cualquiera, y los argumentos segundo y tercero son los límites inferior y superior que delimitan un intervalo. El segundo argumento debe ser menor que el tercer argumento.

La función se comporta de la siguiente forma:

- Si el valor que se le pasa como primer argumento está comprendido en el intervalo delimitado por los argumentos que se le asignan en segundo y en tercer lugar, la función devuelve el valor del primer argumento.
- Si el valor que se le asigna como primer argumento es menor que el límite inferior del intervalo, la función devuelve el límite inferior. Es decir, el número o la fecha que ha recibido como segundo argumento.
- Si el valor que se le asigna como primer argumento es mayor que el límite superior del intervalo, la función devuelve el límite superior. Es decir, el número o la fecha que ha recibido como tercer argumento.

EJEMPLO

Las siguientes llamadas a la función *Bound* () devolverán los valores que se indican en la columna de su derecha:

Llamadas a la función Bound ()	Valores devueltos
Bound (34, 42, 69)	42
Bound (15, 42, 69)	42
Bound (80, 42, 69)	69
Bound ({ 1999-07-22 }, { 1999-07-01 }, { 1999-08-01 })	{ 1999-07-22 }
Bound ({ 1999-06-30 }, { 1999-07-01 }, { 1999-08-01 })	{ 1999-07-01 }

Bound ({ 1999-09-10 }, { 1999-07-01 }, { 1999-08-01 })	{ 1999-08-01 }
--	----------------

Div (Número1, Número2)

Esta función permite obtener el resto de la división correspondiente a dividir el primer argumento entre el segundo. Los casos posibles son:

- Si uno de los argumentos es nulo, el resultado es nulo.
- Si existe un error, devuelve el valor M4_ERROR.
- Si el segundo argumento es 0, devuelve como resultado 0.

EJEMPLO

```
Returns
Div (
    VALUE NUMBER Number,
    VALUE NUMBER Number2
)
```

Format (Número, Precision)

La función *Format* () convierte a cadena el valor que recibe como primer argumento, poniendo como número de cifras decimales tantos caracteres como se indique en el segundo argumento.

Si la función recibe como primer argumento un número que no tiene parte decimal, se agregarán tantos 0 a la derecha del punto decimal como se hayan indicado en la precisión.

El segundo argumento que recibe la función siempre tiene que ser un número positivo. Si la precisión que se indica como segundo argumento es superior a la precisión del número que se indica como primer argumento, la función agregará ceros a la derecha de la parte decimal.

EJEMPLO

Pongamos como ejemplo las siguientes llamadas a la función `Format ( )` que devolverán los siguientes valores:

Llamadas a la función <code>Format ()</code>	Valores devueltos
<code>Format (123.56, 1)</code>	123.5
<code>Format (1689.69, 4)</code>	1689.6900
<code>Format (1689.6915, 2)</code>	1689.69
<code>Format (8915.312, 1)</code>	8915.3

Fraction (Número)

La función *Fraction ()* devuelve la parte decimal del número que se le asigna como argumento.

EJEMPLO

Las siguientes llamadas a la función *Fraction ()* devolverán los valores que se indican en la columna de su derecha:

Llamadas a la función <code>Fraction ()</code>	Valores devueltos
<code>Fraction (12.34)</code>	0.34
<code>Fraction (-90.567)</code>	-0.567
<code>Fraction (10.10)</code>	0.10
<code>Fraction (-890.45)</code>	-0.45

Log (Number, [Base])

Esta función devuelve el logaritmo del número indicado como parámetro, utilizando la base indicada para ello.

Los argumentos de esta función son:

- **Number:** contiene el número del cual se va a obtener el logaritmo.
- **Base:** indica la base en la que se va a representar el número indicado.

Max (Value₁, Value₂, ... , Value_n)

Esta función devuelve el valor más alto de todos los que recibe como argumentos.

La función puede recibir un número ilimitado de argumentos, que pueden ser de tipo numérico o fecha. Todos los valores que se le indiquen como argumentos en una llamada a la función, deben ser del mismo tipo.

La función *Max* () no acepta datos de tipo cadena de caracteres.

EJEMPLO

Las siguientes llamadas a la función *Max* () devolverán los valores que se indican en la columna de la derecha.

Llamadas a la función Max ()	Valores devueltos
Max (5, 69, 42, 35, 15)	69
Max ({ 1999-07-22 }, { 1999-08-01 })	{ 1999-08-01 }
A={ 1999-07-22 } B={ 1999-08-01 } Max (A,B)	{ 1999-08-01 }

Min (Valor₁, Valor₂, ... , Valor_n)

Esta función devuelve el más bajo de todos los valores que recibe como argumentos.

La función puede recibir un número ilimitado de argumentos, que pueden ser de tipo numérico o fecha.

Todos los valores que se indiquen como argumentos en una misma llamada a

la función deben ser del mismo tipo.

La función *Min* () no acepta datos de tipo cadena de caracteres.

EJEMPLO

Las siguientes llamadas a la función *Min* () devolverán los valores que se indican en la columna de la derecha.

Llamadas a la función <i>Min</i> ()	Valores devueltos
<i>Min</i> (5, 69, 42, 35, 15)	5
<i>Min</i> (13, -10, 22)	-10
<i>Min</i> ({ 1999-07-22 }, { 1999-08-01 })	{ 1999-07-22 }
A={ 1999-07-22 } B={ 1999-08-01 } Max (A,B)	{ 1999-07-22 }

Percentage (Porcentaje, Total)

Devuelve el valor obtenido al calcular el porcentaje que recibe como segundo argumento esta función, al número que recibe como primer argumento.

El segundo argumento puede ser cualquier número real.

EJEMPLO

Las siguientes llamadas a la función *Percentage* () devolverían los valores que se indican en la columna de su derecha:

Llamadas a la función <i>Percentage</i> ()	Valores devueltos
<i>Percentage</i> (100, 20)	20
<i>Percentage</i> (1000, 30)	300
<i>Percentage</i> (250, 20)	50

Percentage (-300, 5)	-15
------------------------	-----

Power (Número1, Número2)

Esta función devuelve el resultado obtenido al elevar el número que recibe como primer argumento a la potencia indicada en el número que se le asigna como segundo argumento.

EJEMPLO

Las siguientes llamadas a la función Power () devolverán los valores que se indican en la columna de la derecha:

Llamadas a la función Power ()	Valores devueltos
Power (5, 2)	25
Power (5, -2)	1/25
Power (-5, 3)	-125
Power (-2, -3)	-1/8

Rnd (Max)

Esta función genera un valor aleatorio entre cero y el valor del argumento que se especifique.

Round (Valor, Precisión)

Esta función redondea el valor que recibe como primer argumento, utilizando la precisión que se le indica como segundo argumento.

La función se comporta de distinta forma, dependiendo del tipo de dato del valor que recibe como primer argumento:

-
- Si el primer argumento es un valor de tipo numérico:
 - Si la precisión es mayor que 0, se redondea la parte decimal.

EJEMPLO

- La función Round (123.456, 1) devolvería el valor 123.5
- La función Round (123.456, 2) devolvería el valor 123.46
- La función Round (123.456, 3) devolvería el valor 123.456

-
- Si la precisión es menor que 0, se redondea la parte entera.

EJEMPLO

- La función Round (123.456, -1) devolvería el valor 120.
- La función Round (123.456, -2) devolvería el valor 100.
- La función Round (123.456, -3) devolvería el valor 0.

-
- Si el primer argumento es un valor de tipo cadena de caracteres:
 - Si la precisión (segundo argumento) es mayor que 0, la función devuelve los n primeros caracteres comenzando por la izquierda, siendo n el número indicado como segundo argumento en la llamada a la función.

EJEMPLO

- La función Round ("Empleado", 2) devolverá la cadena de caracteres Em.

-
- Si la precisión es menor que 0, la función devuelve los n primeros caracteres comenzando por la derecha, siendo «n» el número indicado como segundo argumento en la llamada a la función.

EJEMPLO

- La función Round("Empleado", -5) devolverá la cadena de caracteres "leado".
-

- Si el primer argumento es un valor de tipo fecha:

En este caso, la precisión debe estar comprendida entre 1 y 6. Si se utiliza precisión 1, la función devuelve el año; si se utiliza precisión 2, devuelve el año y el mes, y así sucesivamente.

EJEMPLO

Las siguientes funciones devuelven los siguientes valores:

Round ({ 1999-07-22 12:30:15 }, 1)	1999-01-01 00:00:00
Round ({ 1999-07-22 12:30:15 }, 2)	1999-07-01 00:00:00
Round ({ 1999-07-22 12:30:15 }, 3)	1999-07-22 00:00:00
Round ({ 1999-07-22 12:30:15 }, 4)	1999-07-22 12:00:00
Round ({ 1999-07-22 12:30:15 }, 5)	1999-07-22 12:30:00
Round ({ 1999-07-22 12:30:15 }, 6)	1999-07-22 12:30:15

Truncate (Valor, Precisión)

Esta función trunca el valor que recibe como primer argumento, utilizando la precisión que se le indica como segundo argumento.

La función se comporta de distinta forma, dependiendo del tipo de dato del valor que recibe como primer argumento:

- Si el primer argumento es un valor de tipo numérico:
 - Si la precisión es mayor que 0, se trunca la parte decimal.

EJEMPLO

- La función Truncate (123.456, 1) devolvería el valor 123.4
- La función Truncate (123.456, 2) devolvería el valor 123.45
- La función Truncate (123.456, 3) devolvería el valor 123.456

- Si la precisión es menor que 0, se trunca la parte entera.

EJEMPLO

- La función Truncate (123.456, -1) devolvería el valor 120.
- La función Truncate (123.456, -2) devolvería el valor 100.
- La función Truncate (123.456, -3) devolvería el valor 0.

- Si el primer argumento es un valor de tipo cadena de caracteres:
 - Si la precisión (segundo argumento) es mayor que 0, la función devuelve los n primeros caracteres comenzando por la izquierda, siendo n el número indicado como segundo argumento en la llamada a la función.

EJEMPLO

- La función Truncate ("Empleado", 2) devolverá la cadena de caracteres Em.

- Si la precisión es menor que 0, la función devuelve los n primeros caracteres comenzando por la derecha, siendo «n» el número indicado como segundo argumento en la llamada a la función.

EJEMPLO

- La función Truncate("Empleado", -5) devolverá la cadena de caracteres "leado".

- Si el primer argumento es un valor de tipo fecha:

En este caso, la precisión debe estar comprendida entre 1 y 6. Si se utiliza precisión 1, la función devuelve el año; si se utiliza precisión 2, devuelve el año y el mes, y así sucesivamente.

EJEMPLO

Las siguientes funciones devuelven los siguientes valores:

Truncate ({ 1999-07-22 12:30:15 }, 1)	1999-01-01 00:00:00
Truncate ({ 1999-07-22 12:30:15 }, 2)	1999-07-01 00:00:00
Truncate({ 1999-07-22 12:30:15 }, 3)	1999-07-22 00:00:00
Truncate({ 1999-07-22 12:30:15 }, 4)	1999-07-22 12:00:00

Truncate ({ 1999-07-22 12:30:15 }, 5)	1999-07-22 12:30:00
Truncate ({ 1999-07-22 12:30:15 }, 6)	1999-07-22 12:30:15

Funciones de fecha y hora

Las funciones de fecha y hora son:

- AddMonths (Fecha, NúmeroMeses)
- AddYears (Fecha, NúmeroAños)
- CommonTime (InicFecha1 , Fecha de fin1 , InicFecha2 , Fecha de fin2)
- CommonTimeStamp (FechaInicio1 , FechaFin1 , FechaInicio2 , FechaFin2)
- Date30 (Fecha)
- Date30Ex (Fecha, [FebruaryAs30])
- DateAdd (Fecha1, Unidades, TipoUnidad)
- DateDiff (Fecha1, Fecha2, TipoUnidad, [InicioSemana])
- DateToAllNumbers (Fecha, var1, var2, var3, var4, var5, var6)
- DateToNumbers (Fecha, var1, var2, var3)
- DayOfTheWeek (Fecha)
- DaysOfMonth (Fecha)
- DaysOfYear (Número)
- DxCompare (Cadena1, Cadena2)
- DxDate (Día, Mes, Año)
- DxDate30 (Día, Mes, Año)
- DxDate30Ex (Día, Mes, Año, [FebruaryAs30])
- DxDatElso (Fecha)
- DxDatElsoD (Fecha)
- DxDatElsoTS (Fecha)
- DxDay (Fecha)
- DxDaysOfMonth (Mes, Año)
- DxInc (Valor)

-
- DxMonth (Fecha)
 - DxRound (Valor, Precisión)
 - DxRound50 (Valor)
 - DxRoundCent (Valor)
 - DxTruncaCent (Valor)
 - DxYear (Fecha)
 - Months (Fecha1, Fecha2)
 - NumbersToDate (Número1, Número2, ..., Número6)
 - Today ()
 - TodayGMT ()
 - TodayNow ()
 - TodayNowGMT ()
 - Years (Fecha1, Fecha2)

AddMonths (Fecha, NúmeroMeses)

La función *AddMonths ()* suma el número de meses que se le asignan como segundo argumento a la fecha que recibe como primer argumento.

El segundo argumento puede ser un entero positivo o negativo. Si el segundo argumento es negativo, la función restará meses a la fecha que recibe como primer argumento.

EJEMPLO

Las siguientes llamadas a la función *AddMonths ()* devolverán los valores que se indican en la columna de la derecha.

Llamadas a la función AddMonths ()	Valores devueltos
AddMonths ({ 1999-07-01 }, 3)	{ 1999-10-01 00:00:00 }
AddMonths ({ 1999-01-15 22:15:00 }, 5)	{ 1999-06-15 22:15:00 }
AddMonths ({ 1973-07-22 }, 12)	{ 1974-07-22 }
AddMonths ({ 1998-02-02 23:59:59 }, -3)	{ 1999-11-02 23:59:59 }

AddYears (Fecha, NúmeroAños)

La función *AddYears* () suma a la fecha que recibe como primer argumento el número de años que se le asignan como segundo argumento.

El segundo argumento puede ser un entero positivo o negativo. Si el segundo argumento es negativo, la función restará ese número en años a la fecha que recibe como primer argumento.

EJEMPLO

Las siguientes llamadas a la función *AddYears* () devolverán los valores que se indican en la columna de la derecha.

Llamadas a la función AddYears ()	Valores devueltos
AddMonths ({ 1999-07-01 }, 3)	{ 2000-07-01 }
AddMonths ({ 1990-01-15 22:15:00 }, 5)	{ 1995-01-15 22:15:00 }
AddMonths ({ 1973-07-22 }, 12)	{ 1985-07-22 }
AddMonths ({ 1998-02-02 }, -3)	{ 1995-02-02 }

CommonTime (InicFecha1 , Fecha de fin₁ , InicFecha₂ , Fecha de fin₂)

Esta función devuelve el tiempo en común entre dos intervalos de tiempo. Recibe como argumentos cuatro fechas, que se corresponden con la fecha de inicio y de fin del primer y segundo intervalo de tiempo que se quiere comparar. Las fechas deben indicarse en el formato aceptado por el lenguaje LN4.

La función devuelve un valor de tipo fecha {yyyy-mm-dd hh:mm:ss}. Es importante tener en cuenta que se considera el día como unidad.

CommonTimeStamp (FechaInicio₁ , FechaFin₁ , FechaInicio₂ , FechaFin₂)

Esta función devuelve el tiempo compartido entre dos intervalos de tiempo,

usando fechas y horas. La particularidad de esta función estriba en que las diferencias de horas en el mismo día no se cuentan como un día completo. Si alguno de los argumentos es nulo esta función devuelve el valor *NULL*.

Date30 (Fecha)

La función *Date30 ()* devuelve la fecha que correspondería al día que se le asigne como argumento, en el caso de que todos los meses tuviesen 30 días.

La fecha que se indica como argumento debe escribirse en el formato aceptado por el lenguaje LN4 para datos de tipo fecha, variable, o elemento entre otros.

Es una función específica de nómina, que convierte una fecha a su correspondiente fecha en el calendario Juliano (es decir, número de días transcurridos desde el día 1 del año 0), y a continuación divide esta fecha entre 30 para saber cuántos meses y años habrían transcurrido si los meses tuviesen 30 días.

Date30Ex (Fecha, [FebruaryAs30])

Esta función, usada sin especificar el argumento opcional FebruaryAs30, en el caso de que el último mes sea febrero no bisiesto y el día sea 28 ó 29, contabiliza 30 días, y en el caso de que sea febrero bisiesto y el día sea 29, contabiliza 30 días.

Por facilidad de uso, puede ser utilizada con un argumento opcional FebruaryAs30, de modo que si se pasa como M4_TRUE, se comporta como se ha descrito, y si se pasa como M4_FALSE, se comporta como su correspondiente función Date30.

EJEMPLO

```
Comportamiento con el arreglo
date = Date30Ex( {2007-02-28} )
date = Date30Ex( {2007-02-28}, M4_TRUE )
Comportamiento sin el arreglo
date = Date30( {2007-02-28} )
date = Date30Ex( {2007-02-28}, M4_FALSE )
```

DateAdd (Fecha1, Unidades, TipoUnidad)

Esta función muestra el resultado final de sumar un cierto valor a una fecha

indicada como primer argumento. El valor de incremento se indica en el segundo argumento, mientras que el tercero indica el tipo de unidad en el que está expresado dicho valor. Como unidad de incremento se pueden utilizar segundos, minutos, horas, días, semanas, meses o años.

Los comportamientos especiales de esta función son:

- Un argumento nulo devuelve resultado nulo.
- Un argumento incorrecto se intenta convertir a correcto y proseguir. Si falla la conversión, se produce un error y se detiene la ejecución, al igual que el resto de funciones.

EJEMPLO

Fecha 1	Unidad	Tipo de unidad	Resultado
1998-01-15	1	Año	1999-01-15
1996-02-29	1	Año	1997-02-28
1997-01-15	1	Mes	1997-02-15
1996-02-29	1	Mes	1996-02-29
1996-01-30	1	Mes	1996-02-28
1996-01-01	20	Día	1996-01-21
1999-01-01 23:15:00	2	Hora	1999-01-02 01:15:00
1999-01-01 23:15:00	80	Minuto	1999-01-02 00:35:00
1999-09-22	1	Semana	1999-01-29

DateDiff (Fecha1, Fecha2, TipoUnidad, [InicioSemana])

Calcula la diferencia en unidades (por ejemplo, segundos) entre dos fechas

indicadas como argumentos, restándose la primera de la segunda.

El tercer argumento indica el tipo de unidad que se va a considerar, mientras que el último es un argumento opcional que indica el día que comienza la semana. Este argumento hace que la función reste semanas completas, y puede contener cualquier valor comprendido entre 1 y 7, considerándose el 1, lunes, como valor por defecto.

Los comportamientos especiales de la función son los siguientes:

- Si existe algún argumento nulo, la función devuelve como resultado un valor nulo.
- Si el tipo de argumento indicado es incorrecto, la función intenta convertirlo a correcto y seguir. Si falla en la conversión, se produce un error y se detiene la ejecución.
- Si la segunda fecha es anterior a la primera, la función devuelve un número de unidades negativo.

El tipo de unidad *M4_Complete_Week* devuelve el número de semanas completas en el que está el periodo de tiempo pasado como argumento.

Las unidades que se pueden utilizar con esta función se muestran a continuación. Para obtener más información sobre ellas, haga clic en [Macros LN4](#):

Unidades

M4_YEAR

M4_MONTH

M4_DAY

M4_HOUR

M4_MINUTE

M4_SECOND

M4_WEEK

M4_COMPLETE_WEEK

EJEMPLO

Fecha 1	Fecha 2	TipoUnidad	Resultado
1997-01-15	1998-01-01	M4_YEAR	1
1995-02-28	1996-02-29	M4_YEAR	1
1999-02-28	1999-03-30	M4_MONTH	1
1999-03-30	1999-04-30	M4_MONTH	1
1999-01-28	1996-02-28	M4_MONTH	1
1999-01-10	1999-01-15	M4_DAY	5
1998-12-31 22:00:00	1999-01-15	M4_HOUR	12
1999-09-19	1999-09-21 (Tuesday, la semana empieza en lunes)	M4_COMPLETE_W EEK	1
1999-09-19	1999-09-21	M4_WEEK	0

DateToAllNumbers (Fecha, var₁, var₂, var₃, var₄, var₅, var₆)

Esta función recibe como argumento una fecha y seis variables por referencia, que obligatoriamente se deben haber declarado antes de llamar a la función.

Como resultado, se divide la fecha que recibe como primer argumento en años, meses, días, horas, minutos y segundos, y se asignan estos valores a las variables que se han indicado en las posiciones segunda a séptima, respectivamente.

EJEMPLO

Las siguientes llamadas a la variable DateToAllNumbers () realizarán las siguientes asignaciones:

Fecha	Arg 1	Arg 2	Arg 3	Arg 4	Arg 5	Arg 6
{ 1999-07-22 03:50:20 }	1999	7	22	3	50	20
{ 1990-05-03 00:00:00 }	1990	5	3	0	0	0
{ 1992-11-11 00:12:12 }	1992	11	11	0	12	12
{ 1982-06-21 10:56:13 }	1982	6	21	10	56	13

DateToNumbers (Fecha, var₁, var₂, var₃)

Esta función recibe cuatro argumentos: una fecha y tres variables. Como resultado, divide la fecha que recibe como primer argumento en años, meses y días, y asigna estos valores a las variables que se han indicado en segundo, tercer y cuarto lugar respectivamente. Estas variables recogerán datos de tipo entero.

Es necesario haber declarado previamente las variables que se indican como argumentos, ya que son variables por referencia. Además de esta asignación, *DateToNumbers* () devolverá el valor M4_SUCCESS si se ejecutó correctamente o en caso contrario M4_ERROR.

EJEMPLO

Las siguientes llamadas a la función DateToNumbers () realizarían las siguientes asignaciones:

Fecha - Argumento 1	Argumento 2	Argumento 3	Argumento 4
{ 1999-12-25 }	1999	12	25
{ 1990-05-03 }	1990	5	3
{ 1992-11-11 00:12:12 }	1992	11	11
{ 1982-06-21 10:56:13 }	1982	6	21

DayOfTheWeek (Fecha)

La función *DayOfTheWeek* () devuelve el día de la semana (lunes, martes, etc.) correspondiente a la fecha que se le indica como argumento. La función devuelve los siguientes valores:

1	Lunes
2	Martes
3	Miércoles
4	Jueves
5	Viernes
6	Sábado
7	Domingo

La fecha se puede indicar como variable, valor de un elemento, o en un formato de fecha válido para el lenguaje LN4.

EJEMPLO

Las siguientes llamadas a la función *DayOfTheWeek* devolverán los siguientes valores:

DayOfTheWeek ({ 1999-07-25 })	5
DayOfTheWeek ({ 1999-07-22 })	2
DayOfTheWeek ({ 1999-11-23 })	7
DayOfTheWeek ({ 1999-12-25 })	4

DaysOfMonth (Fecha)

Devuelve el número de días del mes que se le asigna como argumento.

EJEMPLO

Las siguientes llamadas a la función *DaysOfMonth* () devolverán los valores que se indican en la columna de la derecha:

Llamadas a la función <i>DaysOfMonth</i> ()	Valores devueltos
<i>DaysOfMonth</i> ({ 1999-11-01 })	30
<i>DaysOfMonth</i> ({ 1999-07-15 })	31
<i>DaysOfMonth</i> ({ 1999-02-20 })	28
<i>DaysOfMonth</i> ({ 2002-02-01 })	29

DaysOfYear (Número)

Devuelve el número de días del año que se le indican como argumento.



Si se le asigna como argumento una constante, debe escribirse el año con cuatro cifras.

EJEMPLO

Las siguientes llamadas a la función *DaysOfYear* () devolverán los valores que se indican en la columna de su derecha:

Llamadas a la función <i>DaysOfYear</i> ()	Valores devueltos
<i>DaysOfYear</i> (1999)	365
<i>DaysOfYear</i> (1993)	365
<i>DaysOfYear</i> (1990)	365
<i>DaysOfYear</i> (2000)	366

DxCompare (Cadena1, Cadena2)

Compara dos cadenas que se indican como argumentos, que pueden tomar como valores fechas o números, e indica qué cadena es mayor.

La función puede devolver los siguientes valores:

- 1: si la primera cadena precede a la segunda.
- -1: si la primera cadena sigue a la segunda.
- 0: si las dos cadenas son iguales.

Si alguna de las cadenas tiene un valor nulo, la función devuelve como resultado un valor nulo.

EJEMPLO

```
DxCompare (  
    VALUE VARIABLE STRING String1,  
    VALUE VARIABLE STRING String2  
)
```

DxDate (Día, Mes, Año)

Esta función devuelve un valor con formato fecha, formado por día, mes y año. Si alguno de los argumentos es nulo, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxDate (  
    VALUE NUMBER Day,  
    VALUE NUMBER Month,  
    VALUE NUMBER Year  
)
```

DxDate30 (Día, Mes, Año)

Esta función devuelve un valor formado por día, mes y año. A diferencia de la

función *DxDate (Día, Mes, Año)*, ésta considera que todos los meses del año tienen 30 días. Si alguno de los argumentos es nulo, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxDate30 (
    VALUE NUMBER Day,
    VALUE NUMBER Month,
    VALUE NUMBER Year
)
```

DxDate30Ex (Día, Mes, Año, [FebruaryAs30])

Esta función, usada sin especificar el argumento opcional FebruaryAs30, en el caso de que el último mes sea febrero no bisiesto y el día sea 28 ó 29, contabiliza 30 días, y en el caso de que sea febrero bisiesto y el día sea 29, contabiliza 30 días.

Por facilidad de uso, puede ser utilizada con un argumento opcional FebruaryAs30, de modo que si se pasa como M4_TRUE, se comporta como se ha descrito, y si se pasa como M4_FALSE, se comporta como su correspondiente función DxDate30.

EJEMPLO

```
Comportamiento con el arreglo
number = DxDate30Ex( 28, 02, 2007 )
number = DxDate30Ex( 28, 02, 2007, M4_TRUE )

Comportamiento sin el arreglo
number = DxDate30( 28, 02, 2007 )
number = DxDate30Ex( 28, 02, 2007, M4_FALSE )
```

DxDatElso (Fecha)

Esta función devuelve como resultado un valor con formato fecha, formado por día, mes y año. A diferencia de la función *DxDate*, esta función devuelve el mismo valor utilizando el formato ISO yyyy-mm-dd. Si alguno de los

argumentos es nulo, la función devuelve como resultado un valor nulo.

EJEMPLO

```
DxDateIso (  
    VALUE DATE AND TIME Date  
)
```

DxDatIsoD (Fecha)

Esta función devuelve un valor con formato fecha, formado por día, mes y año.

A diferencia de la función *DxDatIso*, ésta devuelve el mismo valor pero antecediendo a esta cadena el carácter "d", que indica la particularidad de esta función.

El resultado final se mostrará utilizando el formato { d yyyy-mm-dd }.

Si alguno de los argumentos es nulo, la función devuelve como resultado un valor nulo.

EJEMPLO

```
DxDatIsoD (  
    VALUE DATE AND TIME Date  
)
```

DxDatIsoTS (Fecha)

Esta función devuelve un valor con formato fecha, formado por día, mes y año.

A diferencia de la función *DxDatIso*, ésta devuelve el mismo valor, pero antecediendo a esta cadena el carácter "ts" que indica la particularidad de esta función.

El resultado final se mostrará utilizando el formato { ts yyyy-mm-dd }.

Si alguno de los argumentos es nulo, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxDateIso TS (
    VALUE DATE AND TIME Date
)
```

DxDay (Fecha)

Devuelve el día de una fecha que se indica como argumento. Si la fecha es nula, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxDay (
    VALUE DATE AND TIME Date
)
```

DxDaysOfMonth (Mes, Año)

Esta función devuelve el número de días que tiene un mes de un año concreto. Si alguno de los argumentos es nulo, la función devuelve como resultado un valor nulo.

EJEMPLO

```
DxDaysOfMonth (
    VALUE NUMBER Month,
    VALUE NUMBER Year
)
```

DxInc (Valor)

Esta función permite incrementar un número o una fecha, indicado como argumento, en una unidad. Si el argumento es nulo, esta función devuelve

como resultado un valor nulo.

EJEMPLO

```
DxInc (  
    VALUE VARIANT Value  
)
```

DxMonth (Fecha)

Esta función devuelve el mes correspondiente a una fecha indicada como argumento. Si dicha fecha es nula, la función devuelve como resultado un valor nulo.

EJEMPLO

```
DxMonth (  
    VALUE DATE AND TIME Date  
)
```

DxRound (Valor, Precisión)

Esta función redondea el primer argumento según la precisión indicada en el segundo. Realiza la misma operación que la función Round (Valor, Precisión).

Si el primer argumento es nulo, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxRound (  
    VALUE VARIANT Value  
)
```

DxRound50 (Valor)

Esta función redondea el argumento indicado, en función del valor de éste. La posible casuística es la siguiente:

- Si la parte decimal del argumento está entre .01 y .49, redondeará la parte decimal a .50.
- Si la parte decimal del argumento está entre .51 y .99, redondeará la parte decimal a .0 e incrementará en uno la parte entera.

Si el argumento es nulo, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxRound50 (  
    VALUE VARIANT Value  
)
```

DxRoundCent (Valor)

Esta función redondea a centésimas el argumento indicado y muestra el número final con dos decimales. Si el argumento es nulo, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxRoundCent (  
    VALUE VARIANT Value  
)
```

DxTruncaCent (Valor)

Esta función trunca el número final para mostrarlo únicamente con dos decimales (centésimas).

Si el argumento es nulo, esta función devuelve como resultado un valor nulo.

EJEMPLO

```
DxTruncaCent (
    VALUE VARIANT Value
)
```

DxYear (Fecha)

Esta función tiene como único argumento una fecha, de la cual devuelve el año a la que pertenece. Si el valor de la fecha es nulo, devuelve como resultado un valor nulo.

EJEMPLO

```
DxYear (
    VALUE DATE AND TIME Date
)
```

Months (Fecha1, Fecha2)

La función *Months (Fecha1, Fecha2)* devuelve el número de meses transcurridos entre las dos fechas que recibe como argumentos.

El orden de los argumentos afecta al resultado de la función, es decir, si la fecha que recibe la función como primer argumento es mayor que la fecha que recibe en segundo lugar, la función devolverá valores negativos.

La función no considera el número de días naturales de los meses que se le indican como argumento. Por ejemplo, las dos llamadas a la función *Months ()* que se indican a continuación devolverían el mismo valor a pesar de que enero y febrero no tiene el mismo número de días:

```
Months ( { 1999-01-01 }, { 1999-02-01 } )
Months ( { 1999-02-01 }, { 1999-03-01 } )
```

EJEMPLO

Las siguientes llamadas a la función *Months ()* devolverán estos valores de meses transcurridos como números enteros:

Llamadas a la función Months ()	Valores devueltos
----------------------------------	-------------------

Months ({ 1999-01-01 }, { 1999-03-01 })	2
Months ({ 1999-05-01 }, { 1999-12-10 })	7
Months ({ 1999-03-12 }, { 1999-03-30 })	0
Months ({ 1999-03-30 }, { 1999-03-02 })	-1
Months ({ 2004-03-30 }, { 2003-03-02 })	-13

NumbersToDate (Número₁, Número₂, ..., Número₆)

Convierte a formato fecha los números que recibe como argumento. La función puede recibir un máximo de seis argumentos, cuyo orden se corresponde con el año, mes, día, hora, minutos y segundos de la fecha que se quiere crear.

No es necesario indicar a la función los seis argumentos. Basta con introducir el primer argumento, es decir, el correspondiente al año.

De todas formas, cuando se llame a la función y no se le asignen todos los argumentos, será necesario introducir aquéllos que preceden al argumento que se omite (no es posible introducir un argumento que ocupe un lugar posterior). Es decir, es posible llamar a la función *NumbersToDate* () introduciendo el año y el mes, pero no será posible llamar a la función si se introduce el año y día, sin indicar el mes que se trata.

En el caso de que no se asigne a la función todos los argumentos, la función realizará las siguientes conversiones:

Año - Argumento 1	Obligatorio
Mes - Argumento 2	01 - Enero
Día - Argumento 3	01 - Primer día del mes
Hora - Argumento 4	00
Minuto - Argumento 5	00
Segundo - Argumento 6	00

EJEMPLO

Las siguientes llamadas a la función NumbersToDate () devolverían estas fechas:

Llamadas a la función NumbersToDate ()	Valores devueltos
NumbersToDate (1999, 7, 22, 12, 30, 5)	{ 1999-07-22 12:30:05 }
NumbersToDate (1996, 12, 5, 0, 0, 0)	{ 1996-12-05 00:00:00 }
NumbersToDate (1996, 12, 5)	{ 1996-12-05 00:00:00 }
NumbersToDate (1996, 3)	{ 1996-03-01 00:00:00 }
NumbersToDate (1998)	{ 1998-01-01 00:00:00 }
NumbersToDate (1998, 12, 5, 6)	{ 1998-12-05 06:00:00 }

Today ()

Devuelve la fecha del sistema. No recibe ningún argumento.

EJEMPLO

Si la fecha del sistema es:

22-11-1998

La función Today () devolverá el valor:

{ 1998-11-22 }

TodayGMT ()

Devuelve la fecha del sistema en formato GMT. Puede ser de gran ayuda para la sincronización de servidores en diferentes puntos del mundo.

TodayNow ()

Esta función devuelve la fecha y hora del sistema. No recibe ningún argumento.

EJEMPLO

Si la fecha del sistema es:
22-11-1998 y la hora 15:45:09,
la función `TodayNow ( )` devolverá el valor siguiente:
{ 1998-11-22 15:45:09 }

TodayNowGMT ()

Devuelve la fecha y hora del sistema en formato GMT. Puede ser de gran ayuda para la sincronización de servidores en diferentes puntos del mundo.

Years (Fecha₁, Fecha₂)

Esta función devuelve el número de años transcurridos entre las dos fechas que se le indican como argumentos.

El orden de los argumentos afecta al resultado de la función, es decir, si la fecha que recibe la función como primer argumento es mayor que la fecha que recibe en segundo lugar, la función devolverá valores negativos.

La función devolverá la diferencia obtenida al restar la diferencia entre el año mayor y el menor, sin considerar los meses, días, ni el resto de valores indicados en la fecha.

Por ejemplo, las dos llamadas a la función *Years* que se indican a continuación devolverán el mismo valor:

```
Years ( { 1990-01-01 }, { 1996-01-01 } )  
Years ( { 1990-01-01 }, { 1996-12-01 } )
```

EJEMPLO

Las siguientes llamadas a la función *Years ()* devolverán los valores que recogen las celdas de su derecha:

Llamadas a la función <i>Years ()</i>	Valores devueltos
<code>Years ({ 1990-12-10 }, { 1991-01-01 })</code>	1
<code>Years ({ 1990-12-10 }, { 1996-10-10 })</code>	6

Years ({ 1996-10-09 }, { 1996-07-22 })	0
Years ({ 1998-01-01 }, { 2005-02-03 })	7

Funciones de cambio de moneda

El Meta4Object Engine ofrece la posibilidad de trabajar con distintos tipos de moneda de una forma dinámica y transparente para el usuario.

El funcionamiento de la multimoneda se basa en tres aspectos:

- Meta4Object de cambio de moneda

Es el repositorio de los tipos de cambio entre todas las monedas. El diseño de este Meta4Object permite almacenar los datos como un histórico de los distintos cambios que se han producido.

Por tanto, un cambio de una moneda a otra se define por los identificadores de la moneda origen y destino así como por su fecha de cambio.

El Meta4Object de cambio de moneda gestiona también el uso de monedas intermedias y monedas raíz, y tiene los métodos necesarios para realizar los cambios.

- Elementos de tipo moneda

Existe un tipo de elemento moneda que se reconoce como elemento especial. Los elementos de tipo moneda permiten conectar campos como la fecha de cambio o el tipo de moneda relativo al elemento mediante elementos auxiliares.

Cuando se convierten registros de una moneda a otra, los elementos de tipo moneda se actualizan junto con todos sus campos auxiliares.

- Funciones LN4 para el cambio de monedas

Permiten convertir valores numéricos y registros entre distintas monedas.

Meta4Object de cambio de moneda

Todo el soporte de multimoneda está basado en el Meta4Object de cambio de moneda llamado *Exchange_Rates*.

Básicamente, este Meta4Object permite definir un histórico para cada tipo de moneda con la siguiente información:

- Número de decimales significativos de la moneda

-
- Moneda raíz para una determinada moneda
 - Tipo de conversión al resto de monedas para una fecha de cambio y una fuente de tipo de cambio concretos
 - Monedas auxiliares (intermedias) para realizar las conversiones que no son directas

La estructura del Meta4Object de moneda se describe en los siguientes apartados:

- Nodo Currency
- Nodo Rates
- Nodo Aux_Cur_Ex

Nodo Currency

El nodo *Currency* corresponde al nodo raíz. En él se define la información relativa a cada moneda.

Los campos que identifican este nodo son los siguientes:

- **ID_Currency**: contiene el identificador de la moneda.
- **NM_Currency**: identifica el nombre de la moneda.
- **Dec_Nb**: contiene el número de decimales significativos.
- **Id_Root_Currency**: identifica la moneda raíz (véase nota).



Cuando una moneda tiene una moneda raíz con fecha de inicio anterior a la fecha de cambio, significa que depende de ésta para realizar cualquier cambio. Por ejemplo, si la moneda PES tiene como raíz EUR a partir de 1999, la conversión de PES a cualquier moneda y viceversa se realizará a través de EUR siempre que la fecha de cambio sea posterior a 1999.

- **St_Start_Root_Currency**: identifica la fecha de inicio de la moneda raíz (véase nota).
- **CurGetDecimals**: corresponde a un método del Meta4Object.
- **CurGetExchange**: corresponde a un método del Meta4Object.
- **CurGetExchangeRounded**: corresponde a un método del Meta4Object.

Nodo Rates

Este nodo está conectado por un tipo de conexión Registro-Bloque (RB) con el nodo *Currency*. Este nodo contiene los cambios directos entre la moneda origen y todas las demás.

Los campos que identifican este nodo son los siguientes:

- **ID_Currency**: contiene el identificador de la moneda origen. Está conectado con *Currency*.
- **ID_Refer_Cur**: contiene el identificador de la moneda destino.
- **Ex_type**: identifica la clase de cambio.
- **Dt_Start**: almacena la fecha de inicio del cambio.
- **Dt_End**: guarda la fecha de fin del cambio.
- **Rate**: contiene el tipo de cambio, que es el factor multiplicador que se utiliza para pasar de la moneda origen a la moneda destino.

La clave primaria de la tabla del Meta4Object está compuesta por (*ID_Currency*, *ID_Refer_Cur*, *Ex_type* y *Dt_Start*). Por tanto, siempre que se acceda a un cambio directo, es necesario especificar la moneda origen, la moneda destino, la clase de cambio y la fecha de cambio.

La clase de cambio (*Ex_type*) permite tener distintos tipos de cambio para una moneda en una fecha específica. Esto cobra importancia cuando se cuenta con varias fuentes de financiación, por ejemplo, las cotizaciones de las monedas en diferentes mercados de valores.

Nodo Aux_Cur_Ex

Este nodo está conectado por un tipo de conexión RB con *Currency*, y en él se definen los cambios indirectos entre la moneda origen y la moneda destino.

Los campos que identifican a este nodo son:

- **ID_Currency**: contiene el identificador de la moneda origen. Está conectado con *Currency*.
- **ID_Currency_Des**: contiene el identificador de la moneda destino.
- **ID_Curren_Iterm**: contiene el identificador de la moneda intermedia.
- **Dt_Start**: almacena la fecha de inicio del cambio.
- **Dt_End**: guarda la fecha de fin del cambio.

Elementos de tipo moneda

Si un elemento es de tipo moneda, puede tener asociados mediante elementos auxiliares los siguientes campos (por orden):

- Moneda (*ID_Currency*) del elemento (obligatorio).
- Fecha de cambio del elemento (opcional).
- Tipo de cambio (*Ex_type*) del elemento (opcional).

Las ventajas de la utilización de elementos de tipo moneda son las siguientes

-
- El campo moneda contiene información que le identifica claramente. Es posible tener en el mismo registro varios elementos moneda con identificadores de moneda, fechas de cambio y tipos de cambio distintos.
 - Identifica qué campos de una estructura de nodo contienen valores moneda. Esto es importante para el manejo y presentación de los Meta4Objects.
 - Las funciones *CurExchangeRecord* y *CurExchangeRecordRounded* cambian los elementos moneda a la moneda especificada y actualizan sus campos auxiliares de manera automática.

Funciones LN4 para el cambio de monedas

Sólo se debe llamar a las funciones *CurGetExchange*, *CurGetDecimals* y *CurGetExchangeRounded* a través del Meta4Object de cambio de moneda denominado *Exchange_Rates*. Si se utilizan desde otro Meta4Object, la llamada debe hacerse por nivel 2.

A las funciones *CurExchangeRecord*, *CurGetExchangeCurrency*, *CurExchangeRecordRounded* y *CurGetExchangeCurrencyRounded* se las puede llamar directamente desde cualquier Meta4Object, ya que se encargarse internamente de manejar las llamadas de nivel 2 para tener acceso al Meta4Object.

Las funciones para el cambio de monedas son:

- *CurExchangeRecord* (MonedaDestino, bNuevoRegistro, [FechaCambio], [TipoCambio])
- *CurExchangeRecordRounded* (MonedaDestino, bNuevoRegistro, [FechaCambio], [TipoCambio])
- *CurGetDecimals* (MonedaOrigen)
- *CurGetExchange* (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)
- *CurGetExchangeCurrency* (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)
- *CurGetExchangeCurrencyRounded* (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)
- *CurGetExchangeRounded* (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)

***CurExchangeRecord* (MonedaDestino, bNuevoRegistro, [FechaCambio], [TipoCambio])**

Esta función cambia los valores de todos los elementos de tipo moneda de un

registro a la moneda deseada, teniendo en cuenta para ello el valor del argumento *MonedaDestino*, que indicará la moneda destino. Si se especifican los argumentos opcionales, *FechaCambio* y *TipoCambio*, estos tienen prioridad sobre los elementos auxiliares definidos para indicar la fecha y el tipo de cambio de la moneda.

Los argumentos correspondientes a este elemento son:

- **MonedaDestino**: identifica la moneda destino.
- **BNuevoRegistro**: es un indicador que especifica si se utiliza o no un nuevo registro.
- **[FechaCambio]**: argumento opcional que proporciona la fecha de cambio.
- **[TipoCambio]**: argumento opcional que proporciona el tipo de cambio.

EJEMPLO

```
curExchangeRecord (
    VALUE CURRENCY DestCurrency,
    VALUE NUMBER bNewRecord,
    [VALUE CURRENCY ExchangeDate,]
    [VALUE NUMBER ExchangeType]
)
```

CurExchangeRecordRounded (MonedaDestino, bNuevoRegistro, [FechaCambio], [TipoCambio])

Esta función es igual que *curExchangeRecord* salvo que redondea los valores que se han cambiado, al número de decimales asignado a la moneda destino.

La función tiene como argumentos la moneda destino y un indicador que especifica si se utiliza un nuevo registro, además de dos argumentos opcionales que proporcionan respectivamente la fecha de cambio y el tipo de cambio.

EJEMPLO

```
curExchangeRecordRounded (
    VALUE CURRENCY DestCurrency,
    VALUE NUMBER bNewRecord,
    [VALUE CURRENCY ExchangeDate,]
    [VALUE NUMBER ExchangeType]
)
```

Consulte CurExchangeRecord (MonedaDestino, bNuevoRegistro, [FechaCambio], [TipoCambio]) para más información.

CurGetDecimals (MonedaOrigen)

Esta función devuelve los decimales asociados a la moneda activa. La llamada a esta función debe realizarse desde el Meta4Object *EXCHANGE_RATES*.

EJEMPLO

```
curGetDecimals (
    VALUE CURRENCY Currency
)
```

CurGetExchange (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)

Esta función devuelve el valor resultante del cambio de una moneda a otra en una fecha determinada. La llamada a esta función debe realizarse desde el Meta4Object *EXCHANGE_RATES*, que contiene los tipos de cambio.

Los argumentos que recibe son la moneda origen, la moneda destino, la fecha de cambio, el tipo de cambio y la cantidad de moneda que se va a convertir.

EJEMPLO

```
curGetExchange (
    VALUE CURRENCY OriginCurrency,
    VALUE CURRENCY DestCurrency,
    VALUE DATE AND TIME ExchangeDate,
    VALUE VARIABLE STRING ExchangeType,
    VALUE CURRENCY Value
)
```

CurGetExchangeCurrency (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)

Esta función invoca al elemento *CurGetExchange* del Meta4Object *EXCHANGE_RATES* a través de una llamada interna de nivel 2, teniendo en cuenta una serie de valores que se indican como argumentos.

Dichos argumentos son la moneda origen, la moneda destino, la fecha de cambio, el tipo de cambio y la cantidad de moneda que se va a convertir.

EJEMPLO

```
curGetExchangeCurrency(  
    VALUE CURRENCY OriginCurrency,  
    VALUE CURRENCY DestCurrency,  
    VALUE DATE AND TIME ExchangeDate,  
    VALUE VARIABLE STRING Exchangetype,  
    VALUE NUMBER Value  
)
```

CurGetExchangeCurrencyRounded (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)

Esta función es igual que *curGetExchangeCurrency* salvo que redondea el valor que se ha cambiado al número de decimales asignado a la moneda de destino.

Los argumentos correspondientes a esta función son la moneda origen, la moneda destino, la fecha de cambio, el tipo de cambio, y la cantidad de moneda que se va a convertir.

EJEMPLO

```
curGetExchangeCurrencyRounded(  
    VALUE CURRENCY OriginCurrency,  
    VALUE CURRENCY DestCurrency,  
    VALUE DATE AND TIME ExchangeDate,  
    VALUE VARIABLE STRING ExchangeType,  
    VALUE CURRENCY Value  
)
```

Consulte [CurGetDecimals \(MonedaOrigen\)](#) para más información.

CurGetExchangeRounded (MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor)

Esta función es igual que *curGetExchange* salvo que redondea el valor que se ha cambiado al número de decimales indicado en la moneda de destino.

La llamada a esta función debe realizarse desde el Meta4Object *EXCHANGE_RATES*, que contiene el tipo de cambio que se debe usar.

Los argumentos correspondientes a esta función son la moneda origen, la moneda destino, la fecha de cambio, el tipo de cambio y la cantidad de moneda que se va a convertir.

EJEMPLO

```
curGetExchangeRounded(  
    VALUE CURRENCY OriginCurrency,  
    VALUE CURRENCY DestCurrency,  
    VALUE DATE AND TIME ExchangeDate,  
    VALUE VARIABLE STRING ExchangeType,  
    VALUE CURRENCY Value  
)
```

Consulte [CurGetExchange \(MonedaOrigen, MonedaDestino, FechaCambio, TipoCambio, Valor\)](#) para más información.

Ejemplos de funcionamiento del cambio de moneda

A continuación, se muestra una tabla que contiene una serie de valores ejemplo del Meta4Object *Exchange_Rates*, y posteriormente, se estudiarán distintos ejemplos de cambios de moneda.

Nodo Currency

ID_CURRENCY	DEC_NB	ID_ROOT_CURRENCY	DT_START ROOT_CURRENCY
EUR	5	-	-
DOL	5	-	-
PES	0	EUR	1/1/1999
MAR	2	EUR	1/1/1999
YEN	4	-	-
CUR_EX	5	-	-

Nodo Rates

ID_CURRENCY	ID_REFER_CUR	EX_TYPE	DT_START	DT_END	RATE
EUR	PES	1	1/1/1999	31/12/2001	168
EUR	MAR	1	1/1/1999	31/12/2001	2
EUR	DOL	1	1/1/1999	31/12/2001	1.176
EUR	YEN	1	1/1/1999	31/12/2001	3

ID_CURRENCY	ID_REFER_CUR	EX_TYPE	DT_START	DT_END	RATE
DOL	PES	1	1/1/1990	31/12/1998	145
DOL	MAR	1	1/1/1990	31/12/1998	1.8
DOL	EUR	1	1/1/1999	31/12/2001	0.85
DOL	YEN	1	1/1/1990	31/12/2001	2

ID_CURRENCY	ID_REFER_CUR	EX_TYPE	DT_START	DT_END	RATE
YEN	PES	1	1/1/1990	31/12/1998	70.5
YEN	MAR	1	1/1/1990	31/12/1998	0.9
YEN	EUR	1	1/1/1999	31/12/2001	0.425
YEN	DOL	1	1/1/1990	31/12/2001	0.5
YEN	CUR_EX	1	1/1/1990	31/12/2001	3

ID_CURRENCY	ID_REFER_CUR	EX_TYPE	DT_START	DT_END	RATE
PES	DOL	1	1/1/1990	31/12/1998	0.00689
PES	MAR	1	1/1/1990	31/12/1998	0.0124
PES	YEN	1	1/1/1999	31/12/1998	0.0138

ID_CURRENCY	ID_REFER_CUR	EX_TYPE	DT_START	DT_END	RATE
MAR	DOL	1	1/1/1990	31/12/1998	0.555
MAR	PES	1	1/1/1990	31/12/1998	80.5
MAR	YEN	1	1/1/1999	31/12/1998	1.111

ID_CURRENCY	ID_REFER_CUR	EX_TYPE	DT_START	DT_END	RATE
CUR_EX	YEN	1	1/1/1990	31/12/1998	0.333

Nodo Aux_Cur_Ex

ID_CURRENCY	ID_CURRENCY_D ES	ID_CURRE N_INTERM	DT_START	DT_END
CUR_EX	PES	YEN	1/1/1990	31/12/1999
CUR_EX	MAR	YEN	1/1/1990	31/12/1999
CUR_EX	DOL	YEN	1/1/1990	31/12/2001
CUR_EX	EUR	YEN	1/1/1999	31/12/2001

ID_CURRENCY	ID_CURRENCY_D ES	ID_CURRE N_INTERM	DT_START	DT_END
PES	CUR_EX	YEN	1/1/1990	31/12/1999

ID_CURRENCY	ID_CURRENCY_D ES	ID_CURRE N_INTERM	DT_START	DT_END
MAR	CUR_EX	YEN	1/1/1990	31/12/1999

ID_CURRENCY	ID_CURRENCY_D ES	ID_CURRE N_INTERM	DT_START	DT_END
EUR	CUR_EX	YEN	1/1/1999	31/12/2001

ID_CURRENCY	ID_CURRENCY_D ES	ID_CURRE N_INTERM	DT_START	DT_END
DOL	CUR_EX	YEN	1/1/1990	31/12/2001

A continuación, se muestran los resultados que se obtienen al realizar distintos

cambios de moneda entre las tablas.

Los resultados se indican en una tabla de una columna y dos filas. En la primera fila se muestra el código LN4 utilizado y en la segunda el resultado obtenido de su ejecución.

```
ValueExchanged = CurGetExchange(YEN, DOL, 1/1/2030, 1, 30)
```

```
Cambio(Src=YEN, Dst=DOL, Date=1/1/2030, Type=1) -->1 (Error)  
ValueExchanged = 30
```

```
ValueExchanged = CurGetExchange(DOL, MAR, 1/1/1996, 1, 30)
```

```
Cambio(Src=DOL, Dst=MAR, Date=1/1/1996, Type=1) --> 1.8 (Exito)  
ValueExchanged = 54
```

```
ValueExchanged = CurGetExchange(DOL, MAR, 10/10/1999, 1, 30)
```

```
Cambio(Src=DOL, Dst=MAR, Date=10/10/1999, Type=1)  
--> Cambio(DOL, EUR, 10/10/1999, 1) * Cambio(EUR, MAR, 10/10/1999, 1) -->  
--> 0.85 * 2 -->  
--> 1.7 (Exito)  
ValueExchanged = 51
```

```
ValueExchanged = CurGetExchange(PES, DOL, 10/10/1999, 1, 30)
```

```
Cambio(Src=PES, Dst=DOL, Date=10/10/1999, Type=1)  
--> CambioInverso (EUR, PES, 10/10/1999, 1) * Cambio(EUR, DOL, 10/10/1999,  
1) -->  
--> 1/168 * 1.176 -->  
--> 0.007 (Exito)  
ValueExchanged = 0.21
```

```
ValueExchanged = CurGetExchange(PES, MAR, 10/10/1999, 1, 30)
```

```
Cambio(Src=PES, Dst=MAR, Date=10/10/1999, Type=1)
--> CambioInverso (EUR, PES, 10/10/1999, 1) * Cambio(EUR, MAR, 10/10/
1999, 1) -->
--> 1/168 * 2 -->
--> 0.0119 (Exito)
ValueExchanged = 0.357
```

```
ValueExchanged = CurGetExchange(PES, MAR, 10/10/1995, 1, 30)
```

```
Cambio(Src=PES, Dst=MAR, Date=10/10/1995, Type=1) --> 0.0124 (Exito)
ValueExchanged = 0.372
```

Funciones de archivos

Las funciones de archivos son:

- ClearFiles ()
- CloseFile (Manejador)
- ConcatFile (Fichero, Valor, [AddEOL] Encoding)
- DestroyFile (NombreFichero)
- InitFile (Fichero, Valor, [AddEOL], Encoding)
- OpenFile (NombreFichero, &Manejador)
- ReadFile (Manejador, Tipo, &CadenaDevuelta)

ClearFiles ()

Esta función elimina todos los archivos temporales que aparezcan como abiertos en el sistema del gestor de archivos.

CloseFile (Manejador)

Cierra el archivo al que hace referencia un identificador (Manejador).

EJEMPLO

```
CloseFile (
```

```
VALUE NUMBER Handle  
)
```

ConcatFile (Fichero, Valor, [AddEOL] Encoding)

Esta función abre el archivo de texto plano cuyo nombre se indica en el primer argumento y le agrega los caracteres que se indican como segundo argumento.

El primer argumento debe recoger la ruta completa o relativa de un archivo existente: unidad de disco, lista de directorios, nombre y extensión. Toda esta información se pasará a la función como una cadena de caracteres.

El segundo argumento puede recoger una constante o una variable. Si se introduce un valor de tipo fecha o de tipo número, la función convertirá estos argumentos a una cadena de caracteres antes de modificar el archivo.

El tercer argumento es opcional, y permite agregar una línea después de escribir en el archivo. Esta variable puede tomar los valores *True* o *False* (por defecto es *True*).

El cuarto argumento puede aplicar cuatro valores diferentes:

- Sin valor: Por compatibilidad hacia atrás, se vuelca la cadena en formato ANSI.
- M4_ENCODING_NATIVE: Se vuelca la cadena en formato ANSI en instalaciones ANSI y en formato UTF8 en instalaciones Unicode.
- M4_ENCODING_ANSI: Se vuelca la cadena en formato ANSI.
- M4_ENCODING_UTF8: Se vuelca la cadena en formato UTF8.

EJEMPLO

La siguiente llamada a la función ConcatFile () abre el archivo C:\Meta4\log.txt, y concatena la frase "El salario base asciende a 98.000".

```
InitFile ( "c:\meta4\log.txt", "Resumen salarial")  
StrInt = "El salario base asciende a 98.000"  
ConcatFile ( "c:\meta4\log.txt", StrInt )
```

DestroyFile (NombreFichero)

Borra físicamente un archivo del disco, devolviendo un código de error en caso de que no pueda llevar a cabo la operación.

InitFile (Fichero, Valor, [AddEOL], Encoding)

Crea un archivo de texto plano, y le agrega los caracteres que se le pasan a la función como segundo argumento. Como primer argumento debe indicarse el nombre del archivo y el directorio donde se encuentra en el que cual se desean guardar los datos.

El segundo argumento puede ser una constante, una variable o un dato de tipo número o fecha. La función convertirá este valor en una cadena de caracteres antes de escribir cualquier valor en el archivo.

La función siempre debe recibir los dos argumentos, y no es posible crear un archivo sin escribir ningún valor en él.

El tercer argumento es opcional, y permite agregar una línea después de escribir en el archivo. Esta variable puede tomar los valores *True* o *False* (por defecto es *True*).

El cuarto argumento puede aplicar cuatro valores diferentes:

- Sin valor: Por compatibilidad hacia atrás, se vuelca la cadena en formato ANSI.
- M4_ENCODING_NATIVE: Se vuelca la cadena en formato ANSI en instalaciones ANSI y en formato UTF8 en instalaciones Unicode.
- M4_ENCODING_ANSI: Se vuelca la cadena en formato ANSI.
- M4_ENCODING_UTF8: Se vuelca la cadena en formato UTF8.

EJEMPLO

En este ejemplo la llamada a la función *InitFile ()* creará un nuevo archivo en el subdirectorio \Meta4 de la unidad de disco C, con el nombre LOG.TXT, y escribirá en él el valor que se ha asignado a la variable *StrArchivo*.

```
StrArchivo = "Archivo de Log. Contiene cambios"  
InitFile ( "C:\META4\LOG.TXT", StrArchivo )
```

OpenFile (NombreFichero, &Manejador)

Esta función abre el archivo indicado como primer argumento, pasándole el nombre de una variable por referencia, donde devolverá el identificador que se tendrá que usar posteriormente para leer del archivo y cerrarlo.

EJEMPLO

```
OpenFile (
    VALUE VARIABLE STRING FileName,
    REFERENCE NUMBER Handle
)
```

ReadFile (Manejador, Tipo, &CadenaDevuelta)

Lee del archivo indicado por el primer argumento, asignando una cadena de caracteres al tercer argumento, en función del tipo indicado por el segundo:

- Si es de tipo número, indica el número de caracteres que se van a leer del archivo. El máximo número de caracteres que se pueden pedir es 1 MB. En caso contrario la función devolverá error (-1).
- Si es de tipo carácter, lee hasta el carácter buscado.

El número máximo de bytes que se devuelven será siempre menor de 1 MB. En caso de ser mayor se truncará el resultado.

EJEMPLO

```
handle=0
str = ""
bContinue = M4_TRUE

OpenFile("examplefile.txt", handle)

While bContinue = M4_TRUE

    ret=ReadFile(handle, 4000, str)

    If ret < 4000 then
        bContinue = M4_FALSE
```

```
EndIf  
Wend
```

Funciones de lectura de argumentos

Las funciones de lectura de argumentos son:

- GetArgument (Arg_pos)
- GetArgumentNumber ()

GetArgument (Arg_pos)

Esta función devuelve el valor del argumento cuya posición se ha indicado como argumento. La posición debe ser un valor comprendido entre 1 y n.

EJEMPLO

```
GetArgument (   
    VALUE NUMBER ArgPosition  
 )
```

GetArgumentNumber ()

Esta función devuelve el número de argumentos que se han asignado a un elemento. Puede constar de una parte fija, de una parte variable, o de ambas.

Funciones de fechas de ejecución

Las funciones de fechas de ejecución son:

- AppEndDate ()
- AppRunDate ()
- GetEndDate ()
- GetRunDate ()
- GetRunningEndDate ()

-
- GetRunningRunDate ()
 - GetRunningStartDate ()
 - GetStartDate ()
 - SetEndDate (Fecha)
 - SetRunDate (Fecha)
 - SetStartDate (Fecha)

AppEndDate ()

Esta función devuelve el resultado de la ejecución de la fecha de fin. Su funcionamiento es similar al de la función *GetEndDate*.

AppRunDate ()

Esta función devuelve la fecha de ejecución. Su funcionamiento es similar al de la función *GetRunDate*.

AppStartDate ()

Esta función devuelve el resultado de la ejecución de la fecha de inicio. Su funcionamiento es similar al de la función *GetStartDate*.

GetEndDate ()

Esta función devuelve la fecha de fin del periodo que se está ejecutando. La función *GetAppEndDate* realiza las mismas funciones.

GetRunDate ()

Esta función devuelve la fecha de ejecución del periodo. La función *GetAppRunDate* realiza las mismas funciones.

GetRunningEndDate ()

Similar a la función *GetRunningStartDate()*, esta función devuelve la fecha de fin de ejecución del tramo activo que se está ejecutando en ese momento. La fecha no se puede cambiar, ya que viene definida por el propio proceso de ejecución de tramos.

Esta función es muy útil para la depuración, ya que devuelve la regla concreta que se está ejecutando en ese momento. Por ejemplo, en caso de que en la ejecución en tramos sea necesario que el elemento se ejecute 3 veces; si en el código del elemento está incluida la función `GetRunningEndDate`, se mostrará la fecha de fin de cada tramo que se ejecute.

GetRunningRunDate ()

La función `GetRunningRunDate` devuelve la fecha en la que se llevó a cabo la ejecución del tramo activo o la fecha de corte del tramo que se está ejecutando en ese momento, ya que generalmente se trabaja con dos tramos. Estas fechas no se pueden cambiar, ya que vienen definidas por el propio proceso de ejecución de tramos.



Otra utilidad de esta función consiste en conocer cuál de los dos tramos es el activo en un momento determinado.

Esta función es muy útil para la depuración, ya que devuelve la regla concreta que se está ejecutando en ese momento. Por ejemplo, en caso de que en la ejecución en tramos sea necesario que el elemento se ejecute 3 veces; si en el código del elemento está incluida la función `GetRunningRunDate`, se mostrará la fecha de ejecución o fecha de corte de cada tramo que se ejecute.

La fecha de corte se utiliza cuando se trabaja con métodos, no así las dos anteriores (fecha de inicio y fin) que se utilizan normalmente para conceptos, ya que se ejecutan desde distintos periodos.

GetRunningStartDate ()

Devuelve la fecha de inicio de ejecución del tramo activo que se está ejecutando en ese momento. La fecha no se puede cambiar, ya que viene definida por el propio proceso de ejecución de tramos.

Esta función es muy útil para fines de depuración, ya que devuelve la regla concreta que se está ejecutando en ese momento. Esto puede ser debido a una de las siguientes causas:

- Se hace referencia a un elemento que tiene tramos.
- Se está trabajando con tramos del sistema de asignación de valores.
- Se tiene más de una norma para la misma regla, etc.

Por ejemplo, en caso de que en la ejecución en tramos sea necesario que el elemento se ejecute 3 veces; si en el código del elemento está incluida la

función *GetRunningStartDate*, se mostrará la fecha de inicio de cada tramo que se ejecute.

Esto puede ser interesante para conocer la siguiente información:

- Para conocer por qué se está ejecutando alguna función más de lo esperado.
- El periodo en el que se está ejecutando.
- El origen de alguna dependencia, etc.

GetStartDate ()

Esta función devuelve la fecha de inicio del periodo que se está ejecutando. La función *GetAppStartDate* realiza las mismas funciones.

SetEndDate (Fecha)

Esta función asigna la fecha de fin de un periodo, mediante el argumento que recibe. La función *SetAppEndDate* realiza las mismas funciones. Si la fecha es nula, devuelve como resultado el valor nulo.

EJEMPLO

```
SetEndDate (
    VALUE DATE AND TIME Date
)
```

SetRunDate (Fecha)

Esta función asigna la fecha de ejecución o corte, mediante el argumento que recibe. Si la fecha es nula, esta función asigna el valor nulo.

EJEMPLO

```
SetRunDate (
    VALUE DATE AND TIME Date
)
```

SetStartDate (Fecha)

Esta función asigna la fecha de inicio de un periodo, mediante el argumento que recibe. La función *SetAppStartDate* realiza las mismas funciones. Si la fecha es nula, devuelve como resultado el valor nulo.

EJEMPLO

```
SetStartDate (
    VALUE DATE AND TIME Date)
```

Funciones para manejo de registros

Las funciones recogidas en este apartado permiten realizar diversas operaciones sobre registros pertenecientes a una estructura de nodos.

A continuación se indican las funciones pertenecientes a esta categoría:

- AddFilter (Código/operación, [Estático], [Nombre del argumento _i], [Valor del argumento _i]...)
- AddRegister ()
- AddSortFilter ()
- Begin ()
- BSearch (Índice, Valores...)
- Call (Argumentos..... Nodo, Elemento)
- ChannelAttribCall (Argumentos..... Instancia de Meta4Object, Nodo, Elemento, Atributo)
- ChannelAttribGetSliceValue(Instancia de Meta4Object, Nodo, Elemento, Tramo, Atributo)
- ChannelAttribGetValue (Instancia de Meta4Object, Nodo, Elemento, Atributo)
- ChannelCall (Argumentos..... Instancia de Meta4Object, Nodo, Elemento)
- CleanFilter ()
- CloneRegister
- CopyBranch (FileName , [Record] , [Node] , [M4Obj] ,)
- CopyRegister (Meta4Object, Nodo, [ÍndiceRegistro], [Filtro del elemento origen], [Filtro del elemento destino])

-
- Count ()
 - DeleteAllRegisters ()
 - DeleteRegister ()
 - DestroyAllRegisters ()
 - DestroyBlock ()
 - DestroyRegister ()
 - End ()
 - ExecuteBranch (Ámbito, Nombre del elemento)
 - ExecuteBranch (Ámbito, Nombre del elemento)
 - ExportXML (M4Obj, MapType, XML, Validate, DTD, Public, Handle, UseSchema, Namespace, ...)
 - GetChannelValue (Instancia de Meta4Object, Nodo, Elemento, Registro, [Tramo])
 - GetCurrent ()
 - GetFilterArgument (ID_Filtro, ID_Argumento)
 - GetValue (Nodo, Elemento, Registro, [Tramo])
 - Gonext ()
 - GoPrevious ()
 - InsertRegister ()
 - IsBlockUpdated(ai_bRecursive)
 - IsDeleted ()
 - IsEOF ()
 - IsNew ()
 - IsNewAndDeleted()
 - IsNodeUpdated(ai_bRecursive)
 - IsUpdated ()
 - IsUpdatedDB ()
 - LoadXML (M4Obj, MapType, XML, Validate, First, Last, Handle, UseSchema, [Consistency], ...)
 - MoveTo (ÍndiceRegistro)
 - MoveToEOF ()
 - PasteBranch (FileName , NewRecord , KeepMarks , DeleteFile , [Record] , [Node] , [M4Obj] , ...)

- RefreshFilter ()
- RemoveFilter ()
- SendBlockMark (Modo de envío)
- SendNodeMark (Modo de envío)
- SendRegisterMark (Modo de envío)
- SetChannelValue (Instancia de Meta4Object, Nodo, Elemento, Registro, [Tramo], Valor)
- SetFilterArgument (ID Filtro, ID Arg, Arg Val, ...)
- SetInserted ()
- SetPriority (Prioridad, Nodo, Elemento, Registro, [Tramo])
- SetValue (Nodo, Elemento, Registro, [Tramo], Valor)
- SetValuePriority (Prioridad, Nodo, Elemento, Registro, [Tramo], Valor)
- Sort ([índice])

AddFilter (Código/operación, [Estático], [Nombre del argumento_i], [Valor del argumento_i]...)

Esta función permite agregar un filtro al nodo cargado en memoria, con el primer argumento como filtro.

EJEMPLO

```
AddFilter (
    VALUE VARIANT Code_Operation, [VALUE NUMBER Static,] [VALUE
    VARIABLE STRING ArgId1,] [VALUE VARIANT ArgVal1]
)
```

Con esta función se pueden dar las siguientes posibilidades:

- Si el primer argumento es una cadena, se asume un código LN4 para determinar los registros visibles.
- Si el primer argumento es una macro especificada, se utilizará una operación de filtros.

Con esta macro se podrá mostrar una lista en la que se incluyan ciertos registros y se excluyan otros.

- Si el argumento Estático tiene como valor M4_TRUE, el filtro es estático; si por el contrario es falso (0), el filtro será dinámico. Por defecto es estático.

-
- Si el código del filtro tiene argumentos (código LN4), éstos se pueden indicar como argumentos, así como sus valores iniciales, justamente después de los argumentos Código y Estático.

Esta función devuelve el ID Filtro que debe utilizarse para cambiar o tomar valores como argumentos.

EJEMPLO

Con este código sólo se ven los registros que han sido borrados:

```
AddFilter (OP_DELETED)
```

Pero con éste se ven todos los registros excepto los que han sido borrados:

```
AddFilter (-OP_DELETED)
```

AddRegister ()

La función *AddRegister* () agrega un nuevo registro al nodo en el que se ejecuta, y se sitúa en él. No recibe argumentos.

También permite agregar un registro en un nodo distinto del actual, siempre y cuando se encuentre en el mismo Meta4Object. Para ello, se incluirá el nombre de la estructura de nodo a la que se quiere agregar el registro en el nombre de la función, como por ejemplo:

```
ID_EN.AddRegister ( )
```

A continuación, será necesario asignar valores a los elementos de tipo campo del nuevo registro, que hasta entonces mostrará los valores asignados por defecto.

Para asignar valor a estos elementos, es necesario escribir el nombre del elemento seguido del signo igual "=" y a continuación el valor que se desee asignar. También se puede agregar un registro llamando a un elemento o creando un registro vacío, entre otras posibilidades.

Si se ejecuta la función *AddRegister* () sobre el nodo activo, el puntero que recorre la estructura de nodos se sitúa en el nuevo registro, que será el último registro del nodo.



Si crea un nuevo registro con *AddRegister*() y le asigna valores, éste permanece en memoria hasta que decida almacenarlo en la base de datos. En este caso, los cambios no se harán efectivos hasta que no ejecute la función Persist_tree ().

Si se ejecuta la función *AddRegister ()* sobre otro nodo perteneciente al mismo Meta4Object, el puntero se sitúa en el nodo donde se agrega el registro.

Los valores de los elementos conectados se rellenan automáticamente al agregar el registro, pero sólo se dará este comportamiento si se han conectado los elementos de los nodos padre e hijo entre los que existe una relación de clave externa.

Esto se lleva a cabo en este u otro nodo, dependiendo de dónde se encuentre el puntero en el momento en el que se cree el nuevo registro.

AddSortFilter ()

Esta función permite agregar un filtro para ordenar todos los registros correspondientes a un nodo, de acuerdo con el criterio indicado por el usuario. Admite de dos formas de ejecución, con un índice definido o un número "n" de parejas de elementos y un modo de ordenación:

- *AddSortfilter* (Índice)
- *AddSortfilter* (Item_name, Order [asc/desc]...n pairs)

EJEMPLO

```
AddSortFilter (
    VALUE VARIANT Index_Item, [VALUE NUMBER Order]
)
```

Begin ()

Esta función permite situar el puntero en el primer registro del nodo activo. No recibe argumentos, y es equivalente a la función *MoveTo (0)*.

También permite situarse en el primer registro de otro nodo del mismo Meta4Object. Para ello, es suficiente con incluir en el nombre de la función el de la estructura de nodo con la que se quiere trabajar.

Por ejemplo, para eliminar el primer registro del nodo *NombreNodo* desde el nodo llamado NODO2, se ejecutarían las siguientes órdenes:

```
ID_EN.Begin ( )
ID_EN.DeleteRegister ( )
```

BSearch (Índice, Valores...)

Esta función realiza búsquedas binarias en el bloque activo (conjunto de registros), utilizando para ello uno o varios criterios de búsqueda.



Es necesario que el conjunto de registros esté ordenado. Si no es así, la función lo ordena.

Los argumentos correspondientes a esta función son:

- **Índice:** corresponde al identificador del índice. Ya en su descripción se indica el tipo de ordenación utilizado.
- **Valores:** contiene los elementos que forman parte del índice. Sólo se indican aquellos valores que se deben tener en cuenta para llevar a cabo la búsqueda.

Sólo se puede buscar por igualdad, excepto con el último elemento que puede ser igual, mayor, menor, etc., ya que si no fuera así, no podrían realizarse búsquedas binarias.

EJEMPLO

```
BSearch (  
    VALUE NUMBER Index, VALUE VARIANT Value  
)
```

Call (Argumentos,..., Nodo, Elemento)

La función *Call* () se utiliza para llamar a otros elementos de forma indirecta. Sólo se debe utilizar cuando el método que hace la llamada no conozca el nombre del método que se va a ejecutar en tiempo de diseño, es decir, cuando el nombre de la función se obtenga como resultado de la ejecución de otro método.



En caso de que se conozca el nombre de la función o método al que se quiere llamar en tiempo de diseño, las llamadas se harán de manera directa, escribiendo su nombre, proceso que resulta bastante rápido y más eficiente.

La función *Call* () recibe como argumentos el nombre del método o concepto

que se quiere ejecutar, con todos sus argumentos. Con la función **Call** () sólo se pueden hacer llamadas de tipo método o concepto.

La sintaxis de la función **Call** () depende de si se llama a un método del mismo nodo, o de otro nodo del mismo Meta4Object.

Si se utiliza **Call** () para llamar a un método del nodo desde el que se hace la llamada, en la lista de argumentos se escribirán entre paréntesis todos los argumentos que necesite la función a la que se llama, separados por comas.

```
Call ( argumento1, argumento2, ..., argumenton, nodo, elemento )
```

Si se utiliza **Call** () para llamar a un método de otro nodo del mismo Meta4Object, en la lista de argumentos se escribirá entre paréntesis el nombre del nodo, los argumentos del método (en el caso de que los tenga) y el nombre del método.

EJEMPLO

Para llamar al método *InsertRegister* () del nodo EMPLEADOS, se debe escribir:

```
Call ( "EMPLEADOS", "InsertRegister"
```

ChannelAttribCall (Argumentos,..., Instancia de Meta4Object, Nodo, Elemento, Atributo)

Esta función ejecuta el atributo del elemento indicado como argumento por indirección con el resto de los argumentos. Si se desea leer el valor de un atributo o ejecutar uno por indirección refiriéndose al mismo Meta4Object, es suficiente indicar comillas.

Si por otro lado se quiere utilizar otro Meta4Object distinto al actual, habría que indicar el nombre del Meta4Object que contiene el atributo al que se hace referencia.

El parámetro *Argumentos* corresponde a los argumentos complementarios que se indican como argumentos y que la función debe tener en cuenta a la hora de su ejecución. El número de argumentos es ilimitado, por tanto, se pueden tener tantos como sean necesarios, dependiendo del atributo.

La *Instancia de Meta4Object* identifica la instancia correspondiente al Meta4Object activo que se va a utilizar.

Por último, los argumentos nodo, elemento y atributo identifican respectivamente el nodo y elemento a los que se va a hacer referencia, y a cada uno de los atributos que tiene el elemento que se va a utilizar.

EJEMPLO

```
ChannelAttribCall (  
    VALUE VARIANT Arguments, VALUE VARIABLE STRING Meta4Object, VALUE  
    VARIABLE STRING Node, VALUE VARIABLE STRING Item, VALUE VARIABLE  
    STRING Attribute  
)
```

ChannelAttribGetSliceValue(Instancia de Meta4Object, Nodo, Elemento, Tramo, Atributo)

Existen funciones de indirección para leer atributos del sistema asociados a elementos, como la función ChannelAttribGetValue, o a tramos de elementos, como es el caso de esta función.

La diferencia entre las dos funciones es que ChannelAttribGetSliceValue tiene un argumento más que ChannelAttribGetValue, y es el tramo del que se quiere recuperar el atributo (para el caso de leer los atributos M4CL_ATT_SYS_START_DATE y M4CL_ATT_SYS_END_DATE, por ejemplo).

Los argumentos de la función son el identificador de la instancia correspondiente al Meta4Object que se va a utilizar, los nombres del nodo y elemento a los que se va a hacer referencia, el tramo del que se quiere recuperar el atributo, y el identificador del atributo que tiene el tramo que se va a utilizar.

El tramo puede ser tanto un número que represente el índice del tramo empezando en 0, o una fecha que indique el tramo.

ChannelAttribGetValue (Instancia de Meta4Object, Nodo, Elemento, Atributo)

Esta función de indirección permite leer atributos del sistema asociados a elementos, como por ejemplo el atributo *SysStartDate*.

Los argumentos de la función, que devuelve el atributo del elemento que se ha indicado por indirección, son el identificador de la instancia correspondiente al Meta4Object que se va a utilizar, los nombres del nodo y elemento a los que se va a hacer referencia, y el identificador de cada uno de los atributos que tiene el elemento que se va a utilizar.

Esta función asigna el primer argumento a la cadena vacía, indicando así una

instancia del Meta4Object actual.

Para cada atributo de los que puede tener un elemento, existe una serie de constantes definidas cuyo valor variará dependiendo de cada caso, pero que habrá que indicar siempre en el argumento **Atributo**.

Los únicos atributos que no se pueden ejecutar son aquéllos que tienen argumentos por referencia.

EJEMPLO

```
ChannelAttribGetValue (
    VALUE VARIABLE STRING Meta4Object, VALUE VARIABLE STRING Node,
    VALUE VARIABLE STRING Item, VALUE NUMBER Attribute
)
```

ChannelCall (Argumentos,..., Instancia de Meta4Object, Nodo, Elemento)

Esta función ejecuta un elemento en un nodo de una instancia de Meta4Object que no es el activo. La diferencia de esta función con *Call* estriba en el ámbito de actuación:

- **Call**: hace referencia a un elemento del mismo Meta4Object que el que se encuentra activo.
- **ChannelCall**: hace referencia a un elemento de un Meta4Object distinto del que se encuentra activo, aunque pudiera ser también el mismo Meta4Object.

Para tener acceso a un Meta4Object distinto al activo, se han de realizar llamadas de nivel 2.

Los argumentos correspondientes a esta función son los argumentos de la función que se pretende ejecutar, el nombre del Meta4Object al que se hace referencia, el nombre de la instancia a la que se hace referencia (correspondiente al Meta4Object indicado como argumento), y los nombres del nodo y elemento a los que se hace referencia.

EJEMPLO

```
ChannelCall (
    [VALUE VARIANT Argument1,] VALUE VARIABLE STRING Meta4Object, VALUE
    VARIABLE STRING Node, VALUE VARIABLE STRING Item
)
```

CleanFilter ()

Esta función permite eliminar todos aquellos filtros que se encuentren en memoria agregados a un nodo.

CloneRegister

Esta función crea un nuevo registro, realiza una copia exacta del registro activo, y se sitúa en el nuevo registro.

CopyBranch (FileName , [Record] , [Node] , [M4Obj] , ...)

Esta función permite guardar un registro y sus hijos en un archivo. Admite los siguientes argumentos:

- FileName (cadena): ruta del archivo en el que se han guardado los datos de la rama.
- Record (número): opcional. Registro en el que se copiarán los datos. Si no se especifica nada, se utilizará el registro actual.
- Node (cadena/número): opcional. Nodo en el que se copiarán los datos. Si no se especifica nada, se utilizará el nodo actual.
- M4Obj (cadena): opcional. Meta4Object en el que se copiarán los datos. Si no se especifica nada, se utilizará el Meta4Object actual.

CopyRegister (Meta4Object, Nodo, [ÍndiceRegistro], [Filtro del elemento origen], [Filtro del elemento destino])

Esta función permite copiar en el registro activo, tomando como origen el registro especificado por el Meta4Object, Nodo, ÍndiceRegistro. Se pueden dar las siguientes situaciones:

- Si el valor del Meta4Object está vacío, se utiliza el Meta4Object en el que esté situado.
- Si el *ÍndiceRegistro* es -1 o no se especifica (sin filtros), se usa como activo el nodo **origen**.
- Por defecto, si no se especifica un filtro, sólo se consideran los elementos del registro (todos ellos).

Los argumentos correspondientes a esta función son el nombre del Meta4Object que se va a utilizar, el nombre del nodo al que se hace referencia, el índice que permite localizar el registro (opcional), y los filtros de los nodos origen y destino.



Para más información sobre filtros, haga clic en [Especificación del filtro](#). Un filtro básico puede ser del tipo SCOPE_REGISTER + TYPE_PROPERTY.

EJEMPLO

```
CopyRegister (
    VALUE VARIABLE STRING Meta4Object, VALUE VARIABLE STRING Node,
    [VALUE NUMBER RegisterIndex,] [VALUE NUMBER SourceItemFilter,]
    [VALUE VARIABLE STRING DestItemFilter]
)
```

Count ()

Esta función no recibe argumentos y devuelve el número de registros que están incluidos en el bloque activo.

También se puede aplicar la función *Count ()* para saber cuántos registros incluye otro nodo del mismo Meta4Object. Será suficiente con incluir en el nombre de la función el nombre de la estructura de nodos cuyos registros se quiere contabilizar. Por ejemplo:

```
NoRegistros=ID_EN.Count ( )
```

En este caso, la variable **NoRegistros** recogerá el número de registros que incluye el nodo NombreNodo.

Puede utilizarse esta función junto a una de las sentencias que se exponen en el apartado [Estructuras de control](#).

EJEMPLO

Suponga que tiene una estructura de nodo llamada HORAS_EXTRAS. El nodo HORAS_REALIZADAS de esta estructura recoge las horas extras realizadas por los empleados en el último mes.

Para calcular el número total de horas extras realizadas por todos los empleados, se procedería de la siguiente forma:

```
TotalHorasExtras = 0
```

```
NoHoras=HORAS_EXTRAS.Count ( )  
For n=0 To NoHoras-1  
    TotalHorasExtras=TotalHorasExtras + HORAS_EXTRAS.HORAS_EXTRAS  
Next  
MessageBox ("Aviso", "Se han totalizado " + TotalHorasExtras + " horas  
extras")
```

DeleteAllRegisters ()

Esta función permite marcar como borrados todos los registros de un bloque. No recibe argumentos.

Debe recordar que no se harán efectivos los cambios en la base de datos física hasta que no llame a la función Persist_tree ().

DeleteRegister ()

Esta función no recibe argumentos y permite marcar como borrado el registro del nodo activo sobre el que está situado el puntero.

Para situarse en un registro específico del nodo activo, es necesario utilizar la función MoveTo (ÍndiceRegistro).

La función *DeleteRegister ()* también puede utilizarse para eliminar un registro de otro nodo correspondiente al mismo u otro Meta4Object. Es suficiente con incluir en el nombre de la función el nombre de la estructura de nodo sobre la que se quieren realizar los cambios.

Si quiere eliminar un registro de un nodo distinto al nodo activo, es necesario situarse en el registro que se quiere eliminar.

DestroyAllRegisters ()

Esta función permite eliminar todos los registros del bloque seleccionado. No recibe argumentos.

DestroyBlock ()

Esta función permite eliminar el bloque activo, así como los elementos de bloque que existan. No recibe argumentos.

DestroyRegister ()

Esta función permite eliminar el registro activo en el que está situado el puntero. No recibe argumentos, y después de que se haya ejecutado, el registro siguiente al que ha sido borrado pasa a ser el registro activo.

En caso de que el registro eliminado sea el último de un bloque, no quedará ningún registro como activo.

EJEMPLO

La siguiente tabla muestra el estado de una serie de registros antes de ejecutar la función DestroyRegister.

Registro 1	Registro inactivo	-
Registro 2	Registro activo	Registro a eliminar
Registro 3	Registro inactivo	-

La siguiente tabla muestra el estado de los registros anteriores después de ejecutar la función DestroyRegister.

Registro 1	Registro inactivo	-
Registro 3	Registro activo	-

End ()

Esta función permite mover el registro activo a la última posición del conjunto seleccionado. No recibe argumentos.

ExecuteBranch (Ámbito, Nombre del elemento)

Si se dispone de un Meta4Object con estructura jerárquica, es posible ejecutar toda una rama del Meta4Object, es decir, un nodo y todos los elementos que dependen de él.

Los argumentos correspondientes a esta función son:

- **Ámbito:** representa el ámbito sobre el que se va a ejecutar la función ExecuteBranch.
 - Si *Ámbito* es registro, la función ejecuta el *Nombre del elemento* en todos los nodos que dependan del nodo raíz de un registro.
 - Si *Ámbito* es bloque, la función ejecuta el *Nombre del elemento* en todos los nodos que dependan del nodo raíz de un bloque.
 - Si *Ámbito* es nodo, la función ejecuta el *Nombre del elemento* en todos los nodos.
- **Nombre del elemento:** indica el nombre del elemento al que se va a hacer referencia.

ExportXML (M4Obj, MapType, XML, Validate, DTD, Public, Handle, UseSchema, Namespace, ...)

Esta función permite invocar a la exportación de registros de un Meta4Object a un archivo XML desde LN4 y por nivel 2, sin tener que añadir los correspondientes métodos C++.

Los argumentos de esta función son los siguientes:

- **M4Obj:** instancia de Meta4Object a utilizar. Si se deja a cadena vacía se entiende que se debe trabajar sobre la propia instancia de Meta4Object.
- **MapType:** tipo de mapeo.
- **XML:** nombre del archivo XML
- **Validate:** permite indicar si se va a aplicar validación o no.
- **DTD:** nombre de archivo DTD.
- **Public:** identificador de declaración pública/sistema.
- **Handle:** handle del Meta4Object DMS_M4OBJECT_XML_MAP.
- **UseSchema:** indica si se utiliza XML Schema para la validación.
- **Namespace:** espacio de nombres XML Schema.

FindRegister (Elemento₁, Valor₁, Criterio₁, ..., Elemento_n, Valor_n, Criterio_n, [ÍndiceRegistro])

Esta función busca los registros de un bloque que cumplen una serie de criterios que se han indicado como argumentos.

Para cada condición o criterio se deben indicar tres argumentos, y es posible

escribir un número ilimitado de condiciones.



Si se asignan a la función *FindRegister* () dos o más condiciones, únicamente se recuperarán los registros que cumplan todas ellas; las condiciones se unirán mediante el operador lógico AND.

Los argumentos se deben indicar en el siguiente orden:

- Como primer argumento se ha de indicar el nombre del campo sobre el que se quiere realizar la búsqueda.
- Como segundo argumento se debe indicar el valor que se quiere buscar en el campo indicado como primer argumento.
- En el tercer argumento se debe indicar el criterio que se quiere utilizar para comparar los valores recogidos por el campo con el valor que se introduce como segundo argumento. Los criterios disponibles son los que se muestran en la siguiente tabla:

EQUAL
Localiza un registro en el que la cadena que se busca sea igual a la que se indica.
DISTINCT
Localiza un registro en el que la cadena que se busca sea distinta a la que se indica.
GREATER
Localiza un registro en el que la cadena que se busca sea más grande que la que se indica.
SMALLER
Localiza un registro en el que la cadena que se busca sea más pequeña que la que se indica.
GREATER_OR_EQUAL
Localiza un registro en el que la cadena que se busca sea más grande o igual que la que se indica.
SMALLER_OR_EQUAL
Localiza un registro en el que la cadena que se busca sea más pequeña o igual que la indicada.

EQUAL_OR_NULL
Localiza un registro en el que la cadena que se busca sea nula o igual que la indicada.
DISTINCT_OR_NULL
Localiza un registro en el que la cadena que se busca sea nula o distinta que la indicada.
GREATER_OR_NULL
Localiza un registro en el que la cadena que se busca sea nula o mayor que la indicada.
SMALLER_OR_NULL
Localiza un registro en el que la cadena que se busca sea nula o menor que la indicada.
GREATER_OR_EQUAL_OR_NULL
Localiza un registro en el que la cadena que se busca sea nula, mayor o igual que la indicada.
SMALLER_OR_EQUAL_OR_NULL
Localiza un registro en el que la cadena que se busca sea nula, menor o igual que la indicada.
CASE_EQUAL
Localiza un registro en el que la cadena que se busca sea igual a la indicada, distinguiendo entre mayúsculas y minúsculas.
CASE_DISTINCT
Localiza un registro en el que la cadena que se busca sea distinta a la indicada, distinguiendo entre mayúsculas y minúsculas.
CASE_GREATER
Localiza un registro en el que la cadena que se busca sea mayor que la indicada, distinguiendo entre mayúsculas y minúsculas.
CASE_SMALLER
Localiza un registro en el que la cadena que se busca sea menor que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_GREATER_OR_EQUAL

Localiza un registro en el que la cadena que se busca sea mayor o igual que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_SMALLER_OR_EQUAL

Localiza un registro en el que la cadena que se busca sea menor o igual que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_EQUAL_OR_NULL

Localiza un registro en el que la cadena que se busca sea nula o igual que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_DISTINCT_OR_NULL

Localiza un registro en el que la cadena que se busca sea nula o distinta que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_GREATER_OR_NULL

Localiza un registro en el que la cadena que se busca sea nula o mayor que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_SMALLER_OR_NULL

Localiza un registro en el que la cadena que se busca sea nula o menor que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_GREATER_OR_EQUAL_OR_NULL

Localiza un registro en el que la cadena que se busca sea nula, mayor o igual que la indicada, distinguiendo entre mayúsculas y minúsculas.

CASE_SMALLER_OR_EQUAL_OR_NULL

Localiza un registro en el que la cadena que se busca sea nula, menor o igual que la indicada, distinguiendo entre mayúsculas y minúsculas.

REGULAR_EXPRESSION

Permite establecer una serie de requisitos que debe cumplir el valor del elemento.

Por ejemplo, que el valor de éste empiece por una determinada cadena o un determinado carácter, etc.

REGULAR_EXPRESSION_OR_NULL

Permite establecer una serie de requisitos que debe cumplir el valor del elemento, a menos que se trate de un valor nulo.

Por ejemplo, que el valor del elemento empiece por una determinada cadena o un determinado carácter, etc.

CASE_REGULAR_EXPRESSION

Permite establecer una serie de requisitos que debe cumplir el valor del elemento, distinguiendo entre mayúsculas y minúsculas.

Por ejemplo, que el valor del elemento empiece por una determinada cadena o un determinado carácter, etc.

CASE_REGULAR_EXPRESSION_OR_NULL

Permite establecer una serie de requisitos que debe cumplir el valor del elemento, a menos que se trate de un valor nulo, distinguiendo entre mayúsculas y minúsculas.

Por ejemplo, que el valor del elemento empiece por una determinada cadena o un determinado carácter, etc.

EJEMPLO

En este ejemplo se utiliza el criterio EQUAL. Este criterio permite buscar un registro cuyo nombre sea igual a Laura.

Este criterio puede ser sustituido por cualquier otro de los indicados en la siguiente tabla.

```
FindRegister ("NOMBRE", "Laura", EQUAL)
```

La función *FindRegister* () puede recibir un argumento opcional. Este argumento se debe escribir en última posición, detrás de todas las condiciones, e indica el número de registro a partir del cual se quiere comenzar la búsqueda, entre 0 y n-1.

Si la estructura de nodo incluye al menos un registro que cumpla las condiciones, la función devuelve el índice del primer registro de la estructura de nodo que cumpla las condiciones indicadas.

Si la estructura de nodo no incluye ningún registro que cumpla las condiciones, la función devuelve el valor -1.

EJEMPLO

En el nodo ABSENTISMOS se quieren eliminar los registros correspondientes al empleado 01, con una fecha comprendida entre el 01-07-1999 y el 22-07-1999.

El nodo ABSENTISMOS recoge el código del empleado en el elemento ID_EMPLEADO, y las fechas de inicio y de fin de los periodos de absentismo en los elementos FEC_INI y FEC_FIN.

Para eliminar estos registros, es necesario realizar una búsqueda para saber cuál es su índice. Una vez que se conoce el índice, se puede ejecutar la función DeleteRegister (). A continuación se muestra el código LN4 correspondiente a esta función a modo de ejemplo.

```
Register=FindRegister ( "ID_EMPLEADO", "01", EQUAL, "FEC_INI", {1999-07-01 }, GREATER_OR_EQUAL, "FEC_FIN", { 1999-07-22 }, SMALLER_OR_EQUAL)

If Register = -1 Then
    MessageBox ("Meta4", "No se ha encontrado ningún registro que cumpla las condiciones")
Else
    BuscoaPartirDe=Register
    MoveTo (Register)
    DeleteRegister ( )
    Do
        Register=FindRegister ( "ID_EMPLEADO", "01", EQUAL, "FEC_INI", {1999-07-01 }, GREATER_OR_EQUAL, "FEC_FIN", { 1999-07-22 }, SMALLER_OR_EQUAL, BuscoaPartirDe )
        MoveTo (Register)
        DeleteRegister ( )
        BuscoaPartirDe=Register + 1
    Until IsEOF ( )=M4_TRUE OR Register = -1
EndIf
```

GetChannelValue (Instancia de Meta4Object, Nodo, Elemento, Registro, [Tramo])

Esta función lee el valor de un elemento dado en una instancia. La diferencia con la función GetValue estriba en el ámbito de actuación de la misma:

- **GetValue:** hace referencia a un elemento del mismo Meta4Object que el activo.
- **GetChannelValue:** hace referencia a un elemento de un Meta4Object distinto al activo, aunque pudiera ser también el mismo.

Para tener acceso a un Meta4Object distinto al activo, se han de realizar llamadas de nivel 2.

Los argumentos correspondientes a esta función son una instancia creada en un Meta4Object, los nombres de nodo, elemento y registro a los que se hace referencia, el tramo que se va a considerar.

El argumento *Tramo* es un campo opcional y corresponde al tramo utilizado que va a considerar la función GetChannelValue.

Al utilizar esta función con tramos, existen las siguientes posibilidades:

- Si esta función se ejecuta con tramos y no se especifica el argumento correspondiente, se usa el tramo activo.
- Si el argumento Registro tiene el valor -1, la función lee el registro activo.
- Si se ejecuta con tramos y no se especifica el argumento correspondiente, se utiliza el tramo activo.

EJEMPLO

```
GetChannelValue (  
    VALUE VARIABLE STRING Meta4Object, VALUE VARIABLE STRING Node,  
    VALUE VARIABLE STRING Item, VALUE NUMBER Record, [VALUE NUMBER  
    Slice]  
)
```

GetCurrent ()

Esta función devuelve el índice correspondiente al registro del nodo activo sobre el que está situado el puntero.

Los registros de un nodo se empiezan a contar a partir del número 0. Así, si un

nodo incluye 10 registros, éstos se numerarán del 0 al 9.

EJEMPLO

```
Begin ( )          ` Sitúe el puntero en el primer registro
Registro = GetCurrent ( )  ` Asigna a la variable Registro el valor 0
MoveTo(9)          ` Sitúa el puntero en el décimo registro
Registro= GetCurrent ( )  ` Asigna a la variable Registro el valor
9.
```

GetFilterArgument (ID_Filtro, ID_Argumento)

Esta función muestra el valor actual del argumento indicado. Dicho valor no se puede cambiar, puesto que la operación se realiza con la función SetFilterArgument.

Los argumentos correspondientes son el identificador del filtro utilizado, y el identificador del argumento utilizado cuyo valor se va a mostrar con el uso de esta función. Los argumentos se numeran a partir del 0, y el valor del argumento queda determinado por la posición del registro o su identificador.

EJEMPLO

```
GetFilterArgument (
    VALUE VARIABLE STRING IdFilter, VALUE VARIANT IdArg
)
```

GetValue (Nodo, Elemento, Registro, [Tramo])

Esta función recoge el valor de un campo o de un tramo de un campo, así como del nodo y del registro que aparecen como argumentos. Dichos argumentos son variables o elementos que contienen el nombre del nodo, el nombre del elemento y el índice del registro.

El argumento Tramo es opcional, e identifica el tramo cuyo valor se quiere recuperar. Puede tomar un valor numérico o de tipo fecha. Si se ejecuta con tramos y no se especifica el argumento correspondiente, se usa el tramo activo.

EJEMPLO

```
GetValue (
    VALUE VARIABLE STRING Node, VALUE VARIABLE STRING Item, VALUE
    NUMBER Record, VALUE NUMBER Slice
)
```

Únicamente se debe utilizar la función de indirección *GetValue ()* cuando se desee obtener el valor de un elemento o de un tramo cuyo nombre o nodo se reciba a través de una variable que se desconoce a la hora de escribir el código.

Si se conoce el nombre del elemento, siempre será más fácil obtener su valor directamente mediante una asignación del siguiente tipo:

```
Variable = ID_EN.elemento
```

Para obtener el valor de un tramo, se debe indicar (aun siendo opcional) el tramo en el argumento que recibe la función *GetValue ()* en cuarto lugar.

Gonext ()

Esta función desplaza el puntero que recorre el nodo activo una posición hacia adelante, a partir del registro que se encuentre activo en ese momento.

EJEMPLO

```
Begin ( )          'Sitúa el puntero en el primer registro.
Registro=GetCurrent ( ) 'Asigna a la variable Registro el valor 0.
Gonext ( )         'Mueve el puntero una posición hacia adelante.
Registro=GetCurrent ( ) 'Asigna a la variable Registro el valor 1.
```

GoPrevious ()

Esta función permite mover el puntero que recorre el nodo activo una posición hacia atrás a partir del registro que se encuentra activo en ese momento.

EJEMPLO

```
End ( )          'Sitúa el puntero en el último registro del nodo actual
```

```
Goprevious ( )      `Mueve el puntero hasta el penúltimo registro
```

InsertRegister ()

Esta función inserta un nuevo registro en el nodo actual. A diferencia de *AddRegister ()*, que agrega un registro en blanco al final del nodo, *InsertRegister ()* crea el registro en la posición anterior a la que se encuentra situado el puntero. No recibe argumentos.

Para agregar un nuevo registro a un nodo distinto del actual, es necesario incluir el nombre de la estructura de nodo con la que se quiere trabajar en el nombre de la función. Por ejemplo:

```
ID_EN.InsertRegister ( )
```

Tras crear el registro vacío, se pueden asignar valores a los distintos elementos de tipo campo del nuevo registro. Para asignar valor a un elemento, es suficiente con escribir el nombre del elemento seguido del signo igual y del valor que se le quiere asignar. Si el nuevo registro se ha creado en un nodo distinto del actual, el nombre de los campos debe ir precedido por el nombre del nodo y por un punto. Por ejemplo:

```
ID_EN.elemento = "Valor"
```

Si se ejecuta la función *InsertRegister ()* sobre el nodo activo, el puntero que recorre la estructura de nodos se situará en el nuevo registro.



Recuerde que el nuevo registro permanece en memoria, y que los cambios no se harán efectivos en la base de datos hasta que no llame a la función *Persist_tree ()*.

Si se ejecuta la función *InsertRegister ()* sobre otro nodo del mismo Meta4Object, el puntero permanecerá en la misma posición en la que se encontraba antes de llamar a la función.

EJEMPLO

El siguiente código duplica todos los registros de la estructura de nodos en la que está situado el puntero.

El campo que constituye la clave primaria recoge un valor numérico. El código LN4 rellenará automáticamente el valor de este campo con el valor obtenido al elevar al cuadrado la clave primaria del registro que duplica.

Si ya existe un registro con este valor calculado en su clave primaria, se elevará al cuadrado, y así sucesivamente hasta encontrar un valor único.

```

Exponente = 2
Begin ( )
Do
For n= 0 to ( NumeroRegistro - 1 )
    If Id_Empleado = 1 Then
        IdEmp = 3
    Else
        IdEmp = Power ( Id_Empleado, Exponente )
    EndIf
NomEmp = N_Empleado
EdadEmp = Edad_Empleado
HayOtro = FindRegister ( "Id_Empleado", IdEmp, EQUAL )
While HayOtro <> -1
    Exponente = Exponente + 1
    IdEmp = Power (IdEmp, Exponente )
    HayOtro = FindRegister ( "Id_Empleado", IdEmp, EQUAL )
WEnd
Exponente = 2
InsertRegister ( )
    Id_Empleado = IdEmp
    N_Empleado = NomEmp
    Edad_Empleado = EdadEmp
Gonext ( )
Until IsEOF ( )=M4_TRUE

```

IsBlockUpdated(ai_bRecursive)

Esta función comprueba si alguno de los elementos de ámbito bloque del bloque actual ha sido modificado. Si alguno ha sido modificado retorna M4_TRUE y si no hay modificaciones retorna M4_FALSE. Además, si el argumento de la función es M4_TRUE chequea si alguno de los elementos de todos los registros del bloque actual ha sido modificado.

Los argumentos correspondientes a esta función son los siguientes:

- **ai_bRecursive** : número booleano que indica si se deben comprobar los cambios en los elementos de ámbito registro del bloque

IsDeleted ()

Esta función devuelve el valor M4_TRUE si el registro activo del bloque está marcado como borrado.

IsEOF ()

La función *IsEOF ()* permite saber si el puntero que recorre la estructura de nodo ha alcanzado la variable lógica EOF: En este caso hay dos posibilidades:

- Si el puntero está situado en un registro del nodo activo, *IsEOF ()* devuelve el valor M4_FALSO.
- Si el puntero está situado en la variable lógica EOF, fuera de cualquier registro, la función devuelve el valor M4_TRUE.

La función *IsEOF ()* se utiliza para recorrer los registros de un nodo, junto a las sentencias de control *While...Wend*, y *Do...Until*.

Además, se puede utilizar para recorrer los registros de nodos distintos al nodo activo. Para ello, basta con cualificar el nombre de la función con el nombre de la estructura de nodo con la que se quiere trabajar.

EJEMPLO

```
Total=0
Begin( )
Do
    Total=Total+SALARIO_BASE
    Gonext( )
Until IsEOF( )=M4_TRUE
Total=0
Begin( )
While NOT IsEOF( )=M4_TRUE
    Total=Total+SALARIO_BASE
    Gonext( )
Wend
```

IsNew ()

Esta función devuelve el valor M4_TRUE si el registro activo del bloque está

marcado como nuevo.

IsNewAndDeleted()

Esta función devuelve el valor M4_TRUE si el registro activo del bloque está marcado como nuevo y como borrado.

La función no tiene argumentos y se aplica al registro sobre el que se esté trabajando

IsNodeUpdated(ai_bRecursive)

Esta función comprueba si alguno de los elementos de ámbito nodo del nodo actual ha sido modificado. Si alguno ha sido modificado retorna M4_TRUE y si no hay modificaciones retorna M4_FALSE. Si el argumento de la función es M4_TRUE además chequea si alguno de los elementos de ámbito bloque de todos los bloques del nodo actual ha sido modificado y si alguno de los elementos de todos los registros de todos los bloques del nodo actual ha sido modificado

Los argumentos correspondientes a esta función son los siguientes:

- **ai_bRecursive:** número booleano que indica si se deben comprobar los cambios en los elementos de ámbito bloque y registro del nodo.

IsUpdated ()

Esta función devuelve el valor M4_TRUE si el registro activo del bloque está marcado como actualizado.

IsUpdatedDB ()

Devuelve el valor M4_TRUE si el registro está marcado como que tiene información actualizada que se desea guardar en base de datos.

LoadXML (M4Obj, MapType, XML, Validate, First, Last, Handle, UseSchema, [Consistency], ...)

Esta función permite invocar a la importación de registros de un archivo XML a un Meta4Object desde LN4 y por nivel 2, sin tener que añadir los correspondientes métodos C++.

Los argumentos de esta función son los siguientes:

- **M4Obj**: instancia de Meta4Object a utilizar. Si se deja a cadena vacía se entiende que se debe trabajar sobre la propia instancia de Meta4Object.
- **MapType**: tipo de mapeo.
- **XML**: nombre del archivo XML a importar.
- **Validate**: permite indicar si se va a utilizar validación o no.
- **First**: primer registro del proceso XML.
- **Last**: último registro del proceso XML.
- **Handle**: handle del Meta4Object DMS_M4OBJECT_XML_MAP.
- **UseSchema**: indica si se utiliza XML Schema para la validación.
- **[Consistency]**: argumento opcional para indicar si se va a llevar a cabo una comprobación de consistencia de los elementos.

MoveTo (ÍndiceRegistro)

Esta función mueve el puntero del nodo activo al registro situado en la posición *n* a la que se desea tener acceso. La función recibe como argumento el índice del registro al que se debe mover el puntero.

Los registros de un nodo se numeran comenzando por el número 0. De esta forma, si se quiere mover al primer registro del nodo activo, se ejecuta la orden *MoveTo(0)*; para moverse al registro que ocupa la segunda posición, se ejecuta la orden *MoveTo(1)*, y así sucesivamente.

La función *MoveTo(n)* permite situar el puntero en un registro específico con diferentes propósitos, entre otros:

- Marcar para borrar el registro sobre el que está situado el puntero mediante la función *DeleteRegister ()*.
Este registro se borrará físicamente cuando se realice la próxima transacción.
- Insertar un nuevo registro en blanco a continuación del registro sobre el que está situado el puntero. Se utilizará para ello la función *InsertRegister ()*.

-
- Ver el valor de un elemento de tipo campo o ejecutar un elemento de tipo método o concepto definido en el registro sobre el que se sitúa el puntero.

Esta funcionalidad sólo se empleará si se quiere utilizar un elemento del nodo activo o de cualquier otro nodo.

EJEMPLO

Este ejemplo recorre todos los registros correspondientes al nodo activo, lee los valores del elemento NOMBRE y muestra este valor en una ventana de diálogo.

```
n=0
MoveTo (n)
Do
    MessageBox ("Aviso", NOMBRE)
    n=n+1
    MoveTo ( n )
Until IsEOF ( )=M4_TRUE
```

LN4 incluye otras dos funciones para moverse entre los registros de un nodo de forma secuencial: Gonext () y IsDeleted ().

MoveToEOF ()

Esta función permite mover el puntero del nodo activo a la posición EOF. No debe confundirse la variable lógica EOF con el registro que ocupa la última posición en el nodo. La función no recibe argumentos.

Es posible mover el puntero a la posición EOF de otro nodo del mismo Meta4Object. Esta acción se puede llevar a cabo para cualquier elemento del sistema, tan sólo con incluir en el nombre de la función el nombre de la estructura de nodo con la que se quiere trabajar.

EJEMPLO

Si está situado en el nodo NODO1 y quiere situarse en la posición EOF del nodo NODO2, debería ejecutar la función del siguiente modo.

```
TINodo2.MoveToEOF ( )
```

PasteBranch (FileName , NewRecord , KeepMarks , DeleteFile , [Record] , [Node] , [M4Obj] , ...)

Esta función permite restaurar un registro y sus hijos de un archivo guardado previamente utilizando la función CopyBranch (FileName , [Record] , [Node] , [M4Obj] , ...).

Admite los siguientes argumentos:

- **FileName** (cadena): ruta del archivo en el que se han guardado los datos de la rama.
- **NewRecord** (número): indica si los datos se van a copiar en un nuevo registro (M4_TRUE) o en el registro actual (M4_FALSE).
- **KeepMarks** (número): indica si se van a mantener las marcas de los registros (M4_TRUE) o estos se van a establecer como nuevos (M4_FALSE).
- **DeleteFile** (número): indica si se debe borrar el archivo en el que se han guardado los datos (M4_TRUE) o no (M4_FALSE).
- **Record** (número): opcional. Registro en el que se van a copiar los datos si el nuevo registro es M4_FALSE. Si no se especifica nada, se utilizará el registro actual.
- **Node** (cadena/número): opcional. Nodo en el que se copiarán los datos. Si no se especifica nada, se utilizará el nodo actual.
- **M4Obj** (cadena): opcional. Meta4Object en el que se copiarán los datos. Si no se especifica nada, se utilizará el Meta4Object actual.

RefreshFilter ()

Esta función refresca un filtro estático que se encuentra en memoria agregado a un nodo. Se debe tener en cuenta que si se agregan registros y no se ejecuta la función RefreshFilter, los datos no se actualizarán hasta que no se ejecute esta función.

RemoveFilter ()

Esta función permite eliminar el último filtro cargado en memoria que se haya agregado a un nodo. El filtro es la capacidad de los Meta4Object que permite seleccionar en un nodo aquellos registros que cumplen una determinada condición.

SendBlockMark (Modo de envío)

Esta función permite ver las constantes definidas para *Modo de envío*. Permite marcar el bloque activo para una serialización avanzada. El valor por defecto es No_BRANCH.

SendNodeMark (Modo de envío)

Esta función permite ver las constantes definidas para *Modo de envío*. Permite marcar el nodo activo para una serialización avanzada. El valor por defecto es No_BRANCH.

SendRegisterMark (Modo de envío)

Esta función permite ver las constantes definidas para *Modo de envío*. Permite marcar el registro activo para una serialización avanzada. El valor por defecto es No_BRANCH.

SetChannelValue (Instancia de Meta4Object, Nodo, Elemento, Registro, [Tramo], Valor)

La diferencia de esta función con respecto a SetValue estriba en el ámbito de su actuación:

- **SetValue**: hace referencia a un elemento del mismo Meta4Object que el que se encuentra activo.
- **SetChannelValue**: hace referencia a un elemento de un Meta4Object distinto al que se encuentra activo, aunque pudiera ser también el mismo Meta4Object. Para tener acceso a un Meta4Object distinto al activo, se han de realizar llamadas de nivel 2.

Esta función recibe como argumentos una instancia de un Meta4Object, los nombres del nodo, elemento y registro a los que se hace referencia, el tramo que va a considerar la función, y el valor que se va a asignar a un elemento dado en una instancia

El argumento *Tramo* es un campo opcional. Si la función se ejecuta con tramos y no se especifica el argumento correspondiente, se usa el tramo activo.

EJEMPLO

```
SetChannelValue (
```

```

        VALUE VARIABLE STRING Meta4Object, VALUE VARIABLE STRING Node,
        VALUE VARIABLE STRING Item, VALUE NUMBER Record, [VALUE NUMBER
        Slice,] VALUE VARIANT Value
    )

```

SetFilterArgument (ID_Filtro, ID_Arg, Arg_Val, ...)

Esta función permite cambiar el valor actual del argumento indicado. Para ello recibe como argumentos el identificador del filtro utilizado, el identificador del argumento utilizado cuyo valor se mostrará al utilizar esta función, y el valor del argumento que se va a asignar.

Los argumentos se numeran a partir del 0 y se pueden utilizar uno o varios; y el argumento puede ser el especificado por un identificador o posición.

EJEMPLO

```

SetFilterArgument (
    VALUE VARIABLE STRING IdFilter, VALUE VARIANT IdArg0, VALUE VARIANT
    ArgValue0
)

```

SetInserted ()

Esta función marca un registro como nuevo. Se puede utilizar cuando se necesiten copiar registros de una tabla a otra.

Los registros de la tabla origen se marcan como insertados, y no se copiarán en la tabla destino hasta el momento en que se lleve a cabo la función *Persist* ().

Sólo es posible asignar una marca cuando se va a insertar un nuevo registro. El ámbito de aplicación de esta función es sólo para registros que no dispongan de marcas.

SetPriority (Prioridad, Nodo, Elemento, Registro, [Tramo])

Esta función cambia directamente el valor correspondiente a la prioridad de un elemento. A diferencia de la función SetValuePriority, ésta no modifica el valor del elemento.

Sus argumentos son la prioridad que se va a asignar al elemento, y los nombres de nodo, elemento, registro y tramo a los que se hace referencia. El argumento de Tramo es opcional. Si se ejecuta la función con tramos y no se especifica su argumento correspondiente, se usa el tramo activo.

EJEMPLO

```
SetPriority (
    VALUE NUMBER Priority, VALUE VARIABLE STRING Node, VALUE VARIABLE
    STRING Item, VALUE NUMBER Record, [VALUE NUMBER Slice]
)
```

SetValue (Nodo, Elemento, Registro, [Tramo], Valor)

Esta función permite asignar un valor a un nodo, un elemento o un registro, o a uno de sus tramos.

La función *SetValue* () recibirá como argumentos el nombre del nodo, el nombre del elemento, el nombre del registro y el valor que se quiere asignar al elemento. El argumento *[Tramo]* corresponde al nombre del tramo, y es un argumento opcional, ya que si no se quiere asignar un valor a un tramo, se omite. Este argumento puede tomar un valor numérico o de tipo fecha.

Únicamente se utilizará la función *SetValue* () cuando se desee asignar un valor a un elemento de un nodo cuyos nombres se reciben como variables o elementos.

En este último caso, conocemos el elemento en tiempo de ejecución, pero no en tiempo de diseño. Es decir, se debe utilizar *SetValue* () cuando se tiene que hacer una asignación de valor a un elemento cuyo nombre, nodo o tramo se desconoce.

Si se conoce el nombre del elemento y el nodo en el que se encuentra en tiempo de diseño, no se debe utilizar la función *SetValue* (). En su lugar, se le debe asignar un valor directamente, mediante una asignación del siguiente tipo:

```
Nodo.elemento = Valor
```

EJEMPLO

El siguiente código realiza las siguientes acciones:

- Se indica a qué grupo de usuarios pertenece el usuario SUPERV.

- A continuación, se busca en la estructura de nodo **PERMISOS_CONCEPTO_GRUPO** las tablas de valor y los conceptos sobre los que tiene permisos el grupo de usuarios al que pertenece el usuario SUPERV.

Sean, por ejemplo, los siguientes registros:

Tabla	Concepto	Grupo
VAL_CATEGORIA	600	ADMIN
VAL_CONVENIO	560	ADMIN

El primer registro indica que los usuarios incluidos en el grupo ADMIN pueden modificar el valor del concepto 600 en la tabla de valor VAL_CATEGORIA.

El segundo registro indica que los usuarios incluidos en el grupo ADMIN pueden modificar el valor del concepto 560 en la tabla de valor VAL_CONVENIO.

- Tras consultar esta estructura de nodo, el método busca en la estructura de nodo **AcumuladoLargo** el valor que se ha asignado en la última paga calculada a los conceptos sobre los que tenga permiso el usuario SUPERV en la tabla PERMISOS_VALORES_CONCEPTO.

Se supone que los registros del nodo **AcumuladoLargo** están ordenados de menor a mayor según los valores del campo FECHA_PAGO.

- Por último, en las tablas de valor sobre las que tenga permiso el usuario, se guardan los valores recuperados del acumulado.

En este ejemplo se utilizarán las siguientes estructuras de nodo:

Grupos_Usuarios

Grupo	Usuario
ADMIN	M25
ADMIN	M25
SYSTEM	M25

Permisos_Valores_Concepto

Tabla_Vvalor	Concepto	Grupo
VAL_CATEGORIA	600	ADMIN
VAL_CONVENIO	560	ADMIN
VAL_CONVENIO	670	SYSTEM
VAL_CONVENIO	900	SYSTEM

Acumulado_Largo

ID_EMPLEADO	FEC_IMPU	FEC_PAGO	600	560		1002
01	25-01-97	25-01-97	13000	2500		90000
02	25-01-97	25-01-97	15000	6700		98000
....						
01	25-02-98	25-02-98	12000	3000		100000
02	25-02-98	25-02-98	34000	5600		120000

El método se ejecuta desde el nodo asociado a la estructura de nodo
Permisos_Valores_Concepto.

```
strGrupoUsuario = "ADMIN"
intRegistro = FindRegister ( "GRUPO", strGrupoUsuario, EQUAL)
If intRegistro = -1 Then
    MessageBox ( "Meta4", "No existe ningún permiso para este grupo de
    usuario")
Else
    intBuscoaPartirDe=intRegistro
    MoveTo (intRegistro)
    strTablaValor = TABLA_VALOR
    strConcepto = CONCEPTO
    Acumulado.End ( )
    dtUltimaPaga=Acumulado.FEC_PAGO
    intRegAcum=Acumulado.FindRegister ("ID_EMPLEADO", "01", EQUAL,
    "FEC_IMPU", dtUltimaPaga, EQUAL, "FEC_PAGO", dtUltimaPaga, EQUAL)
    If intRegAcum = -1 Then
        MessageBox("Meta4", "No se ha calculado este empleado en la
        última paga")
    Else
        Acumulado.MoveTo (intRegAcum)
        GetValue ("ACUMULADO", strConcepto, -1, intValorConcepto)
        Call ( strTablaValor, "InsertRegister" )
        SetValue ( strTablaValor, "Concepto", strConcepto )
        SetValue ( strTablaValor, "Valor", intValorConcepto )
        SetValue ( strTablaValor, "Fec_ini", Today ( ) )
        SetValue ( strTablaValor, "Fec_fin", NullValue ( ) )
    EndIf
Do
    intRegistro = FindRegister ( "GRUPO", strGrupoUsuario, EQUAL,
    intBuscoaPartirDe)
```

```

    intBuscoaPartirDe=intRegistro
    MoveTo (intRegistro)
    strTablaValor = TABLA_VALOR
    strConcepto = CONCEPTO
    Acumulado.End ( )
    dtUltimaPaga=Acumulado.FEC_IMPU
intRegAcum=Acumulado.FindRegister ("ID_EMPLEADO", "01", EQUAL,
"FEC_IMPU", dtUltimaPaga, EQUAL, "FEC_PAGO", dtUltimaPaga, EQUAL)
If intRegAcum = -1 Then
    MessageBox("Meta4", "No se ha calculado el empleado en la última
paga")
Else
    Acumulado.MoveTo (intRegAcum)
    GetValue ("ACUMULADO", strConcepto, intValorConcepto)
    Call ( strTablaValor, "InsertRegister" )
    SetValue ( strTablaValor, "Concepto", strConcepto )
    SetValue ( strTablaValor, "Valor", intValorConcepto )
    SetValue ( strTablaValor, "Fec_ini", Today ( ) )
    SetValue ( strTablaValor, "Fec_fin", NullValue ( ) )
EndIf
Until IsEOF ( )=M4TRUE OR IntRegister = -1
EndIf

```

SetValuePriority (Prioridad, Nodo, Elemento, Registro, [Tramo], Valor)

Esta función asigna un valor indicado como argumento al nombre del elemento, asignándole una determinada prioridad entre 0 y 3 (con 0 para nivel de prioridad máximo y 3 para nivel de prioridad mínimo).

Sus argumentos son la prioridad que tiene asignada un elemento, los nombres del nodo, elemento, registro y tramo al que se hace referencia, y el valor que se pretende asignar al elemento.



La prioridad no sólo se asigna a elementos, sino también a tramos. Sólo es posible compactar dos elementos o tramos si ambos tienen las mismas prioridades.

El argumento del tramo es opcional. Si el argumento *Registro* es -1, se utiliza el

tramo activo. Si en la ejecución con tramos el argumento correspondiente a tramos no se especifica, se utilizaría el del tramo activo por defecto.

Cuando con esta función se intenta sobrescribir el valor de un elemento se comprueba el valor asignado al argumento **Prioridad**, momento en el cual pueden suceder dos cosas:

- Si la prioridad del elemento es menor o igual que aquella con la que se compara, se sobrescribe el valor del elemento con el valor que se pasa como argumento.
- Si la prioridad del elemento es mayor que aquella con la que se compara, no se sobrescribe el valor del elemento.

EJEMPLO

```
SetValuePriority (  
    VALUE NUMBER Priority, VALUE VARIABLE STRING Node, VALUE VARIABLE  
    STRING Item, VALUE NUMBER Record, [VALUE NUMBER Slice]  
)
```

Sort ([índice])

A los nodos se les puede asignar una serie de índices que permitan clasificarlos de forma ascendente o descendente, tomando como referencia uno o varios elementos. Si no se indica un tipo de índice, se utilizará el que esté definido por defecto (sólo uno podrá estar marcado para que actúe por defecto).

La función Sort altera físicamente en memoria el orden de los registros, a diferencia del AddSortFilter, que sólo actúa a nivel de navegación.

EJEMPLO

```
Sort (  
    [VALUE NUMBER Index]  
)
```

Funciones para la base de datos lógica

Las funciones que pertenecen a esta categoría permiten realizar operaciones de diverso tipo en la base de datos lógica.

A continuación se indican las funciones pertenecientes a esta categoría:

- BeginDBTransaction
- BeginTransaction ()
- Delete_blk ()
- Delete_prg ()
- EndDBTransaction (M4_COMMIT / M4_ROLLBACK / EXECUTE_POSTVALIDATION)
- EndTransaction (M4_COMMIT / M4_ROLLBACK / EXECUTE_POSTVALIDATION / M4_ROLLBACK_RESUME)
- ExecuteDirectSQL (ID_Conexión, Máscara....)
- ExecuteRealSQL (ID_Conexión....)
- ExecuteSQL ()
- GetAutoloadMode ()
- GetNodeAutoloadMode (Modo)
- Insert_blk ()
- Insert_prg ()
- Load_blk ()
- Load_prg ()
- Persist_tree ()
- SetAutoloadMode (Modo)
- SetLoadRange ([M4Obj] , [Node] , RangeSize , RangeStart , ...)
- SetLoadTimeOut([M4ObjectInstance], [Node], TimeOut)
- SetNodeAutoloadMode (Modo)
- Update_blk ()
- Update_prg ()

Puede obtener información adicional sobre transacciones en los siguientes apartados:

- Tipos de transacciones
- Métodos transaccionales

Tipos de transacciones

Antes de comenzar la descripción de las funciones pertenecientes a este apartado, vamos a hacer una distinción previa entre los dos tipos de transacciones posibles:

-
- Por un lado, están las transacciones básicas de base de datos. Para trabajar con éstas, se dispone de las siguientes funciones:
 - *BeginDBTransaction ()*
 - *EndDBTransaction (Acometer/Vuelta atrás/Ejecutar postvalidación)*
 - Por otro, están las transacciones de Meta4Object Engine, que permiten trabajar con métodos en los que se desconoce a priori si van a existir errores o no en su ejecución, como la función *Persist()*. Si los métodos fallan, se permite continuar ejecutando el código.

Las funciones para trabajar con estas transacciones son:

- *BeginVMTransaction ()*
- *EndVMTransaction (Acometer/Vuelta atrás)*

Además, existe otro conjunto de funciones, que son las más usadas y que inician simultáneamente una transacción de Meta4Object Engine y de base de datos. Se trata de *BeginTransaction ()* y *EndTransaction ()*. Estas funciones están definidas como elementos del sistema, así que se puede tener acceso a ellas desde cualquier estructura de nodo.

Métodos transaccionales

Los métodos transaccionales son aquéllos que inician una transacción de Meta4Object Engine antes de iniciar su ejecución. Si se produce algún error en el método, se detendrá su ejecución, de modo que el funcionamiento del método origen que le llamó no se vea afectado, ya que éste prosigue su ejecución y realiza la instrucción siguiente a la llamada al elemento transaccional.

En el caso de que se produjese algún error en el elemento transaccional, la función devolvería como resultado M4_ERROR, de igual modo que en las variables por referencia, si las hay.

Un ejemplo de método transaccional es el método Persist_tree ().

BeginDBTransaction

Esta función inicia una transacción de base de datos.

BeginTransaction ()

Esta función inicia una transacción de Meta4Object Engine y de base de datos. Si se produce un error cuando el Meta4Object Engine está ejecutando el código, la función evita que se detenga la ejecución.

Por ejemplo, la función **Persist ()** puede fallar porque exista un cambio en el

valor de la clave primaria.

Esta función se utiliza tanto cuando se lleva a cabo una transacción de base de datos como de ejecución.

Delete_blk ()

Esta función elimina de la base de datos todos los registros marcados como "borrados". Sólo elimina los registros borrados en el bloque activo del nodo desde el que se llama a la función. Para ejecutarla correctamente, siga estos pasos:

- Inicie una transacción en la base de datos física.
- Ejecute la función *Delete_blk ()*.
- Finalice la transacción guardando los cambios, para que se reflejen físicamente en la base de datos.

EJEMPLO

```
Begin Transaction
  Delete_blk ( )
End Transaction (Commit)
```

Delete_prg ()

Esta función elimina los registros que se hayan borrado en el bloque activo del nodo desde el que se llama a la función y todos aquellos que estén conectados y dependan de ellos.

Para ejecutar correctamente esta función, siga estos pasos:

- Inicie una transacción en la base de datos física.
- Ejecute la función *Delete_prg ()*
- Para finalizar la transacción, guarde permanentemente los cambios para que se reflejen físicamente en la base de datos.

EJEMPLO

```
Begin Transaction
  Delete_prg ( )
End Transaction (Commit)
```

EndDBTransaction (M4_COMMIT / M4_ROLLBACK / EXECUTE_POSTVALIDATION)

Esta función finaliza una transacción de base de datos. Hasta que no se realiza una transacción de base de datos, la información no se guarda físicamente.

La función recibe un único argumento, con tres posibles valores:

- **M4_COMMIT**: guarda en la base de datos los cambios realizados.
- **M4_ROLLBACK**: facilita, en caso de producirse un error en la ejecución de una función, que todos los cambios realizados desde el inicio de la transacción se deshagan.
- **EXECUTE_POSTVALIDATION**: este argumento comprueba el funcionamiento de las postvalidaciones. Esta posibilidad tiene su máximo significado en operaciones de importación de datos.

EJEMPLO

```
EndDBTransaction (
    VALUE NUMBER Commit/RollBack
)
```

EndTransaction (M4_COMMIT / M4_ROLLBACK / EXECUTE_POSTVALIDATION / M4_ROLLBACK_RESUME)

Esta función finaliza una transacción de Meta4Object y de base de datos. Hay que tener en cuenta que si se trabaja con base de datos, los datos no se guardan físicamente en ella hasta que no se haga commit de la transacción de base de datos en la que se guarden.

La función recibe un único argumento, con varios posibles valores:

- **M4_COMMIT**: Finaliza la transacción en curso de Meta4Object Engine y de base de datos aplicando los cambios y continua la ejecución de LN4 normalmente.

- **M4_ROLLBACK**: en caso de que el usuario detecte un error en la ejecución de una función, este argumento le permite provocar que se deshagan todos los cambios de base de datos realizados desde el inicio de la transacción; no guarda los datos y finaliza la ejecución de LN4.
- **EXECUTE_POSTVALIDATION**: este argumento comprueba el funcionamiento de las postvalidaciones. Tiene especial importancia en operaciones de importación de datos.
- **M4_ROLLBACK_RESUME**: este argumento actúa de forma similar al argumento Rollback con la diferencia de que en este caso no finaliza la ejecución del LN4, esta se retoma a partir del final de la transacción (si la hubiese).



Tenga en cuenta que este comportamiento no se cumple en las transacciones anidadas, ya que la aplicación no soporta este tipo de transacciones (solo existe una transacción, que es la primera que se abre).

EJEMPLO

```
EndTransaction (  
    VALUE NUMBER Commit/RollBack  
)
```

ExecuteDirectSQL (ID_Conexión, Máscara,...)

Esta función ejecuta una sentencia SQL con una conexión lógica entre identificadores y una máscara de validación.

EJEMPLO

```
ExecuteDirectSQL (  
    VALUE NUMBER IdConnection, VALUE VARIABLE STRING Mask  
)
```

ExecuteRealSQL (ID_Conexión,...)

Ejecuta una sentencia SQL con un identificador lógico de conexión.

EJEMPLO

```
ExecuteRealSQL (
    VALUE NUMBER IdConnection
)
```

ExecuteSQL ()

Esta función ejecuta una sentencia de APIs de SQL desde un Meta4Object. Acepta como argumentos cero u otros valores, cada uno de los cuales corresponde a un argumento.

GetAutoloadMode ()

Esta función devuelve el modo de carga automática que tiene asignado el Meta4Object activo.

Para obtener más información sobre este apartado, consulte las [Macros de autocarga](#).

GetNodeAutoloadMode (Modo)

Esta función devuelve el modo de autocarga que tiene asignado el nodo activo. Por defecto, devuelve el nodo activo cargado.

Para obtener más información sobre este apartado, consulte las [Macros de autocarga](#) que han sido definidas.

Insert_blk ()

Esta función inserta en la base de datos aquellos registros que hayan sido creados en el bloque activo del nodo desde el que se llama a la función.

Para ejecutar correctamente la función, siga estos pasos:

- Inicie una transacción en la base de datos física.

- Ejecute la función *Insert_blk ()*.
- Por último, guarde permanentemente los cambios para que se reflejen en la base de datos.

EJEMPLO

```
Begin Transaction
  Insert_blk ( )
End Transaction (Commit)
```

Insert_prg ()

Esta función inserta en la base de datos aquellos registros que se hayan creado en el bloque activo del nodo desde el que se llama a la función y los que dependan de esos registros.

Para ejecutar correctamente esta función, siga estos pasos:

- Inicie una transacción en la base de datos física.
- Ejecute la función *Insert_prg ()*.
- Por último, guarde permanentemente los cambios para que se reflejen físicamente en la base de datos.

EJEMPLO

```
Begin Transaction
  Insert_prg ( )
End Transaction (Commit)
```

Load_blk ()

Esta función realiza una carga de datos desde la base de datos sobre el bloque que reúne el conjunto de registros activo.

EJEMPLO

```
Begin ( )
  Load_blk ( )
```

```
End ( )
```

Load_prg ()

Esta función llama a la función Update_blk () y realiza la carga de datos correspondiente a un nodo y a cada uno de los elementos conectados.

EJEMPLO

```
Begin ( )  
    Load_prg ( )  
End ( )
```

Persist_tree ()

Esta función guarda en la base de datos todos los cambios que se hayan efectuado en el bloque activo del nodo sobre el que esté situado, así como en los bloques de los nodos hijos con los que esté conectado.

La función ejecuta las siguientes acciones:

- Inicia una transacción en la base de datos física, para lo que realiza una llamada a la función *Begin Transaction ()*.
- Posteriormente, realiza una de las siguientes acciones en función de la operación que se haya realizado previamente.
 - Borra de la base de datos física todos aquellos registros que se hayan marcado para borrar, bien desde la interfaz o mediante llamadas a la función *DeleteRegister ()*.
 - Actualiza en la base de datos física todos aquellos registros que hayan sido actualizados, desde la interfaz del programa.
 - Inserta en la base de datos física todos aquellos registros que se hayan agregado desde la memoria, bien desde la interfaz del programa o mediante llamadas a la función *InsertRegister ()*.
- Por último, cierra la transacción y, dependiendo de los resultados, ejecuta una de las siguientes acciones:
 - Guarda permanentemente todos los cambios y los refleja en la base de datos, si éstos se han llevado a cabo satisfactoriamente (*Commit*).

- Restaura el estado anterior, si no se ha podido realizar correctamente alguno de los pasos anteriores (*Rollback*).

SetAutoloadMode (Modo)

La carga automática es una característica específica de los Meta4Objects, que permite que su contenido se cargue en memoria a medida que éste se va necesitando. De este modo, cuando se intenta tener acceso a un registro, el sistema comprueba si se ha visitado anteriormente:

- En caso negativo, procederá a la carga inicial de la información que contiene el registro.
- En caso contrario, el tiempo necesario para el acceso será más breve, ya que dispone de la información cargada en memoria.

Esta función aporta entre otras las siguientes ventajas:

- En el caso de un Meta4Object muy grande, no se necesita cargar completo en memoria, sino únicamente las partes que se visiten.
- El acceso a las partes ya visitadas con anterioridad es más rápido.

Los modos de carga automática se pueden establecer en dos niveles:

- **Nivel de nodo:** se explica en la función *SetNodeAutoloadMode (Modo)*.
- **Nivel de Meta4Object:** se puede indicar como nivel de carga el bloque. Desde el nivel de Meta4Object se puede especificar que se considere únicamente la configuración del nodo, de lo contrario, ésta no se tiene en cuenta.

Existen los siguientes tipos de carga automática:

- Se puede asignar a la carga automática el valor **off**, lo que significa que la función está desactivada por defecto y en caso de que se quiera contar con ella, el usuario debe hacer la llamada.

Si no se llama al *Autoload* (porque el Meta4Object no lo tenga definido previamente), el Meta4Object queda vacío, sin datos.

- Se puede establecer la carga automática en modo **propagante**: cuando se accede a un bloque, sólo se carga la rama a la que pertenece dicho bloque, puesto que se necesita tener acceso a los hijos del bloque, pero no es necesario que estén cargados los demás bloques. Esto aporta una gran ventaja, ya que permite minimizar el número de transacciones a realizar.

Este proceso se consigue llamando a la función *Load_prg* y no a la función *Load_blk*, que es la que se utilizaría cargar el bloque entero.

- Se puede establecer la carga automática a nivel de **bloque**, lo que permite cargar en memoria un bloque entero.

-
- Se puede asignar a la carga automática **nodesays**, lo que permite realizar la operación que el nodo tenga definida previamente.

EJEMPLO

```
SetAutoloadMode (
    VALUE NUMBER Mode
)
```

SetLoadRange ([M4Obj] , [Node] , RangeSize , RangeStart , ...)

Esta función permite llevar a cabo la carga de registros por rangos. Ofrece la posibilidad de realizar cargas incrementales sin carga parcial.

Los argumentos de esta función son:

- Instance: opcional. Indica la instancia de nivel 2 sobre la que va a aplicar la función.
 - Si no se pasa este parámetro se trabaja sobre el propio Meta4Object.
 - Si se pasa debe ser una cadena. También se soporta cadena vacía para el propio Meta4Object.
 - Si se pasa un valor que no es una cadena o que no es una instancia válida se da error.
- Node: opcional si no viene Instance. Indica el nodo sobre el que se va a aplicar la función.
 - Si no se pasa este parámetro se trabaja sobre el propio nodo.
 - Si se pasa puede ser cadena, número o nulo. Si se pasa otro valor se da error.
 - Si es cadena debe ser el identificador de un nodo existente.
 - Si se pasa cadena vacía o un identificador no válido se da error.
 - Si es número es el índice de un nodo existente. Si el índice no existe se da error.
 - Si es nulo se trabaja sobre el propio nodo si se está trabajando sobre el propio Meta4Object.
- RangeSize: indica el máximo número de filas a cargar.
 - Si es 0 indica que no se debe hacer la carga por rangos.
 - Si es negativo no se da error y se hace la conversión a su entero positivo.
- RangeStart: indica la primera fila a cargar.

- Si es 0 indica la primera fila de la sentencia.
- Si es negativo no se da error y se hace la conversión a su entero positivo.



Tenga en cuenta que esta función no soporta el uso de la cláusula UNION dentro de la sentencia de carga.

A continuación se muestra un ejemplo sencillo de carga de registros de 100 en 100 empezando en el primero:

EJEMPLO

```

DestroyBlock()
SetLoadRange( 100, 0 )

do
    DestroyAllRegisters()
    LOAD_BLK()
    count = Count()

    for i = 0 to count - 1
        'Do something
    next
until count <> 100

```

SetLoadTimeout([M4ObjectInstance], [Node], Timeout)

En la base de datos lógica existe un mecanismo configurable para el bloqueo de sentencias complejas. Este mecanismo permite especificar un límite de tiempo a partir del cual se parará la ejecución de la sentencia. El mecanismo controla el tiempo de ejecución (no el de recuperación de los registros), y evita que si una sentencia es compleja y el gestor requiere muchos recursos y tiempo para ejecutarla, el servidor de aplicaciones se quede bloqueado.

La función que permite especificar el tiempo máximo en segundos para la carga de un bloque concreto es SetLoadTimeout([M4ObjectInstance], [Node], Timeout). Este valor se tiene en cuenta siempre que el tiempo máximo global esté activado y sea mayor que el valor particular especificado.

Así por ejemplo, para establecer un tiempo máximo de espera para la ejecución

("timeout") de 5 segundos:

EJEMPLO

```
DestroyBlock()  
SetLoadTimeOut( 5 )  
LOAD_BLK()
```

SetNodeAutoloadMode (Modo)

Esta función establece la carga automática a nivel de nodo. La carga automática lleva a cabo las acciones que el nodo le indica.

Cada vez que la carga automática necesita realizar alguna acción, pregunta al nodo el tipo de carga que tiene asignado.

En los nodos puede haber diferentes clases de carga automática:

- **Bloque:** esta clase de carga automática permite cargar un bloque entero.
- **Propagante:** esta clase de carga automática permite cargar ciertos elementos dependientes de un bloque concreto, así como todos los elementos que dependen de ellos.
- **Off:** esta clase de carga automática permite evitar la carga de un determinado nodo, independientemente de su motivo.



En cada nodo se puede tener un modo distinto de carga automática. Para obtener más información sobre este apartado, consulte las macros [*Macros de autocarga*](#).

EJEMPLO

```
SetNodeAutoloadMode (  
    VALUE NUMBER Mode  
)
```

Update_blk ()

Esta función actualiza en la base de datos todos aquellos registros que hayan tenido cambios.

Sólo actualiza los registros que se hayan modificado en el bloque activo del nodo desde el que se llama a la función.

Para ejecutar correctamente esta función, siga estos pasos:

- Inicie una transacción en la base de datos física.
- Ejecute la función *Update_blk ()*
- Para finalizar la transacción, guarde permanentemente los cambios en la base de datos.

EJEMPLO

```
Begin Transaction
  Update_blk ( )
End Transaction (Commit)
```

Update_prg ()

Esta función actualiza en la base de datos todos los registros que hayan tenido cambios, así como los registros relacionados.

Sólo actualiza aquellos registros modificados en el bloque activo del nodo desde el que se llama a la función y todos aquellos que dependan de ellos.

Para ejecutar correctamente la función, siga estos pasos:

- Inicie una transacción en la base de datos física.
- Ejecute la función *Update_prg ()*
- Para finalizar la transacción, guarde permanentemente los cambios para que se reflejen en la base de datos.

EJEMPLO

```
Begin Transaction
  Update_prg ( )
End Transaction (Commit)
```

Funciones de Meta4Objects

Todas las funciones correspondientes a este apartado tienen como ámbito de aplicación un Meta4Object, por lo que no se aplican a nivel de nodo. La sintaxis para ejecutar cualquiera de estas funciones es la siguiente:

nombre de Meta4Object o nombre de instancia ! Función ()

Las funciones de Meta4Objects básicas son:

- AddAsPartner ()
- CheckModify ()
- GetHandle ()
- GetProperty (Propiedad)
- IsDataCacheable ()
- Load ()
- LoadFromFile (NombreArchivo)
- Persist ()
- PersistToFile ([NombreArchivo])
- Release ()
- RemoveAsPartner ()
- SetProperty (Propiedad, Valor)
- Unload ()

AddAsPartner ()

Cuando se realizan transacciones cliente/servidor, en ocasiones es necesario utilizar Meta4Objects compañeros, puesto que es posible que un Meta4Object requiera el uso de otros, no sólo cuando se realizan llamadas de nivel 2.

Si posteriormente se quiere eliminar de la lista cualquiera de estos Meta4Objects seleccionados como compañeros, se debe utilizar la función Funciones básicas de llamadas de nivel 2.

CheckModify ()

Esta función indica si ha habido alguna modificación en el Meta4Object. Se utiliza con frecuencia en herramientas que deben preguntar al usuario si ha habido cambios en un Meta4Object antes de proceder a cerrarlo, para que, en caso afirmativo, se guarden físicamente en la base de datos.

GetHandle ()

Esta función retorna el identificador numérico único de una instancia de Meta4Object. Este identificador numérico se calcula en el momento que se crea la instancia del Meta4Object y no cambia a lo largo de toda la vida de esa instancia. Este número sirve para identificar de manera unívoca una instancia de Meta4Object. En LN4 puede utilizarse con otras funciones de Ln4 como InsertChannel.

Esta función no tiene argumentos y se aplica al Meta4Object sobre el que se esté trabajando.

GetProperty (Propiedad)

Con esta función se pueden conocer los valores que tienen asignadas las propiedades existentes, como por ejemplo, la fecha de ejecución. Recibe como único argumento el nombre de la propiedad cuyos valores se quiere consultar (PropertyName).

Ésta es una función de Meta4Object, de forma que la sintaxis que se debe emplear para invocarla es:

Value = Meta4ObjectAlias!GetProperty("property name")

En caso no existir esa propiedad se devolvería el valor nulo.

IsDataCacheable ()

Permite conocer si un determinado Meta4Object tiene configurados los datos para que puedan almacenarse en la memoria caché. La propiedad *Data Cacheable* puede tener los siguientes valores:

- 1: indica que los datos se pueden almacenar en la caché.
- 0: indica que los datos no se pueden almacenar en la caché.

Load ()

La función carga desde la base de datos contenidos relativos a un Meta4Object, lo que significa que llama a la función Load_prg dentro de todos los nodos raíz.

LoadFromFile (NombreArchivo)

Esta función carga datos de un Meta4Object desde un archivo. No se puede aplicar sobre el Meta4Object que se está ejecutando, ya que devolvería un código de error.

Persist ()

Esta función guarda físicamente en la base de datos contenidos relativos a un Meta4Object.

PersistToFile ([NombreArchivo])

Esta función guarda datos de un Meta4Object en un archivo. Si el archivo especificado no existe, la función lo crea.

Si la función se ejecuta correctamente, devuelve el nombre; si se produce algún error, devuelve nulo. El nombre de archivo es opcional, ya que si no se indica, se asigna uno por defecto.

Release ()

Esta función cierra un Meta4Object que esté en memoria. Tiene una limitación, ya que no se puede ejecutar sobre el mismo Meta4Object desde el que se ejecuta, sino que se tiene que ejecutar sobre otro.

RemoveAsPartner ()

Para realizar transacciones cliente/servidor, se necesitan Meta4Objects compañeros. Esta función elimina de la lista determinados Meta4Objects compañeros que hayan sido agregados previamente mediante la función *Funciones básicas de llamadas de nivel 2*.

SetProperty (Propiedad, Valor)

Los Meta4Objects pueden tener propiedades. Esta función asigna valores a las propiedades existentes, o crea otras nuevas, como por ejemplo, la fecha de ejecución.

Unload ()

Descarga todos los datos que se tengan en memoria para poder ejecutar con éxito la función *Load ()* y poder cargar nueva información.

Esta función tiene una limitación, ya que no se puede ejecutar sobre el mismo Meta4Object desde el que se ejecuta, sino que debe realizarse sobre cualquier otro.

Funciones básicas de llamadas de nivel 2

Las funciones básicas de llamadas de nivel 2 son:

- ApplyBookMark (BookMark)
- CloseChannel (IdInstancia)
- DefineInstance (Instancia, Meta4Object, PolíticaCarga, PolíticaCompartición)
- DefineInstanceEx (Instancia, Meta4ObjectNombre, PolíticaCarga, PolíticaCompartición, [IDOrganización], [TipoOrganización], [CSTipoAcceso], [DataLanguage], [Role]....)
- EraseAllL2Instances
- GetBookMark (Type)
- OpenChannel (Meta4Object, IDInstancia, Ámbito, ModoApertura)
- OpenChannelEx (Meta4Object, IdInstancia, Ambito, ModoApertura, [OrganizaciónID], [TipoOrganización], [CSTipoAcceso])
- ReleaseBookMark (BookMark)

ApplyBookMark (BookMark)

Esta función, junto a las funciones GetBookMark (Type) y ReleaseBookMark (BookMark), se utiliza para la gestión de marcadores (o *bookmarks*) sobre los Meta4Objects. Un marcador es un objeto tecnológico que almacena en memoria la posición actual de los registros de todos los nodos de un

Meta4Object. El objetivo es poder posicionarse nuevamente en estos registros después de un desposicionamiento (por movimiento, borrado, recarga, etcétera).

La función ApplyBookMark toma el *handle* que devuelve la función GetBookMark y se reposicionan todos los nodos en las posiciones actuales almacenadas.

Véase un ejemplo completo de uso de las tres funciones de manejo de marcadores en [GetBookMark \(Type\)](#).

CloseChannel (IdInstancia)

Esta función cierra el Meta4Object correspondiente a una instancia, cuyo identificador se ha indicado como argumento.

EJEMPLO

```
CloseChannel (
    VALUE VARIABLE STRING IdInstance
)
```

DefineInstance (Instancia, Meta4Object, PolíticaCarga, PolíticaCompartición)

Esta función define una instancia de un Meta4Object de forma dinámica. Sólo funciona para ejecuciones Just In Time (JIT), y sus argumentos son los siguientes:

- **Instancia:** corresponde al identificador de la instancia.
- **Meta4Object:** contiene el identificador del Meta4Object del que se va a abrir una instancia.
- **PolíticaCarga:** indica la política de carga del Meta4Object.
- **PolíticaCompartición:** muestra la política de compartición de la instancia.

Esta función utiliza las [Macros de autocarga para llamadas de nivel 2](#).

EJEMPLO

```
DefineInstance (
```

```

VALUE VARIABLE STRING Instance,
VALUE VARIABLE STRING Meta4Object,
VALUE NUMBER PolíticaCarga,
VALUE NUMBER SharePolicy
)

```

DefineInstanceEx (Instancia, Meta4ObjectNombre, PolíticaCarga, PolíticaCompartición, [IDOrganización], [TipoOrganización], [CSTipoAcceso], [DataLanguage], [Role],...)

Esta función define una instancia correspondiente a un Meta4Object de forma dinámica. Además, permite definir la organización, tipo de organización, tipo de acceso, lenguaje y rol.

Sólo funciona para ejecución *Just In Time* (JIT).

Los argumentos correspondientes a esta función son los siguientes:

- **Instancia:** corresponde al identificador de la instancia.
- **Meta4Object:** contiene el identificador del Meta4Object del que se va a abrir una instancia.
- **PolíticaCarga :** indica la política de carga del Meta4Object.
- **PolíticaCompartición:** contiene la política de compartición.
- **IDOrganización:** contiene el identificador unívoco de la organización definida.
- **TipoOrganización:** indica el tipo de organización.
- **CSTipoAcces:** indica el tipo de acceso al Meta4Object.
- **DataLanguage:** indica el lenguaje en el que están definidos los datos.
- **Role:** indica el rol de aplicación con el que se va a realizar la llamada.

Esta función utiliza las Macros de autocarga para llamadas de nivel 2 y las Macros de tipo de acceso.

EraseAllL2Instances

Esta función permite eliminar todas las instancias que se hayan abierto con llamadas de nivel 2.

GetBookMark (Type)

Esta función, junto a las funciones [ApplyBookMark \(BookMark\)](#) y [ReleaseBookMark \(BookMark\)](#), se utiliza para la gestión de marcadores (o *bookmarks*) sobre los Meta4Objects. Un marcador es un objeto tecnológico que almacena en memoria la posición actual de los registros de todos los nodos de un Meta4Object. El objetivo es poder posicionarse nuevamente en estos registros después de un desposicionamiento (por movimiento, borrado, recarga, etcétera).

Al ejecutar la función desde cualquier Meta4Object, detecta la posición actual de todos sus nodos, almacena esas posiciones y las referencia mediante un *handle* que devuelve para que lo utilicen las funciones [ApplyBookMark](#) y [ReleaseBookMark](#).

La función tiene un parámetro opcional para indicar el tipo de marcador deseado:

- De posición: [M4_BOOKMARK_POSITION](#)
- De clave primaria: [M4_BOOKMARK_PK](#)

Para obtener más información, haga clic en [Macros LN4](#).

A continuación, se ofrece un completo ejemplo de uso de las tres funciones de manejo de marcadores:

EJEMPLO

```
MoveTo(3)
HIJO.MoveTo(3)
MessageBox(ToString(GetCurrent()), ToString(HIJO.GetCurrent()))

iBookMark = GetBookMark(M4_BOOKMARK_POSITION)

Begin()
MessageBox(ToString(GetCurrent()), ToString(HIJO.GetCurrent()))

ApplyBookMark(iBookMark)
MessageBox(ToString(GetCurrent()), ToString(HIJO.GetCurrent()))

ReleaseBookMark(iBookMark)

MoveTo(3)
HIJO.MoveTo(3)
```

```
MessageBox(ToString(GetCurrent()), ToString(HIJO.GetCurrent()))

iBookMark = GetBookMark(M4_BOOKMARK_PK)

Begin()
MessageBox(ToString(GetCurrent()), ToString(HIJO.GetCurrent()))

ApplyBookMark(iBookMark)
MessageBox(ToString(GetCurrent()), ToString(HIJO.GetCurrent()))

ReleaseBookMark(iBookMark)
```

InsertChannel

Esta función crea un nuevo enlace de nivel 2 en el Meta4Object sobre el que se esté trabajando. El enlace de nivel 2 se hace a la instancia dada por el primer argumento que es un identificador numérico único que se debe haber obtenido con una llamada a la función `GetHandle`. Este enlace de nivel 2 se asocia al identificador de instancia dado por el segundo argumento. Si ya existe un enlace de nivel 2 para ese identificador de instancia, no se realiza la operación y se devuelve un error.

Los argumentos correspondientes a esta función son los siguientes:

- **ai_iM4ObjHandle**: número que representa un identificador numérico único de una instancia de Meta4Object.
- **ai_sInstancelId**: cadena que representa el identificador de la instancia de nivel 2.

OpenChannel (Meta4Object, IDInstancia, Ámbito, ModoApertura)

Esta función abre una instancia de Meta4Object utilizando una llamada de Meta4Object Engine de nivel 2.

Si el Meta4Object no está cargado, la función permite sobrescribir los argumentos *Ámbito* y *ModoApertura*, es decir, el ámbito y el modo en el que se abre el Meta4Object, cuyos posibles valores están definidos como constantes.

Los argumentos correspondientes a esta función son los siguientes:

- **Meta4Object**: contiene el identificador del Meta4Object del que se va a abrir una nueva instancia.

-
- **IDInstancia:** contiene el identificador de la instancia.
 - **Ámbito:** ámbito de la instancia abierta. Indica la política de compartición. Los posibles valores que puede tomar este argumento están definidos como constantes.
Para obtener más información haga clic en [Macros LN4](#).
 - **ModoApertura:** indica el modo en que se abre una instancia en el Meta4Object. Los posibles valores que puede tomar este argumento están definidos como constantes.
Para obtener más información, haga clic en [Macros LN4](#).

EJEMPLO

```
OpenChannel (  
    VALUE VARIABLE STRING Meta4Object,  
    VALUE VARIABLE STRING IdInstance,  
    VALUE NUMBER Scope,  
    VALUE NUMBER OpenMode  
)
```

OpenChannelEx (Meta4Object, IdInstancia, Ambito, ModoApertura, [OrganizaciónID], [TipoOrganización], [CSTipoAcceso])

Esta función modifica el tipo de organización, tipo de acceso, ámbito, modo, etc., con el que se abre un Meta4Object que esté cargado en memoria.

La descripción que está en la base de datos se utiliza para cargar el Meta4Object, que tendrá como características aquellas que estén definidas en la base de datos.

Los argumentos correspondientes a esta función son los siguientes:

- **Meta4Object:** contiene el identificador del Meta4Object del que se va a abrir una nueva instancia.
- **IDInstancia:** contiene el identificador de la instancia.
- **Ámbito:** ámbito de la instancia abierta. Indica la política de compartición. Los posibles valores que puede tomar este argumento están definidos como constantes.
Para obtener más información, haga clic en [Macros LN4](#).

- **ModoApertura:** indica el modo en que se abre una instancia en el Meta4Object. Los posibles valores que puede tomar este argumento están definidos como constantes.

Para obtener más información, haga clic en [Macros LN4](#).

- **IDOrganización:** es un argumento opcional e indica el identificador de la organización del Meta4Object.
- **TipoOrganización:** es un argumento opcional e indica el tipo de organización del Meta4Object.
- **CSTipoAcceso:** es un argumento opcional e indica el tipo de acceso del Meta4Object.

Esta función utiliza las [Macros de tipo de acceso](#).

ReleaseBookMark (BookMark)

Esta función, junto a las funciones [ApplyBookMark \(BookMark\)](#) y [GetBookMark \(Type\)](#), se utiliza para la gestión de marcadores (o *bookmarks*) sobre los Meta4Objects. Un marcador es un objeto tecnológico que almacena en memoria la posición actual de los registros de todos los nodos de un Meta4Object. El objetivo es poder posicionarse nuevamente en estos registros después de un desposicionamiento (por movimiento, borrado, recarga, etcétera).

Esta función permite eliminar los marcadores creados con la función [GetBookMark](#) cuando ya no se necesitan para liberar la memoria que ocupaban. Si no se llama a esta función, se eliminarán al salir del Meta4Object correspondiente.

Véase un ejemplo completo de uso de las tres funciones de manejo de marcadores en [GetBookMark \(Type\)](#).

Funciones para ejecución de código JIT

La tecnología JIT (Just in Time) permite ejecutar un código determinado durante la ejecución de otro código. De ahí que las funciones que se recogen en este apartado permitan realizar operaciones de compilación de código LN4.

A esta categoría pertenecen las siguientes funciones:

- [clcExecuteJIT \(Manejador, ValArg1, ValArg2,...\)](#)
- [clcCompile \(\)](#)
- [clcExecuteJIT \(Manejador, ValArg1, ValArg2,...\)](#)
- [clcExecuteLN4JIT \(CódigoLN4, Arg1, ValArg1, Arg2, ValArg2...\)](#)

-
- clcGetPolish ()
 - clcOldTItolDTI ()
 - clcGetPolish ()
 - clcReleaseJIT (Manejador)
 - clcRestoreSource ()
 - clcSourceTold ()
 - clcSourceToNumber ()
 - clcSourceToString ()
 - CompileBlock ()
 - LocalTest ()

clcCheckSyntax ()

Esta función permite realizar una comprobación de la sintaxis y devuelve si esta es correcta o no.

clcCompile ()

Esta función permite compilar el elemento activo.

clcExecuteJIT (Manejador, ValArg1, ValArg2,...)

Esta función ejecuta el código LN4 al que se hace referencia mediante Manejador, que ha sido previamente compilado por la función clcPrepareJIT (CódigoLN4, &Manejador, Arg1, Arg2,...).

La función utiliza el resto de los argumentos como argumentos del código.

clcExecuteLN4JIT (CódigoLN4, Arg1, ValArg1, Arg2, ValArg2...)

Esta función compila y ejecuta el código LN4 que se indica como primer parámetro y que se utiliza en los distintos argumentos *Arg1*, *Arg2*, etc y los diferentes valores correspondientes a esos argumentos *ValArg1*, *ValArg2*, etc.

clcGetPolish ()

Esta función compila el código fuente LN4 desde el elemento correspondiente e

inserta código en polaco en el elemento en cuestión.

clcOldTltoIDTI ()

Esta función cambia la fórmula LN4, sustituyendo los códigos de identificación de estructura de nodo. Sólo debe utilizarse después de la función `ExecuteBranch`.

clcPrepareJIT (CódigoLN4, &Manejador, Arg1, Arg2,...)

Esta función compila el código LN4 que se indica como primer parámetro, utilizando el resto de argumentos, y devuelve como resultado una referencia a la fórmula compilada, que se podrá utilizar en una ejecución posterior con la función `clcExecuteJIT (Manejador, ValArg1, ValArg2,...)`.

La referencia a la fórmula compilada se devuelve en `&Manejador`, y el carácter `&` significa que esta variable indica por referencia el valor correspondiente a ese argumento.

clcReleaseJIT (Manejador)

Esta función borra el código LN4 al que se referencia mediante `Manejador`, y que se ha compilado previamente con la función `clcPrepareJIT (CódigoLN4, &Manejador, Arg1, Arg2,...)`.

clcRestoreSource ()

Esta función modifica la fórmula LN4, transformándola del formato de base de datos a su formato habitual.

clcSourceTold ()

Esta función devuelve la fórmula LN4, con sus elementos modificados de forma que indiquen su `ID_ITEM`.

clcSourceToNumber ()

Esta función retorna la fórmula LN4, con sus elementos modificados de forma que indiquen su `ID_SYNONYM` (con #).

clcSourceToString ()

Esta función devuelve la fórmula LN4, con sus elementos modificados de forma que indiquen su N_SYNONYM (con @).

CompileBlock ()

Esta función compila todos los elementos del bloque activo definido en las reglas.

LocalTest ()

Esta función ejecuta la fórmula real según su código polaco. La función interpreta asimismo el código.

Funciones de traza

Estas funciones permiten escribir en el sistema log mensajes de diverso tipo (error, aviso, depuración, etc.).

- DumpChannel ()
- DumpNode ()
- DumpRegister ()
- DumpStatistics (NombreArchivo, Meta4Object, [Nivel])
- GetErrorString (Módulo, Submódulo, Código, [Argumento 1])
- GetLog (Posición, &Tipo, &Módulo, &Submódulo, &Código, &TextoAdicional)
- GetLogSize ()
- PurgeLog (NumElimina)
- SetLog (Tipo, Módulo, Submódulo, Código, [Argumento])

DumpChannel ()

La función *DumpChannel ()* se utiliza únicamente con fines de depuración. Esta función no recibe ningún argumento y guarda en un archivo de traza, C:\TEMP\VMCHANNEL.LOG, el Meta4Object que está activo cuando se llama a la función.

El archivo de traza se guarda en el subdirectorío temporal del puesto de trabajo cliente.

La función *DumpChannel* () no puede utilizarse para guardar registros a un archivo distinto del archivo de traza predeterminado.



Si se trabaja en entornos Unicode el formato del archivo resultante debe ser UTF-8.

DumpNode ()

La función *DumpNode* () se utiliza únicamente con fines de depuración. Esta función no recibe ningún argumento y guarda en un archivo de traza, *C:\TEMP\VMNODE.LOG*, el nodo que está activo cuando se llama a la función.

El archivo de traza se guarda en el subdirectorío temporal del puesto de trabajo cliente.

La función *DumpNode* () no puede utilizarse para guardar registros a un archivo distinto del archivo de traza predeterminado.



Si se trabaja en entornos Unicode el formato del archivo resultante debe ser UTF-8.

DumpRegister ()

La función *DumpRegister* () se utiliza únicamente con fines de depuración. Esta función no recibe ningún argumento y guarda en un archivo de traza, *C:\TEMP\VMREG.LOG*, el registro del nodo que está activo cuando se llama a la función.

El archivo de traza se guarda en el subdirectorío temporal del puesto de trabajo cliente.

La función *DumpRegister* () no puede utilizarse para guardar registros a un archivo distinto del archivo de traza predeterminado.



Si se trabaja en entornos Unicode el formato del archivo resultante debe ser UTF-8.

DumpStatistics (NombreArchivo, Meta4Object, [Nivel])

Esta función vuelca en un archivo estadísticas del Meta4Object (número de bloques, registros, nodos, etc). Sus argumentos son:

- **NombreArchivo:** contiene el nombre de archivo destino en el que se van a guardar las estadísticas que se obtengan del Meta4Object.
- **Meta4Object:** corresponde al nombre del Meta4Object cuyas estadísticas se van a elaborar.
- **Nivel:** este argumento es opcional e indica el nivel de profundidad en Meta4Objects invocados por llamadas de nivel 2.



Si se trabaja en entornos Unicode el formato del archivo resultante debe ser UTF-8.

EJEMPLO

```
DumpStatistics (  
    VALUE FIXED STRING FileName,  
    VALUE FIXED STRING Meta4Object,  
    [VALUE NUMBER Level]  
)
```

GetErrorString (Módulo, Submódulo, Código, [Argumento 1])

Esta función devuelve el texto asociado a un error, formateando la cadena indicada en función de los argumentos introducidos y del idioma en que esté.

Los argumentos correspondientes a la función son:

- **Módulo:** identifica el módulo de error o aviso. El valor de este argumento puede variar entre 0 y 255.
- **Submódulo:** identifica el submódulo de error o aviso. El valor de este argumento puede variar entre 0 y 255.
- **Código:** identifica el código de error o aviso. El valor de este argumento puede variar entre 0 y 65.536.
- **[Argumento 1]:** indica el primer argumento que se debe sustituir.

EJEMPLO

```
GetErrorString(  
    VALUE NUMBER Module,  
    VALUE NUMBER SubModule,  
    VALUE NUMBER Code,  
    [VALUE VARIANT Argument1]  
)
```

GetLog (Posición, &Tipo, &Módulo, &Submódulo, &Código, &TextoAdicional)

Esta función lee un mensaje de error del sistema de errores.

Los argumentos correspondientes a esta función son los siguientes:

- **Posición:** contiene la posición del mensaje. Se pueden dar dos situaciones:
 - Si la posición es positiva, adquiere valores desde el principio, es decir, los más recientes.
 - Si la posición es negativa, adquiere valores desde el final, es decir, los más antiguos.
- **&Tipo:** contiene por referencia el tipo de mensaje (de error o de aviso).
- **&Módulo:** identifica por referencia el módulo de error o aviso. El valor de este argumento puede variar entre 0 y 255.
- **&Submódulo:** identifica por referencia el submódulo de error o aviso. El valor de este argumento puede variar entre 0 y 255.
- **&Código:** identifica por referencia el código de error o aviso. El valor de este argumento puede variar entre 0 y 65.536.
- **&TextoAdicional:** contiene por referencia el mensaje de error definido en el Meta4Object de errores, según el tipo de error o aviso.

EJEMPLO

```
GetLog (  
    VALUE NUMBER Position,  
    REFERENCE NUMBER Type,  
    REFERENCE NUMBER Module,  
    REFERENCE NUMBER SubModule,  
    REFERENCE NUMBER Code,
```

```
REFERENCE VARIABLE STRING AdditionalText  
)
```

GetLogSize ()

Esta función devuelve el número de mensajes de error que están en la cola de errores en memoria.

PurgeLog (NumElimina)

Esta función elimina mensajes de error del sistema de errores, ya que están en memoria.

Se pueden dar dos situaciones:

- Si el valor es positivo, la función elimina empezando desde el principio, es decir, los errores más antiguos.
- Si el valor es negativo, la función elimina empezando desde el final, es decir, los errores más recientes.

EJEMPLO

```
PurgeLog (  
    VALUE NUMBER NumPurges  
)
```

SetLog (Tipo, Módulo, Submódulo, Código, [Argumento])

Esta función agrega un mensaje de tipo error o aviso al sistema log.

Los argumentos correspondientes a esta función son los siguientes:

- **Tipo:** contiene el tipo de mensaje. Puede ser de error, aviso o depuración.
- **Módulo:** identifica el módulo de error o aviso. El valor de este argumento puede variar entre 0 y 255.
- **Submódulo:** identifica el submódulo de error o aviso. El valor de este argumento puede variar entre 0 y 255.

- **Código:** identifica el código de error o aviso. El valor de este argumento puede variar entre 0 y 65.535.
- **Argumento:** es un argumento opcional y corresponde a los argumentos del mensaje de error definido en el Meta4Object de errores. Es una descripción textual según el tipo de error o aviso.

EJEMPLO

```
SetLog (  
    VALUE NUMBER Type,  
    VALUE NUMBER Module,  
    VALUE NUMBER SubModule,  
    VALUE VARIABLE STRING Code,  
    [VALUE VARIANT Argument]  
)
```

Funciones para transacciones de memoria

Las funciones utilizadas para las transacciones de memoria son:

- CheckPointBlock ()
- CheckPointChannel ()
- CheckPointNode ()
- CheckPointRegister ()
- UndoBlock ()
- UndoChannel ()
- UndoNode ()
- UndoRegister ()

CheckPointBlock ()

Esta función realiza transacciones en memoria e inicia otras nuevas a nivel de bloque.

Cuando se están realizando cambios en el Meta4Object se guarda el estado en el que estaba antes de llevar a cabo la última transacción. De este modo, si se ejecuta la función *CheckPointBlock*, los cambios realizados se guardan en el Meta4Object.

Si se realiza un *UndoBlock* se deshacen estos cambios. De este modo, el estado del Meta4Object es el mismo que después de ejecutar la función *CheckPointBlock* por última vez.

CheckPointChannel ()

Esta función realiza transacciones en memoria e inicia otras nuevas a nivel de Meta4Object.

Cuando se están realizando cambios en el Meta4Object, se guarda el estado en el que estaba antes de llevar a cabo la última transacción. De este modo, si se ejecuta la función *CheckPointChannel*, los cambios realizados se guardan en el Meta4Object.

Si se realiza un *UndoChannel* se deshacen estos cambios. De este modo, el estado del Meta4Object es el mismo que después de ejecutar la función *CheckPointChannel* por última vez.

CheckPointNode ()

Esta función realiza transacciones en memoria e inicia otras nuevas a nivel de nodo.

Cuando se están realizando cambios en el Meta4Object se guarda el estado en el que estaba antes de llevar a cabo la última transacción. De este modo, si se ejecuta la función *CheckPointNode*, los cambios realizados se guardan en el Meta4Object.

Si se realiza un *UndoNode*, se deshacen estos cambios. De este modo, el estado del Meta4Object es el mismo que después de ejecutar la función *CheckPointNode* por última vez.

CheckPointRegister ()

Esta función realiza transacciones en memoria e inicia otras nuevas a nivel de registro.

Cuando se están realizando cambios en el Meta4Object, se guarda el estado en el que estaba antes de llevar a cabo la última transacción. De este modo, si se ejecuta la función *CheckPointRegister*, los cambios realizados se guardan en el Meta4Object.

Si se realiza un *UndoBlock ()*, se deshacen estos cambios. De este modo, el estado del Meta4Object es el mismo que después de ejecutar la función *CheckPointRegister* por última vez.

UndoBlock ()

Esta función deshace los últimos cambios que se hayan realizado en el Meta4Object a nivel de bloque.

UndoChannel ()

Esta función deshace los últimos cambios que se hayan realizado en el Meta4Object a nivel de Meta4Object.

UndoNode ()

Esta función deshace los últimos cambios que se hayan realizado en el Meta4Object a nivel de nodo.

UndoRegister ()

Esta función deshace los últimos cambios que se hayan realizado en el Meta4Object a nivel de registro.

Funciones de lectura de metadatos

Estas funciones permiten obtener información de metadatos, y en todas ellas el argumento filtro se entiende como un criterio según el cual se establece una selección de elementos de un nodo.

Las funciones básicas de lectura de metadatos son:

- GetChannelId
- GetNodeData ([M4Obj] , [Node] , DataType , [DumpError] , ...)
- GetNodeIndex (Nodo)
- GetNodeItemArgument (IdArg, TipoArg, Prec, IsRefArg, ArgPos, Nodo, Posición, Filtro) o (IdArg, TipoArg, Prec, IsRefArg, ArgPos, Nodo, Posición, IDElemento)
- GetNodeItemArgumentNumber (NúmeroFijo, HasVarArg, NúmeroRef, Nodo, Posición, Filtro) o (NúmeroFijo, HasVarArg, NúmeroRef, Nodo, IDElemento)
- GetNodeItemArgumentPos (Nodo, Posición, Filtro, IDArg) o (Nodo, IDElemento, IDArg)

-
- GetNodeItemData([M4Obj], Nodo, Posición, Filtro, TipoDato) o ([M4Obj],
Nodo, IDElemento, TipoDato)
 - GetNodeItemId (Nodo, Posición, Filtro)
 - GetNodeItemIndex (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemInternalType (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemM4Type (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemPrecision (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemPrecisionEx (Precision, Nodo, Posición, Filtro) o (Precision,
Nodo, IDElemento)
 - GetNodeItemReadFieldId (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemReadObjectAlias (Nodo, Posición, Filtro) o (Nodo,
IDElemento)
 - GetNodeItemReadObjectId (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemScale (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemScope (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemSliceTotalize (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemType (Nodo, Posición, Filtro) o (Nodo, Elemento)
 - GetNodeItemWriteFieldId (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeItemWriteObjectAlias (Nodo, Posición, Filtro) o (Nodo,
IDElemento)
 - GetNodeItemWriteObjectId (Nodo, Posición, Filtro) o (Nodo, IDElemento)
 - GetNodeNumberOfItems (Nodo, Filtro)
 - GetNodeReadObjectId (NodeId o NodeIndex)
 - GetNodeWriteObjectId (NodeId o NodeIndex)
 - GetThisItemId
 - GetThisNodeId

GetChannelData ([M4Obj] , DataType , ...)

Esta función devuelve información de los metadatos de un Meta4Object. La función es de argumentos variables, y como mínimo hay que indicar el tipo de dato que se pide.

Los argumentos correspondientes a esta función son los siguientes:

- **M4Obj:** permite indicar un alias de Meta4Object para consultar metadatos de otro Meta4Object que no sea el actual. Este argumento es opcional y si no se especifica, la función opera sobre el Meta4Object actual.
- **DataType:** indica cual es el tipo solicitado de metadatos (obligatorio). Los tipos de datos admitidos vienen dados por las Macros de tipo de metadato de un Meta4Object. Son las siguientes:

M4_M4OBJECT_NODE_NUMBER
M4_M4OBJECT_CREATION_TYPE
M4_M4OBJECT_NAME
M4_M4OBJECT_ID
M4_M4OBJECT_INSTANCE_NUMBER
M4_M4OBJECT_INSTANCE_HANDLE

GetChannelId

Esta función devuelve el identificador del Meta4Object activo.

GetNodeData ([M4Obj] , [Node] , DataType , [DumpError] , ...)

Esta función devuelve la información relativa a los metadatos del nodo. El tipo de información se especifica a través del argumento DataType.

Los argumentos correspondientes a esta función son los siguientes:

- **M4Obj:** contiene el identificador de la instancia de nivel 2 de Meta4Object que se va a utilizar. Este argumento es opcional y si no se especifica, la función opera sobre el Meta4Object actual.
- **Node:** contiene el identificador del nodo que se va a utilizar, también se puede utilizar el índice de nodo.
- **DataType:** indica cual es el tipo de metadatos del nodo a solicitar. Los tipos de datos admitidos vienen dados por las Macros de tipo de metadato de un nodo. Son las siguientes:

M4_NODE_READ_OBJECT_ID
M4_NODE_WRITE_OBJECT_ID
M4_NODE_ID
M4_NODE_NAME
M4_NODE_INDEX
M4_NODE_IS_ROOT
M4_NODE_IS_REAL_ROOT
M4_NODE_PARENT

M4_NODE_SENTENCE

M4_NODE_MAX_NUM_ROWS_DB

M4_NODE_AUTO_FILTER

- **DumpError:** Indica si se vuelca error o no si el dato buscado no existe en el nodo.

GetNodeIndex (Nodo)

Esta función devuelve el índice asociado a un nodo.

EJEMPLO

```
GetNodeIndex (  
    VALUE VARIABLE STRING Node  
)
```

GetNodeItemArgument (IdArg, TipoArg, Prec, IsRefArg, ArgPos, Nodo, Posición, Filtro) o (IdArg, TipoArg, Prec, IsRefArg, ArgPos, Nodo, Posición, IDElemento)

Esta función devuelve la información sobre el argumento ubicado en la posición especificada en el argumento *ArgPos* del elemento.

Los argumentos correspondientes a esta función son los siguientes:

- **IDArg:** parámetro de salida que contiene el ID del argumento.
- **TipoArg:** parámetro de salida que indica el tipo correspondiente al argumento.
- **Prec:** parámetro de salida que corresponde a la precisión del elemento.
- **IDRefArg:** parámetro de salida que indica si el argumento es de entrada o salida.
- **ArgPos:** indica la posición del argumento indicado como argumento.
- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro.
- **IDElemento:** Identificador del elemento al que se hace referencia.

- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemArgumentPos (
    VALUE VARIABLE STRING Node,
    VALUE VARIANT Item,
    [VALUE NUMBER Filter,]
    VALUE VARIABLE STRING ArgId
)
```

GetNodeItemArgumentNumber (NúmeroFijo, HasVarArg, NúmeroRef, Nodo, Posición, Filtro) o (NúmeroFijo, HasVarArg, NúmeroRef, Nodo, IDElemento)

Esta función permite conocer el número de argumentos del elemento. Los argumentos propios de la función son los siguientes:

- **NúmeroFijo:** parámetro de salida que contiene el número de argumentos fijos de los que se consta.
- **HasVarArg :** parámetro de salida que indica si el elemento tiene un número variable de argumentos.
- **NúmeroRef:** parámetro de salida que indica el número de argumentos de entrada y salida.
- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro.
- **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemArgumentPos (
    VALUE VARIABLE STRING Node,
    VALUE VARIANT Item,
```

```
[VALUE NUMBER Filter,]  
VALUE VARIABLE STRING ArgId  
)
```

GetNodeItemArgumentPos (Nodo, Posición, Filtro, IDArg) o (Nodo, IDElemento, IDArg)

Esta función permite conocer la posición de un argumento correspondiente a un elemento. Sus argumentos son los siguientes:

- **Nodo**: contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento**: este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.
- **IDArg**: es el identificador del argumento del que se pide la posición. Este argumento es del elemento.

EJEMPLO

```
GetNodeItemArgumentPos (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter,]  
    VALUE VARIABLE STRING ArgId  
)
```

GetNodeItemData([M4Obj], Nodo, Posición, Filtro, TipoDato) o ([M4Obj], Nodo, IDElemento, TipoDato)

Esta función devuelve la información sobre el tipo de metadato solicitado.

La sintaxis admitida es:

```
GetNodeItemData([M4Obj], Nodo, Pos, Filtro, TipoDato)
GetNodeItemData([M4Obj], Nodo, IDElemento, TipoDato)
```

Los argumentos correspondientes a esta función son los siguientes:

- **M4Obj**: contiene el identificador de la instancia de nivel 2 de Meta4Object que se va a utilizar. Este argumento es opcional y si no se especifica la función opera sobre el Meta4Object actual.
- **Nodo**: contiene el identificador del nodo que se va a utilizar, también se puede utilizar el índice de nodo.

- **Elemento**

Este segundo argumento puede tomar uno de dos valores posibles:

- **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
- **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.
- **TipoDato**: indica cual es el tipo de metadatos a solicitar. Los tipos de datos admitidos vienen dados por las siguientes macros que no deben ir entre comillas:

M4_ITEM_TYPE

M4_ITEM_SCOPE

M4_ITEM_M4TYPE

M4_ITEM_PRECISION

M4_ITEM_INDEX

M4_ITEM_SCALE

M4_ITEM_INTERNAL_TYPE

M4_ITEM_READ_OBJECT_ALIAS

M4_ITEM_WRITE_OBJECT_ALIAS

M4_ITEM_READ_FIELD_ID

M4_ITEM_WRITE_FIELD_ID

M4_ITEM_READ_OBJECT_ID

M4_ITEM_WRITE_OBJECT_ID

M4_ITEM_SLICE_TOTALIZE

M4_ITEM_NAME

M4_ITEM_ORDER

M4_ITEM_SYNONYM

M4_ITEM_ID

M4_ITEM_NUMBER

M4_ITEM_IS_PK

EJEMPLO

Devolver el tipo de elemento de un elemento determinado:

```
...
type = GetNodeItemType( "MI_NODO", "MI_ELEMENTO")
type = GetNodeItemData( "MI_NODO", "MI_ELEMENTO", M4_ITEM_TYPE )
type = GetNodeItemData( "MI_M4OBJ", "MI_NODO", "MI_ELEMENTO",
M4_ITEM_TYPE )
...
```

GetNodeItemId (Nodo, Posición, Filtro)

Esta función devuelve el identificador del elemento del nodo que está en la posición indicada del filtro que se va a aplicar.

EJEMPLO

```
GetNodeItemId (
    VALUE VARIABLE STRING Node,
    VALUE NUMBER Position,
    VALUE NUMBER Filter
)
```

GetNodeItemIndex (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función devuelve el índice correspondiente a un elemento. En esta función se entiende por índice el ordinal que posee cada uno de los elementos en su nodo según el filtro aplicado.

Los argumentos correspondientes a esta función:

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.

- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemIndex (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

Si no existe el elemento en el nodo, la función devuelve un mensaje de aviso.

GetNodeItemInternalType (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función permite conocer el tipo interno de un determinado elemento, como por ejemplo de Fecha de inicio, Fecha de fin, etc. Sus argumentos son los siguientes:

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemInternalType (  
    VALUE VARIABLE STRING Node,
```

```
VALUE VARIANT Item,  
[VALUE NUMBER Filter]  
)
```

GetNodeItemM4Type (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función devuelve el tipo básico de un elemento de un nodo. Sus argumentos son los siguientes:

- **Nodo**: contiene el nombre del nodo que se va a utilizar.
- **Elemento**: este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemM4Type (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

Devuelve los siguientes posibles valores de tipos Meta4:

- 0: nulo
- 1: cadena fija
- 2: cadena variable
- 3: largo
- 4: fecha
- 5: timestamp

- 6: número
- 7: variante
- 8: moneda
- 9: número variante
- 10: blob
- 11: cadena binaria
- 12: tipo de hora

GetNodeItemPrecision (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función devuelve la precisión de un elemento. Sus argumentos son los siguientes:

- **Nodo**: contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento**: este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

Esta función tiene diferentes aplicaciones, por ejemplo:

- Si el elemento es alfanumérico, permite saber el número de caracteres que puede tener el elemento.
- Si el elemento es numérico, la precisión, junto con la escala, permite saber el número de decimales con los que se va a guardar ese número en la base de datos.

EJEMPLO

```
GetNodeItemPrecision (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

GetNodeItemPrecisionEx (Precision, Nodo, Posición, Filtro) o (Precision, Nodo, IDElemento)

Esta función devuelve por referencia la precisión de un elemento. Devuelve como resultado la constante M4_TRUE si se ejecuta correctamente, o M4_FALSE si se produce algún error.

Los argumentos correspondientes a esta función son los siguientes:

- **Precisión:** indica la precisión del elemento especificado. La precisión se puede utilizar para diferentes fines.
 - Si el elemento es alfanumérico, esta función permite saber el número de caracteres que puede tener el elemento.
 - Si el elemento es numérico, la precisión, junto con la escala, permite saber el número de decimales con los que se va a guardar ese número en la base de datos.
- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemPrecisionEx (  
    REFERENCE NUMBER Prec,  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

GetNodeItemReadFieldId (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función permite conocer el ID del campo de lectura de un elemento. Sus argumentos son los siguientes:

- **Nodo**: contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento**: este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemReadFieldId (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

GetNodeItemReadObjectAlias (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función permite conocer el alias del objeto de lectura de un elemento. Sus argumentos son los siguientes:

- **Nodo**: contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento**: este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemReadObjectAlias (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

GetNodeItemReadObjectId (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función permite conocer el ID del objeto de lectura de un elemento. Sus argumentos son los siguientes:

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro.
 - Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemReadObjectId (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

GetNodeItemScale (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función devuelve la escala correspondiente a un elemento. Este valor es muy importante y se le puede dar diferentes usos, por ejemplo, se puede utilizar

cuando se tenga un elemento numérico, ya que la escala y la precisión permiten saber el número de decimales con los que se va a guardar ese número en la base de datos.

Los argumentos correspondientes a esta función son:

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemScale (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

GetNodeItemScope (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función devuelve el ámbito de un elemento determinado. Sus argumentos son los siguientes:

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemScope (
    VALUE VARIABLE STRING Node,
    VALUE VARIANT Item,
    [VALUE NUMBER Filter]
)
```

Devuelve los siguientes posibles valores:

- 1: nodo
- 2: bloque
- 3: registro

GetNodeItemSliceTotalize (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función devuelve el total de tramos de un elemento determinado.

GetNodeItemType (Nodo, Posición, Filtro) o (Nodo, Elemento)

Esta función devuelve el tipo del elemento. Se le pueden indicar valores de dos formas:

- Identificador del nodo, posición y filtro del elemento
- Identificador del nodo y del elemento

Los argumentos correspondientes a esta función son los siguientes:

- **Nodo**: contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento**: este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo.

Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemType (
```

```

VALUE VARIABLE STRING Node,
VALUE VARIANT Item,
[VALUE NUMBER Filter]
)

```

Devuelve los siguientes posibles valores: método, concepto, campo y propiedad.

GetNodeItemWriteFieldId (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función permite conocer el ID del campo de escritura de un elemento. Sus argumentos son los siguientes:

- **Nodo**: contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento**: este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición**: Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento**: Identificador del elemento al que se hace referencia.
- **Filtro**: contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```

GetNodeItemWritefieldId (
  VALUE VARIABLE STRING Node,
  VALUE VARIANT Item,
  [VALUE NUMBER Filter]
)

```

GetNodeItemWriteObjectAlias (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función permite conocer el alias del objeto de escritura de un elemento.

Sus argumentos son los siguientes:

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemWriteObjectAlias (  
    VALUE VARIABLE STRING Node,  
    VALUE VARIANT Item,  
    [VALUE NUMBER Filter]  
)
```

GetNodeItemWriteObjectId (Nodo, Posición, Filtro) o (Nodo, IDElemento)

Esta función permite conocer el ID del objeto de escritura de un elemento. Sus argumentos son los siguientes:

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Elemento:** este segundo argumento puede tomar uno de dos valores posibles:
 - **Posición:** Posición del elemento al que se hace referencia, en cuyo caso es necesario que se indique un filtro en el siguiente argumento.
 - **IDElemento:** Identificador del elemento al que se hace referencia.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo. Este argumento es obligatorio únicamente cuando se indique la posición del elemento al que se hace referencia.

EJEMPLO

```
GetNodeItemWriteObjectId (  
    VALUE VARIABLE STRING Node,
```

```
REFERENCE VARIANT Item,
[VALUE NUMBER Filter]
)
```

GetNodeNumberOfItems (Nodo, Filtro)

Esta función devuelve el número de elementos que existen en el nodo que se pasa como primer argumento.

EJEMPLO

```
GetNodeNumberOfItems (
    VALUE VARIABLE STRING Node,
    VALUE NUMBER Filter
)
```

El filtro que hay que aplicar se usa para visualizar los elementos de un ámbito específico y de un tipo específico, o de una combinación de los dos.

- **Nodo:** contiene el identificador del nodo o el índice que se va a utilizar.
- **Filtro:** contiene el filtro que se va a aplicar sobre el nodo.

Para esto, use los macros de tipo y de ámbito disponibles. Los posibles valores para el filtro son los siguientes:

- M4 SCOPE_ALL: Elementos de todos los ámbitos
- M4 SCOPE_REGISTER: Elementos de registro (solamente)
- M4 SCOPE_BLOCK: Elementos de bloque (solamente)
- M4 SCOPE_NODE: Elementos de nodo (solamente)
- M4 TYPE_ALL: Todos los tipos de elemento
- M4 TYPE_METHOD: Elementos de tipo método
- M4 TYPE_PROPERTY: Elementos de tipo propiedad
- M4 TYPE_FIELD: Elementos de tipo campo
- M4 TYPE_CONCEPT: Elementos de tipo concepto

Puede aplicar uno o más criterios para el filtro a través del uso del operador booleano OR.

EJEMPLO

Para recuperar sólo los métodos cuyo ámbito sea Bloque del nodo llamado minodo:

```
GetNodeNumberOfItems( "minodo", M4_SCOPE_BLOCK + M4_TYPE_METHOD )
```

GetNodeReadObjectId (NodeId o NodeIndex)

Esta función devuelve el identificador del objeto lógico de lectura de nodo.

GetNodeWriteObjectId (NodeId o NodeIndex)

Esta función retorna el identificador del objeto lógico de escritura de nodo.

GetThisItemId

Esta función devuelve el identificador del método que se está ejecutando.

GetThisNodeId

Esta función devuelve el identificador del nodo activo.

Funciones de nómina y del sistema de asignación de valores

Las funciones que se engloban dentro de este apartado ofrecen facilidades específicas a la hora de realizar cálculos de nómina:

- ApplyTable (NodoValores, NodoHistórico, NodoOrganización, [HistóricoCP, Valor sCP], iNumCP, [ApplyAlways])
- YTDSearchFiltered (Meta4Object, Nodo, FechaInicio, FechaFin, FechaEntrada, FechaPago, PagoFrec, TipoPago, TipoBúsqueda, RevBeh, SelPays, TotOp, [IDElemento, Valor, Función])

Además, existe una serie de funciones del sistema de asignación de valores que se utiliza mucho en la nómina.

- TransferApplyValue (Host Meta4Object, id, Host_nodo_id, [DMD, MonedaDestino, FechaIntercambio, TipoIntercambio])

- PrSetDates (FechaInicio, FechaFin, [FechaEjecución])
- PrSetDates (FechaInicio, FechaFin, [FechaEjecución])
- ApplyTable (NodoValores, NodoHistórico, NodoOrganización, [HistóricoCP, Valor sCP], iNumCP, [ApplyAlways])
- TransferApplyValue (Host, Meta4Object, id, Host_nodo_id, [DMD, MonedaDestino, FechaIntercambio, TipoIntercambio])

Consulte Sistema de asignación de valores para más información acerca del sistema y la forma de trabajar con las funciones indicadas.

Sistema de asignación de valores

Desde la herramienta Diseñador de modelo de datos, se puede gestionar un tipo especial de tablas: las tablas de valor. La función principal de estas tablas es la asignación de determinados valores asociados a conceptos.



Las tablas de valor son tablas fechadas, en las que todo se guarda como si se tratara de un histórico. Al igual que en los históricos, estas tablas tienen unas columnas fijas y otros que dependerán del nivel.

Los conceptos se pueden agrupar en función de la tipología del concepto, o la creación, función o ejecución del mismo. Por ejemplo: puede agrupar en Devengos todos los conceptos definidos como devengos genéricos que percibirá el empleado: salario base, dietas, plus de horas extras, etc.

Un dominio de metadatos (o DM) es, dentro de una jerarquía, el nivel superior en el que se agrupan los conceptos propios de aquellos módulos en los que es imprescindible la asignación de un valor para su ejecución.

La principal aplicación de los DM se encuentra relacionada con el cálculo de la nómina, aunque se extiende su funcionalidad a otros procesos dentro de la gestión de los recursos humanos.

Con idea de garantizar al máximo la integridad en el cálculo de la nómina y la confidencialidad en otros procesos de la gestión propia de los recursos humanos, la tecnología base de Meta4 PeopleNet estructura un subsistema de seguridad basado en la creación de grupos de usuarios con permisos para modificar el valor asignado a los conceptos de nómina. Desde la tabla Grupos DMD, se van a definir los posibles grupos que va a tener asociado un DM, es decir, los permisos sobre los DM.



No olvide que al adscribir un DM a un grupo, el DM heredar  todos los permisos concedidos al grupo.

El sistema de asignaci n de valores es un proceso que consta de una serie de funciones, cuyo orden l gico de ejecuci n es el siguiente:

- TransferApplyValue (Host_Meta4Object, id, Host_nodo_id, [DMD, MonedaDestino, FechaIntercambio, TipoIntercambio])
- PrSetDates (FechaInicio, FechaFin, [FechaEjecuci n])
- PrSetDates (FechaInicio, FechaFin, [FechaEjecuci n])
- ApplyTable (NodoValores, NodoHist rico, NodoOrganizaci n, [Hist ricoCP, Valor sCP], iNumCP, [ApplyAlways])
- TransferApplyValue (Host_Meta4Object, id, Host_nodo_id, [DMD, MonedaDestino, FechaIntercambio, TipoIntercambio])

La transferencia se produce entre distintos Meta4Objects, pudiendo realizarse tambi n en el mismo Meta4Object, aunque su fin no sea  ste.

Con todas las funciones citadas, se realiza el proceso del sistema de asignaci n de valores. Todas las funciones, salvo *TransferApplyValue*, realizan la fase 1, y cuando por  ltimo se ejecuta  sta, se completa la fase 2.

En la fase 1 se utiliza un Meta4Object especial que consta de tablas, hist ricos, organigramas, informaci n sobre el empleado, informaci n sobre la empresa, etc. En esta fase, se leen tablas de valores, como el salario o las ausencias, que hayan ido incluyendo los usuarios. Esta informaci n queda depositada en un nodo intermedio.



El pivotado de informaci n consiste en cambiar la distribuci n de la informaci n, de forma que en vez de aparecer en filas, los valores aparezcan en columnas.

- En la fase 2 se toma esta informaci n, se pivota, se transfiere a los nodos seg n la relaci n de dominio de metadatos y componentes del dominio de metadatos que exista y, por  ltimo, se deposita en un registro correspondiente al nodo de c lculo.

ApplyTable (NodoValores, NodoHist rico, NodoOrganizaci n, [Hist ricoCP, Valor sCP], iNumCP, [ApplyAlways])

Esta funci n tiene como  mbito de aplicaci n las tablas, y ofrece diversos modos de ejecuci n:

- Sólo se indica el valor de las tablas de valores que son aplicables por fechas, DM, etc.
- Se indica también la tabla de históricos. Ésta aplica valores según las fechas del histórico. Para ello, se utiliza la relación de claves primarias, ya que de este modo en la tabla de históricos un elemento tendrá un valor igual al de la tabla de valores.

Por ejemplo, el valor del elemento "departamento" de la tabla de histórico tiene que ser igual al valor del elemento "Pertenece al dpto" de la tabla de valores.

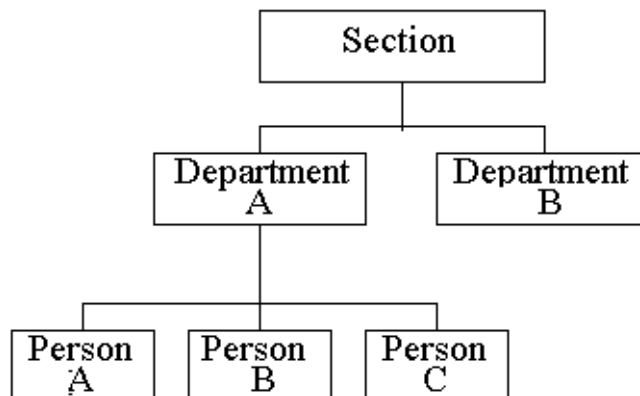
Si se cumple esta condición, siempre comparando valores, significa que el registro de la tabla de valores es aplicable.

- La última posibilidad es por organigrama; en el Meta4Object del sistema de asignación de valores hay un nodo especial que contiene el organigrama, por ejemplo, un organigrama de la empresa.

El sistema de asignación de valores actúa en función de este organigrama, de tal forma que se conecta a un nodo del organigrama, en el que intenta aplicar el valor en orden ascendente. A la hora de aplicar valores, tienen preferencia aquellos elementos más inferiores en la jerarquía, ascendiendo hasta el nodo raíz.

EJEMPLO

Imagine que tiene el siguiente organigrama:



El objetivo de este ejemplo es cambiar el salario base del individuo A que trabaja en el departamento A correspondiente a una sección.

Para llevar a cabo esta operación, el proceso necesario es el siguiente:

- Acceder a la tabla de históricos y comprobar que el individuo A existe, al igual que sucede en el organigrama de la organización.
- Acceder a la tabla de valores y ver el salario que tiene asignado el individuo A. Aquí se pueden dar dos situaciones:

-
- El individuo A existe en la tabla de valores, por lo que se procede a cambiar el salario base sustituyendo el valor existente por el nuevo.
 - El individuo A no existe en la tabla de valores, por lo que habría que ver en el organigrama de la organización el elemento del que depende, en este caso el Departamento A, y tener acceso de nuevo a la tabla de valores para consultar el salario que le corresponde de forma general a este departamento. En caso de no existir tampoco en la tabla de valores, hay que encontrar un elemento en el organigrama del que dependa el Individuo A que tenga su correspondencia en la tabla de valores, momento en el cual se procederá a cambiar el salario base por el actual.

Los argumentos correspondientes a esta función son los siguientes:

- **NodoValores:** indica el nodo de la tabla de valores.
- **NodoHistórico:** indica el nodo de la tabla de históricos. Puede ser una cadena vacía si no se están utilizando nodos históricos y organigramas.
- **NodoOrganización:** indica el nodo de la tabla de organigrama. Puede ser una cadena vacía si no se están utilizando nodos históricos y organigramas.
- **HistóricoCP:** este argumento opcional indica el nombre de un elemento en la tabla de históricos.
- **Valor sCP:** este argumento opcional indica el nombre de un elemento en la tabla de valores.
- **iNumCP:** indica el número de pares de claves primarias que se indican.
- **ApplyAlways:** cuando se va a aplicar un registro que ya existe, este argumento opcional indica si se añade uno nuevo para no eliminar el valor antiguo o si, por el contrario, en lugar de crear uno nuevo, se eliminan los valores que hubiera. Sus posibles valores son:
 - TRUE: aunque se vaya a aplicar un registro que ya existe se añade uno nuevo y no se machaca el valor antiguo.
 - FALSE (valor por defecto): si el registro ya existe no se crea uno nuevo, sino que se machacan los valores antiguos.

EJEMPLO

```
ApplyTable (  
    VALUE VARIABLE STRING ValuesNode, VALUE VARIABLE STRING  
    HistoricNode, VALUE VARIABLE STRING OrgNode, [VALUE VARIABLE STRING  
    HistoricPK,] [VALUE VARIABLE STRING ValuesPK,] VALUE NUMBER iNumPK  
)
```

CreateOrgChart ()

Esta función permite preparar un determinado organigrama para trabajar con él. De esta forma, es más fácil comprobar que no existen ciclos y trabajar con tramos.

Esta función se tiene que ejecutar sobre el nodo que se especifique como correspondiente al del organigrama.

ExecuteGroup (Nombre de grupo)

Esta función ejecuta un grupo de conceptos en el orden lógico en el que deben hacerlo. Cuando se define un elemento, se le puede asignar un grupo determinado con un identificador único. En un grupo no deben existir bucles, ya que la función *ExecuteGroup ()* no acabaría nunca su ejecución.

EJEMPLO

```
ExecuteGroup (
    VALUE VARIABLE STRING GroupName
)
```

PrInitApplyValue (Id_DM, Grupo)

La tabla *DMD_components* contiene todos los componentes que se pueden aplicar utilizando esta función. Cualquier otro componente que exista y que no aparezca en esta lista, no se podrá aplicar con esta función.

Con esta función se indica con qué dominio de metadatos y con qué grupo DM se va a trabajar, y se asignan valores en la tabla. Posteriormente, se fuerza la carga de la tabla. Si el filtro es acorde con las fechas, la tabla queda filtrada por el grupo DM indicado.



El grupo DM es una agrupación de conceptos dentro del dominio de metadatos.

Los argumentos correspondientes a esta función son el identificador de la tabla

DM que va a estar formada por una serie de conceptos, y el identificador del grupo de conceptos que forman parte del DM.

EJEMPLO

```
prInitApplyValue (  
    VALUE VARIABLE STRING IdDMD, VALUE VARIABLE STRING Group  
)
```

PrSetDates (FechaInicio, FechaFin, [FechaEjecución])

Esta función permite indicar las fechas de ejecución (fechas de inicio y fin del tramo) para un individuo concreto cuya nómina se quiere calcular, así como la fecha de corte o de ejecución (esta última es un argumento opcional).

La función se puede utilizar para filtrar tablas. Se utilizan las fechas en el nodo intermedio para determinar el periodo con el que se va a trabajar.

EJEMPLO

```
prSetDates (  
    VALUE DATE AND TIME StartDate, VALUE DATE AND TIME EndDate, [VALUE  
    DATE AND TIME RunDate]  
)
```

PrSetPeriods (FechaInicio, FechaFin, FechaEjecución, ModoTramo)

Esta función permite indicar las fechas de ejecución del sistema de asignación de valores.

Los argumentos que recibe son las fechas de inicio y fin del tramo, la fecha de ejecución o corte y el modo de tramo que se va a utilizar.

EJEMPLO

```
prSetPeriods (  
    VALUE DATE AND TIME StartDate, VALUE DATE AND TIME EndDate, VALUE  
    DATE AND TIME WorkDate, VALUE NUMBER SliceMode
```

```
)
```

ResetexecuteGroup()

Esta función recibe como argumento un grupo de ejecución y recorre todos los elementos (conceptos) de ese grupo poniendo su prioridad de ejecución al valor mínimo (3). Normalmente se emplea en cálculos de nómina con retroactividad. En este tipo de cálculos se realiza el cálculo de los conceptos de nómina más de una vez por empleado, y de una vez a otra se llama a esta función para dejar los conceptos en un estado “limpio”.

EJEMPLO

```
ResetExecuteGroup( "3" )
```

TransferApplyValue (Host_Meta4Object, id, Host_nodo_id, [DMD, MonedaDestino, FechaIntercambio, TipoIntercambio])

Esta función recibe el nombre de fase 2 del sistema de asignación de valores. Puede ejecutarse con o sin tramos, dependiendo de cómo se esté ejecutando la nómina. Se puede asignar prioridades a cada uno de los elementos.

EJEMPLO

```
TransferApplyValue (
    VALUE VARIABLE STRING HostM4ObjetId, VALUE VARIABLE STRING
    HostNodeId, [VALUE VARIABLE STRING DMD,] [VALUE VARIABLE STRING
    DestCurrency,] [VALUE DATE ExchangeDate,] [VALUE VARIABLE STRING
    ExchangeType]
)
```

YTDSearch (Meta4Object, Nodo, FechaInicio, FechaFin, FechaEntrada, FechaPago, PagoFrec, TipoPago, TipoBúsqueda, RevBeh, SelPays, TotOp)

Esta función calcula el acumulado de una estructura de nodo de amplias dimensiones que esté cargada en memoria y que conste de todos los datos correspondientes al acumulado.

La función va leyendo la información nodo a nodo, rellenando varias estructuras de nodo pequeñas con datos que cumplan un criterio específico.

Esta función no está implementada para llamadas de nivel 2, es decir, no es posible tener acceso a otros Meta4Objects distintos al actual.

EJEMPLO

```
YTDSearch (  
    VALUE VARIABLE STRING Meta4Object, VALUE VARIABLE STRING Node,  
    VALUE DATE AND TIME StartDate, VALUE DATE AND TIME EndDate, VALUE  
    DATE AND TIME ImputeDate, VALUE DATE AND TIME PayDate, VALUE  
    VARIABLE STRING PayFrequency, VALUE VARIABLE STRING PayType, VALUE  
    NUMBER SearchType, VALUE NUMBER RevBehaviour, VALUE NUMBER SelPays,  
    VALUE NUMBER TotOperation  
)
```

YTDSearchFiltered (Meta4Object, Nodo, FechaInicio, FechaFin, FechaEntrada, FechaPago, PagoFrec, TipoPago, TipoBúsqueda, RevBeh, SelPays, TotOp, [IDElemento, Valor, Función])

Esta función filtra las búsquedas en la estructura de nodo origen, por criterios de fecha, tales como periodo. Además, calcula el acumulado de una estructura de nodo leyendo los nodos de uno en uno.

Se puede utilizar con condiciones adicionales sobre el nodo, seleccionando aquellos registros que están en el bloque activo, del mismo modo que la función *FindRegister* (elemento, valor, función...).

También permite especificar condiciones adicionales que debe cumplir el registro para que se le tenga en cuenta cuando se ejecute el algoritmo *YTD Search*.

Para obtener información adicional, haga clic en uno de los siguientes apartados:

- Especificación del filtro
- Ejecución con tramos
- Diferentes comportamientos con distintas llamadas

Especificación del filtro

El usuario puede indicar argumentos en grupos de tres, como en la función *FindRegister* (), mediante los que se especifica el *ID_Elemento*, o valor que se usa para comparar con la función en uso.

Al utilizar filtros, sólo se seleccionan los registros que cumplan las condiciones especificadas, que podrían ser hasta 20 distintas.

Ejecución con tramos

Si se va a llevar a cabo una ejecución con tramos, en el filtro que se utilice se puede indicar el valor de uno de ellos. De este modo, se harán varias llamadas a la función *YTDSearchFiltered*, una por tramo.

Por tanto, en el nodo destino se dispondrá de los resultados requeridos para ver diferentes valores de un elemento a lo largo del tiempo.

EJEMPLO

El elemento `node.filter_item` tiene dos tramos y se llama a la función del siguiente modo:

```
YTDSearchFiltered(..., "MY_ITEM", node.filter_item, EQUAL)
```

Se llamará a las funciones dos veces, como es normal en una ejecución con tramos. En el nodo destino se recogerán los resultados según el formato que se ha especificado.

Diferentes comportamientos con distintas llamadas

En la función existen varios argumentos opcionales [*IDElemento*, *Valor*, *Función*], en los que se debe especificar cómo se desea que aparezcan los resultados en el nodo destino.

Si se indica este argumento, se deberá hacer en la última posición. De lo contrario, sólo se guardará un registro.

Funciones para estadísticas

Las funciones para estadísticas son:

-
- SttGetLevel ()
 - SttPersistToFile (NombreArchivo)
 - SttSetLabel (Etiqueta)
 - SttSetLevel (Nivel)

SttGetLevel ()

Esta función permite obtener el nivel de las estadísticas (Stt) activas. El nivel de las estadísticas es una máscara entera donde cada bit corresponde a una opción estadística:

- 0: Sin Stt
- 1: Stt de elementos (bit 1)
- 2: Stt de tiempos de elementos (bit 2)
- 4: Stt de tiempos de base de datos (bit 3)
- 8: Stt de sentencias SQL (bit 4)
- 16: Stt de los valores (bit 5)

SttPersistToFile (NombreArchivo)

Esta función guarda todas las estadísticas del Meta4Object en el archivo especificado, siempre que las estadísticas estén activas (Nivel = 0).

EJEMPLO

```
sttPersistToFile (  
    VALUE VARIABLE STRING FileName  
)
```

SttSetLabel (Etiqueta)

Esta función asigna una etiqueta a las estadísticas activas y todas sus dependencias.

SttSetLevel (Nivel)

Esta función permite establecer el nivel de las estadísticas (Stt) activas. El nivel de las estadísticas es una máscara entera donde cada bit corresponde a una opción estadística:

- 0: Sin Stt
- 1: Stt de elementos (bit 1)
- 2: Stt de tiempos de elementos (bit 2)
- 4: Stt de tiempos de base de datos (bit 3)
- 8: Stt deSQL (bit 4)
- 16: Valores (bit 5)

Funciones avanzadas

Las funciones avanzadas son:

- CrossTab (IDMeta4Object, IDHost, Columna1, [columna2, ...], [Índice])
- LookUp (PCP1, PCP2...ES1, SCP1, SI1, SCP2..., NúmeroSecundario, ElementoFiltroaAplicar, NodoUsuario, FechaInicioApl, FechaFinApl, FechaInicioCorr, FechaFinCorr)
- Pivot (IDMeta4Object, IDHost, DominioDestino, [Índice])

CrossTab (IDMeta4Object, IDHost, Columna1, [columna2, ...], [Índice])

Esta función permite crear una tabla de referencias cruzadas de un nodo de un Meta4Object a otro nodo del mismo Meta4Object o de otro Meta4Object.

Utiliza la información del componente de la tabla de referencias cruzadas del dominio para realizar el ajuste real.

LookUp (PCP1, PCP2...ES1, SCP1, SI1, SCP2..., NúmeroSecundario, ElementoFiltroaAplicar, NodoUsuario, FechaInicioApl, FechaFinApl, FechaInicioCorr, FechaFinCorr)

Esta función está orientada a proporcionar un servicio funcional, consistente en filtrar un nodo. Se la puede llamar desde las funciones *LIST* y *Validate*, que son métodos generados con la herramienta Diseñador del modelo de datos, mediante los cuales se pueden filtrar nodos.

Sus argumentos correspondientes son:

- **PCPn**: es un argumento opcional, y contiene la clave primaria que permite conocer el contenido de los elementos primarios.
- **ESn**: es un argumento opcional, que permite conocer el contenido de los elementos secundarios. Siempre se indica en números pares, ya que debe contener el identificador del elemento y el contenido por el que se va a buscar. En este caso, el argumento *SIn* contiene el identificador del elemento.
- **SCPn**: este argumento opcional indica el contenido del elemento por el que se va a buscar. El identificador del elemento se indica en el argumento *SIn*.
- **NúmeroSecundario**: este argumento indica el número de parejas de elementos secundarios que existen. No es necesario indicar el número de elementos primarios, ya que se obtiene de los secundarios.
- **ElementoFiltroaAplicar**: este argumento es obligatorio y de tipo cadena; en caso de no ser así, generaría un error. Contiene el identificador del elemento que se quiere invocar, el cual permite generar la sentencia SQL que se pretende ejecutar sobre el *Meta4Object*.

Este valor constituye realmente un método al cual se invoca. Su comportamiento varía dependiendo de si el *Meta4Object* se puede almacenar en la caché.

- **NodoUsuario**: este argumento es el identificador de un nodo; sirve para agregar una funcionalidad adicional a la función *ElementoFiltroaAplicar*, por ejemplo, incluir un filtro adicional.

En este caso, el argumento indica el identificador del nodo donde va a estar el elemento que calcula ese filtro adicional.

El identificador del elemento que se va a invocar debe ser *APPLY_FILTER*. Este argumento puede tener como valor nulo o "", en cuyo caso no se tendría en cuenta.

- **FechaInicioApl**: este argumento filtra datos utilizando como criterio la fecha de inicio. Siempre aparece, aunque su valor pueda ser nulo, "" o cualquier otro dato que no sea fecha, en cuyo caso no se tendría en cuenta.
- **FechaFinApl**: este argumento filtra datos por fecha de fin y siempre aparece, aunque su valor puede ser nulo, "" o cualquier otro dato que no sea fecha, en cuyo caso no se tendría en cuenta.
- **FechaInicioCorr**: este argumento filtra datos por fecha de inicio de corrección y siempre aparece, aunque su valor puede ser nulo, "" o cualquier otro dato que no sea fecha, en cuyo caso no se tendría en cuenta.
- **FechaFinCorr**: este argumento filtra datos por fecha de fin de corrección y siempre aparece, aunque su valor puede ser nulo, "" o cualquier otro dato que no sea fecha, en cuyo caso no se tendría en cuenta.

Puede obtener información adicional sobre los siguientes temas:

- Aplicación de fechas
- Filtro en RAM
- Tratamiento del argumento ElementoFiltroaAplicar
- Tratamiento del elemento definido por el usuario
- Funcionalidad añadida de LookUp

Aplicación de fechas

Existen dos modos de aplicación:

- Si el Meta4Object no se puede almacenar en la memoria caché, se comprueba si existen elementos marcados con un tipo interno determinado de filtro de fecha. Si es así, se rellenan las fechas, siempre que existan, con los valores que aparecen en los argumentos *FechaInicioApl*, *FechaFinApl*, *FechaInicioCorr* y *FechaFinCorr*, para que posteriormente la base de datos lógica los aplique en las sentencias que ejecute.
- Si el Meta4Object se puede almacenar en la caché, se compone un filtro en RAM similar a los filtros que aplicaría la BDL.

Filtro en RAM

Independientemente de si el Meta4Object se puede almacenar en la caché o no, se construye una parte del filtro en RAM. Esta parte del filtro es para la parte *LIKE*, es decir, se toman los argumentos secundarios indicados y se aplica un filtro en RAM para cada uno de ellos.

Para cada pareja de secundarios que exista, se construye una parte del filtro en la que se concatenan mediante el operador booleano AND.

Tratamiento del argumento ElementoFiltroaAplicar

Posteriormente, se invoca al elemento contenido en *ElementoFiltroaAplicar*, que actuará de una forma u otra en función de si el Meta4Object se puede almacenar en la caché.

A *ElementoFiltroaAplicar* se le asignan los argumentos primarios y un indicador que determina si el elemento se puede almacenar en la caché:

- 0: indica que el elemento no se puede almacenar en la caché.
- 1: indica que el elemento se puede almacenar en la caché.

La función *ElementoFiltroaAplicar* aplica un filtro al nodo donde se invoca. Si el Meta4Object no se puede almacenar en la caché, genera un filtro SQL con el que se rellena el *SysSentence*. Si se puede almacenar en la caché, el método crea un filtro en RAM, parecido al SQL, no rellena el *SysSentence* y devuelve como resultado de su ejecución el filtro en RAM que se aplicará posteriormente.

SysSentence es un elemento especial que permite sobrescribir una sentencia determinada asociada a una estructura de nodo o agregar un nuevo filtro que se concatena a la sentencia principal de la estructura de nodo.

Tratamiento del elemento definido por el usuario

Como siguiente paso se invoca el elemento definido por el usuario. Si el nodo elegido por el usuario hace las mismas cosas que el *ElementoFiltroaAplicar*, se indicaría aquí. Se define en el argumento *NodoUsuario*.

Por último, se indicaría si el *Meta4Object* se puede almacenar en la caché o no. Es un método que no va a tener más que un argumento que puede ser del siguiente tipo:

- 0: indica que el *Meta4Object* no se puede almacenar en la caché.
- 1: indica que el *Meta4Object* se puede almacenar en la caché.

Con ese método, si el *Meta4Object* no se puede almacenar en la caché, el usuario tendrá que aplicar un filtro o sino devolver un filtro en RAM para que posteriormente se aplique.

Funcionalidad añadida de LookUp

La función *LookUp* concatena con el operador AND todas las operaciones anteriores. Si el *Meta4Object* se pueda almacenar en la caché, habrá varios AND y si no se puede almacenar en la caché, habrá uno que será el LIKE, que permitirá buscar una cadena concreta indicada como argumento.

Por último, se procede a realizar la carga del bloque donde se ha invocado la función *LookUp*.

Si el *Meta4Object* no se puede almacenar en la caché, se invoca al elemento *Load_prg*, con lo cual se desencadena la carga del bloque de hijos filtrado por los filtros SQL que hayan podido aplicar los elementos *APPLY_FILTER*.

Si el *Meta4Object* se puede almacenar en la caché, la función comprueba si la información está cargada en memoria.

Si no lo está, se llama a la función *Load ()* del *Meta4Object*. Después de estas operaciones, el *Meta4Object* queda cargado de un determinado modo.

Por último, se aplican dos filtros en RAM:

- Filtro en RAM del LIKE, para realizar búsquedas literales de un patrón indicado.
- Resto de filtros, sólo si el *Meta4Object* se almacena en la caché.

Como resultado final, la función *LookUp* devuelve el número de registros que quedan después de aplicar los filtros.

La función tiene dos excepciones:

- Si el método al que tenemos que invocar con *Apply_list* y el *Meta4Object* no se puede almacenar en la caché, no se invoca el método.
- La función borra los filtros de anteriores ejecuciones del *LookUp* antes de aplicar los filtros nuevos.

EJEMPLO

```
LookUp (
    VALUE VARIABLE STRING PrimaryPK1,
    VALUE VARIABLE STRING SecondaryPK,
    VALUE VARIABLE STRING SecondaryValue,
    VALUE NUMBER SecondaryNumber,
    VALUE VARIABLE STRING ApplyFiterItem,
    VALUE VARIABLE STRING UserNode,
    VALUE DATE AND TIME AppStartDate,
    VALUE DATE AND TIME AppEndDate,
    VALUE DATE AND TIME CorrStartDate,
    VALUE DATE AND TIME CorrEndDate
)
```

A continuación se detallan dos ejemplos de la función *LookUp*:

- EJEMPLO 1
- EJEMPLO 2

EJEMPLO 1

En este ejemplo se utiliza el método *LIST*, que invoca a la función *LookUp*. Los argumentos que se indican son los siguientes:

- **ARG_ID_OBJECT**: indica el valor de la clave primaria. El método *LIST* dispone además de otro argumento que no se muestra en este ejemplo y que se utiliza para indicar el valor de la clave primaria secundaria.
- **ID_Índice**: contiene el identificador de la columna en la que se va a realizar la búsqueda.
- **ARG_ID_INDEX**: indica el valor de la cadena buscada.
- **APPLY_LIST**: indica el método que se invoca desde la función *LookUp*.
- **GetArgument (1)**: indica el nodo activo del usuario.
- **GetArgument (2)**: indica la fecha de inicio.

- **GetArgument (3):** contiene la fecha de fin.

El código correspondiente de este ejemplo es:

```
'LIST
iReturn = LookUp(ARG_ID_OBJECT, "ID_INDEX", ARG_ID_INDEX, 1,
"APPLY_LIST", GetArgument(1), GetArgument(2), GetArgument(3), "", "")
RETURN iReturn
```

EJEMPLO 2

En este ejemplo, el método *APPLY_FILTER_LIST* invoca la función *LookUp*.

En la primera parte del código, hasta la sentencia *ELSE* se crea un filtro RAM, y a partir de esa sentencia, se ejecuta la sentencia *SELECT* de SQL.

Ambas operaciones se realizan sobre el Meta4Object.

```
'APPLY_FILTER_LIST
iLastArg = GetArgumentNumber()
sSQLParam = ""
sArgParam = ""
sRAMParam = ""
sEmpty = ""
sFROM = "FROM &SDD_INDEX SDD_INDEX "
If GetArgument(iLastArg) = TRUE Then
  Template
  If IsNull(ARG) <> TRUE And Arg <> sEmpty Then
    sValue = "ITEM = " + Chr({HoldChr1}) + ToString(ARG, 0) +
    Chr({HoldChr2})
    If sRAMParam <> sEmpty Then
      sRAMParam = " AND " + sRAMParam
    EndIf
    sRAMParam = sRAMParam + sValue
  EndIf
  If IsNull(ARG_ID_OBJECT) <> TRUE And ARG_ID_OBJECT <> sEmpty Then
    sValue = "ID_OBJECT = " + CHR(34) + ToString(ARG_ID_OBJECT, 0) +
    CHR(34)
    If sRAMParam <> sEmpty Then
      sRAMParam = sRAMParam + " And "
    EndIf
    sRAMParam = sRAMParam + sValue
  EndIf
  sReturn = sRAMParam
Else
```

```

Template
If IsNull(ARG) <> TRUE And Arg <> sEmpty Then
    sValue = "ALIAS.FIELD = ?(type, prec, scale)"
    If sSQLParam <> sEmpty Then
        sSQLParam = " AND " + sSQLParam
    EndIf
    sSQLParam = sSQLParam + sValue
    If sArgParam <> sEmpty Then
        sArgParam = "|" + sArgParam
    EndIf
    sArgParam = sArgParam + ToString(ARG, 0)
EndIf
If IsNull(ARG_ID_OBJECT) <> TRUE And ARG_ID_OBJECT <> sEmpty Then
    sValue = "SDD_INDEX.ID_OBJECT= ?(2, 30, 0)"
    If sSQLParam <> sEmpty Then
        sSQLParam = sSQLParam + " And "
    EndIf
    sSQLParam = sSQLParam + sValue
    If sArgParam <> sEmpty Then
        sArgParam = sArgParam + "|"
    EndIf
    sArgParam = sArgParam + ToString(ARG_ID_OBJECT, 0)
EndIf
If sSQLParam <> "" Then
    sSQLParam = " WHERE " + sSQLParam
EndIf
SYS_SENTENCE_A = sFROM + sSQLParam
SYS_PARAM_A = sArgParam
sReturn = ""
EndIf
RETURN sReturn

```

Pivot (IDMeta4Object, IDHost, DominioDestino, [Índice])

Esta función pivota un Meta4Object a través de un dominio de metadatos. La acción de pivotado consiste en cambiar las filas por columnas.

Los metadatos correspondientes a cada uno de los campos tienen una propiedad llamada campo de dominio, que debe contener el nombre de un elemento. Estos elementos del campo de dominio suelen estar marcados como tipo interno componentes del dominio, para distinguirlos del resto de los elementos. Si coincide el valor, se actúa sobre él y se indica el valor.

Los argumentos correspondientes a esta función son:

- **IDMeta4Object:** en él se especifica el Meta4Object donde se encuentra el nodo con los registros a pivotar. Se pueden dar dos situaciones:
 - Si el argumento está vacío, "", se asume que el nodo origen está en el propio Meta4Object que se encuentra activo.
 - Si el argumento no está vacío, el nodo donde se encuentra el elemento pivote debe tener definido el uso del otro Meta4Object por la interfaz de nivel 2, de modo que el argumento IDMeta4Object pueda corresponderse con un alias del Meta4Object del nodo donde está el elemento pivote, o de igual modo se pueda corresponder con una instancia del Meta4Object donde está el elemento pivote.
- **IDHost:** es un argumento que nunca puede estar vacío. En él se especifica el nodo donde se encuentran los registros que se van a pivotar. Se pueden dar dos situaciones:
 - Si existe IDMeta4Object como primer argumento, especifica el nombre del nodo de ese Meta4Object donde están los registros que se van a pivotar.
 - Si no existe IDMeta4Object como primer argumento, el argumento especifica uno de los siguientes elementos:
 - El alias del nodo donde están los registros a pivotar para el nodo donde se ejecuta la función de pivotado
 - El nombre de la estructura de nodo donde están los registros que se van a pivotar
 - El nombre del nodo, si no es ninguno de los anteriores
- **DominioDestino:** especifica el DM destino sobre el que se realiza el pivotado. Está compuesto por *DMD_fields*. Si un elemento del DM destino no pertenece a este DM, no se lleva a cabo la aplicación del pivotado para él.

Si el valor de este argumento es (""), se realiza el pivotado para los elementos de cualquier DM.

- **Índice:** es un argumento opcional. Especifica el índice que se utilizará para ordenar los registros que se van a pivotar.

Además, indica cuál es la clave primaria que se va a utilizar para los registros. Si este argumento no aparece, se emplea el índice por defecto del nodo con los registros que se van a pivotar.

Para obtener información adicional, haga clic en [Funcionamiento del pivotado](#).

Funcionamiento del pivotado

En el caso de un nodo que contiene un conjunto de registros expuestos en forma de acumulado corto, el algoritmo de pivotado genera en un nodo destino

el mismo número de registros que existen en el nodo origen con diferente clave primaria.

La clave primaria del nodo origen viene determinada por un índice que debe estar definido en ese nodo. Los elementos del índice son los elementos que marcan la clave primaria.

Lo primero que hace el algoritmo es ordenar el nodo origen según un índice concreto. Esto no es necesario, pero hace que el algoritmo sea más rápido posteriormente.

Después de esto, ya es posible pivotar el registro. Para saber cuál es la columna del nodo origen que indica el componente DM, se debe marcar únicamente ese elemento con el tipo interno *DMD_component*.

El algoritmo recorre todas las columnas del nodo origen, excepto la del *DMD_component*, intentando pivotar su contenido. Para ello:

1. El DM de la columna destino debe ser igual al indicado como tercer argumento, o a cualquier otro si ese argumento está vacío.
2. El *DMD_component* de la columna destino debe ser igual al valor de la columna *DMD_component* origen (marcada como 18).
3. El *DMD_field* de la columna destino debe coincidir con el nombre de la columna origen que se va a pivotar.

El algoritmo recorre todas las columnas origen intentando pivotarlas sobre cualquier columna destino que cumpla las condiciones especificadas anteriormente.

No existe límite en las columnas destino que pueden cumplir la condición de búsqueda.



Un pivotado incremental es la operación de asignar valor sólo en aquellas celdas destino cuyo valor sea distinto al que se desea asignar, no así en aquellas celdas en las que coincida.

Si una celda destino ya existe y tiene un valor idéntico al que se va a asignar, no se llevará a cabo la asignación, lo que permitirá realizar pivotados incrementales.

EJEMPLO

Considere la siguiente tabla origen.

ID_Concept	ID_HR	OR_HR_Rol e	Valor
1001	4497	1	5
1011	4497	1	8
1021	4497	1	4
1001	4498	1	6
1011	4498	1	16
1001	2234	1	2
1021	2234	1	32

Las columnas marcadas como clave primaria mediante el índice serían ID_HR y OR_HR_ROLE. La columna ID_Concept tiene 18 como tipo interno.

Considere una tabla destino del siguiente tipo.

ID_HR	OR_HR_Rol e	CELL1	CELL2	CELL3	CELL4
2234	1	2	<NULL>	32	2
4497	1	5	8	4	5
4498	1	6	16	<NULL>	6

Esta tabla sería el resultado de aplicar la función de pivotado sobre un DM (por ejemplo *payroll*), asumiendo lo siguiente:

- El elemento *Cell1* tiene un DM *payroll*, un *DMD_component* 1001 y un *DMD_field value*.
- El elemento *Cell2* tiene un DM *payroll*, un *DMD_component* 1011 y un *DMD_field value*.
- El elemento *Cell3* tiene un DM *payroll*, un *DMD_component* 1021 y un *DMD_field value*.
- El elemento *Cell4* tiene un DM *payroll*, un *DMD_component* 1001 y un *DMD_field value*.

Por ejemplo, si hubiese otra columna en el origen, se pivotaría sobre el elemento *Cell5*, si éste tuviese un DM *payroll*, un *DMD_component* 1011 y un *DMD_field moneda*.

EJEMPLO

```
Pivot (
    VALUE VARIABLE STRING Meta4Object,
    VALUE VARIABLE STRING HostId,
    VALUE VARIABLE STRING TargetDMD,
    [VALUE NUMBER Index]
)
```

Macros LN4

Las macros del lenguaje LN4 son una serie de constantes que han sido definidas internamente por Meta4[®].

Estas macros tienen asignado un modo determinado de actuación, y se invocan escribiendo directamente el nombre de la constante.

Existen macros de diverso tipo:

- Macros básicas
- Macros de búsqueda
- Macros para cuadros de diálogo
- Macros para gestionar botones en cuadros de diálogo
- Macros de funciones de totalización
- Macros para transacciones en base de datos
- Macros para manipulación de cadenas
- Macros de tipo y ámbito de elementos
- Macros de constantes CHR
- Macros de constantes de indirección de atributos (lectura)
- Macros de constantes de indirección de atributos (ejecución)
- Macros de autocarga
- Macros de autocarga para llamadas de nivel 2
- Macros de tipo de acceso
- Macros de constantes de log
- Macros de constantes de ordenación
- Macros de valores de modo de envío
- Macros de constantes de fecha
- Macros de tipos variantes
- Macros de políticas de compartición
- Macros de funciones de fecha (suma/diferencia)
- Macros matemáticas
- Macros de metadatos de elementos
- Macros de tipo de metadato de un Meta4Object
- Macros de tipo de metadato de un nodo

- Macros de Unicode
- Macros de comportamiento societario en niveles 2
- Macros para el manejo de marcadores

Macros básicas

Las macros básicas son:

- M4_TRUE
- M4_FALSE
- M4_SHORT_DATE_FORMAT
- M4_LONG_DATE_FORMAT
- M4_SUCCESS
- M4_ERROR
- M4_ZERO
- M4_POSITIVE
- M4_NEGATIVE

M4_TRUE

Esta macro simboliza el valor booleano true.

M4_FALSE

Esta macro simboliza el valor booleano FALSE.

M4_SHORT_DATE_FORMAT

Es el formato de fecha corto. A diferencia de la constante *M4_Long_Date_Format*, esta macro no agrega segundos, minutos ni horas.

M4_LONG_DATE_FORMAT

Es el formato de fecha largo. A diferencia de la constante *M4_Short_Date_Format*, esta macro agrega las horas, minutos y segundos.

M4_SUCCESS

Esta macro indica cuándo una función se ha ejecutado correctamente.

M4_ERROR

Esta macro indica cuándo una función no se ha ejecutado correctamente.

M4_ZERO

Esta macro simboliza el valor cero.

M4_POSITIVE

Esta macro simboliza un valor positivo.

M4_NEGATIVE

Esta macro simboliza un valor negativo.

Macros de búsqueda

Las macros de búsqueda son:

- M4_EQUAL
- M4_DISTINCT
- M4_GREATER
- M4_SMALLER
- M4_GREATER_OR_EQUAL
- M4_SMALLER_OR_EQUAL
- M4_EQUAL_OR_NULL
- M4_DISTINCT_OR_NULL
- M4_GREATER_OR_NULL
- M4_SMALLER_OR_NULL
- M4_GREATER_OR_EQUAL_OR_NULL
- M4_SMALLER_OR_EQUAL_OR_NULL

- M4_CASE_EQUAL
- M4_CASE_DISTINCT
- M4_CASE_GREATER
- M4_CASE_SMALLER
- M4_CASE_GREATER_OR_EQUAL
- M4_CASE_SMALLER_OR_EQUAL
- M4_CASE_EQUAL_OR_NULL
- M4_CASE_DISTINCT_OR_NULL
- M4_CASE_GREATER_OR_NULL
- M4_CASE_SMALLER_OR_NULL
- M4_CASE_GREATER_OR_EQUAL_OR_NULL
- M4_CASE_SMALLER_OR_EQUAL_OR_NULL
- M4_REGULAR_EXPRESSION
- M4_REGULAR_EXPRESSION_OR_NULL
- M4_CASE_REGULAR_EXPRESSION
- M4_CASE_REGULAR_EXPRESSION_OR_NULL

M4_EQUAL

Esta macro permite buscar un registro en el que el valor que se busca sea igual a la cadena indicada.

M4_DISTINCT

Esta macro permite buscar un registro en el que el valor que se busca sea distinto a la cadena indicada.

M4_GREATER

Esta macro permite buscar un registro en el que el valor que se busca sea mayor que el indicado.

M4_SMALLER

Esta macro permite buscar un registro en el que el valor que se busca sea menor que el indicado.

M4_GREATER_OR_EQUAL

Esta macro permite buscar un registro en el que el valor que se busca sea mayor o igual que el indicado.

M4_SMALLER_OR_EQUAL

Esta macro permite buscar un registro en el que el valor que se busca sea menor o igual que el indicado.

M4_EQUAL_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea igual que el indicado o nulo.

M4_DISTINCT_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea distinto que el indicado o nulo.

M4_GREATER_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea mayor que el indicado o nulo.

M4_SMALLER_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea menor que el indicado o nulo.

M4_GREATER_OR_EQUAL_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea mayor o igual que el indicado, o nulo.

M4_SMALLER_OR_EQUAL_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea

menor o igual que el indicado, o nulo.

M4_CASE_EQUAL

Esta macro permite buscar un registro en el que el valor que se busca sea igual al indicado, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_DISTINCT

Esta macro permite buscar un registro en el que el valor que se busca sea distinto al indicado, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_GREATER

Esta macro busca un registro en el que el valor que se busca sea mayor que el indicado, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_SMALLER

Esta macro busca un registro en el que el valor que se busca sea menor que el indicado, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_GREATER_OR_EQUAL

Esta macro permite buscar un registro en el que el valor que se busca sea mayor o igual que el indicado, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_SMALLER_OR_EQUAL

Esta macro permite buscar un registro en el que el valor que se busca sea menor o igual que el indicado, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_EQUAL_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea igual que el indicado o nulo, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_DISTINCT_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea distinto que el indicada o nulo, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_GREATER_OR_NULL

Esta macro busca un registro en el que el valor que se busca sea mayor que el indicado o nulo, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_SMALLER_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea menor que el indicado o nulo, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_GREATER_OR_EQUAL_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea mayor o igual que el indicado, o nulo, distinguiendo entre mayúsculas y minúsculas.

M4_CASE_SMALLER_OR_EQUAL_OR_NULL

Esta macro permite buscar un registro en el que el valor que se busca sea menor o igual que el indicado o nula, distinguiendo entre mayúsculas y minúsculas.

M4_REGULAR_EXPRESSION

Esta macro permite establecer una serie de requisitos que debe cumplir el valor del elemento. Por ejemplo, que el valor de un elemento empiece por una determinada cadena o un determinado carácter, etc.

M4_REGULAR_EXPRESSION_OR_NULL

Esta macro permite establecer una serie de requisitos que debe cumplir el valor del elemento, a no ser que se trate de un valor nulo. Los requisitos pueden ser, por ejemplo, que el valor del elemento empiece por una determinada cadena o un determinado carácter, etc.

M4_CASE_REGULAR_EXPRESSION

Esta macro permite establecer una serie de requisitos que debe cumplir el valor del elemento. Por ejemplo, que el valor del elemento empiece por una determinada cadena o un determinado carácter, etc., distinguiendo entre mayúsculas y minúsculas.

M4_CASE_REGULAR_EXPRESSION_OR_NULL

Esta macro permite establecer una serie de requisitos que debe cumplir el valor del elemento, a menos que se trate de un valor nulo. Los requisitos pueden ser, por ejemplo, que el valor del elemento empiece por una determinada cadena o un determinado carácter, etc., distinguiendo entre mayúsculas y minúsculas.

Macros para cuadros de diálogo

Las macros que a continuación se exponen se utilizan para indicar los botones que se desean que aparezcan en el cuadro de diálogo.

- M4_BTN_OK
- M4_BTN_OK_CANCEL
- M4_BTN_ABORT_RETRY_IGNORE
- M4_BTN_YES_NO_CANCEL
- M4_BTN_YES_NO
- M4_BTN_RETRY_CANCEL

M4_BTN_OK

Esta macro indica que aparezca el botón Aceptar en el cuadro de diálogo que se muestra al usuario.

M4_BTN_OK_CANCEL

Esta macro indica que aparezcan los botones Aceptar y Cancelar en el cuadro de diálogo que se muestra al usuario.

M4_BTN_ABORT_RETRY_IGNORE

Esta macro indica que aparezcan los botones Anular, Reintentar e Ignorar en el

cuadro de diálogo que se muestra al usuario.

M4_BTN_YES_NO_CANCEL

Esta macro indica que aparezcan los botones Sí, No y Cancelar en el cuadro de diálogo que se muestra al usuario.

M4_BTN_YES_NO

Esa macro indica que aparezcan los botones Sí y No en el cuadro de diálogo que se muestra al usuario.

M4_BTN_RETRY_CANCEL

Esa macro indica que aparezcan los botones Reintentar y Cancelar en el cuadro de diálogo que se muestra al usuario.

Macros para gestionar botones en cuadros de diálogo

Las macros que a continuación se exponen se utilizan para saber el botón que el usuario ha presionado de todos los que aparecen en el cuadro de diálogo.

- M4_OK
- M4_CANCEL
- M4_ABORT
- M4_RETRY
- M4_IGNORE
- M4_YES
- M4_NO

M4_OK

Esta macro indica que el usuario ha presionado el botón Aceptar situado en el cuadro de diálogo que se ha mostrado en pantalla.

M4_CANCEL

Esta macro indica que el usuario ha presionado el botón Cancelar situado en el

cuadro de diálogo que se ha mostrado en pantalla.

M4_ABORT

Esta macro indica que el usuario ha presionado el botón Anular situado en el cuadro de diálogo que se ha mostrado en pantalla.

M4_RETRY

Esta macro indica que el usuario ha presionado el botón Reintentar situado en el cuadro de diálogo que se ha mostrado en pantalla.

M4_IGNORE

Esta macro indica que el usuario ha presionado el botón Ignorar situado en el cuadro de diálogo que se ha mostrado en pantalla.

M4_YES

Esta macro indica que el usuario ha presionado el botón Sí, situado en el cuadro de diálogo que se ha mostrado en pantalla.

M4_NO

Esta macro indica que el usuario ha presionado el botón No, situado en el cuadro de diálogo que se ha mostrado en pantalla.

Macros de funciones de totalización

Cuando se define un elemento, se le asigna una función. Esto es de gran utilidad, ya que cuando se totaliza y no se indica ninguna otra particularidad, se utilizaría esa función definida como función por defecto.

En este tipo de funciones, existen dos atributos clave:

- **SystotalSlices**: totaliza los tramos de un elemento y los totaliza.
- **SysTotalItems**: totaliza cada uno de los elementos en cada uno de sus registros y los totaliza.

A continuación, se detallan otras funciones definidas por las que se puede

totalizar:

- M4_TOTAL_COUNT
- M4_TOTAL_SUM
- M4_TOTAL_AVG
- M4_TOTAL_MAX
- M4_TOTAL_MIN
- M4_TOTAL_FIRST
- M4_TOTAL_LAST

M4_TOTAL_COUNT

Esta macro simboliza la totalización del recuento.

M4_TOTAL_SUM

Esta macro simboliza la totalización de la suma.

M4_TOTAL_AVG

Esta macro simboliza la totalización de la media.

M4_TOTAL_MAX

Esta macro simboliza la totalización del máximo.

M4_TOTAL_MIN

Esta macro simboliza la totalización del mínimo.

M4_TOTAL_FIRST

Esta macro simboliza la totalización y considera el resultado como primer valor.

M4_TOTAL_LAST

Esta macro simboliza la totalización y considera el resultado como último valor.

Macros para transacciones en base de datos

En este apartado se reúnen macros que son de gran utilidad a la hora de realizar operaciones durante una transacción de base de datos iniciada previamente.

- M4_COMMIT
- M4_ROLLBACK
- M4_EXECUTE_POSTVALIDATION

M4_COMMIT

Esta macro simboliza que los cambios realizados en una transacción de base de datos que se ha iniciado previamente se guardan permanentemente.

M4_ROLLBACK

Esta macro simboliza que se desechan los cambios realizados en una transacción de base de datos que se ha iniciado previamente.

Por ejemplo, puede utilizarse cuando interese deshacer los cambios realizados desde el inicio de una transacción en caso de producirse un error en la ejecución de una función, o bien para detener una ejecución en un momento determinado por cualquier motivo.

M4_EXECUTE_POSTVALIDATION

Esta macro comprueba si las validaciones posteriores funcionan correctamente o no. Esta posibilidad es de gran utilidad para casos de importación.

Macros para manipulación de cadenas

En este apartado, se reúnen las macros utilizadas para eliminar espacios de una cadena de texto especificada. Estas macros se utilizan con la función *Funciones para manejo de registros*.

- M4_TRIM_LEFT
- M4_TRIM_ALL
- M4_TRIM_RIGHT
- M4_LOWERCASE

-
- M4_UNCHANGED
 - M4_UPPERCASE

M4_TRIM_LEFT

Esta macro indica que se eliminan los espacios que existan en el lado izquierdo de la cadena.

M4_TRIM_ALL

Esta macro indica que se eliminan los espacios que existan a ambos lados de la cadena.

M4_TRIM_RIGHT

Esta macro indica que se eliminan los espacios que existan en el lado derecho de la cadena.

M4_LOWERCASE

Esta macro indica que se convierte a minúsculas una cadena dada.

M4_UNCHANGED

Esta macro indica que una cadena se deja tal cual se escribió.

M4_UPPERCASE

Esta macro indica que se convierte a mayúsculas una cadena dada.

Macros de tipo y ámbito de elementos

Las macros de tipo y ámbito de elementos son:

- M4_SCOPE_ALL
- M4_SCOPE_REGISTER
- M4_SCOPE_BLOCK

- M4_SCOPE_NODE
- M4_TYPE_ALL
- M4_TYPE_METHOD
- M4_TYPE_PROPERTY
- M4_TYPE_FIELD
- M4_TYPE_CONCEPT

M4_SCOPE_ALL

Con esta macro se indica que el ámbito de un elemento puede ser el registro, el bloque y el nodo.

M4_SCOPE_REGISTER

Con esta macro se indica que el ámbito de un elemento va a ser de tipo registro.

M4_SCOPE_BLOCK

Con esta macro se indica que el ámbito de un elemento va a ser de tipo bloque.

M4_SCOPE_NODE

Con esta macro se indica que el ámbito de un elemento va a ser de tipo nodo.

M4_TYPE_ALL

Con esta macro se indica que el tipo de elemento puede ser Método, Propiedad, Campo o Concepto.

M4_TYPE_METHOD

Con esta macro se indica que el tipo de elemento va a ser Método.

M4_TYPE_PROPERTY

Con esta macro se indica que el tipo de elemento va a ser Propiedad.

M4_TYPE_FIELD

Con esta macro se indica que el tipo de elemento va a ser Campo.

M4_TYPE_CONCEPT

Con esta macro se indica que el tipo de elemento va a ser Concepto.

Macros de constantes CHR

Son macros útiles cuando se quiere escribir a nivel de código un carácter, como puede ser un tabulador, comillas, nueva línea, etc.

- M4_NEW_LINE
- M4_TAB
- M4_DOUBLE_QUOTE
- M4_CR

M4_NEW_LINE

Esta macro se utiliza cuando se quiere escribir una nueva línea.

M4_TAB

Esta macro se utiliza cuando se quiere introducir en una cadena un tabulador.

M4_DOUBLE_QUOTE

Esta macro se utiliza cuando se quiere escribir en una cadena unas comillas.

M4_CR

Esta macro se utiliza cuando se quiere escribir una nueva línea y un retorno de carro.

Macros de constantes de indirección de atributos (lectura)

Estas macros se utilizan cuando se quiera leer un atributo concreto. Para cada uno de los atributos hay una macro específica.

Para obtener más información sobre estas macros, haga clic en [Atributos y Atributos-Métodos](#).

Este tipo de constantes se refieren a los atributos que se ejecutan por indirección. Por ejemplo, los correspondientes a ChannelAttribCall, engloban:

- [M4_ATT_SYS_SLICE_NUMBER](#)
- [M4_ATT_SYS_START_DATE](#)
- [M4_ATT_SYS_END_DATE](#)
- [M4_ATT_SYS_FIRST_SLICE](#)
- [M4_ATT_SYS_LAST_SLICE](#)
- [M4_ATT_SYS_OLD_VALUE](#)
- [M4_ATT_SYS_BLOB_DESCRIPTION](#)
- [M4_ATT_SYS_BLOB_MASK](#)
- [M4_ATT_SYS_AUX_ITEM_ID](#)

M4_ATT_SYS_SLICE_NUMBER

Se aplica a elementos e indica el número de tramos que contiene un elemento.

M4_ATT_SYS_START_DATE

Se aplica a tramos y corresponde a la fecha de inicio del tramo.

M4_ATT_SYS_END_DATE

Se aplica a tramos y corresponde a la fecha de fin del tramo.

M4_ATT_SYS_FIRST_SLICE

Se aplica a elementos y corresponde al índice del primer tramo del elemento.

M4_ATT_SYS_LAST_SLICE

Se aplica a elementos y corresponde al índice del último tramo del elemento.

M4_ATT_SYS_OLD_VALUE

Se aplica a elementos y tramos, e indica el valor antiguo del elemento.

M4_ATT_SYS_BLOB_DESCRIPTION

Se aplica a elementos y devuelve el nombre del elemento.

M4_ATT_SYS_BLOB_MASK

Se aplica a elementos, e indica la máscara BLOB del elemento.

M4_ATT_SYS_AUX_ITEM_ID

Se aplica a elementos, e indica el identificador del elemento auxiliar de un elemento.

Para obtener más información sobre esta y otras macros, haga clic en [Atributos y Atributos-Métodos](#).

Macros de constantes de indirección de atributos (ejecución)

Estas macros se utilizan cuando se quiere ejecutar por indirección un atributo concreto. Para cada uno de los atributos hay una macro concreta.

Por ejemplo, los correspondientes a *ChannelAttribValue*, engloban:

- M4_ATT_SYS_ADD_NEW_SLICE
- M4_ATT_SYS_CREATE_SLICE
- M4_ATT_SYS_SPLIT_SLICE
- M4_ATT_SYS_TOTALIZE_ITEMS
- M4_ATT_SYS_TOTALIZE_SLICES
- M4_ATT_SYS_CREATE_BLOB_FILE

- M4_ATT_SYS_SET_BLOB_DESCRIPTION

M4_ATT_SYS_ADD_NEW_SLICE

Se aplica a elementos e indica que se ha agregado un nuevo tramo al elemento.

M4_ATT_SYS_CREATE_SLICE

Se aplica a elementos y tramos, e indica que se ha agregado un nuevo tramo al elemento y que se le ha asignado un valor.

M4_ATT_SYS_SPLIT_SLICE

Se aplica a tramos, e indica que el tramo se ha dividido en dos utilizando la fecha, manteniendo así el valor acorde al comportamiento.

M4_ATT_SYS_TOTALIZE_ITEMS

Se aplica a elementos, e indica que adquiere como valor la totalización de este elemento.

M4_ATT_SYS_TOTALIZE_SLICES

Se aplica a tramos, e indica que adquiere como valor la totalización de este elemento.

M4_ATT_SYS_CREATE_BLOB_FILE

Se aplica a elementos, e indica la creación de un archivo para el elemento en el directorio con esta extensión.

M4_ATT_SYS_SET_BLOB_DESCRIPTION

Se aplica a elementos, e indica la asignación de la descripción de un elemento.

Macros de autocarga

La carga automática es una característica de los Meta4Object, que permite que se carguen en memoria a medida que se van utilizando.

- M4_AUTOLOAD_OFF
- M4_AUTOLOAD_BLOCK
- M4_AUTOLOAD_PRG
- M4_AUTOLOAD_NODESAYS

M4_AUTOLOAD_OFF

Esta macro significa que por defecto esa función no está activa y en caso de que se quiera utilizar, el usuario debe hacer la llamada.

M4_AUTOLOAD_BLOCK

Esta macro significa la carga completa de un bloque entero.

M4_AUTOLOAD_PRG

Esta macro indica que la propiedad *Autoload* es de tipo propagante, es decir, que se carga el bloque y sus hijos.

M4_AUTOLOAD_NODESAYS

La carga automática hace lo que el nodo tiene configurado. Cada vez que se hace uso de la carga automática, se consulta la configuración del nodo para ver el tipo de carga que tiene predefinido.

Macros de autocarga para llamadas de nivel 2

La carga automática es una característica de los Meta4Object, que permite que se carguen en memoria a medida que se van utilizando. Las siguientes macros se utilizan en la carga automática para llamadas de nivel 2, con las funciones DefineInstance (Instancia, Meta4Object, PolíticaCarga, PolíticaCompartición) y DefineInstanceEx (Instancia, Meta4ObjectNombre, PolíticaCarga, PolíticaCompartición, [IDOrganización], [TipoOrganización], [CSTipoAcceso], [DataLanguage], [Role],...):

- M4_OPEN_MODE_NO_LOAD
- M4_OPEN_MODE_LOAD
- M4_OPEN_MODE_AUTOLOAD_BLOCK
- M4_OPEN_MODE_AUTOLOAD_PRG
- M4_OPEN_MODE_AUTOLOAD_NODESAYS

M4_OPEN_MODE_NO_LOAD

Esta macro indica que no se cargue el Meta4Object.

M4_OPEN_MODE_LOAD

Esta macro indica que se cargue todo el Meta4Object.

M4_OPEN_MODE_AUTOLOAD_BLOCK

Esta macro indica que no se cargue el Meta4Object y poner el modo de autocarga a modo bloque.

M4_OPEN_MODE_AUTOLOAD_PRG

Esta macro indica que no se cargue el Meta4Object y poner el modo de autocarga a modo bloque e hijos.

M4_OPEN_MODE_AUTOLOAD_NODESAYS

Esta macro indica que no se cargue el Meta4Object y poner el modo de autocarga al modo que esté indicado en el nodo.

Macros de tipo de acceso

Las macros para el acceso C/S por nivel 2 se utilizan con las funciones OpenChannelEx (Meta4Object, IdInstancia, Ambito, ModoApertura, [OrganizaciónID], [TipoOrganización], [CSTipoAcceso]) y DefineInstanceEx (Instancia, Meta4ObjectNombre, PolíticaCarga, PolíticaCompartición, [IDOrganización], [TipoOrganización], [CSTipoAcceso], [DataLanguage], [Role],...). Son las siguientes:

-
- M4_CSTYPE_FRONT
 - M4_CSTYPE_BACK
 - M4_CSTYPE_BOTH
 - M4_CSTYPE_REAL_BOTH
 - M4_CSTYPE_FATHER

M4_CSTYPE_FRONT

El Meta4Object que se levante se creará como parte cliente. Este uso está totalmente desaconsejado por su peligrosidad.

M4_CSTYPE_BACK

El Meta4Object que se levante se creará como parte servidora. Este uso está totalmente desaconsejado por su peligrosidad.

M4_CSTYPE_BOTH

El Meta4Object que se levante se creará como parte cliente en entornos cliente y como parte servidora en entornos de servidor. Este uso es el recomendado.

M4_CSTYPE_REAL_BOTH

El Meta4Object que se levante se creará como no separable.

M4_CSTYPE_FATHER

El Meta4Object que se levante se creará con el mismo tipo cliente servidor que el Meta4Object que lo invoca.

Macros de constantes de log

Las macros de constantes de log son:

- M4_ERRORLOG
- M4_WARNINGLOG
- M4_DEBUGINFOLOG

M4_ERRORLOG

Esta macro representa los mensajes de error.

M4_WARNINGLOG

Esta macro representa los avisos generados.

M4_DEBUGINFOLOG

Esta macro representa los procesos de depuración.

Macros de constantes de ordenación

Bajo este apartado se recogen todas las macros utilizadas conjuntamente con la función de ordenación *AddSortFilter* (), tales como:

- M4_ASCENDING
- M4_DESCENDING

M4_ASCENDING

Esta macro es un criterio de clasificación ascendente.

M4_DESCENDING

Esta macro es un criterio de clasificación descendente.

Macros de valores de modo de envío

Las macros de valores de modo de envío son:

- M4_SEND_NO_BRANCH
- M4_SEND_BRANCH
- M4_SEND_RESET

M4_SEND_NO_BRANCH

Esta macro indica que un nodo ha sido marcado para enviar, pero no así sus hijos.

M4_SEND_BRANCH

Esta macro indica que se ha marcado un nodo y sus hijos para enviar.

M4_SEND_RESET

Esta macro indica que se ha deshabilitado un nodo y, por lo tanto, no se va a enviar.

Macros de constantes de fecha

Las macros de constantes de fecha son:

- M4_MINUS_INF
- M4_PLUS_INF

M4_MINUS_INF

Esta macro es el límite establecido como fecha mínima inferior. Su valor es 01-01-1800.

M4_PLUS_INF

Esta macro es el límite establecido como fecha máxima superior. Su valor es 01-01-4000.

Macros de tipos variantes

El uso de estas macros corresponde a la función *GetVariantType*, en la cual se indica un valor, y la función utilizada dice si es una cadena, fecha o número.

Estas funciones son de gran utilidad en aquellas ocasiones en las que se pasa un argumento y se desconoce su tipo, en función del cual actuará de un modo u otro.

- M4_VARIANT_TYPE_NULL
- M4_VARIANT_TYPE_NUMBER
- M4_VARIANT_TYPE_STRING
- M4_VARIANT_TYPE_DATE

M4_VARIANT_TYPE_NULL

El tipo de variante utilizada en esta macro es nulo.

M4_VARIANT_TYPE_NUMBER

El tipo de variante utilizada en esta macro es número.

M4_VARIANT_TYPE_STRING

El tipo de variante utilizada en esta macro es cadena.

M4_VARIANT_TYPE_DATE

El tipo de variante utilizada en esta macro es fecha.

Macros de políticas de compartición

Este tipo de macros se utilizan para las funciones siguientes:

- DefineInstance (Instancia, Meta4Object, PolíticaCarga, PolíticaCompartición)
- OpenChannel (Meta4Object, IdInstancia, Ámbito, ModoApertura)
- DefineInstanceEx (Instancia, Meta4ObjectNombre, PolíticaCarga, PolíticaCompartición, [IDOrganización], [TipoOrganización], [CSTipoAcceso], [DataLanguage], [Role],...)
- OpenChannelEx (Meta4Object, IdInstancia, Ambito, ModoApertura, [OrganizaciónID], [TipoOrganización], [CSTipoAcceso])

Estas macros disponen de diferentes tipos de compartición:

- M4_INSTANCE_NOT_SHARED
- M4_INSTANCE_LOCAL_SHARED
- M4_INSTANCE_GLOBAL_SHARED

M4_INSTANCE_NOT_SHARED

Esta macro indica que es una instancia no compartida.

M4_INSTANCE_LOCAL_SHARED

Esta macro indica que es una instancia compartida localmente.

M4_INSTANCE_GLOBAL_SHARED

Esta macro indica que es una instancia compartida globalmente.

Macros de funciones de fecha (suma/diferencia)

Este tipo de funciones indican:

- La unidad destino en la que se quiere obtener la diferencia entre dos fechas.
- El tipo de unidad con la que se va a incrementar una fecha.

Las macros de funciones de fecha son:

- M4_YEAR
- M4_MONTH
- M4_DAY
- M4_WEEK
- M4_WEEKDAY
- M4_HOUR
- M4_MINUTE
- M4_SECOND

M4_YEAR

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en años.

M4_MONTH

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en meses.

M4_DAY

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en días.

M4_WEEK

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en semanas completas.

M4_WEEKDAY

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en semanas.

Se entiende como semanas distintas un periodo que cubre dos días por ejemplo, perteneciendo un día a una semana y otro día a otra. Siempre es independiente del número de días.

M4_HOUR

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en horas.

M4_MINUTE

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en minutos.

M4_SECOND

Indica que la diferencia entre dos fechas o el incremento de una fecha se va a dar en segundos.

Macros matemáticas

Las macros matemáticas son:

- M4_MATH_PI
- M4_MATH_E

M4_MATH_PI

Representa al valor de la constante *Pi* que es 3.1415

M4_MATH_E

Representa el valor de la constante neperiana *e* que es 2.71

Macros de metadatos de elementos

El uso de estas macros corresponde a la función GetNodeItemData, en la cual se indica un elemento concreto y una macro que describe un tipo de metadato cuyo valor se requiere.

Las macros de metadatos de elementos son:

- M4_ITEM_TYPE
- M4_ITEM_SCOPE
- M4_ITEM_M4TYPE
- M4_ITEM_PRECISION
- M4_ITEM_INDEX
- M4_ITEM_SCALE
- M4_ITEM_INTERNAL_TYPE
- M4_ITEM_READ_OBJECT_ALIAS
- M4_ITEM_WRITE_OBJECT_ALIAS
- M4_ITEM_READ_FIELD_ID
- M4_ITEM_WRITE_FIELD_ID
- M4_ITEM_READ_OBJECT_ID
- M4_ITEM_WRITE_OBJECT_ID
- M4_ITEM_SLICE_TOTALIZE
- M4_ITEM_NAME

- M4_ITEM_ORDER
- M4_ITEM_SYNONYM
- M4_ITEM_ID
- M4_ITEM_NUMBER

M4_ITEM_TYPE

Con esta macro se indica que quiere recuperar el tipo del elemento, que puede ser método, propiedad, campo o concepto.

M4_ITEM_SCOPE

Con esta macro se indica que quiere recuperar el ámbito del elemento que puede ser registro, bloque o nodo.

M4_ITEM_M4TYPE

Con esta macro se indica que quiere recuperar el dato acerca del tipo M4, que puede ser nulo, cadena de longitud fija, cadena de longitud variable, campo long, fecha, fecha y hora, número, variant, moneda, variant numérico, fichero, campo long binario o hora.

M4_ITEM_PRECISION

Con esta macro se indica que quiere recuperar la precisión del elemento.

M4_ITEM_INDEX

Con esta macro se indica que quiere recuperar el índice del elemento.

M4_ITEM_SCALE

Con esta macro se indica que quiere recuperar la escala del elemento.

M4_ITEM_INTERNAL_TYPE

Con esta macro se indica que quiere recuperar el tipo interno del elemento.

M4_ITEM_READ_OBJECT_ALIAS

Con esta macro se indica que quiere recuperar el alias del objeto de lectura del elemento.

M4_ITEM_WRITE_OBJECT_ALIAS

Con esta macro se indica que quiere recuperar el alias del objeto de escritura del elemento.

M4_ITEM_READ_FIELD_ID

Con esta macro se indica que quiere recuperar el identificador del campo de lectura del elemento.

M4_ITEM_WRITE_FIELD_ID

Con esta macro se indica que quiere recuperar el identificador del campo de escritura del elemento.

M4_ITEM_READ_OBJECT_ID

Con esta macro se indica que quiere recuperar el identificador del objeto de lectura del elemento.

M4_ITEM_WRITE_OBJECT_ID

Con esta macro se indica que quiere recuperar el identificador del objeto de escritura del elemento.

M4_ITEM_SLICE_TOTALIZE

Con esta macro se indica que quiere recuperar el tipo de totalización que tiene el elemento para cuando se totalizan sus tramos.

M4_ITEM_NAME

Con esta macro se indica que quiere recuperar el nombre del elemento.

M4_ITEM_ORDER

Con esta macro se indica que quiere recuperar el orden del elemento dentro del nodo.

M4_ITEM_SYNONYM

Con esta macro se indica que quiere recuperar el sinónimo del elemento.

M4_ITEM_ID

Con esta macro se indica que quiere recuperar el identificador del elemento.

M4_ITEM_NUMBER

Con esta macro se indica que quiere recuperar el número de elementos del nodo.

M4_ITEM_IS_PK

Con esta macro se indica que quiere recuperar si un elemento es PK del nodo o no. 1 si es pk y 0 si no es pk.

Macros de tipo de metadato de un Meta4Object

El uso de estas macros corresponde a la función GetChannelData, que devuelve información de los metadatos de un Meta4Object. La función es de argumentos variables, y como mínimo hay que indicar el tipo de dato que se pide.

Las macros de tipo de metadato de un Meta4Object son:

- M4_M4OBJECT_NODE_NUMBER
- M4_M4OBJECT_CREATION_TYPE
- M4_M4OBJECT_NAME
- M4_M4OBJECT_ID

-
- M4_M4OBJECT_INSTANCE_NUMBER
 - M4_M4OBJECT_INSTANCE_HANDLE

M4_M4OBJECT_NODE_NUMBER

Indica el número de nodos del Meta4Object cuyos metadatos se quieren recuperar.

M4_M4OBJECT_CREATION_TYPE

Indica el tipo de creación del Meta4Object cuyos datos se quieren recuperar.

M4_M4OBJECT_NAME

Indica el nombre del Meta4Object cuyos datos se quieren recuperar.

M4_M4OBJECT_ID

Indica el identificador del Meta4Object cuyos datos se quieren recuperar.

M4_M4OBJECT_INSTANCE_NUMBER

Indica el número de instancia del Meta4Object cuyos datos se quieren recuperar.

M4_M4OBJECT_INSTANCE_HANDLE

Indica el handle de instancia del Meta4Object cuyos datos se quieren recuperar. En el caso de esta macro, la función GetChannelData admite un parámetro nuevo al final (sin contar el tipo de metadato), que permite especificar la instancia que se quiere a partir de su posición, empezando en 0.

A continuación se muestran sendos ejemplos de uso de esta macro:

EJEMPLO 1

Ejemplo de uso con una instancia de otro Meta4Object (por ejemplo el de nómina):

```

        MsgBox( "Id", GetChannelData( "ZZ_JESUS_PRUEBA",
M4_M4OBJECT_ID ) )

        x = GetChannelData( "ZZ_JESUS_PRUEBA",
M4_M4OBJECT_INSTANCE_NUMBER )
        MsgBox( "Id", ToString( x, 0 ) )

        for i = 0 to x - 1
            MsgBox( "Id", GetChannelData( "ZZ_JESUS_PRUEBA", i,
M4_M4OBJECT_INSTANCE_HANDLE ) )
        next

```

EJEMPLO 2

Ejemplo de uso con una instancia del mismo Meta4Object:

```

        MsgBox( "Id", GetChannelData( M4_M4OBJECT_ID ) )

        x = GetChannelData( M4_M4OBJECT_INSTANCE_NUMBER )
        MsgBox( "Id", ToString( x, 0 ) )

        for i = 0 to x - 1
            MsgBox( "Id", GetChannelData( i,
M4_M4OBJECT_INSTANCE_HANDLE ) )
        next

```

Macros de tipo de metadato de un nodo

El uso de estas macros corresponde a la función `GetNodeData`, que devuelve la información relativa a los metadatos del nodo. Todas ellas, salvo `M4_NODE_IS_REAL_ROOT` y `M4_NODE_PARENT`, proceden de las propiedades del nodo y de su estructura de nodo.

Las macros de tipo de metadato de nodo son:

- M4_NODE_READ_OBJECT_ID
- M4_NODE_WRITE_OBJECT_ID
- M4_NODE_ID
- M4_NODE_NAME

-
- M4_NODE_INDEX
 - M4_NODE_IS_ROOT
 - M4_NODE_IS_REAL_ROOT
 - M4_NODE_PARENT
 - M4_NODE_SENTENCE
 - M4_NODE_MAX_NUM_ROWS_DB
 - M4_NODE_AUTO_FILTER

M4_NODE_READ_OBJECT_ID

Con esta macro se indica que se quiere recuperar el identificador del objeto lógico de lectura.

M4_NODE_WRITE_OBJECT_ID

Con esta macro se indica que se quiere recuperar el identificador del objeto lógico de escritura.

M4_NODE_ID

Con esta macro se indica que se quiere recuperar el identificador del nodo.

M4_NODE_NAME

Con esta macro se indica que se quiere recuperar el nombre traducido del nodo.

M4_NODE_INDEX

Con esta macro se indica que se quiere recuperar el índice del nodo (orden del nodo dentro de su Meta4object).

M4_NODE_IS_ROOT

Con esta macro se indica que se quiere recuperar si el nodo está marcado en el diseñador como nodo raíz.

M4_NODE_IS_REAL_ROOT

Con esta macro se indica que se quiere recuperar si realmente el nodo no tiene padres.

M4_NODE_PARENT

Con esta macro se indica que se quiere recuperar el identificador del nodo padre.

M4_NODE_SENTENCE

Con esta macro se indica que se quiere recuperar la sentencia lógica de lectura del nodo.

M4_NODE_MAX_NUM_ROWS_DB

Con esta macro se indica el número de registros que como máximo se recuperarán de base de datos al cargar el nodo, aunque haya más registros en base de datos. Se define su valor cuando se quiere evitar que se carguen demasiado registros de base de datos.

M4_NODE_AUTO_FILTER

Esta macro permite obtener la información del tipo de filtro automático de un nodo, ya que se devuelve el valor definido en el nodo. Los posibles valores para los que no se ha definido macro son:

- M4CL_NODE_AUTO_FILTER_NONE: 0
- M4CL_NODE_AUTO_FILTER_APPLICATION: 1
- M4CL_NODE_AUTO_FILTER_CORRECTION: 2
- M4CL_NODE_AUTO_FILTER_APP_CORR: 3

Macros de Unicode

El uso de estas macros corresponde a la función Encoding, en la cual se indica el comportamiento de ficheros en entornos Unicode o ANSI según el valor que se les asigne.

Las macros de Unicode son:

-
- M4_ENCODING_NATIVE
 - M4_ENCODING_ANSI
 - M4_ENCODING_UTF8

M4_ENCODING_NATIVE

Se vuelca la cadena en formato ANSI en instalaciones ANSI y en formato UTF8 en instalaciones Unicode.

M4_ENCODING_ANSI

Se vuelca la cadena en formato ANSI.

M4_ENCODING_UTF8

Se vuelca la cadena en formato UTF8.

Macros de comportamiento societario en niveles 2

El uso de estas macros, que indican el comportamiento societario en nivel 2, corresponde a la función DefineInstanceEx.

Las macros son las siguientes:

- M4_ORGANIZATION_L2_TYPE_NONE
- M4_ORGANIZATION_L2_TYPE_MONO
- M4_ORGANIZATION_L2_TYPE_MULTI
- M4_ORGANIZATION_L2_TYPE_FATHER
- M4_ORGANIZATION_L2_TYPE_ANY

M4_ORGANIZATION_L2_TYPE_NONE

Esta macro indica que el comportamiento es independiente de la sociedad.

M4_ORGANIZATION_L2_TYPE_MONO

Esta macro indica que no se admiten datos de varias sociedades.

M4_ORGANIZATION_L2_TYPE_MULTI

Esta macro indica que se admiten datos de varias sociedades.

M4_ORGANIZATION_L2_TYPE_FATHER

Esta macro indica que el comportamiento es el mismo que el del Meta4Object de origen.

M4_ORGANIZATION_L2_TYPE_ANY

Esta macro indica que el comportamiento puede ser cualquiera.

Macros para el manejo de marcadores

El uso de estas macros, que indican el tipo de marcador que se va a aplicar, corresponde a la función GetBookMark (Type).

Las macros son las siguientes:

- M4_BOOKMARK_POSITION
- M4_BOOKMARK_PK

M4_BOOKMARK_POSITION

Esta macro indica que en el marcador se guarda la posición actual ocupada en todos los nodos del Meta4Object. Es el valor que se aplica por defecto.

El proceso es más rápido y ocupa menos memoria que si se guarda el valor de la clave primaria sobre el que posicionarse, como en el caso de la macro M4_BOOKMARK_PK, pero no es tan fiable. Por eso, solo se recomienda utilizar esta macro cuando no hay reordenaciones o recarga de base de datos o se añaden o eliminan registros en los nodos del Meta4Object.

M4_BOOKMARK_PK

Esta macro indica que en el marcador se guarda el valor de la clave primaria (PK) sobre el que posicionarse.

El proceso es más lento y ocupa más memoria que si se guarda la posición actual ocupada en todos los nodos del Meta4Object, como en el caso de la macro M4_BOOKMARK_POSITION, pero es más fiable. Por eso, se recomienda utilizar esta macro cuando se crean o destruyen registros, porque eso puede implicar que haya reordenaciones en los nodos del Meta4Object (recarga de base de datos, por ejemplo).

Uso de Meta4Objects especializados con LN4

Hay unas funcionalidades que invocan otros Meta4Objects especializados mediante una llamada de nivel 2, pasándoles datos que se debe procesar de un fin determinado.

Actualmente existen unas funcionalidades en la aplicación Meta4 que utiliza este mecanismo:

- Enviar, responder y manipular correo electrónico
- Trabajar con Microsoft Word y Excel desde la aplicación



Consulte los manuales de conectividad correspondientes para más detalles.

- Imprimir PDF desde un Meta4Object de datos
- Uso del API para modificar documentos PDF
- Uso del API para visualizar documentos en Word y convertirlos a PDF
- Uso del API para mostrar y guardar un documento en el gestor documental
- Uso del API para activar la gestión de anexos
- Uso del API para personalizar la información de firma en los documentos PDF
- Uso del API para crear parámetros de aplicación
- Uso del API para acceder a las ejecuciones de una planificación multisocietaria
- Uso del API para transferir la información de rol-sociedad de una sociedad a otra
- Uso del API para abrir usuarios del portal en una sociedad
- Uso del API para dar de baja usuarios asociados a personas
- Uso del API para reabrir usuarios asociados a una persona en una fecha
- Uso del API para borrar los usuarios asociados a una persona
- Uso del API para borrar un rol-sociedad a un usuario
- Uso del API para la descarga asíncrona de ficheros de forma segura
- Uso del API para ejecutar gráficos
- Uso del API para reducir el tamaño y peso de las imágenes
- Uso del API para ejecutar cartas y salida directa a Excel
- Uso del API para manejar archivos Excel en el servidor de aplicaciones

-
- [Uso del API para la apertura parcial de documentos Excel](#)
 - [Uso del API para proteger archivos Excel contra modificaciones](#)
 - [Uso del API para eliminar hojas de cálculo](#)
 - [Uso del API para crear un archivo Excel vacío](#)
 - [Uso del API para convertir archivos Excel en PDF](#)
 - [Uso del API para transcodificar objetos lógicos](#)
 - [Uso del API para transcodificar valores de Meta4Objects](#)
 - [Uso del API para comprimir y descomprimir archivos en formato ZIP](#)
 - [Uso del API para encriptar, desencriptar, firmar y verificar archivos](#)
 - [Uso del API para generar la información asociada a los componentes de un concepto final](#)
 - [Cómo pasar valores a parámetros para consultas nuevas](#)
 - [Cómo llamar a una consulta desde una opción de menú](#)

Imprimir PDF desde un Meta4Object de datos

Esta funcionalidad es útil cuando quiere rellenar un formulario PDF con los datos del repositorio, por ejemplo, para generar contratos para unas personas utilizando un formulario impreso existente.

Los archivos del tipo FDF, XFDF o XFA permiten almacenar la información de los campos, incluyendo una referencia al documento PDF. Cuando se abre un archivo XFDF, FDF o XFA con el Acrobat Reader, o con el Acrobat, automáticamente se funden los datos del XFDF o XFA en el formulario PDF. Este archivo fundido se puede mandar a imprimir.

El archivo XFDF no sólo se generará con una referencia al documento PDF sino a partir de un Meta4Object de datos.



Esta funcionalidad sólo está disponible para Windows e impresión en cliente.

Para ello, los pasos generales a seguir son:

1. Crear el Meta4Object de datos que debe exportar los datos a un archivo XFDF.
2. Crear el código de una regla en un método que concatena los datos en un archivo XFDF. Consulte [Crear código para generar un XFDF desde un Meta4Object de datos](#)

3. Crear otra regla en el mismo método u en otro que realiza una llamada nivel 2 al método PRINT_PDF del Meta4Object PRINT_PDF, pasándole unos argumentos requeridos. Consulte [Llamar al método PRINT_PDF](#)

Crear código para generar un XFDF desde un Meta4Object de datos

Para generar la salida de datos a un archivo del tipo XFDF desde un Meta4Object el código debería incluir lo siguiente:

1. Declaración de los atributos del archivo XFDF:
 - La ruta absoluta del archivo XFDF a generar.
 - La ruta absoluta de la plantilla o formulario PDF a utilizar.
 - Las líneas de la cabecera que describirán el tipo de archivo, *character set* a utilizar, y la referencia a la ruta completa de la plantilla o formulario.
 - Los campos.
2. Inicio y concatenación del archivo de salida XFDF:
 - Inicie y concatene el archivo XFDF con los datos que debería figurar en la cabecera del archivo.
 - Concatene el código XML de cada campo uno por uno hasta finalizar. Los valores de estos campos suelen ser del elemento de una estructura de nodo.
3. Finalización del archivo declarando el cierre de los campos, y el código XFDF.

A continuación se muestra un ejemplo del código:

EJEMPLO

```
lRutaXFDF = "C:\temp\fichero.xfdf" //El archivo XFDF con la ruta absoluta
lRutaPlantilla = "C:\temp\formulario.pdf" // El formulario pdf
lPrimLin = "<?xml version="+chr(34)+"1.0"+chr(34)+" encoding="+chr(34)+"ISO-8859-1"+chr(34)+"?>"
lSegLin = "<xfdf xmlns="+chr(34)+"http://ns.adobe.com/xfdf/"
"+chr(34)+" xml:space="+chr(34)+"preserve"+chr(34)+">"
lTerLin = " <f href="+chr(34)+ lRutaPlantilla +chr(34)+"/>"
lCuarLin = " <fields>"
InitFile(lRutaXFDF,lPrimLin)
ConcatFile(lRutaXFDF,lSegLin )
ConcatFile(lRutaXFDF,lTerLin )
```

```

ConcatFile(lRutaXFDF,lCuarLin )
//Por cada campo
sCampo = "ID_CAMPO_1" //El identificador del campo en el formulario
pdf
sValor = "27" //El valor que queramos meter en el fichero, normalmente
se trae de TI.ITEM
lLinCampo = "<field
name="+chr(34)+sCampo+chr(34)+"><value>"+sValor+"</value></field>"
ConcatFile(lRutaXFDF,lLinCampo)
//Finalizar el fichero
lPenUltLin = " </fields>"
lUltLin     = "</xfdf>"
ConcatFile(lRutaXFDF,lPenUltLin )
ConcatFile(lRutaXFDF,lUltLin )

```

A continuación se muestra un ejemplo de un archivo XFDF:

EJEMPLO

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xfdf xmlns="http://ns.adobe.com/xfdf/" xml:space="preserve">
<f href="C:\MiDirectorio\MiDocumento\mi_informe.pdf"/>
  <fields>
    <field name="Apellido1"><value>López</value></field>
    <field name="Apellido2"><value>Romero</value></field>
    <field name="Nombre"><value>Alberto</value></field>
    <field name="Dir1"><value>Centro Europa Empresarial - Edf. Roma</
value></field>
    <field name="Dir2"><value>C/ Rozabella, 8</value></field>
    <field name="CodPost"><value>28230</value></field>
    <field name="Ciudad"><value>Las Rozas</value></field>
    <field name="Pais"><value>España</value></field>
  </fields>
</xfdf>

```

Llamar al método PRINT_PDF

El método PRINT_PDF del Meta4Object PRINT_PDF manda a imprimir el archivo XFDF o XFA utilizando los siguientes argumentos requeridos:

1. **AI_XFDF_FILE_PATH**: la ruta absoluta al archivo a imprimir.
2. **AI_COPY_NUM**: el número de copias a imprimir.
3. **AI_CONCURRENT_COPY_NUM**: el número de copias a imprimir concurrentemente en la cola de la impresora. Por ejemplo, si el valor es 3 significa que en un momento dado sólo habrán 3 archivos con datos imprimiéndose a la vez, y cuando se libera uno se va abriendo otro archivo con datos. De este modo se evita que no se tengan muchas copias imprimiéndose concurrentemente y abiertas n instancias del programa que imprime, el Acrobat Reader, por ejemplo.
4. **AI_PRINTER_NAME**: el nombre de impresora donde se debe ejecutar el proceso.

Uso del API para modificar documentos PDF

Para poder modificar documentos PDF se ofrece el Meta4Object PDF. En los siguientes apartados se describe la funcionalidad que se ofrece en este Meta4Object:

- Estructura de nodo PDF
- Propiedades de los métodos anteriores

Estructura de nodo PDF

En la estructura de nodo denominada PDF, perteneciente al nodo SCH_DOCUMENT_EDITOR, se encuentra la funcionalidad para poder modificar un documento PDF.

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- **OPEN_DOCUMENT**: este método nos permite abrir un documento PDF.
- **GET_PAGE_NUMBER**: este método nos permite recuperar el número de páginas del documento previamente abierto.
- **GET_PAGE_HEIGHT**: este método permite obtener la altura de la página actual.
- **GET_PAGE_WIDTH**: este método permite obtener el ancho de la página actual.
- **SCALE_PAGE**: este método sirve para escalar la página actual.
- **SET_CURRENT_PAGE**: este método permite establecer la página actual del documento PDF sobre la que se va a trabajar.
- **SET_FONT**: este método permite establecer características de la fuente.
- **SET_COLOR**: este método permite establecer el color de la fuente.

-
- **ADD_TEXT**: este método permite añadir un texto en un área.
 - **ADD_IMAGE**: este método permite añadir una imagen en unas coordenadas.
 - **ADD_NEW_PAGE**: este método permite añadir una nueva página.
 - **ADD_RECTANGLE**: este método permite añadir un rectángulo.
 - **CHECK_STATUS**: este método sirve para verificar el estado del documento.
 - **PROC_XFDF**: este método permite generar un archivo PDF a partir de un fichero de tipo XFDF (XML Forms Data Format).
 - **PROC_XFA**: este método permite generar un archivo PDF a partir de un fichero de tipo XFA (XML Forms Architecture).
 - **PROTECT_PDF**: este método permite proteger un archivo PDF por medio de una contraseña.
 - **CLOSE_DOCUMENT**: este método permite guardar y cerrar un documento previamente abierto.

En los siguientes apartados se describen estos métodos con más detalle.

Método OPEN_DOCUMENT

Este método permite abrir un documento PDF para ser modificado. Recibe como argumentos:

- **AI_TYPE_FILE**: tipo de fichero.
- **AI_SOURCE_FILE_PATH**: ruta completa al fichero que se quiere modificar.
- **AI_TARGET_FILE_PATH**: ruta completa al fichero de salida, sobre el que se volcarán los cambios.

El tipo de fichero, en el caso de documentos PDF, debe ser la cadena PDF.

El documento origen y destino pueden ser iguales o pueden ser diferentes, según si el usuario quiere guardar los cambios en el fichero original o en otro distinto. Ambos ficheros, tanto el de entrada como el de salida, deben existir. Lógicamente, el fichero de salida debe tener permisos de escritura.

Método GET_PAGE_NUMBER

Este método permite obtener el número de páginas de un documento previamente abierto.

Método GET_PAGE_HEIGHT

Este método permite obtener la altura de la página actual.

Método GET_PAGE_WIDTH

Este metodo sirve para obtener la anchura de la página actual.

Método SCALE_PAGE

Este método sirve para ajustar la escala de la página actual. Los argumentos que recibe son los siguientes:

- **AI_X_POS**: coordenada x.
- **AI_Y_POS**: coordenada y.
- **AI_FACTOR**: factor de escala aplicado.

Método SET_CURRENT_PAGE

Este método permite establecer la página actual de un documento previamente abierto. La página actual será la página sobre la que se realizarán las futuras modificaciones.

- **AI_PAGE_NUM**: página actual.

Método SET_FONT

Este método permite establecer diferentes características de la fuente.

- **AI_FONT_NAME**: nombre de la fuente (Helvetica, Courier, etc.) o ruta hasta el archivo de la fuente (C:\Windows\Fonts\ahronbd.ttf, etc.).
- **AI_FONT_SIZE**: tamaño de la fuente.
- **AI_FONT_STYLE**: estilo de la fuente (negrita, cursiva, subrayado, etc.).

Método SET_COLOR

Este método permite establecer el color de la fuente.

- **AI_RED**: el valor de rojo del nuevo color.
- **AI_GREEN**: el valor de verde del nuevo color.
- **AI_BLUE**: el valor azul del nuevo color.

Método ADD_TEXT

Este método permite añadir un texto en un área especificada. Se permite que el texto sea un texto multilínea. Los argumentos son:

- **AI_TEXT**: texto a escribir.
- **AI_X_POS**: coordenada x.
- **AI_Y_POS**: coordenada y.

-
- **AI_WIDTH**: ancho.
 - **AI_HEIGHT**: alto.
 - **AI_LEADING**: distancia entre líneas.
 - **AI_HALIGNMENT**: alineamiento.
 - **AI_ROTATION**: rotación.

Las coordenadas x, y, el ancho y el alto están referenciados a la esquina inferior izquierda de la página.

Método ADD_IMAGE

Este método permite añadir una imagen en una posición del documento. Los argumentos que reciben son los siguientes:

- **AI_FILE_PATH**: ruta al fichero que contiene la imagen que se quiere insertar.
- **AI_X_POS**: coordenada x.
- **AI_Y_POS**: coordenada y.
- **AI_WIDTH**: ancho de la imagen.
- **AI_HEIGHT**: alto de la imagen.

Método ADD_NEW_PAGE

Este método permite añadir una nueva página a un documento previamente abierto. Los argumentos que recibe son los siguientes:

- **AI_WIDTH**: ancho de la nueva página.
- **AI_HEIGHT**: alto de la nueva página.

Método ADD_RECTANGLE

Este método se emplea para añadir un rectángulo en un documento previamente abierto. Recibe los siguientes elementos:

- **AI_X_POS**: coordenada x del rectángulo.
- **AI_Y_POS**: coordenada y del rectángulo.
- **AI_WIDTH**: anchura del rectángulo.
- **AI_HEIGHT**: altura del rectángulo.
- **AI_LINE_WIDTH**: ancho de línea aplicado.
- **AI_LINE_STYLE**: estilo de línea aplicado.

Método CHECK_STATUS

Este método sirve para verificar el estado del documento.

Método PROC_XFDF

Este método permite generar un archivo PDF a partir de un fichero de tipo XFDF (XML Forms Data Format). Recibe los siguientes argumentos:

- **AI_INPUT_XFDF**: fichero XFDF a convertir.
- **AI_INPUT_PDF**: fichero PDF al que aplicar la transformación.
- **AI_OUTPUT_PDF**: fichero PDF transformado.

Método PROC_XFA

Este método sirve para generar un archivo PDF a partir de un fichero de tipo XFA (XML Forms Architecture). Recibe como argumentos:

- **AI_INPUT_XFA**: fichero XFA a convertir.
- **AI_INPUT_PDF**: fichero PDF al que aplicar la transformación.
- **AI_OUTPUT_PDF**: fichero PDF transformado.

Método PROTECT_PDF

Este método permite generar los informes de Meta4 en ficheros PDF protegidos por contraseña, de forma que solo los puedan abrir aquellos usuarios que dispongan de la contraseña. Recibe los siguientes argumentos:

- **AI_USER_PASSWORD**: contraseña de usuario.
- **AI_INPUT_PDF**: nombre del fichero de entrada.
- **AI_OUTPUT_PDF**: nombre del fichero de salida.

A continuación se ofrece un ejemplo típico de llamada a este método:

EJEMPLO

```
PDF!SCH_DOCUMENT_EDITOR.PROTECT_PDF("username",  
  "Z:\m4pdf_engine\filename.pdf", "Z:\m4pdf_engine\filename.pdf")
```

Método CLOSE_DOCUMENT

Este método permite guardar y cerrar un documento previamente abierto. Una

vez que un documento se ha cerrado ya no se puede seguir trabajando sobre el mismo.

Propiedades de los métodos anteriores

A continuación se describen los valores de constantes que pueden usarse para dar valores a los argumentos de los métodos anteriores y facilitar así su uso:

1. Propiedades de tipo de documento:

PDF_TYPE = "PDF"

TYPE_DOCUMENT

2. Propiedades de alineamiento de texto:

ALIGN_BOTTOM: Valor por defecto: 6

ALIGN_CENTER: Valor por defecto: 1

ALIGN_JUSTIFIED: Valor por defecto: 3

ALIGN_JUSTIFIED_ALL: Valor por defecto: 8

ALIGN_LEFT: Valor por defecto: 0

ALIGN_RIGHT: Valor por defecto: 2

ALIGN_TOP: Valor por defecto: 4

3. Propiedades de estilo de fuentes:

BOLD: Valor por defecto: 1

BOLDITALIC: Valor por defecto: 3

ITALIC: Valor por defecto: 2

NORMAL: Valor por defecto: 0

UNDERLINE: Valor por defecto: 4

4. Posibles fuentes:

"Courier"

"Courier-Bold"

"Courier-Oblique"

"Courier-BoldOblique"

"Helvetica"

"Helvetica-Bold"

"Helvetica-Oblique"

"Helvetica-BoldOblique"

"Symbol"

"Times-Roman"
"Times-Bold"
"Times-Italic"
"Times-BoldItalic"
"ZapfDingbats"

Uso del API para visualizar documentos en Word y convertirlos a PDF

En Rich Web, es posible generar documentos Word de MailMerge incrustados en una presentación. Para ello se dispone de una tarea que automatiza la generación de documentos WORD y PDF. Esta funcionalidad se puede utilizar ejecutando la tarea SRTC_MANAGE_MERGE_DOC o a través de la presentación del mismo nombre SRTC_MANAGE_MERGE_DOC.

Tanto la tarea como la presentación tienen los mismos parámetros de entrada y salida que se describen en los siguientes apartados:

- Parámetros de entrada
- Parámetros de salida

Parámetros de entrada

Los parámetros de entrada son los siguientes:

- **FILENAME:** parámetro obligatorio. Cadena con la ruta y el nombre del fichero de datos para la combinación de datos de MailMerge.
- **ID_TEMPLATE:** parámetro obligatorio. Cadena con el identificador de plantilla a utilizar para el MailMerge. Esta plantilla debe haber sido guardada previamente en PeopleNet desde la opción de "Archivos de plantillas".
- **CONVERT_PDF:** parámetro opcional. Valor numérico que indica si se desea convertir a PDF el documento resultante de la combinación de datos. Valores posibles:
 - 1: Convertir a PDF.
 - 0: No Convertir (Defecto).
- **SHOW_AFTER_CONVERT:** parámetro opcional. Valor numérico que por defecto tiene el valor 0 y sirve para cambiar la secuencia de ejecución. En la secuencia normal de ejecución primero se ejecuta la combinación de datos, se genera el documento resultante y se muestra en el visor, donde se puede modificar y guardar con o sin cambios en el gestor documental. Si se ha indicado en el parámetro CONVERT_PDF, se guardará en el gestor

documental en formato PDF. Si a este parámetro se le pone valor 1 la conversión a PDF se realizará antes de visualizarse, por tanto se visualizará el documento resultante en formato PDF sin posibilidad de modificaciones.

- **TITLE:** parámetro opcional. Cadena con el título a establecer al documento que se guarda en el gestor documental.
- **AUTHOR:** parámetro opcional. Cadena con el autor a establecer al documento que se guarda en el gestor documental.
- **DESCRIPTION:** parámetro opcional. Cadena con la descripción a establecer al documento que se guarda en el gestor documental.
- **CLASS:** parámetro opcional. Cadena con el tipo de documento que se guarda en el gestor documental. En caso de activar la gestión de anexos, se refiere al documento anexo.
- **ENABLE_ATTACH_MNG:** parámetro obligatorio. Valor numérico que indica si se quiere activar o no la gestión de anexos:
 - 0: Desactivar gestión de anexos (valor por defecto).
 - 1 Activar gestión de anexos.
- **PARENT_CLASS:** parámetro opcional. Cadena con el tipo de documento padre que se guarda en el gestor documental.
- **PARENT_TITLE:** parámetro opcional. Cadena con el título del documento padre que se guarda en el gestor documental.

Parámetros de salida

Los parámetros de salida son los siguientes:

- **RESULT:** valor True si no ha habido errores y el documento se ha guardado en el gestor documental y False si ha habido algún error o se cancela.
- **ID_DOCUMENT:** identificador del documento que se ha guardado en el gestor documental. Sólo vendrá relleno si RESULT = True. En caso de activar la gestión de anexos, se refiere al identificador del documento anexo.
- **ID_DOCUMENT_PARENT:** identificador del documento padre que se ha guardado en el gestor documental. Sólo vendrá relleno si RESULT = True y se activó la gestión de anexos (ENABLE_ATTACH_MNG=1).

Recuerde que, cuando se trabaja desde presentaciones, todos los parámetros deberán ser establecidos como valores de contexto.

Ejemplo de llamada desde una presentación

Para ilustrar este caso práctico, en primer lugar procedemos a dar valores a los

parámetros obligatorios y a las opciones deseadas. En el ejemplo se indica que se quiere conversión a PDF del documento, que se visualice el documento después de convertirlo y que se active la gestión de anexos.

EJEMPLO

```
BEGIN Action_set setContextValFile
    Set = "*C*FILENAME := c:\fichero.txt"
END

BEGIN Action_set setContextValTemplate
    Set = "*C*ID_TEMPLATE := MERGE_TEST"
END

BEGIN Action_set setContextValTitle
    Set = "*C*TITLE := Titulo del documento"
END

BEGIN Action_set setContextValConvertPDF
    Set = "*C*CONVERT_PDF := [*F*1]"
END

BEGIN Action_set setContextValShowAfterConvert
    Set = "*C*SHOW_AFTER_CONVERT := [*F*1]"
END

BEGIN Action_set setContextValAttachMng
    Set = "*C*ENABLE_ATTACH_MNG := [*F*1]"
END
```

A continuación se llama a la presentación:

```
BEGIN Extends exVManageMergeDoc
    Idinstance = "SRTC_MANAGE_MERGE_DOC"
END
```

Por último, se recoge el resultado:

```
BEGIN Action_message msg11
    Text = "res: [*C*RESULT]" ? True: Se ha guardado / False: No
se ha guardado
END

BEGIN Action_message msg12
    Text = "doc: [*C*ID_DOCUMENT], doc padre:
[*C*ID_DOCUMENT_PARENT] " ? Identificador del documento si se ha
guardado.
END
```

Utilidad de conversión de Word a PDF

Adicionalmente, PeopleNet pone a disposición de los desarrolladores el Meta4Object SCH_DOCUMENT_CONVERTER para proporcionar, mediante LN4, acceso a la conversión de archivos y, en nuestro caso particular, de DOC/DOCX a PDF.

Posee las estructuras de nodo SCH_DOCUMENT_CONVERTER (que contiene exclusivamente el método CONVERT) y SCH_JAVA_API (de uso interno).

El método CONVERT recibe como parámetros las rutas completas del documento de entrada (DOC/DOCX en nuestro caso) y del documento de salida (PDF en nuestro caso). Devuelve como código de retorno M4_SUCCESS si la conversión ha sido satisfactoria y M4_ERROR en caso contrario.

Uso del API para mostrar y guardar un documento en el gestor documental

Para invocar la funcionalidad de mostrar y guardar un documento en el gestor documental desde cualquier presentación, seleccione la tarea genérica BP_WF_SHOW_ADD_DOC_ACTIVITY.

A continuación, como parámetros de entrada, pase la siguiente información:

- Nombre y ruta del documento
- Título del documento
- Autor
- Descripción
- Tipo de documento

El sistema devolverá como parámetros de salida los siguientes datos:

- Si se ha guardado el documento
- ID del documento

Generalmente, el sistema mostrará el contenido del documento incrustado en la pantalla, aunque puede darse el caso de que se ejecute una aplicación externa para visualizar el documento. En cualquiera de los dos casos, el sistema ofrece dos posibilidades:

- **Guardar:** esta opción permite almacenar el documento en el gestor documental.
- **Salir:** haga clic en esta opción si prefiere cerrar la ventana sin realizar ninguna operación.

Uso del API para activar la gestión de anexos

La gestión de anexos está disponible desde clientes Rich Web y se puede utilizar ejecutando las tareas que permiten visualizar y guardar un documento en el gestor documental (BP_SHOW_ADD_DOC_ACTIVITY o SRTC_MANAGE_MERGE_DOC). Mediante esta funcionalidad, se permite indicar el documento del que es anexo el documento o carta que se está visualizando y grabar ambos en el gestor documental. La funcionalidad también está disponible a través de sus respectivas presentaciones SRTC_WF_EASY_TASK_SHOW_ADD_DOC y SRTC_MANAGE_MERGE_DOC.

Se ha añadido un nuevo parámetro de entrada, ENABLE_ATTACH_MNG, que habilitará un panel donde el usuario podrá indicar el documento del que es anexo y un parámetro nuevo de salida, ID_DOCUMENT_PARENT, donde se devuelve el identificador del documento padre que se ha guardado en el gestor documental.

La configuración de la gestión de anexos relativa a la tarea SRTC_MANAGE_MERGE_DOC es similar y su configuración se explica en el apartado [Uso del API para visualizar documentos en Word y convertirlos a PDF](#).

En los siguientes apartados se describen todos los parámetros correspondientes a la tarea BP_SHOW_ADD_DOC_ACTIVITY:

- Parámetros de entrada
- Parámetros de salida



Cuando un usuario está ejecutando las tareas que permiten visualizar y guardar un documento en el gestor documental (BP_SHOW_ADD_DOC_ACTIVITY o SRTC_MANAGE_MERGE_DOC) y el desarrollador ha habilitado la opción ENABLE_ATTACH_MNG = 1, la pantalla de **Visualizar y guardar documento en el gestor documental** presenta en la parte inferior un panel donde se puede establecer el nombre del documento del que es anexo. Para este documento se tendrá que indicar la ruta donde se encuentra. Cuando el usuario pulse "Guardar" el sistema guardará el documento visualizado y su anexo en el gestor documental.

Parámetros de entrada

Los parámetros de entrada son los siguientes:

- **FILENAME:** parámetro obligatorio. Cadena con la ruta y el nombre del archivo a mostrar.
- **TITLE:** parámetro opcional. Cadena con el título a establecer en el documento que se guarda en el gestor documental.
- **AUTHOR:** parámetro opcional. Cadena con el autor a establecer en el documento que se guarda en el gestor documental.
- **DESCRIPTION:** parámetro opcional. Cadena con la descripción a establecer en el documento que se guarda en el gestor documental.
- **CLASS:** parámetro opcional. Cadena con el tipo de documento que se guarda en el gestor documental. En caso de activar la gestión de anexos se refiere al documento anexo.
- **ENABLE_ATTACH_MNG:** parámetro obligatorio. Valor numérico que indica si se quiere activar o no la gestión de anexos: 0: Desactivar gestión de anexos (valor por defecto); 1: Activar gestión.
- **PARENT_CLASS:** parámetro opcional. Cadena con el tipo de documento padre que se guarda en el gestor documental.
- **PARENT_TITLE:** parámetro opcional. Cadena con el título del documento padre que se guarda en el gestor documental.

Parámetros de salida

Los parámetros de salida son los siguientes:

- **RESULT:** valor True si no ha habido errores y el documento se ha guardado en el gestor documental y False si ha habido algún error o se cancela.
- **ID_DOCUMENT:** identificador del documento que se ha guardado en el gestor documental. Sólo vendrá relleno si RESULT = True. En caso de activar la gestión de anexos, se refiere al identificador del documento anexo.
- **ID_DOCUMENT_PARENT:** identificador del documento padre que se ha guardado en el gestor documental. Sólo vendrá relleno si RESULT = True y se ha activado previamente la gestión de anexos (ENABLE_ATTACH_MNG=1).

Recuerde que, cuando se trabaja desde presentaciones, todos los parámetros deberán ser establecidos como valores de contexto.

Ejemplo de llamada desde una presentación

Para ilustrar este ejemplo de uso, empezamos por dar valores a los parámetros obligatorios y al título del documento.

EJEMPLO

```
BEGIN Action_set setContextValFile
    Set = "*C*FILENAME := c:\fichero.txt"
END

BEGIN Action_set setContextValTitle
    Set = "*C*TITLE := Titulo del documento"
END

BEGIN Action_set setContextValAttachMng
    Set = "*C*ENABLE_ATTCH_MNG := [*F*1]"
END
```

A continuación se llama a la presentación:

```
BEGIN Extends exVManageMergeDoc
    Idinstance = "SRTC_WF_EASY_TASK_SHOW_ADD_DOC"
END
```

Por último, se recoge el resultado:

```
BEGIN Action_message msg11
    Text = "res: [*C*RESULT]" ? True: Se ha guardado / False: No
se ha guardado
END

BEGIN Action_message msg12
    Text = "doc: [*C*ID_DOCUMENT], doc parent
[*C*ID_DOCUMENT_PARENT] " ? Identificador de los documentos si se han
guardado.
END
```

Uso del API para personalizar la información de firma en los documentos PDF

PeopleNet pone a disposición de los desarrolladores el Meta4Object SRTC_MERGE_DOC_AND_SIGN para personalizar la información de firma en documentos PDF. En los siguientes apartados se describe la funcionalidad que aporta este Meta4Object:

- Estructura de nodo SRTC_MERGE_DOC_AND_SIGN
- Proceso para cambiar los valores por defecto de las propiedades de la estructura de nodo
- Proceso para cambiar la información de cada firmante

Estructura de nodo SRTC_MERGE_DOC_AND_SIGN

En la estructura de nodo SRTC_MERGE_DOC_AND_SIGN se encuentra la funcionalidad necesaria para poder generar un documento PDF con sus datos de firma.

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- **ADD_LEFT_MARGIN_TEXT:** este método se encarga de añadir el texto en vertical en el margen izquierdo de la página.
- **ADD_SIGNERS_DATA:** este método se encarga de añadir la información de cada firmante. También calcula la coordenada y en la que se debe posicionar cada firmante.
- **GENERATE_SIGNER_DATA:** genera el texto correspondiente a la información de un firmante.
- **GET_SCALE_FACTOR:** calcular el factor de escala para una página para poder insertar el texto de los firmantes.
- **GET_SIGN_DATA_HEIGHT:** calcula la altura del área de los firmantes.
- **MERGE_DOC_AND_SIGN:** genera un blob donde se deja el fichero pdf con la información de la firma.

En los siguientes apartados se describen algunos de estos métodos con más detalle. En esta estructura de nodo también se tienen las siguientes propiedades:

- **PR_FONT_NAME:** nombre de la fuente. Valor por defecto: Helvética.
- **PR_FONT_SIZE:** tamaño de la fuente. Valor por defecto: 8.
- **PR_FONT_STYLE:** estilo de la fuente. Valor por defecto: 1.
- **PR_RED:** composición de rojo del color de la fuente. Valor por defecto: 0.

- **PR_GREEN**: composición de verde del color de la fuente. Valor por defecto: 0.
- **PR_BLUE**: composición de azul del color de la fuente. Valor por defecto: 0.
- **PR_PAGE_X_POS**: coordenada x donde se posiciona la página escalada. Valor por defecto: 10.
- **PR_PAGE_Y_POS**: coordenada y donde se posiciona la página escalada.
- **PR_TEXT_X_POS**: coordenada x donde se posiciona el área de los firmantes. Valor por defecto: 20.
- **PR_TEXT_Y_POS**: coordenada y donde se posiciona el área de cada firmante.
- **PR_TEXT_HEIGHT**: altura del área de cada firmante. Valor por defecto 5.
- **PR_TEXT_WIDTH**: ancho del área de cada firmante. Valor por defecto 180.
- **PR_TEXT_ALIGNMENT**: alineamiento horizontal de cada área de firmantes. Valor por defecto 0.
- **PR_TEXT_ROTATION**: rotación de cada área de firmantes. Valor por defecto 0.
- **PR_TEXT_Y_INCR**: distancia entre las áreas de cada firmante y la cabecera. Valor por defecto 1.
- **PR_TEXT_Y_BOTTOM**: distancia entre los datos de los firmantes y el final de página. Valor por defecto 10.
- **PR_TEXT_Y_TOP**: distancia entre la cabecera y la página original escalada. Valor por defecto 1.
- **PR_TEXT_LEADING**: distancia entre las distintas líneas de cada área de firmante. Valor por defecto 5.
- **PR_LEFT_MARGIN_TEXT_LEADING**: distancia entre las distintas líneas del área del margen izquierdo. Valor por defecto 5.
- **PR_LEFT_MARGIN_TEXT**: texto que se muestra en el margen izquierdo de la página.
- **PR_LEFT_MARGIN_TEXT_X_POS**: coordenada x del área del margen izquierdo de la página. Valor por defecto 2.
- **PR_LEFT_MARGIN_TEXT_Y_POS**: coordenada y del área del margen izquierdo de la página. Valor por defecto 50.
- **PR_LEFT_MARGIN_TEXT_HEIGHT**: altura del área del margen izquierdo de la página. Valor por defecto 200.
- **PR_LEFT_MARGIN_TEXT_WIDTH**: ancho del área del margen izquierdo de la página. Valor por defecto 7.
- **PR_LEFT_MARGIN_TEXT_ALIGNMENT**: alineamiento horizontal del área del margen izquierdo de la página. Valor por defecto 1.

-
- **PR_LEFT_MARGIN_TEXT_ROTATION**: rotación del área del margen izquierdo de la página. Valor por defecto 90.
 - **PR_SIGN_HEADER_ALIGNMENT**: alineamiento horizontal del texto de la cabecera. Valor por defecto 0.
 - **PR_SIGN_HEADER_HEIGHT**: altura del texto de la cabecera. Valor por defecto 5.
 - **PR_SIGN_HEADER_ROTATION**: rotación del texto de la cabecera. Valor por defecto 0.
 - **PR_SIGN_HEADER_TEXT**: texto que se muestra en la cabecera.
 - **PR_SIGN_HEADER_WIDTH**: ancho del área del texto de la cabecera. Valor por defecto 180.
 - **PR_SIGN_HEADER_X_POS**: coordenada x del texto de la cabecera. Valor por defecto 10.
 - **PR_SIGN_HEADER_Y_POS**: coordenada y donde se posiciona el texto de la cabecera.

Proceso para cambiar los valores por defecto de las propiedades de la estructura de nodo

Si se diese la circunstancia en la que el usuario quiere modificar algunos/todos los valores de las propiedades del Meta4Object, la recomendación es hacer un nuevo método en el que se establezcan los valores deseados y conectarlo con el tipo de precedencia “Antes” al método MERGE_DOC_AND_SIGN.

Proceso para cambiar la información de cada firmante

Para cada firmante, la información que se muestra es la siguiente:

- Nombre y apellidos de la persona.
- Fecha de la firma.
- Identificador del certificado.

En el caso de que el usuario quiera añadir o eliminar información, debe sobrescribir el método GENERATE_SIGNER_DATA. Este método compone la cadena que se va a mostrar como información del firmante. Si se quiere añadir otro tipo de información como, por ejemplo, el puesto, se puede añadir en este método.

Este método recibe como argumentos:

- **AI_ID_PERSON**: ID_Person.
- **AI_NAME**: gb name.

- **AI_DT_SIGN**: fecha firma.
- **AI_ID_CERTIFICATE**: ID certificado.

Uso del API para crear parámetros de aplicación

PeopleNet dispone de un API para la creación de parámetros de aplicación. Este API se encuentra en el Meta4Object EDT_PARAMS y consta de dos métodos, uno para la creación de claves y otro para la asignación de valores.

Uno de los usos funcionales de este API puede ser la configuración de las sustituciones de texto. Este proceso se realiza añadiendo parámetros de aplicación con la palabra clave de la sustitución bajo la ruta FORMS/CHANGETEXT.

A continuación se muestra un ejemplo de uso:

1. Método para la creación de un parámetro:

```
EDT_PARAMS!SAV_APPLICATION.ADDKEY (ID_APPLICATION,
ID_SECTION, ID_KEY)
```

EJEMPLO

```
sKey = "SCO_PERSON_BUFFER_ESP_1"

If EDT_PARAMS!SAV_APPLICATION.LOAD_APPLICATION ("FORMS",
"CHANGETEXT", sKey ) = M4_ERROR Then

    EDT_PARAMS!SAV_APPLICATION.ADD_KEY( "FORMS",
    "CHANGETEXT", sKey )
EndIf

EDT_PARAMS!Persist()
```

2. Método para asignar valor a la clave creada:

```
EDT_PARAMS!SAV_APPLICATION.SETVALUE (ID_APPLICATION,
ID_SECTION, ID_KEY, ID_CONCEPT, VALUE_CONCEPT, VALUE)
```

EJEMPLO

```
sKey = "SCO_PERSON_BUFFER_ESP_1"
sConcept = "ID_ORGANIZATION"
sValueConcept = "S001"
```

```
sValue = "sco_person_buffer_1=Grupo sanguíneo"

EDT_PARAMS!SAV_APPLICATION.SET_VALUE("FORMS", "CHANGETEXT", sKey
,sConcept , sValueConcept, sValue)

EDT_PARAMS!Persist()
```

Uso del API para acceder a las ejecuciones de una planificación multisocietaria

PeopleNet pone a disposición de los desarrolladores un API que permite acceder a las ejecuciones de una planificación multisocietaria y consultar cómo han terminado. Entre otras cosas, permitirá saber si ha finalizado correctamente, el estado concreto de los procesos terminados y los recursos (archivos) asociados a los mismos.

El API se sitúa en el Meta4Object `SRTC_JS_SCHEDULINGS_API` y permite tener accesibles las ejecuciones de una tarea planificada de forma multisocietaria. Para cada ejecución se debe tener disponible el estado de la ejecución, la sociedad de ejecución y sus recursos.

Por defecto, la sociedad de ejecución se toma del contexto de la planificación. Para los casos especiales en los que la sociedad de ejecución se establezca a través de un parámetro (por ejemplo, en la planificación múltiple de informes) se da una función para indicar que la sociedad se lea de un parámetro.

En los siguientes apartados se describe la funcionalidad que se ofrece en este Meta4Object:

- Estructura de nodo `SRTC_JS_SCHEDULINGS_API`
- Estructura de nodo `SRTC_SCHEDULINGS_BY_PARAM`
- Estructura de nodo `SRTC_SCHEDULING_RESOURCES`

Estructura de nodo `SRTC_JS_SCHEDULINGS_API`

La estructura de nodo `SRTC_JS_SCHEDULINGS_API` contiene las funciones generales del API.

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- Método CALC_ORGANIZATION_FROM_PARAM: este método permite establecer que el cálculo de la sociedad de ejecución se lea de un parámetro de la tarea en vez de leerse el contexto de la planificación que es el defecto.
- Método LOAD_SCHED_EXE_BY_TASK_PARAM: este método sirve para cargar las ejecuciones de una tarea en el nodo SRTC_SCHEDULINGS_BY_PARAM.
- Método DELETE_SCHED_TASK: este método permite borrar una planificación.

Método CALC_ORGANIZATION_FROM_PARAM

Este método se utiliza para establecer que el cálculo de la sociedad de ejecución se lea de un parámetro de la tarea en lugar de leerse el contexto de la planificación que es el defecto. El método tiene definidos los siguientes parámetros:

- **ARG_PARAM_NAME**: identificador del parámetro de la tarea que puede contener la sociedad.
- **ARG_ORG_SEP**: cadena con el separador del parámetro de la sociedad dentro del valor del parámetro.

Así, por ejemplo, en los informes de ejecución múltiple la sociedad va en el parámetro PARAM_REPORT con el separador @@.

Método LOAD_SCHED_EXE_BY_TASK_PARAM

Este método sirve para cargar las ejecuciones de una tarea en el nodo SRTC_SCHEDULINGS_BY_PARAM. Incluye los siguientes parámetros:

- **ARG_ID_TASK**: identificador de la tarea a chequear. Se corresponderá con la tarea utilizada para la planificación del informe.
- **ARG_PARAM_NAME**: identificador del parámetro de la tarea que va a contener un valor único para que se pueda identificar.
- **ARG_PARAM_VALUE**: valor del parámetro. Este valor deberá ser único para no mezclar ejecuciones diferentes.
- **ARG_ID_SUBTASK**: identificador de la subtarea de la tarea compuesta ARG_ID_TASK por la que filtrar.

Los posibles códigos de retorno son: M4_SUCCESS, si la tarea se ha cargado correctamente, y M4_ERROR en caso contrario.

Asimismo, se deja accesible en el nodo SRTC_SCHEDULINGS_BY_PARAM la información de las ejecuciones.

Método DELETE_SCHED_TASK

Este método se utiliza para borrar una planificación. Incluye el siguiente parámetro:

- **ARG_ID_SCHED_TASK**: identificador de la planificación a borrar.

Los posibles códigos de retorno son: M4_SUCCESS, si el borrado de la planificación se ha completado con éxito, y M4_ERROR en caso contrario.

Estructura de nodo SRTC_SCHEDULINGS_BY_PARAM

La estructura de nodo SRTC_JS_SCHEDULINGS_BY_PARAM contiene todas las ejecuciones solicitadas.

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- Método LOAD_RESOURCES: método de registro para cargar los recursos de la ejecución en la que se encuentra el nodo posicionado

Método LOAD_RESOURCES

Método de registro para cargar los recursos de la ejecución en la que se encuentra el nodo posicionado. Los recursos se cargarán en el nodo SRTC_SCHEDULING_RESOURCES.

El método tiene definidos los siguientes elementos:

- **ID_SCHED_TASK**: identificador de la planificación
- **ID_TASK_EXE**: identificador de la ejecución
- **STATUS**: cadena con el estado de la ejecución. Los posibles valores son:
 - “0” : Ejecutado sin errores
 - “1” : Esperando
 - “2” : En ejecución
 - “3” : Cancelando
 - “4” : Expirado
 - “6” : Interrumpido
 - “7” : Cancelado
 - “10” : Abortando
 - “11” : Abortado
 - “20” : Ejecutado con errores
- **ID_ORGANIZATION**: identificador de la sociedad de ejecución

Estructura de nodo SRTC_SCHEDULING_RESOURCES

La estructura de nodo SRTC_SCHEDULING_RESOURCES contiene todas los recursos asociados a una ejecución.

Esta estructura de nodo incluye el siguiente elemento:

- RESOURCE: ítem blob con el recurso.

Ejemplo de consulta del estado de una ejecución

En este apartado se ofrece un ejemplo de uso de este API para consultar el estado de ejecución de una planificación:

EJEMPLO

```
// *****
// Method          : LOAD_SCHED_AND_RESOURCES
// Arguments:
//   ARG_ID_TASK:    Identificador de tarea planificada. (Tarea
//                   compleja)
//   ARG_TASK_PARAM_NAME: Identificador del parámetro de la tarea
//                   por el que filtrar
//   ARG_TASK_PARAM_VALUE: Valor del parámetro por el que filtrar
//   ARG_ID_SUBTASK : Id subtarea dentro de la tarea compleja por la
//                   que filtrar
// Description      : Dada una tarea compleja (ARG_ID_TASK) hay que
//                   cargar las ejecuciones de una o varias planificaciones
//                   identificadas por el valor de uno de sus parámetros
//                   (ARG_TASK_PARAM_NAME=ARG_TASK_PARAM_VALUE ).
// Se filtrarán por las ejecuciones de la subtarea: ARG_ID_SUBTASK
//
// *****

'Constantes de los estados
'0 Processed Ejecutado
'1 Waiting Esperando
'2 Processing En ejecución
'3 Cancelling Cancelando
'4 Expired Expirado
'6 Interrupted Interrumpido
'7 Cancelled Cancelado
```

```

'10 Aborting Abortando
'11 Aborted Abortado
'20 Ejecutado con errores

sResultFile = "c:\temp\"+ARG_ID_TASK + "_" + ARG_PARAM_VALUE + ".txt"

initFile(sResultFile,"Ejecución de la tarea: "+ARG_ID_TASK + "
Subtarea: " +ARG_ID_SUBTASK + " Parámetro: " + ARG_PARAM_NAME + "
Valor: " + ARG_PARAM_VALUE, M4_TRUE)

'Indicar que se calcule la sociedad a partir del parámetro del informe
'-----
--

SRTC_JS_SCHEDULINGS_API!SRTC_JS_SCHEDULINGS_API.CALC_ORGANIZATION_FRO
M_PARAM("PARAM_REPORT","@@")

'Solicitar la carga de las ejecuciones
'-----

iRet
=SRTC_JS_SCHEDULINGS_API!SRTC_JS_SCHEDULINGS_API.LOAD_SCHED_EXE_BY_TA
SK_PARAM(ARG_ID_TASK,ARG_PARAM_NAME,ARG_PARAM_VALUE,ARG_ID_SUBTASK)
if iRet = M4_SUCCESS then
    iExeCount =
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.Count()
    if iExeCount = 0 then
        messagebox("", "No hay ejecuciones que cumpla el filtro")
        return M4_ERROR
    endif

    'Para cada ejecución comprobar el estado y solicitar la
carga de recursos
    '-----
    -----

    for i = 0 to iExeCount -1
        SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.moveto(i)
        sIdSchedTask = ToString(
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.ID_SCHED_TASK,0)
        sIdExeTask= ToString(
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.ID_TASK_EXE,0)
        if SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.STATUS = "0"
then

```

```
ConcatFile(sResultFile , "Ejecución correcta para la sociedad :  
" +  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.ID_ORGANIZATION + "  
Id Planificación: " + sIdSchedTask + " Id Ejecución: " + sIdExeTask,  
M4_TRUE )  
  
if  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.LOAD_RESOURCES() =  
M4_SUCCESS then  
ConcatFile(sResultFile , " Recursos de la sociedad : " +  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.ID_ORGANIZATION,  
M4_TRUE )  
  
'Para cada recurso recuperar el archivo  
'-----  
  
for j = 0 to  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULING_RESOURCES.Count() -1  
  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULING_RESOURCES.moveto(j)  
ConcatFile(sResultFile , " Recurso: " +  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULING_RESOURCES.RESOURCE, M4_TRUE )  
next  
else  
ConcatFile(sResultFile , " Error obteniendo recursos de  
la sociedad : " +  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.ID_ORGANIZATION,  
M4_TRUE )  
endif  
else  
ConcatFile(sResultFile , "Ejecución incorrecta para la sociedad  
: " +  
SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.ID_ORGANIZATION + "  
Status: " + SRTC_JS_SCHEDULINGS_API!SRTC_SCHEDULINGS_BY_PARAM.STATUS  
+" Id Planificación: " + sIdSchedTask + " Id Ejecución: " +  
sIdExeTask , M4_TRUE )  
endif  
next  
else  
messagebox("", "No hay ejecuciones que cumpla el filtro")  
endif
```

Uso del API para transferir la información de rol-sociedad de una sociedad a otra

La aplicación dispone de un API para desarrolladores que permite transferir la información rol-sociedad del usuario asociado a un empleado desde una sociedad a otra. Esto puede ser útil, por ejemplo, para utilizarlo en la funcionalidad de cambio de sociedad de un empleado.

A partir de un identificador de persona y una sociedad de origen, se busca un usuario asociado para transferir los roles a la sociedad destino. Los roles válidos que se transferirán serán los establecidos según el/los tipos indicados o si no se establece un tipo, se transferirán todos.

Este método se encuentra definido en:

- Meta4Object: SRTC_SIMPLE_USER_MAINTENANCE
- Nodo: SRTC_USERS_API_FUNCTIONS
- Método TRANSFER_PERSON_ROLES_BET_ORG

Método TRANSFER_PERSON_ROLES_BET_ORG

Este método sirve para transferir los roles de una persona de una sociedad a otra. Los argumentos del método son:

- **ARG_ID_PERSON**: ID Persona a transferir (obligatorio)
- **ARG_SOURCE_ORG**: ID Sociedad origen (obligatorio)
- **ARG_TARGET_ORG**: ID Sociedad destino (obligatorio)
- **ARG_TRANSFER_DATE**: Fecha de transferencia (obligatorio)
- **ARG_ROLES_TYPE_LIST**: Lista de tipos de roles a ejecutar. Separada por comas.

Ejemplo: "1,2": Roles del SSE y SSM (opcional)

- "1": Roles del ESS (roles definidos en el parámetro:)
- "2": Roles del MSS (roles definidos en el parámetro:)
- "3": Roles de otro tipo (roles definidos en el parámetro:)
- **ARG_SHOW_ERRORS**: (opcional)
 - 1: Mostrar errores (defecto) (M4_TRUE en LN4)
 - 0: No mostrar errores (M4_FALSE en LN4)

- **MANAGER_TRANSFER_AS_EMPLOYEE [OUT]**: este argumento de salida indica si se ha transferido un responsable como empleado por no poderse hacer de forma automática. Sus posibles valores son M4_TRUE/ M4_FALSE.

La función retorna M4_SUCCESS si se ha realizado la transferencia de los roles con éxito y si no se hecho la transferencia se indica la causa indicando el tipo de error concreto.

Si hay errores se mete el mensaje de error en la fila en función de si se ha indicado que se muestren o no errores (ARG_SHOW_ERRORS) y se devuelve el código de error correspondiente:

- M4_SUCCESS: Operación realizada con éxito
- -1: Error en persistencia
- -2: Error, argumentos nulos
- -3: Sociedad origen no existe
- -4: Sociedad destino no existe
- -5: La sociedad origen es igual a la sociedad destino.
- -6: Sociedad origen y destino de distinto nivel
- -7: Usuario no existe
- -8: Existe más de un usuario asociado a la persona
- -9: No existen roles para ese usuario y sociedad origen que transferir

A continuación se ofrece un ejemplo de uso de este método:

EJEMPLO

Transferir de la persona "M10105" desde la sociedad "0001" a la sociedad "0002" con fecha "1/1/2011" los roles del ESS y del MSS ("1,2") y muestra errores.

```
iRet =  
SRTC_SIMPLE_USER_MAINTENANCE!SRTC_USERS_API_FUNCTIONS.TRAN  
SFER_PERSON_ROLES_BET_ORG("M10105","0001","0002",1/1/2011,"1,2"  
,M4_TRUE)
```

Uso del API para abrir usuarios del portal en una sociedad

La aplicación dispone de un método para abrir el usuario asociado a una persona con perfil de empleado o manager, en una sociedad y un rango de fechas. A partir de un identificador de persona, se busca el usuario asociado. El

usuario se abre sólo si está cerrado a la fecha de control.

Este método se encuentra definido en

- Meta4Object: SRTC_SIMPLE_USER_MAINTENANCE
- Nodo: SRTC_USERS_API_FUNCTIONS
- Método : OPEN_USER_BY_PERSON_WITH_ROLE

Los argumentos del método son:

- **ARG_ID_PERSON**: ID Persona asociado al usuario a abrir (obligatorio)
- **ARG_ID_ORGANIZATION**: identificador de la sociedad donde se le quiere dar permisos (obligatorio).
- **ARG_USER_TYPE**: (1: ESS, 2: MSS) tipo de usuario empleado/ responsable para asignarle los roles que corresponda (obligatorio).
- **ARG_USER_TYPE_ROLE**: identificador del rol con el que reabrir el usuario, en el caso de que haya varios roles para el tipo del empleado. Si no se pasa este argumento, se toma el rol que corresponda. Si se quiere reabrir como responsable y hay varios roles, se reabrirá como empleado y se indicará la situación en el parámetro de salida:
MANAGER_TRANSFER_AS_EMPLOYEE
- **ARG_DT_START**: fecha de apertura del usuario (obligatorio).
- **ARG_DT_END**: fecha de fin de apertura del usuario (obligatorio).
- **ARG_CLOSE_CHECK_DATE**: fecha de control de cierre del usuario. Normalmente se corresponde con la fecha de apertura (obligatorio).
- **ARG_SHOW_ERRORS**:
 - 1: Mostrar errores (defecto) (M4_TRUE en LN4).
 - 0: No mostrar errores (M4_FALSE en LN4).
- **MANAGER_TRANSFER_AS_EMPLOYEE [OUT]**: M4_TRUE/M4_FALSE. Indica si se ha transferido un responsable como empleado por no poder hacerse de forma automática.



El método permite la asignación al usuario de un perfil a futuro distinto del actual. Esta situación puede ocurrir, por ejemplo, cuando se da de baja a futuro un periodo de un usuario tipo mánager y luego se quiere dar de alta como empleado en un futuro posterior. El API permite convivir perfiles de empleado y mánager.

Se retorna M4_SUCCESS si se ha abierto el usuario con éxito; en caso contrario se indica la causa y se especifica el tipo de error concreto.

Si hay errores, se mete el mensaje de error en la fila en función de si se ha indicado que se muestren o no errores (ARG_SHOW_ERRORS) y se devuelve

el código de error correspondiente:

- 0: M4_SUCCESS: Operación realizada con éxito.
- 1: Error en persistencia.
- 2: Error, argumentos nulos.
- 3: Sociedad inválida.
- 4: Usuario no existe.
- 5: Más de un usuario asociado a la persona.
- 6: Usuario abierto a la fecha de control.
- 7: No se ha podido identificar el perfil a establecer. Caso de varios perfiles para el mismo tipo de usuario.
- 8: Perfil inválido. El valor del parámetro ARG_USER_TYPE_ROLE no se corresponde con ningún perfil válido para el tipo de usuario.
- 9: El usuario está bloqueado.

MANAGER_TRANSFER_AS_EMPLOYEE: 1; si un empleado se abre como empleado a pesar de haberse indicado como responsable. Esto se puede dar en los casos en los que haya varios roles de responsable y no se ha indicado uno concreto.

EJEMPLO

Abrir el usuario asociado de la persona "M1206" como empleado del portal

ID_PERSON = "M1206"

bShowErrors = M4_TRUE

dDtStart = Today()

dDtEnd = Today() + 100

iUserType = 1 'ESS

sIdOrganization = "DEM4"

bManagerOpenAsEmployee = M4_FALSE

iRet =

SRTC_SIMPLE_USER_MAINTENANCE!SRTC_USERS_API_FUNCTIONS.OPEN
_USER_BY_PERSON_WITH_ROLE(sID_PERSON,sIdOrganization,iUserType,"",d
DtStart,dDtEnd,dDtStart,bShowErrors,"", bManagerOpenAsEmployee)

Uso del API para dar de baja usuarios asociados a personas

PeopleNet pone a disposición de los desarrolladores un método que permite, a

partir de un identificador de persona, dar de baja su usuario asociado. Esto implica establecer la fecha de fin al usuario y a todos los roles que se tengan abiertos a la fecha de baja.

Sin embargo, el API incluye un argumento para indicar que no se cierren los roles del ESS/MSS (**ARG_EXCLUDE_ESSMSS_ROLES** = **M4_TRUE**). Así, si el usuario tiene roles del ESS/MSS, estos roles no se cierran y por tanto no se cierra el usuario, sino solo el resto de roles.

Este método se encuentra definido en:

- **Meta4Object:** **SRTC_SIMPLE_USER_MAINTENANCE**
- **Nodo:** **SRTC_USERS_API_FUNCTIONS**
- **Método:** **CLOSE_USER_BY_PERSON**

Los argumentos del método son:

- **ARG_ID_PERSON:** ID Persona a transferir (obligatorio)
- **ARG_CLOSE_DATE:** Fecha de transferencia (obligatorio)
- **ARG_SHOW_ERRORS** (opcional):
 - 1: Mostrar errores (defecto) (**M4_TRUE** en LN4)
 - 0: No mostrar errores (**M4_FALSE** en LN4)
- **ARG_EXCLUDE_ESSMSS_ROLES** (opcional):
 - **M4_FALSE** (defecto)
 - **M4_TRUE:** excluir los roles del ESS/MSS y si hay roles del ESS/MSS que no se cierran, no se cierra el usuario.

Se retorna **M4_SUCCESS** si se ha realizado la baja con éxito y, en caso contrario, se indica la causa indicando el tipo de error concreto.

Si hay errores, se mete el mensaje de error en la fila en función de si se ha indicado que se muestren o no los errores (**ARG_SHOW_ERRORS**) y se devuelve el código de error correspondiente:

- 0: **M4_SUCCESS:** Operación realizada con éxito
- -1: Error en persistencia
- -2: Error, argumentos nulos
- -7: El usuario no existe
- -8: Existe más de un usuario asociado a la persona

EJEMPLO

Dar de baja a la persona con identificador “M10105” con fecha “1/1/2011”.

```
iRet =  
SRTC_SIMPLE_USER_MAINTENANCE!SRTC_USERS_API_FUNCTIONS.CLOS  
E_USER_BY_PERSON("M10105"," 1/1/2011",M4_TRUE)
```

Uso del API para reabrir usuarios asociados a una persona en una fecha

PeopleNet pone a disposición de los desarrolladores un método que permite reabrir los usuarios asociados a una persona que estén cerrados a una fecha dada.

A partir de un identificador de persona se buscan los usuarios asociados en la sociedad de ejecución para reabrirlos.

Un usuario se reabrirá si su fecha de fin coincide con la fecha que se indica. Una vez que se reabre un usuario, se reabrirán aquellos perfiles tales que la fecha de fin coincida con la indicada.

Este método se encuentra definido en:

- Meta4Object: SRTC_SIMPLE_USER_MAINTENANCE
- Nodo: SRTC_USERS_API_FUNCTIONS
- Método: REOPEN_USER_BY_PERSON

Los argumentos del método REOPEN_USER_BY_PERSON son:

- **ARG_ID_PERSON**: ID Persona a transferir (obligatorio)
- **ARG_CLOSE_DATE**: fecha de cierre que debe tener el usuario y sus perfiles para reabrirlo (obligatorio)
- **ARG_SHOW_ERRORS** (opcional):
 - 1: Mostrar errores (defecto). (M4_TRUE en LN4)
 - 0: No mostrar errores. (M4_FALSE en LN4)

Se retorna M4_SUCCESS si se ha realizado la baja con éxito y en caso contrario se indica la causa indicando el tipo de error concreto.

Si hay errores se mete el mensaje de error en la fila en función de si se ha indicado que se muestren o no errores (ARG_SHOW_ERRORS) y se devuelve el código de error correspondiente:

- 0: M4_SUCCESS: Operación realizada con éxito.
- -1: Error en persistencia.
- -2: Error, argumentos nulos.

-
- 7: No existen usuarios asociados a la persona.
 - 8: No se han reabierto usuarios porque existen usuarios pero no con la fecha de cierre indicada.

EJEMPLO

Reabrir usuarios de la persona "M10105".

iRet =

```
SRTC_SIMPLE_USER_MAINTENANCE!SRTC_USERS_API_FUNCTIONS.REOPEN_USER_BY_PERSON("M10105",Today(),M4_TRUE)
```

Uso del API para borrar los usuarios asociados a una persona

La aplicación dispone de un método para borrar los usuarios asociados a una persona.

A partir de un identificador de persona se buscan los usuarios asociados en la sociedad de ejecución para borrarlos.

Borrar el usuario implica borrar sus perfiles, alias, usuarios de depuración, etcétera.

Este método se encuentra definido en:

- Meta4Object: SRTC_SIMPLE_USER_MAINTENANCE
- Nodo: SRTC_USERS_API_FUNCTIONS
- Método: DELETE_USER_BY_PERSON

Los argumentos del método DELETE_USER_BY_PERSON son:

- **ARG_ID_PERSON**: ID Persona a transferir (obligatorio)
- **ARG_SHOW_ERRORS** (opcional):
 - 1: Mostrar errores (defecto). (M4_TRUE en LN4)
 - 0: No mostrar errores. (M4_FALSE en LN4)

Se retorna M4_SUCCESS si se ha realizado la baja con éxito y en caso contrario se indica la causa indicando el tipo de error concreto.

Si hay errores, se mete el mensaje de error en la fila en función de si se ha indicado que se muestren o no errores (ARG_SHOW_ERRORS) y se devuelve el código de error correspondiente:

- 0: M4_SUCCESS: Operación realizada con éxito.
- -1: Error en persistencia.
- -2: Error, argumentos nulos.
- -7: Usuario no existente.

EJEMPLO

Borrar usuarios de la persona "M10105".

iRet =

SRTC_SIMPLE_USER_MAINTENANCE!SRTC_USERS_API_FUNCTIONS.DELETE_USER_BY_PERSON("M10105",M4_TRUE).

Uso del API para borrar un rol-sociedad a un usuario

PeopleNet dispone de un método que permite borrar un rol-sociedad a un usuario. Se permite, además, borrar o cerrar el usuario a una fecha si el usuario se queda sin roles tras el borrado.

Para los roles del ESS/MSS se considera el rol indicado y los roles internos asociados según la definición de la lista de roles del ESS/MSS en el parámetro de aplicación.

Este método se encuentra definido en

- Meta4Object: SRTC_SIMPLE_USER_MAINTENANCE
- Nodo: SRTC_USERS_API_FUNCTIONS
- Método: DELETE_ROLE_ORGANIZATION

Los argumentos del método DELETE_ROLE_ORGANIZATION son:

- **ARG_ID_APP_USER**: ID Usuario (obligatorio).
- **ARG_ID_ORGANIZATION**: ID Sociedad (obligatorio).
- **ARG_ID_APP_ROLE**: ID rol (obligatorio).
- **DELETE_USER_IF_NO_ROLE** (opcional): M4_TRUE/M4_FALSE. Indica si hay que borrar el usuario cuando se queda sin roles.
- **USER_END_DATE_IF_NO_ROLE** (opcional): Fecha de cierre del usuario si se queda sin roles. Tiene efecto si DELETE_USER_IF_NO_ROLE = M4_FALSE.

Se retorna M4_SUCCESS si se ha realizado la operación con éxito y M4_ERROR en caso contrario.

EJEMPLO

```
Borrar el rol M4EMPLOYEE para la sociedad 0001 para el usuario
"0001CANTO01" y cerrar el usuario a fecha de hoy si se queda sin roles.
sldAppUser = "0001CANTO01"
sldOrg="0001"
sldAppRole="M4EMPLOYEE"
iDeleteUserWithOutRole = M4_FALSE
dDateUserCloselIfNoRoles = Today()
'iRet =
SRTC_SIMPLE_USER_MAINTENANCE!SRTC_USERS_API_FUNCTIONS.DELE
TE_ROLE_ORGANIZATION(sldAppUser, sldOrg,
sldAppRole,iDeleteUserWithOutRole,dDateUserCloselIfNoRoles )
```

Uso del API para crear usuarios desde el alta de empleados

Con el fin de permitir a los desarrolladores automatizar el proceso de alta de un empleado sin tener que acudir a la herramienta de mantenimiento de usuarios para crear el usuario de aplicación, la aplicación dispone de un método en el Meta4Object API de usuarios SRTC_SIMPLE_USER_MAINTENANCE. A partir de un identificador de persona, se busca si no existe y se crea; si ya existe, se actualiza el usuario según la información indicada.

Este método se encuentra definido en:

- Meta4Object: SRTC_SIMPLE_USER_MAINTENANCE
- Nodo: SRTC_USERS_API_FUNCTIONS
- Método: OPEN_OR_CREATE_USER_BY_PERSON

Los argumentos del método OPEN_OR_CREATE_USER_BY_PERSON son:

- **ARG_ID_PERSON** (obligatorio): identificador de la persona.
- **ARG_ID_ORGANIZATION** (obligatorio): identificador de la sociedad.
- **ARG_USER_TYPE** (obligatorio): tipo de usuario 1: ESS, 2: MSS.
- **ARG_USER_TYPE_ROLE**: rol concreto a asociarle al usuario. Para el caso de un nuevo usuario, si no se indica nada se coge el primer perfil del tipo de usuario que corresponda. Si el usuario ya existe y hay un grupo de perfiles se da error de identificación de perfil.
- **ARG_DT_START** (obligatorio): fecha de inicio del usuario.
- **ARG_DT_END** (obligatorio): fecha de fin del usuario.

- **ARG_CLOSE_CHECK_DATE**: fecha de chequeo de cierre para los casos de usuarios existentes. Por defecto es ARG_DT_START.
- **ARG_CONSIDER_SSOWEB**:
 - 1 (Defecto): tener en cuenta el parámetro de aplicación USE_SSOW_AS_USER_ID_ALIAS. Si está activo se crea con SSO Web pero sin alias.
 - 0: crear sin SSO Web independientemente de lo que diga el parámetro.
- **ARG_SHOW_ERRORS**: mostrar o no los errores. Por defecto se muestran.
- **(Ref) MANAGER_AS_EMPLOYEE**: para el caso de usuarios abiertos en los que se solicitan permisos como mánager y, como el Sistema no puede hacerlo, los abre como empleado. Por ejemplo, para el caso de que haya varios roles de mánager.

Se retorna M4_SUCCESS si se ha realizado la operación con éxito y, en caso contrario, se indica la causa y el tipo de error concreto.

Si hay errores, se mete el mensaje de error en la fila en función de si se ha indicado que se muestren errores o no (ARG_SHOW_ERRORS) y se devuelve el código de error correspondiente:

- 0: M4_SUCCESS: operación realizada con éxito.
- -1: error en persistencia.
- -2: argumentos obligatorios no rellenos.
- -3: error en ID Organización (OPEN).
- -4: el usuario no existe (OPEN).
- -5: existe más de un usuario asociado a la persona (OPEN).
- -7: error identificando el perfil (OPEN).
- -8: perfil inválido (NEW/OPEN).
- -9: usuario bloqueado (OPEN).
- -10: error de la sociedad de ejecución (NEW/OPEN).
- -11: la persona no tiene correo (NEW).
- -12: la persona tiene correo, pero está en uso (NEW).
- -13: error en el envío de correo con la contraseña (NEW).

Uso del API para establecer conexión por SSO e impedir conexión con usuario y contraseña

La aplicación pone a disposición de los desarrolladores un método API que permite establecer el mecanismo de conexión unificada o Single Sign-On

(SSO) como procedimiento de conexión de un usuario. Además, opcionalmente, permite impedir la conexión por usuario y contraseña a aquellos usuarios para los que se establece la conexión SSO.

Se busca el usuario asociado y, si el parámetro de aplicación SECURITY/USER_CREATION/USE_SSOW_AS_USER_ID_ALIAS está activo, se establece al usuario el acceso por SSO.

Este método se encuentra definido en:

- Meta4Object: SRTC_SIMPLE_USER_MAINTENANCE
- Nodo: SRTC_USERS_API_FUNCTIONS
- Método: SET_SSOWEB_BY_PERSON

Los argumentos del método SET_SSOWEB_BY_PERSON son:

- **ARG_ID_PERSON**: ID Persona (obligatorio).
- **ARG_SHOW_ERRORS** (opcional):
 - 1: mostrar errores (defecto) (M4_TRUE en LN4).
 - 0: no mostrar errores (M4_FALSE en LN4).

Se retorna M4_SUCCESS tanto si la operación se ha realizado con éxito como si no ha sido posible por estar configurado el parámetro de aplicación USE_SSOW_AS_USER_ID_ALIAS = 0.

Si hay errores, se mete el mensaje de error en la fila en función de si se ha indicado que se muestren errores o no (ARG_SHOW_ERRORS) y se devuelve el código de error correspondiente:

- M4_SUCCESS: operación realizada con éxito.
- -1: error en persistencia.
- -2: error, argumentos nulos.
- -4: el usuario no existe.
- -5: existe más de un usuario asociado a la persona.



Si el parámetro de aplicación SECURITY/USER_CREATION/AVOID_USR_PWD_LOGIN_WITH_SSOW = 1 (por defecto es 0) se impedirá que el usuario acceda mediante usuario y contraseña, es decir, solo podrá acceder por SSO.

Uso del API para la descarga asíncrona de ficheros de forma segura

PeopleNet dispone de la capacidad de descargar archivos ubicados en el

servidor de aplicaciones al puesto cliente, de manera desatendida y segura, mediante el protocolo http/https. Esta capacidad se ofrece como API público de desarrollo para poder ser utilizado desde cualquier módulo que lo requiera. La descarga se realiza a través del navegador web instalado en los puestos cliente, ofreciendo la posibilidad de abrir el archivo o guardarlo en una ruta determinada.

Para disponer de este mecanismo de descarga, es obligatorio configurar ciertos parámetros en el servidor de aplicaciones, así como tener desplegada la aplicación web m4gateway.

El API de desarrollo permite la descarga de archivos individuales o la descarga conjunta de archivos agrupados en un único archivo zip. La interfaz de desarrollo permite realizar las siguientes acciones:

- Comprobar la disponibilidad del mecanismo.
- Solicitar la preparación de un archivo para su descarga.
- Invocar la descarga de un archivo ya preparado.
- Solicitar la creación de un grupo de archivos.
- Asignar archivos ya preparados a un grupo.
- Invocar la descarga de un grupo.

El API se encuentra implementado en el nodo SRTC_FILES_DOWNLOAD_API del Meta4Object SRTC_FILES_DOWNLOAD. Los seis métodos que lo componen son:

- Método API_CHECK_ASYNC_CONFIG(Ref AO_AVAILABLE)
- Método API_PREPARE_FILE_ASYNC(AI_FILE_PATH, AI_FILE_NAME, Ref AO_FILE_KEY)
- Método API_GET_FILE_ASYNC(AI_FILE_KEY)
- Método API_CREATE_FILE_GROUP_ASYNC(Ref AO_GROUP_KEY)
- Método API_ADD_FILE_TO_GROUP_ASYNC(AI_FILE_KEY, AI_GROUP_KEY)
- Método API_GET_FILE_GROUP_ASYNC(AI_GROUP_KEY)

Método API_CHECK_ASYNC_CONFIG(Ref AO_AVAILABLE)

Este método devuelve por referencia la disponibilidad o no del mecanismo de descarga mediante http/s (M4_TRUE/M4_FALSE).

En caso de error en la ejecución retorna M4_ERROR, en caso contrario retorna M4_SUCCESS.

Método API_PREPARE_FILE_ASYNC(AI_FILE_PATH, AI_FILE_NAME, Ref AO_FILE_KEY)

Este método prepara el archivo indicado en el argumento AI_FILE_PATH para su descarga, por lo que la ruta debe ser accesible desde el servidor de aplicaciones. Como resultado, devuelve por referencia una clave de acceso a utilizar en la solicitud de descarga.

El argumento AI_FILE_NAME es opcional y en caso de indicarse un valor no nulo, se interpretará como el nombre de archivo a mostrar en el navegador Web al iniciar la apertura/descarga. Resulta de utilidad cuando AI_FILE_PATH es un ítem de tipo BLOB y conocemos el nombre original de archivo.

En caso de error en la ejecución retorna M4_ERROR, en caso contrario retorna M4_SUCCESS.

Método API_GET_FILE_ASYNC(AI_FILE_KEY)

Este método invoca al navegador Web para que abra/descargue el archivo referenciado por la clave AI_FILE_KEY, obtenida mediante una llamada previa a API_PREPARE_FILE_ASYNC.

En caso de error en la ejecución retorna M4_ERROR, en caso contrario retorna M4_SUCCESS.

Método API_CREATE_FILE_GROUP_ASYNC(Ref AO_GROUP_KEY)

Este método crea un nuevo grupo para poder realizar una descarga múltiple de archivos. Como resultado, devuelve por referencia una clave de acceso, necesaria para agregar los archivos que compondrán el grupo.

En caso de error en la ejecución retorna M4_ERROR, en caso contrario retorna M4_SUCCESS.

Método API_ADD_FILE_TO_GROUP_ASYNC(AI_FILE_KEY, AI_GROUP_KEY)

Este método añade la clave de un archivo en un determinado grupo, ambos identificados por su clave. El archivo debe haberse preparado para la descarga previamente.

En caso de error en la ejecución retorna M4_ERROR, en caso contrario retorna

M4_SUCCESS.

Método API_GET_FILE_GROUP_ASYNC(AI_GROUP_KEY)

Este método invoca al navegador Web para que abra/descargue el grupo referenciado por la clave AI_GROUP_KEY. El archivo resultante consiste en la agrupación y compresión de todos aquellos que componen el grupo.

En caso de error en la ejecución retorna M4_ERROR, en caso contrario retorna M4_SUCCESS.

Uso del API para ejecutar gráficos

Utilizando el API podrá:

- Ejecutar salida a gráfico predefinido.
- Ejecutar un gráfico y obtener la imagen bmp.
- Ejecutar un gráfico desde una presentación dando valores a los parámetros.

Para ello debe definir una tarea específica como se indica a continuación:

- Ejecutar salida a gráfico predefinido

El API también puede utilizarse para ejecutar un gráfico desde una opción de menú o desde donde queramos. Para ello hay que crear una tarea, con código de cliente Windows y con los siguientes parámetros:

1. CMFREF , Cadena, entrada, 1.
2. SBLOCKNAME , Cadena, entrada, 2.
3. SPARAMETER , Cadena, entrada, 3.

Código cliente Windows:

- SysTask.TaskCMFLoader
- SetConstStr CMFREF, "GrTool.CmfGraph";
- SetConstStr SPARAMETER, "ID_T3,ID_REPORT";
- SetConstStr SBLOCKNAME, "";

EJEMPLO

```
SysTask.TaskCMFLoader
SetConstStr CMFREF, "GrTool.CmfGraph";
SetConstStr SPARAMETER, "ZZ_EXECUTE_CHART,ZZ_EXECUTE_CHART";
```

```
SetConstStr SBLOCKNAME, "";
```

En este caso, como se muestra en el ejemplo, hay que dejar sin valor el parámetro SBLOCKNAME.

■ Ejecutar un gráfico y obtener la imagen bmp

El procedimiento para ejecutar un gráfico, no mostrarlo y guardar el resultado en un archivo de imagen bmp es muy similar al que se aplica para ejecutar una salida a gráfico predefinido, solo que en este caso, como valor del parámetro SBLOCKNAME, se debe indicar la ruta completa del fichero donde se quiere generar la imagen bmp.

EJEMPLO

```
SysTask.TaskCMFLoader  
SetConstStr CMFREF, "GrTool.CmfGraph";  
SetConstStr SPARAMETER, "ZZ_EXECUTE_CHART,ZZ_EXECUTE_CHART";  
SetConstStr SBLOCKNAME, "C:\Temp\prueba.bmp";
```

■ Ejecutar un gráfico desde una presentación dando valores a los parámetros

También es posible ejecutar un gráfico desde una presentación dando valores a los parámetros, para lo que hay que adaptar el API tal y como se muestra en el siguiente ejemplo.

EJEMPLO

```
BEGIN Action_set asPARAMETER  
  Set = "*C*SPARAMETER:= ZZ_EXECUTE_CHART,ZZ_EXECUTE_CHART"  
END  
BEGIN Action_set ascParams1  
  Set = "*C*PARAMETERS_DEFAULT:=PARAMETERS_DEFAULT"  
END  
BEGIN Action_set ascParams2  
  Set =  
  "*C*PARAM_VALUE:=ZZ_EXECUTE_CHART_STD_PERSON;PARAMETRO;1"  
END  
BEGIN Action_bp Action_bpMN_QUERY_PREVIEW
```

```
ldbp = "ZZ_EJECUTAR_GRAFICO"  
Load = False  
Localcontext = False  
END
```

Uso del API para reducir el tamaño y peso de las imágenes

La aplicación dispone de un método que permite reducir el tamaño y peso de las imágenes que se almacenan en el servidor. Para ello hay que introducir la ruta donde se encuentra la imagen a redimensionar, la ruta de la nueva imagen, la anchura y altura de la misma, si se quiere mantener la relación de aspecto actual, las dimensiones propuestas y/o si se quiere utilizar el modo de renderizado de alta calidad.

Si no se le proporciona la ruta de la nueva imagen (argumento `AI_TARGET_IMAGE`) esta se creará en el directorio temporal con un nombre aleatorio y la ruta se devolverá en dicho parámetro, que es por referencia.

Este método se encuentra definido en:

- Meta4Object: `SRTC_IMAGE_RESIZER`
- Nodo: `SRTC_IMAGE_RESIZER`
- Método: `RESIZE`

Los argumentos del método son:

- **AI_SOURCE_IMAGE**: ruta completa a la imagen original.
- **AI_TARGET_IMAGE**: ruta completa a la imagen nueva a generar. Es un parámetro por referencia. Si se deja vacío el sistema genera el archivo automáticamente en el directorio temporal y devuelve la ruta en este parámetro.
- **AI_WIDTH**: anchura propuesta para la nueva imagen (en píxeles).
- **AI_HEIGHT**: altura propuesta para la nueva imagen (en píxeles).
- **AI_KEEP_ASPECT_RATIO**: indica si se mantiene la relación de aspecto o no (1/0 o `M4_TRUE/M4_FALSE`). Por defecto vale 0.
- **AI_KEEP_DIMENSIONS**: indica si se mantiene la dimensión propuesta (1/0 o `M4_TRUE/M4_FALSE`) cuando `AI_KEEP_ASPECT_RATIO` = 1. En caso de que tanto este argumento como `AI_KEEP_ASPECT_RATIO` tengan valor 1, para que la imagen pueda mantener a la vez las dimensiones propuestas y la relación de aspecto, se genera un marco que rodea la imagen. Por defecto vale 0.

-
- **AI_ENABLE_HQ_RENDERING:** indica si se permite el modo de renderizado de alta calidad o no (1/0 o M4_TRUE/M4_FALSE). Por defecto vale 0.

Uso del API para ejecutar cartas y salida directa a Excel

Utilizando el API también podrá:

- Ejecutar cartas en Word.
- Ejecutar salida a Excel directa.

En ambos casos también hay que definir una tarea específica para ello:

- Ejecutar cartas en Word

En primer lugar, debe crear una tarea Word para colgarla desde una opción de menú del árbol principal de PeopleNet, aunque existe igualmente la posibilidad de ejecutar la carta desde una presentación. Un ejemplo del procedimiento detallado se muestra a continuación:

EJEMPLO

Primero hay que definir una tarea:

- Id Tarea: Throw_Word

Con código de cliente Visual Basic y sin parámetros.

Código ejecutable:

- TiComple.TaskThrowWord
- SetConstStr ID_LETTER,"Tu carta";



Para insertar el retorno de carro en este editor hay que pulsar CTRL+M.

A continuación hay que definir una opción de menú:

- Id_Menu: Tu menu
- Id_Icono: DMSEDT16_A
- Id_Tarea: Throw_Word

-
- Ejecutar salida a Excel directa

En este caso se debe crear una tarea Excel para colgarla desde una opción de menú del árbol principal de PeopleNet (también se podría ejecutar desde una presentación). A continuación se muestra un ejemplo:

EJEMPLO

En primer lugar se debe definir una tarea:

- Id Tarea: Throw_Excel

Con código de cliente Visual Basic y sin parámetros.

Código ejecutable:

- TiComple.TaskThrowExcel
- SetConstStr ID_QUERY,"Tu Meta4Object";



Para insertar el retorno de carro en este editor hay que pulsar CTRL+M.

Para la definición de la opción de menú:

- Id_Menu: Tu menu
- Id_Icono: EXCEL32_A
- Id_Tarea: Throw_Excel

Uso del API para manejar archivos Excel en el servidor de aplicaciones

La aplicación dispone de un API para desarrolladores que permite manejar archivos Excel en el servidor de aplicaciones. Se utilizará como intermediaria la componente Java Apache POI, diseñada para acceder y manipular ficheros Excel.

El API se encuentra implementado en el Meta4Object `SNTC_EXCEL_JAVA`. Este contiene un nodo para el libro o workbook (`SNTC_EXCEL_JAVA_WORKBOOK`) y un nodo hijo con conector registro-bloque en representación de las hojas o worksheet (`SNTC_EXCEL_JAVA_WORKSHEET`). Al abrir un archivo Excel, se corresponderá con un registro en el nodo workbook. Cada una de las hojas abiertas se corresponderá con un registro en el nodo worksheet.

Paralelamente, se proporciona un API de desarrollo adicional, idéntico al descrito en este apartado, pero no basado en Java. Este API se encuentra implementado en el Meta4Object `SNTC_EXCEL_CPP` y cuenta con la misma estructura, un nodo para los workbooks abiertos y un nodo para las worksheets. La interfaz de los métodos principales del API de desarrollo es exactamente igual y como añadido a la implementación Java, pueden ejecutarse tanto en la parte cliente, como en la servidora. Al igual que la implementación Java, no requiere de la instalación del producto Office Excel, ya que trabaja directamente sobre los archivos de esta aplicación.

En los siguientes apartados se describe la funcionalidad que aportan estos Meta4Objects:

- Estructura de nodo `SNTC_EXCEL_JAVA_WORKBOOK`
- Estructura de nodo `SNTC_EXCEL_JAVA_WORKSHEET`

Estructura de nodo `SNTC_EXCEL_JAVA_WORKBOOK`

En esta estructura de nodo, se tienen definidos los siguientes métodos para abrir, cerrar, o guardar un fichero Excel:

- Método `OPEN_WORKBOOK`: abrirá el fichero indicado por la ruta especificada. Retornará `M4_SUCCESS` si se puede abrir, y `M4_ERROR` si no se puede.
- Método `SAVE_WORKBOOK_AS`: intentará guardar el workbook en la ruta indicada por un argumento.
- Método `CLOSE_WORKBOOK`: función que habrá que invocar para liberar los recursos correspondientes.

A continuación se ofrece un ejemplo de uso de los métodos a nivel de workbook:

EJEMPLO

```
'Abrir el fichero excel
sFile = "z:\\source\\xls\\testexcel.xls"
iRetOpen = SNTC_EXCEL_JAVA_WORKBOOK.OPEN_WORKBOOK (sFile)
if (iRetOpen <> M4_SUCCESS) then
    MessageBox("Cannot open workbook", sFile)
    return M4_ERROR
endif

/* hacer las operaciones necesarias, que se explican en la estructura
de nodo SNTC_EXCEL_JAVA_WORKSHEET */
```

```
'Guardar como  
iRetSave =  
SNTC_EXCEL_JAVA_WORKBOOK.SAVE_WORKBOOK_AS("z:\\source\\xls\\testexcel  
_save.xls")
```

Método OPEN_WORKBOOK

Este método abre el fichero indicado por la ruta especificada. Retorna M4_SUCCESS si se puede abrir, y M4_ERROR si no se puede.

Método SAVE_WORKBOOK_AS

Este método intenta guardar el workbook en la ruta indicada por un argumento.

Método CLOSE_WORKBOOK

Método que hay que invocar para liberar los recursos correspondientes.

Estructura de nodo SNTC_EXCEL_JAVA_WORKSHEET

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- Método GET_WORKSHEET: una vez abierto el workbook, se trata de establecer la hoja sobre la que se realizarán las operaciones.
- Método GET_VALUE: una vez abiertos el libro y la hoja, permite recuperar el valor de una fila y columna.
- Método SET_VALUE: una vez abiertos el libro y la hoja, permite establecer el valor de una fila y columna.
- Método SET_COLUMN_HIDDEN: permite ocultar una columna.
- Método SET_ROW_HIDDEN: sirve para ajustar la altura de una fila a cero.
- Método COPY_CELL: permite copiar los contenidos de una celda.
- Método COPY_ROW: sirve para copiar los contenidos de una fila.

Método GET_WORKSHEET

Una vez abierto el workbook, se trata de establecer la hoja sobre la que se realizarán las operaciones. Cada vez que queramos posicionarnos en una hoja o cambiar de hoja, se llamará a este método.

A continuación se ofrece un ejemplo de uso del método:

EJEMPLO

```
`Abrir la hoja de identificador 0
`-----

iIndexWorksheet = 0
iRetOpenWorkSheet = SNTC_EXCEL_JAVA_WORKSHEET.GET_WORKSHEET
(iIndexWorksheet)
if (iRetOpenWorkSheet <> M4_SUCCESS) then
    MessageBox("Cannot open worksheet", iIndexWorksheet)
    return M4_ERROR
endif
```

Método GET_VALUE

Una vez abiertos el libro y la hoja, este método permite recuperar el valor de una fila y columna. Para ello es necesario pasar el identificador de fila, columna, más un argumento de salida de tipo variant.

A continuación se ofrece un ejemplo de uso del método:

EJEMPLO

```
'Cadena
OUT = "cadena"
iRetGet = SNTC_EXCEL_JAVA_WORKSHEET.GET_VALUE(1, 0, OUT)
MessageBox (iRetGet, OUT)
iRetGet = SNTC_EXCEL_JAVA_WORKSHEET.GET_VALUE(1, 1, OUT)
MessageBox (iRetGet, OUT)

'Números
OUT = 0
iRetGet = SNTC_EXCEL_JAVA_WORKSHEET.GET_VALUE(1, 3, OUT)
MessageBox (iRetGet, OUT)

'Fechas
OUT = TodayNow()
iRetGet = SNTC_EXCEL_JAVA_WORKSHEET.GET_VALUE(1, 2, OUT)
MessageBox (iRetGet, OUT)
```

Método SET_VALUE

Una vez abiertos el libro y la hoja de Excel, este método permite establecer el valor de una fila y columna. Para ello es necesario pasar el identificador de fila, de columna, y un argumento de entrada de tipo variant. El comportamiento podría variar en ciertas combinaciones de tipos de datos, dependiendo de la naturaleza de las celdas de destino.

A continuación se ofrece un ejemplo de uso del método:

EJEMPLO

```
iRetSet = SNTC_EXCEL_JAVA_WORKSHEET.SET_VALUE(2, 0, "Irene")  
iRetSet = SNTC_EXCEL_JAVA_WORKSHEET.SET_VALUE(2, 1, "García")
```

Método SET_COLUMN_HIDDEN

Este método permite ocultar una columna de una hoja de Excel.

A continuación se ofrece un ejemplo de uso de este método, en el que se oculta la columna C (el valor 1 es para ocultar):

EJEMPLO

```
iRetGet = SNTC_EXCEL_JAVA_WORKSHEET.SET_COLUMN_HIDDEN(2, 1)
```

Método SET_ROW_HIDDEN

Este método permite ajustar a cero la altura de una fila de una hoja de Excel.

A continuación se ofrece un ejemplo de uso de este método, en el que se asigna una altura cero a la fila 9 (el valor 1 es para ello). Tenga en cuenta que en este API las filas se empiezan a numerar por 0.

EJEMPLO

```
iRetGet = SNTC_EXCEL_JAVA_WORKSHEET.SET_ROW_HIDDEN(8, 1)
```

Método COPY_CELL

Este método sirve para copiar los contenidos de una celda. Como el método copia el formato, se puede utilizar indirectamente para dar valores a las celdas realizando una inicialización de formato basada en otras.

Método COPY_ROW

Este método se utiliza para copiar los contenidos de una fila. Como el método copia el formato, se puede emplear indirectamente para dar valores a las filas realizando una inicialización de formato basada en otras.

Tenga en cuenta que el comportamiento de lo que se hará con la fila destino es el siguiente: si la fila existe en destino (el API de POI no devuelve nulo), empuja en una unidad todas las filas hacia abajo. En caso contrario, se crea una fila nueva.

Uso del API para la apertura parcial de documentos Excel

Existe una funcionalidad exclusiva del API no basado en Java, que permite la apertura parcial de documentos Excel. Resulta de interés cuando se pretende acceder en modo lectura a un determinado rango dentro de un documento relativamente grande.

Debe tenerse en cuenta que la operación de guardado solo incluirá este rango, eliminando el resto del documento.

La funcionalidad se encuentra implementada:

- En el Meta4Object EXCEL_CPP, nodo EXCEL_WORKBOOKS:

```
OpenWorkbookPartially (AI_FILE_PATH, AI_SHEET, AI_FIRST_ROW,  
AI_LAST_ROW)
```

- En el Meta4Object SNTC_EXCEL_CPP, nodo SNTC_EXCEL_CPP_WORKBOOK:

```
OPEN_WORKBOOK_PARTIALLY (AI_FILE_PATH, AI_SHEET, AI_FIRST_ROW,  
AI_LAST_ROW)
```

Abre la hoja con índice AI_SHEET del documento AI_FILE_PATH. Al igual que en el resto de métodos del API Excel, los índices para hoja, fila y columna se interpretan comenzando por 1 en los métodos de EXCEL_CPP y por 0 en los de SNTC_EXCEL_CPP.

Si se desea cargar una hoja completa debe especificar el valor -1 en los argumentos AI_FIRST_ROW y AI_LAST_ROW.

La implementación de las funciones de API de apertura y guardado cuenta con una funcionalidad basada en el uso de archivos temporales, que permite reducir el consumo de memoria de estas operaciones. Para ello es necesario

establecer el valor del parámetro de aplicación CONFIGURATION / FORMS / ACTIVATE_TMP_FILES_IN_EXCEL_CPP = 1. El valor por defecto es 0 (desactivado).

La razón de no salir activado de forma automática es porque requiere que el software antivirus de la máquina donde se ejecuta excluya del escaneo las carpetas donde se abren y guardan los archivos porque puede colisionar con esta nueva funcionalidad.

Uso del API para proteger archivos Excel contra modificaciones

Existe una funcionalidad exclusiva del API no basado en Java, que permite especificar opcionalmente una contraseña para proteger los archivos Excel contra modificaciones.

Esta funcionalidad está disponible en el Meta4Object EXCEL_CPP, nodo EXCEL_CPP_WORKSHEETS, en el método existente PROTECT([password]).

La protección se realiza sobre la hoja actual. En caso de disponer de un libro con varias hojas habría que protegerlas una a una. La desprotección no requiere la contraseña, simplemente elimina la información de protección de la hoja.

Uso del API para eliminar hojas de cálculo

Existe una funcionalidad exclusiva del API no basado en Java, que permite eliminar la hoja de cálculo con la que se está trabajando.

La funcionalidad se encuentra implementada:

- En el Meta4Object EXCEL_CPP, estructura de nodo EXCEL_CPP_WORKSHEETS, en el método DELETE_WORKSHEET:
- En el Meta4Object SNTC_EXCEL_CPP, estructura de nodo SNTC_EXCEL_CPP_WORKSHEET, en el método DELETE_WORKSHEET.

Uso del API para crear un archivo Excel vacío

Existe una funcionalidad exclusiva del API no basado en Java, que permite crear archivos Excel sin ninguna hoja de cálculo. Una vez creado el archivo vacío se debe añadir al menos una hoja para que este sea funcional, ya que en caso contrario no será compatible con la aplicación MS Excel.

Esta funcionalidad está disponible en el Meta4Object EXCEL_CPP, nodo

EXCEL_CPP_WORKBOOKS, en el método existente NewEmptyWorkbook.

Uso del API para convertir archivos Excel en PDF

La aplicación dispone de un API que permite convertir un documento Excel a PDF. Para ello se debe indicar la ruta al fichero origen (el archivo convertido se guardará en la misma ruta). Además, se puede indicar si se quiere deshabilitar el cumplimiento con PDF A/1B (por defecto los pdfs se generan en modo PDF A/1B).

El API se encuentra implementado en el Meta4Object `SRTC_EXCEL_CONVERTER` y contiene los siguientes métodos, definidos en la Estructura de nodo `SRTC_EXCEL_CONVERTER`.

- Método `API_CONVERT_TO_PDF`
- Método `API_CONVERT_TO_PDF_A`

Asimismo, los APIs de Excel de alto nivel (Meta4Object `EXCEL_CPP`) y bajo nivel (Meta4Object `SNTC_EXCEL_CPP`) incluyen la posibilidad de convertir un fichero Excel a PDF utilizando el API del Meta4Object `SRTC_EXCEL_CONVERTER`. Contienen los siguientes métodos homónimos, definidos en la Estructura de nodo `EXCEL_CPP` y Estructura de nodo `SNTC_EXCEL_CPP`, respectivamente:

- Método `CONVERT_TO_PDF`
- Método `CONVERT_TO_PDF`

Estructura de nodo `SRTC_EXCEL_CONVERTER`

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- Método `API_CONVERT_TO_PDF`
- Método `API_CONVERT_TO_PDF_A`

Método `API_CONVERT_TO_PDF`

Este método genera un fichero PDF a partir del fichero Excel indicado como parámetro.

El argumento del método es:

- **`AI_SOURCE_PATH`**: ruta al archivo Excel de origen que se va a convertir a PDF.

Método API_CONVERT_TO_PDF_A

Este método genera un fichero PDF cumpliendo con los requisitos del formato PDF/A-1B a partir del fichero Excel indicado como parámetro.

El argumento del método es:

- **AI_SOURCE_PATH**: ruta al archivo Excel de origen que se va a convertir a PDF cumpliendo con el formato PDF/A-1B.

Estructura de nodo EXCEL_CPP

En esta estructura de nodo, perteneciente al API de alto nivel de Excel (Meta4Object EXCEL_CPP), se tiene definido el siguiente método:

- Método CONVERT_TO_PDF

Método CONVERT_TO_PDF

Este método convierte el archivo Excel indicado como parámetro a PDF con formato PDF/A-1B. Para ello invoca al Método CONVERT_TO_PDF del Meta4Object SNTC_EXCEL_CPP. El PDF resultante tendrá el mismo nombre que el fichero Excel y se deja en la misma ruta.

El argumento del método es:

- **AI_EXCEL_FILE_PATH**: ruta al archivo Excel que se va a convertir a PDF con conformidad PDF/A habilitada.

Estructura de nodo SNTC_EXCEL_CPP

En esta estructura de nodo, perteneciente al API de bajo nivel de Excel (Meta4Object SNTC_EXCEL_CPP), se tiene definido el siguiente método:

- Método CONVERT_TO_PDF

Método CONVERT_TO_PDF

Este método convierte el archivo Excel indicado como parámetro a PDF con formato PDF/A-1B. Para ello invoca al Método API_CONVERT_TO_PDF del Meta4Object SRTC_EXCEL_CONVERTER. El PDF resultante tendrá el mismo nombre que el fichero Excel y se deja en la misma ruta.

El argumento del método es:

- **AI_EXCEL_FILE_PATH**: ruta al archivo Excel que se va a convertir a PDF con conformidad PDF/A habilitada.

Uso del API para transcodificar objetos lógicos

La aplicación pone a disposición de los desarrolladores un API para transcodificar objetos lógicos. Este API contiene los métodos Transcodificar Valores (CSP_PRE_TRANSCOD_VAL) y Transcodificar Valores extendida (CSP_PRE_TRANSCOD_VAL_EXT), que permiten transcodificar objetos lógicos con una o varias claves primarias, respectivamente.

Ambos métodos están definidos en la estructura de nodo CSP_PRE_TRANSCOD_VAL2 del Meta4Object Gestión de valores de la transcodificación (CSP_PRE_TRANSCOD_VAL2).

En el siguiente apartado se describe la funcionalidad que aporta este Meta4Object:

- Estructura de nodo CSP_PRE_TRANSCOD_VAL2

Estructura de nodo CSP_PRE_TRANSCOD_VAL2

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- Método CSP_PRE_TRANSCOD_VAL
- Método CSP_PRE_TRANSCOD_VAL_EXT



Se recomienda utilizar preferentemente el método CSP_PRE_TRANSCOD_VAL_EXT para transcodificar objetos lógicos.

Método CSP_PRE_TRANSCOD_VAL

Este método funciona con objetos lógicos que transcodifican una única clave primera.

Los argumentos del método son:

- **SISTEMA:** ID del sistema de transcodificación usado.
- **TABLA:** ID del objeto lógico configurado.
- **ARG_TRANSCOD:** Valor que se quiere transcodificar.
- **ARG_RETORNO:** Tipo de valor transcodificado que se desea devolver.
Posibles valores:
 - **1:** ID Externo
 - **2:** Nombre Externo
 - **3:** ID Meta4

- **4:** Nombre en Meta4
- **5:** Lo que esté relleno (el ID si lo está)
- **ARG_TIPO:** Tipo de valor que se quiere transcodificar (el del valor del ARG_TRANSCOD). Posibles valores:
 - **1:** ID Externo
 - **2:** Nombre Externo
 - **3:** ID Meta4
 - **4:** Nombre en Meta4

Método CSP_PRE_TRANSCOD_VAL_EXT

Este método permite transcodificar objetos lógicos con una o varias claves primarias.

Los argumentos del método son:

- **ARG_SYSTEM:** ID del sistema de transcodificación usado.
- **ARG_TABLE:** ID del objeto lógico configurado.
- **ARG_VALUE_TYPE:** Tipo de valor que se quiere transcodificar (el del valor de los ARG_VALUE). Posibles valores:
 - **1:** ID Externo
 - **2:** Nombre Externo
 - **3:** ID Meta4
 - **4:** Nombre en Meta4
- **ARG_RETURN_VALUE_TYPE:** Tipo de valor transcodificado que se desea obtener. Posibles valores:
 - **1:** ID Externo
 - **2:** Nombre Externo
 - **3:** ID Meta4
 - **4:** Nombre en Meta4
 - **5:** Lo que esté relleno en la configuración (el ID si lo está)
- **ARG_VALUE_1:** Valor de la clave primaria (o nombre) 1.
- **ARG_RETURN_VALUE_1:** Valor transcodificado de la clave primaria 1.
- **ARG_VALUE_2:** Valor de la clave primaria (o nombre) 2.
- **ARG_RETURN_VALUE_2:** Valor transcodificado de la clave primaria 2.
- **ARG_VALUE_3:** Valor de la clave primaria (o nombre) 3.
- **ARG_RETURN_VALUE_3:** Valor transcodificado de la clave primaria 3.
- **ARG_VALUE_4:** Valor de la clave primaria (o nombre) 4.

-
- **ARG_RETURN_VALUE_4:** Valor transcodificado de la clave primaria 4.

Uso del API para transcodificar valores de Meta4Objects

La aplicación dispone de un API para desarrolladores que permite modificar los valores de los elementos de un Meta4Object por los valores transcodificados basándose en los metadatos y devuelve el valor transcodificado de los elementos del Meta4Object a partir de dichos metadatos.

Este API contiene el método Modificar elemento con transcodificación (SRTC_CHANGE_ITEM_TRANSCOD), que modifica los valores de los elementos de un Meta4Object por los valores transcodificados, y el método Valor de la transcodificación (SRTC_ITEM_VALUE_TRANSCOD), que devuelve el valor transcodificado de los elementos del Meta4Object sin modificar el valor del elemento en el Meta4Object.

Los dos métodos buscan los objetos del Meta4Object y revisan a través de las relaciones si alguno de los valores debe transcodificarse. Por ejemplo, supongamos que se ha definido la tabla STD_LU_GENDER como transcodificable. Entonces, en el mantenimiento de la persona, en la tabla STD_PERSON, como el campo ID_GENDER está relacionado con STD_LU_GENDER, sería transcodificable.

Ambos métodos están definidos en la estructura de nodo SRTC_DYN_TRANSCOD del Meta4Object Gestión de transcodificación dinámicamente (SRTC_MNG_DYN_TRANSCOD).

En el siguiente apartado se describe la funcionalidad que aporta este Meta4Object:

- Estructura de nodo SRTC_DYN_TRANSCOD

Estructura de nodo SRTC_DYN_TRANSCOD

En esta estructura de nodo, se tienen definidos los siguientes métodos:

- Método SRTC_CHANGE_ITEM_TRANSCOD
- Método SRTC_ITEM_VALUE_TRANSCOD

Método SRTC_CHANGE_ITEM_TRANSCOD

Este método modifica los valores de los elementos de un Meta4Object por los valores transcodificados, basándose en los metadatos (valores, objeto de lectura y de escritura). Permite transcodificar los identificadores.

Los argumentos del método son:

- **ARG_ID_HANDLE:** Handle del Meta4Object que se quiere transcodificar.
- **ARG_ID_T3:** ID del Meta4Object que se quiere transcodificar.
- **ARG_ID_NODE:** ID del nodo que se quiere transcodificar.
- **ARG_ID_ITEM:** ID del elemento que se quiere transcodificar.
- **ARG_SYSTEM_TRANSCOD:** ID del sistema de transcodificación usado.
- **ARG_DIRECTION:** Sentido de la transcodificación. Posibles valores:
 - - 1: de Meta4 a la aplicación externa.
 - - 2: de la aplicación externa a Meta4.
- **ARG_BOOKMARK:** Posicionamiento del registro que queremos transcodificar.
- **ARG_SET_VALUE: (M4_TRUE/M4_FALSE).** Indica si se quiere o no modificar el valor del elemento.
- **ARG_VALUE_TRANSCOD:** Devuelve el valor del elemento transcodificado por si no se ha modificado el elemento.

Método SRTC_ITEM_VALUE_TRANSCOD

Este método devuelve el valor transcodificado de los elementos de un Meta4Object, basándose en los metadatos (objeto de lectura y de escritura). La diferencia con el método SRTC_CHANGE_ITEM_TRANSCOD reside en que no lee el valor del elemento, sino el que se pasa por argumento, y tampoco modifica dicho valor en el Meta4Object.

Los argumentos del método son:

- **ARG_ID_HANDLE:** Handle del Meta4Object que se quiere transcodificar.
- **ARG_ID_T3:** ID del Meta4Object que se quiere transcodificar.
- **ARG_ID_NODE:** ID del nodo que se quiere transcodificar.
- **ARG_ID_ITEM:** ID del elemento que se quiere transcodificar.
- **ARG_SYSTEM_TRANSCOD:** ID del sistema de transcodificación usado.
- **ARG_DIRECTION:** Sentido de la transcodificación. Posibles valores:
 - - 1: de Meta4 a la aplicación externa
 - - 2: de la aplicación externa a Meta4.
- **ARG_VALUE:** Valor que se quiere transcodificar (no es el que tiene el elemento en ese momento).
- **ARG_VALUE_TRANSCOD:** Devuelve el valor del elemento transcodificado por si no se ha modificado el elemento.

Uso del API para comprimir y descomprimir archivos en formato ZIP

PeopleNet dispone de un API para comprimir y descomprimir archivos en formato ZIP. El API se compone de varios métodos, que están implementados en una librería tipo DLL. Por tanto, para poder usarlos desde LN4, es necesario crear los siguientes métodos:

- Método M4GENERATEZIP
- Método M4EXPANDZIP

Método M4GENERATEZIP

Este método permite comprimir archivos en formato ZIP. Sus propiedades identificativas, argumentos y reglas son:

- Retorno: M4_SUCCESS/M4_ERROR
- Ámbito = Bloque
- Tipo básico = Número (Parte entera = 18, Parte decimal = 0)
- Argumentos variables = Sí
- Argumentos fijos:
 1. ZIP_PATH
 - Posición = 0
 - Tipo argumento = Valor
 - Tipo básico = Cadena variable
 - Longitud = 255
 2. FILE1
 - Posición = 1
 - Tipo argumento = Valor
 - Tipo básico = Cadena variable
 - Longitud = 255
- Regla (R1):
 - Tipo: DLL
 - Recurso: m4doc
 - Método: M4GenerateZip

Método M4EXPANDZIP

Este método permite descomprimir archivos en formato ZIP. Sus propiedades identificativas, argumentos y reglas son:

- Retorno: M4_SUCCESS/M4_ERROR
- Ámbito = Bloque
- Tipo básico = Número (Parte entera = 18, Parte decimal = 0)
- Argumentos variables = Sí
- Argumentos fijos:
 1. ZIP_PATH
 - Posición = 0
 - Tipo argumento = Valor
 - Tipo básico = Cadena variable
 - Longitud = 255
 2. OUTPUT_DIR
 - Posición = 1
 - Tipo argumento = Valor
 - Tipo básico = Cadena variable
 - Longitud = 255
- Regla (R1):
 - Tipo: DLL
 - Recurso: m4doc
 - Método: M4ExpandZip

Ejemplos de uso del API para comprimir y descomprimir archivos ZIP

A continuación se muestran unos ejemplos de uso de los métodos M4GenerateZip y M4ExpandZip:

1. Generación de un archivo ZIP añadiendo dos archivos y poniendo una contraseña de protección de apertura ("eee" en el ejemplo):

EJEMPLO

```
res = 0
```

```
res = M4GENERATEZIP("-p", "eee", "c:/temp/pruebaln4.zip", "c:/temp/prueba.doc", "c:/temp/fromalfresco2.ppt")
```

```
return res
```

Si se especifica el argumento “-p”, quiere decir que se quiere proteger el archivo ZIP con una contraseña que obligatoriamente irá en el siguiente argumento (“eee”, en el ejemplo).

2. Inclusión de nuevos archivos en un archivo ZIP existente:

EJEMPLO

```
res = 0

res = M4GENERATEZIP("c:/temp/pruebaln4.zip", "-a", "-r", "edudu.doc",
" c:/temp/prueba.doc", "-r" , "otro.ppt", "c:/temp/fromalfresco2.ppt")

return res
```

En el primer argumento se indica el archivo ZIP sobre el que se quieren añadir archivos, y a continuación, se debe indicar un argumento fijo “-a” (para indicar que se van a añadir archivos) y a continuación los archivos a añadir. Si delante de cada archivo se establece el argumento “-r” y un nombre de archivo, indicará que en el ZIP se renombrará el archivo con ese nombre.

3. Descompresión de un archivo ZIP:

EJEMPLO

```
res = CREATE_DIR("C:\TEMP\EDUDU2")

res = M4EXPANDZIP("c:/temp/pruebaln4.zip", "c:/temp/testzip")

return res
```

Uso del API para encriptar, desencriptar, firmar y verificar archivos

PeopleNet ofrece un API de desarrollo, mediante el que cualquier proceso puede hacer uso de las funciones de encriptación, desencriptación, firma y verificación de archivos. Asimismo, ciertas funcionalidades del producto (procesos de exportación de datos, por ejemplo) pueden hacer uso de este API, requiriendo por lo tanto de la configuración de las claves públicas y privadas.

El archivo resultante de la encriptación puede desencriptarse en PeopleNet mediante la invocación al método de desencriptación del API de desarrollo. La implementación de este método localiza la clave privada de Meta4, que se habrá registrado previamente con el identificador "M4_PGP_CERT".

Los métodos públicos del API de desarrollo se encuentran en el Meta4Object SRTC_PGP_CRYPT, en el nodo SRTC_PGP_CRYPT, y son los siguientes:

- Método API_ENCRYPT_FILE(ARG_FILE, ARG_KEY, ARG_OUT_FILE)
- Método API_DECRYPT_FILE(ARG_FILE, ARG_EXT, ARG_OUT_FILE)
- Método API_SIGN_FILE(ARG_FILE, ARG_OUT_FILE)
- Método API_VERIFY_FILE(ARG_FILE, ARG_KEY_ID, ARG_EXT, ARG_OUT_FILE)
- Método API_ENCRYPT_SIGN_FILE(ARG_FILE, ARG_KEY_ID, ARG_OUT_FILE)
- Método API_DECRYPT_VERIFY_FILE(ARG_FILE, ARG_EXT, ARG_KEY_ID, ARG_OUT_FILE)

Todas las operaciones también se encuentran disponibles en la versión de bajo nivel del API, donde cada uno de los métodos recibe como argumento la ruta a los archivos de clave necesarios. Los métodos de bajo nivel se identifican por incluir KF en su nombre y son los siguientes:

- Método API_KF_ENCRYPT_FILE(ARG_FILE, ARG_KEY_FILE, ARG_OUT_FILE)
- Método API_KF_DECRYPT_FILE(ARG_FILE, ARG_KEY_FILE, ARG_PWD, ARG_EXT, ARG_OUT_FILE)
- Método API_KF_SIGN_FILE(ARG_FILE, ARG_KEY_FILE, ARG_PWD, ARG_OUT_FILE)
- Método API_KF_VERIFY_FILE(ARG_FILE, ARG_KEY_FILE, ARG_EXT, ARG_OUT_FILE)
- Método API_KF_ENCRYPT_SIGN_FILE(ARG_FILE, ARG_ENCR_KEY_FILE, ARG_SIGN_KEY_FILE, ARG_PWD, ARG_OUT_FILE)

-
- Método API_KF_DECRYPT_VERIFY_FILE(ARG_FILE, ARG_DECR_KEY_FILE, ARG_PWD, ARG_VERIFY_KEY_FILE, ARG_EXT, ARG_OUT_FILE)

Los argumentos de salida son cadenas con la ruta a los archivos resultantes de la ejecución. Estos archivos están soportados por propiedades de tipo BLOB en el Meta4Object del API, por lo que una vez destruido, los archivos se eliminarán. El Meta4Object usuario del API de desarrollo es responsable de mantener su propia referencia al archivo mediante una propiedad de tipo BLOB, si requiere acceder después de destruir el Meta4Object.

Método API_ENCRYPT_FILE(ARG_FILE, ARG_KEY, ARG_OUT_FILE)

Este método permite llevar a cabo la encriptación de archivos. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a encriptar.
- ARG_KEY: identificador de una clave pública registrada en el sistema, con la que encriptar el archivo.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo encriptado.

Método API_DECRYPT_FILE(ARG_FILE, ARG_EXT, ARG_OUT_FILE)

Este método permite descryptar archivos que han sido encriptados por el cliente mediante la clave pública de Meta4. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a descryptar.
- ARG_EXT: extensión del archivo resultante de la descryptación.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo descryptado.

Método API_SIGN_FILE(ARG_FILE, ARG_OUT_FILE)

Este método permite firmar un archivo en nombre de Meta4, esto es, con su clave privada. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a firmar.

- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta al archivo firmado.

Método API_VERIFY_FILE(ARG_FILE, ARG_KEY_ID, ARG_EXT, ARG_OUT_FILE)

Este método sirve para verificar un archivo firmado por una entidad externa. Para ello es necesario que la clave pública de la entidad se haya registrado previamente en el sistema. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a verificar.
- ARG_KEY_ID: identificador de una clave pública registrada en el sistema, con la que verificar el archivo.
- ARG_EXT: extensión del archivo sin firma una vez verificado.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta al archivo sin firma.

Método API_ENCRYPT_SIGN_FILE(ARG_FILE, ARG_KEY_ID, ARG_OUT_FILE)

Este método encripta un archivo con una clave pública previamente registrada y firma con la clave privada de Meta4. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a encriptar y firmar.
- ARG_KEY_ID: identificador de una clave pública registrada en el sistema, con la que encriptar el archivo.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo encriptado y firmado.

Método API_DECRYPT_VERIFY_FILE(ARG_FILE, ARG_EXT, ARG_KEY_ID, ARG_OUT_FILE)

Este método desencripta un archivo con la clave privada de Meta4 y valida la firma con una clave pública previamente registrada. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a desencriptar y verificar.
- ARG_EXT: extensión del archivo resultante de la desencriptación.
- ARG_KEY_ID: identificador de una clave pública registrada en el sistema, con la que verificar la firma.

-
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo descriptado.

Método API_KF_ENCRYPT_FILE(ARG_FILE, ARG_KEY_FILE, ARG_OUT_FILE)

Este método permite llevar a cabo la encriptación de archivos mediante la clave pública que recibe. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a encriptar.
- ARG_KEY_FILE: ruta del archivo de clave pública con el que encriptar el archivo.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo encriptado.

Método API_KF_DECRYPT_FILE(ARG_FILE, ARG_KEY_FILE, ARG_PWD, ARG_EXT, ARG_OUT_FILE)

Este método permite descriptar archivos mediante la clave privada que recibe. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a descriptar.
- ARG_KEY_FILE: ruta del archivo de clave privada con el que descriptar el archivo.
- ARG_PWD: contraseña de la clave privada.
- ARG_EXT: extensión del archivo resultante de la descriptación.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo descriptado.

Método API_KF_SIGN_FILE(ARG_FILE, ARG_KEY_FILE, ARG_PWD, ARG_OUT_FILE)

Este método sirve para firmar un archivo en nombre de la entidad representada por la clave privada que recibe. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a firmar.
- ARG_KEY_FILE: ruta del archivo de clave privada con el que firmar el archivo.
- ARG_PWD: contraseña de la clave privada.

- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta al archivo firmado.

Método API_KF_VERIFY_FILE(ARG_FILE, ARG_KEY_FILE, ARG_EXT, ARG_OUT_FILE)

Este método sirve para verificar un archivo mediante la clave pública que recibe. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a verificar.
- ARG_KEY_FILE: ruta del archivo de clave pública con el que verificar el archivo.
- ARG_EXT: extensión del archivo sin firma una vez verificado.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta al archivo sin firma.

Método API_KF_ENCRYPT_SIGN_FILE(ARG_FILE, ARG_ENCR_KEY_FILE, ARG_SIGN_KEY_FILE, ARG_PWD, ARG_OUT_FILE)

Este método encripta y firma un archivo con los ficheros de clave pública y privada especificados. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a encriptar y firmar.
- ARG_ENCR_KEY_FILE: ruta del archivo de clave pública con el que encriptar el archivo.
- ARG_SIGN_KEY_FILE: ruta del archivo de clave privada con el que firmar el archivo.
- ARG_PWD: contraseña de la clave privada.
- ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo encriptado y firmado.

Método API_KF_DECRYPT_VERIFY_FILE(ARG_FILE, ARG_DECR_KEY_FILE, ARG_PWD, ARG_VERIFY_KEY_FILE, ARG_EXT, ARG_OUT_FILE)

Este método desencripta y verifica la firma de un archivo, con los ficheros de clave privada y pública especificados. Dispone de los siguientes argumentos:

- ARG_FILE: ruta del archivo a desencriptar y verificar.

-
- ARG_DECR_KEY_FILE: ruta del archivo de clave privada con el que descriptar el archivo.
 - ARG_PWD: contraseña de la clave privada.
 - ARG_VERIFY_KEY_FILE: ruta del archivo de clave pública con el que verificar el archivo.
 - ARG_EXT: extensión del archivo resultante de la descriptación.
 - ARG_OUT_FILE: argumento de salida, donde se devuelve la ruta del archivo descriptado.

Uso del API para generar la información asociada a los componentes de un concepto final

La aplicación ofrece un API de desarrollo para generar la información asociada a los componentes de un concepto final, con el objetivo de facilitar la creación de conceptos finales de forma programática. Para estos componentes se generará la información del nodo auxiliar, del nodo de revisión, del nodo de diferencias y sus respectivas reglas de revisión y diferencias. Se regeneran las reglas de plantillas asociadas al concepto final.

Esta funcionalidad está disponible en:

- Meta4Object: SRTC_API_GENERATE_PI
- Nodo: SRTC_API_GENERATE_PI
- Método: GENERATE_PI

Los argumentos del método son:

- ARG_ID_T3: identificador del Meta4Object de nómina.
- ARG_ID_PI1: identificador del concepto final.
- ARG_ID_PI2: identificador del concepto final (opcional).

El método devuelve los siguientes valores:

- 0: M4_SUCCESS: Operación realizada con éxito.
- -1: M4_ERROR: Error en la generación.

Cómo pasar valores a parámetros para consultas nuevas

Desde las presentaciones de la aplicación puede realizar una llamada a una consulta pasándole parámetros.

A continuación se muestra un ejemplo de llamada a una consulta desde una presentación pasando parámetros:

EJEMPLO

```
BEGIN Button Button1
  Idchannel = "DEM1_UT_PERSONA_REL_0825122219"
  Idnode = "DEM1_UT_PERSON_STD_PERSON"
  Left = 157
  Top = 112
  Width = 112
  Height = 44
  Text = "Button1"
BEGIN Evclick Evclick1
  BEGIN Action_set asIDT3
    Set = "*C*ID_T3:=[*F*"DEM1_UT_PERSONA_REL_0825122219"]"
  END
  BEGIN Action_set asIDReport
    Set = "*C*ID_REPORT:=[*F*"DEM1_UT_PERSONA_REL_0825122219"]"
  END
  BEGIN Action_set DEFINE_PRESENTATION
    Set = "*C*#SYS_BLOCK_CONTEXT_VAL#:=SetParamConcatString
DEM1_UT_PERSON_STD_PERSON,PARAM_1,[*O*/Presentation1/Form1/
txtIdPersonFilter.Text];SetParamConcatString
DEM1_UT_PERSON_STD_PERSON,PARAM_2,[*O*/Presentation1/Form1/
TxtIsVeteranFilter.Text]"
  END
  BEGIN Action_task CONTEXTO
    Class = "M4setval.setcontextval"
  END

  BEGIN Action_bp Action_bpMN_QUERY_EDIT
    Idbp = "EXAMPLE_EXE_QUERY_FROM_MENU"
    Load = False
    Localcontext = False
  END
END
END
```

Cómo llamar a una consulta desde una opción de menú

Desde la aplicación puede realizar una llamada a una consulta desde una opción de menú.

Para colgar una consulta de una opción de menú, debe crear una tarea para asociarla a la opción de menú con las siguientes características:

- La tarea debe ser de tipo ejecución de presentación (por ejemplo, la tarea `EXAMPLE_EXE_QUERY_FROM_MENU`).
- Debe tener los siguientes parámetros:
 - `#SYS_ID_PRESENTATION#`, ordinal 1, de tipo cadena y de entrada. Valor por defecto `"SRTC_QR_LIST_MY_QUERIES_RPT_DESKTOP"`.
 - `ID_T3`, ordinal 2 de tipo cadena y de entrada. Valor por defecto: el identificador de la consulta a ejecutar.
 - `ID_REPORT`, ordinal 3 de tipo cadena y de entrada. Valor por defecto: el identificador del informe asociado a la consulta a ejecutar. Si es una consulta nueva será el mismo identificador que el del `Meta4Object`.
 - `ID_LETTER`, ordinal 4 de tipo cadena y de entrada. Valor por defecto vacío si no es una carta. Debe rellenarse con el identificador si es una carta.
 - `DESKTOP_CALL`, ordinal 5 de tipo cadena y de entrada. Valor por defecto: 1.
 - Código Cliente Windows: `StForm.PTCMFTask`



Si la tarea es susceptible de tener varios roles en ejecución y suma de roles, es preferible meterla como excepción para que trabaje con todos los roles y no lo pida al ejecutarla. Luego, al ejecutar la consulta, ya se pedirá.

